Zcim Project

Digital Research CP/M® 2.2

Operating System Manual

version 2025-08-02

Printing

The *CP/M Operating System Manual* **was** printed in the United States of America.

First Edition: 1976

Second Edition: July 1982

Third Edition: September 1983

Zcim Project edition: version 2025-08-02

Contents

List of Tables

1. CP/M Features and Facilities

1.1. Introduction

CP/M is a monitor control program for microcomputer system development that uses floppy disks or Winchester hard disks for backup storage. Using a computer system based on the Intel 8080 microcomputer, CP/M provides an environment for program construction, storage, and editing, along with assembly and program checkout facilities. CP/M can be easily altered to execute with any computer configuration that uses a Zilog Z80 or an Intel 8080 Central Processing Unit (CPU) and has at least 20K bytes of main memory with up to 16 disk drives. A detailed discussion of the modifications required for any particular hardware environment is given in Section 6. Although the standard Digital Research version operates on a single-density Intel MDS 800, several different hardware manufacturers support their own input-output (I/O) drivers for CP/M.

The CP/M monitor provides rapid access to programs through a comprehensive file management package. The file subsystem supports a named file structure, allowing dynamic allocation of file space as well as sequential and random file access. Using this file system, a large number of programs can be stored in both source and machine executable form.

CP/M 2 is a high-performance, single console operating system that uses table-driven techniques to allow field reconfiguration to match a wide variety of disk capacities. All fundamental file restrictions are removed, maintaining upward compatibility from previous versions of release 1.

Features of CP/M 2 include field specification of one to sixteen logical drives, each containing up to eight megabytes. Any particular file can reach the full drive size with the capability of expanding to thirty-two megabytes in future releases. The directory size can be field-configured to contain any reasonable number of entries, and each file is optionally tagged with Read-Only and system attributes. Users of CP/M 2 are physically separated by user numbers, with facilities for file copy operations from one user area to another. Powerful relative-record random access functions are present in CP/M 2 that provide direct access to any of the 65536 records of an eight-megabyte file.

CP/M also supports ED, a powerful context editor, ASM, an Intel-compatible assembler, and DDT, debugger subsystems. Optional software includes a powerful Intel-compatible macro assembler, symbolic debugger, along with various high-level languages. When coupled with CP/M's Console Command Processor (CCP), the resulting facilities equal or exceed similar large computer facilities.

CP/M is logically divided into several distinct parts:

BIOS (Basic I/O System), hardware-dependent

BDOS (Basic Disk Operating System)

CCP (Console Command Processor)

TPA (Transient Program Area)

The BIOS provides the primitive operations necessary to access the disk drives and to interface standard peripherals: teletype, CRT, paper tape reader/punch, and user-defined peripherals. You can tailor peripherals for any particular hardware environment by patching this portion of CP/M. The BDOS provides disk management by controlling one or more disk drives containing independent file directories. The BDOS implements disk allocation strategies that provide fully dynamic file construction while minimizing head movement across the disk during access. The BDOS has entry points that include the following primitive operations, which the program accesses:

SEARCH looks for a particular disk file by name.

OPEN opens a file for further operations.

CLOSE closes a file after processing.

RENAME changes the name of a particular file.

READ reads a record from a particular file.

WRITE writes a record to a particular file.

SELECT selects a particular disk drive for further operations.

The CCP provides a symbolic interface between your console and the remainder of the CP/M system. The CCP reads the console device and processes commands, which include listing the file directory, printing the contents of files, and controlling the operation of transient programs, such as assemblers, editors, and debuggers. The standard commands that are available in the CCP are listed in Section 1.2.1.

The last segment of CP/M is the area called the Transient Program Area (TPA). The TPA holds programs that are loaded from the disk under command of the CCP. During program editing, for example, the TPA holds the CP/M text editor machine code and data areas. Similarly, programs created under CP/M can be checked out by loading and executing these programs in the TPA.

Any or all of the CP/M component subsystems can be overlaid by an executing program. That is, once a user's program is loaded into the TPA, the CCP, BDOS, and BIOS areas can be used as the program's data area. A bootstrap loader is programmatically accessible whenever the BIOS portion is not overlaid; thus, the user program need only branch to the bootstrap loader at the end of execution and the complete CP/M monitor is reloaded from disk.

The CP/M operating system is partitioned into distinct modules, including the BIOS portion that defines the hardware environment in which CP/M is executing. Thus, the standard system is

easily modified to any nonstandard environment by changing the peripheral drivers to handle the custom system.

1.2. Functional Description

You interact with CP/M primarily through the CCP, which reads and interprets commands entered through the console. In general, the CCP addresses one of several disks that are on-line. The standard system addresses up to sixteen different disk drives. These disk drives are labeled A through P. A disk is logged-in if the CCP is currently addressing the disk. To clearly indicate which disk is the currently logged disk, the CCP always prompts the operator with the disk name followed by the symbol >, indicating that the CCP is ready for another command. Upon initial start-up, the CP/M system is loaded from disk A, and the CCP displays the following message:

`CP/M VER` *x.x*

where *x.x* is the CP/M version number. All CP/M systems are initially set to operate in a 20K memory space, but can be easily reconfigured to fit any memory size on the host system (see Section 1.6.9). Following system sign-on, CP/M automatically logs in disk A, prompts you with the symbol A>, indicating that CP/M is currently addressing disk A, and waits for a command. The commands are implemented at two levels: built-in commands and transient commands.

1.2.1. General Command Structure

Built-in commands are a part of the CCP program, while transient commands are loaded into the TPA from disk and executed. The following are built-in commands:

ERA erases specified files.

DIR lists filenames in the directory.

REN renames the specified file.

SAVE saves memory contents in a file.

TYPE types the contents of a file on the logged disk.

Most of the commands reference a particular file or group of files. The form of a file reference is specified in Section 1.2.2.

1.2.2. File References

A file reference identifies a particular file or group of files on a particular disk attached to CP/M. These file references are either unambiguous (*ufn*) or ambiguous (*afn*). An unambiguous file reference uniquely identifies a single file, while an ambiguous file reference is satisfied by a number of different files.

File references consist of two parts: the primary filename and the filetype. Although the filetype is optional, it usually is generic. For example, the filetype ASM is used to denote that the file is an

assembly language source file, while the primary filename distinguishes each particular source file. The two names are separated by a period, as shown in the following example:

filename.*typ*

In this example, *filename* is the primary filename of eight characters or less, and *typ* is the filetype of no more than three characters. As mentioned above, the name

filename

is also allowed and is equivalent to a filetype consisting of three blanks. The characters used in specifying an unambiguous file reference cannot contain any of the following special characters:

```
<   >   .   ,   ;   :   =   ?   *   [   ]   %   |   (   )   /   \
```

while all alphanumerics and remaining special characters are allowed.

An ambiguous file reference is used for directory search and pattern matching. The form of an ambiguous file reference is similar to an unambiguous reference, except the symbol `?` can be interspersed throughout the primary and secondary names. In various commands throughout CP/M, the `?` symbol matches any character of a filename in the `?` position. Thus, the ambiguous reference

```
X?Z.C?M
```

matches the following unambiguous filenames:

```
XYZ.COM
```

and

```
X3Z.CAM
```

The wildcard character can also be used in an ambiguous file reference. The `*` character replaces all or part of a filename or filetype. Note that

```
*.*
```

equals the ambiguous file reference

```
????????.???
```

while

```
filename.*
```

and

```
*.typ
```

are abbreviations for

```
filename.???
```

and

```
????????.typ
```

respectively. As an example,

```
A>DIR *.*
```

is interpreted by the CCP as a command to list the names of all disk files in the directory. The following example searches only for a file by the name `X.Y`:

```
A>DIR X.Y
```

Similarly, the command

```
A>DIR X?Y.C?M
```

causes a search for all unambiguous filenames on the disk that satisfy this ambiguous reference.

The following file references are valid unambiguous file references:

```
X
X.Y
XYZ
XYZ.COM
GAMMA
GAMMA.1
```

As an added convenience, the programmer can generally specify the disk drive name along with the filename. In this case, the drive name is given as a letter `A` through `P` followed by a colon (`:`). The specified drive is then logged-in before the file operation occurs. Thus, the following are valid file references with disk name prefixes:

```
A:X.Y
P:XYZ.COM
B:XYZ
B:X.A?M
C:GAMMA
C:*.ASM
```

All alphabetic lower-case letters in file and drive names are translated to upper-case when they are processed by the CCP.

1.3. Switching Disks

The operator can switch the currently logged disk by typing the disk drive name, `A` through `P`, followed by a colon when the CCP is waiting for console input. The following sequence of prompts and commands can occur after the CP/M system is loaded from disk `A`:

```
CP/M VER 2.2
A>DIR↵                          List all the files on disk A
A:SAMPLE   ASM SAMPLE   PRN
A>B:↵                           Switch to disk B
B>DIR *.ASM↵                    List all ASM files on B
B:DUMP     ASM FILES    ASM
B>A>↵                           Switch back to A
A>
```

1.4. Built-in Commands

The file and device reference forms described can now be used to fully specify the structure of the built-in commands. Assume the following abbreviations in the description below:

ufn unambiguous file name reference

afn ambiguous file name reference

1.4.1. ERA Command

Syntax:

ERA *afn*

The ERA (erase) command removes files from the currently logged- in disk, for example, the disk name currently prompted by CP/M preceding the >. The files that are erased are those that satisfy the ambiguous file reference *afn*. The following examples illustrate the use of ERA:

`ERA X.Y`	The file named X.Y on the currently logged disk is removed from the disk directory and the space is returned.
`ERA X.*`	All files with primary name X are removed from the current disk.
`ERA *.ASM`	All files with secondary name ASM are removed from the current disk.

`ERA X?Y.C?M`	All files on the current disk that satisfy the ambiguous reference `X?Y.C?M` are deleted.
`ERA *.*`	Erase all files on the current disk. In this case, the CCP prompts the console with the message `ALL FILES (Y/N)?` which requires a `Y` response before files are actually removed.
`ERA B:*.PRN`	All files on drive `B` that satisfy the ambiguous reference `????????.PRN` are deleted, independently of the currently logged disk.

1.4.2. `DIR` Command

Syntax:

`DIR` *afn*

The `DIR` (directory) command causes the names of all files that satisfy the ambiguous filename *afn* to be listed at the console device. As a special case, the command

```
DIR
```

lists the files on the currently logged disk (the command `DIR` is equivalent to the command `DIR *.*`). The following are valid `DIR` commands:

```
DIR X.Y
DIR X?Y.C?M
DIR ??.Y
```

Similar to other CCP commands, the *afn* can be preceded by a drive name. The following `DIR` commands cause the selected drive to be addressed before the directory search takes place:

```
DIR B:
DIR B:X.Y
DIR B:*.A?M
```

If no files on the selected disk satisfy the directory request, the message

```
NO FILE
```

appears at the console.

1.4.3. `REN` Command

Syntax

`REN` *ufn1=ufn2*

The REN (rename) command allows you to change the names of files on disk. The file satisfying *ufn2* is changed to *ufn1*. The currently logged disk is assumed to contain the file to rename (*ufn2*). You can also type a left-directed arrow (5FH) instead of the equal sign if the console supports this graphic character (normally rendered as an underscore "_" on modern systems). The following are examples of the REN command:

`REN X.Y=Q.R`	The file `Q.R` is changed to `X.Y`.
`REN XYZ.COM=XYZ.XXX`	The file `XYZ.XXX` is changed to `XYZ.COM`.

The operator precedes either *ufn1* or *ufn2* (or both) by an optional drive address. If *ufn1* is preceded by a drive name, then *ufn2* is assumed to exist on the same drive. Similarly, if *ufn2* is preceded by a drive name, then *ufn1* is assumed to exist on the drive as well. The same drive must be specified in both cases if both *ufn1* and *ufn2* are preceded by drive names. The following REN commands illustrate this format:

`REN A:X.ASM=Y.ASM`	The file `Y.ASM` is changed to `X.ASM` on drive `A`.
`REN B:ZAP.BAS=ZOT.BAS`	The file `ZOT.BAS` is changed to `ZAP.BAS` on drive `B`.
`REN B:A.ASM=B:A.BAK`	The file `A.BAK` is renamed to `A.ASM` on drive `B`.

If *ufn1* is already present, the REN command responds with the error

```
FILE EXISTS
```

and not perform the change. If *ufn2* does not exist on the specified disk, the message

```
NO FILE
```

is printed at the console.

1.4.4. SAVE Command

Syntax:

SAVE *n ufn*

The SAVE command places *n* pages (256-byte blocks) onto disk from the TPA and names this file *ufn*. In the CP/M distribution system, the TPA starts at 0100H (hexadecimal) which is the second page of memory. The SAVE command must specify 2 pages of memory if the user's program occupies the area from 0100H through 02FFH. The machine code file can be subsequently loaded and executed. The following are examples of the SAVE command:

`SAVE 3 X.COM`	Copies 0100H through 03FFH to `X.COM`.

`SAVE 40 Q`	Copies `0100H` through `28FFH` to `Q`. Note that 28 is the page count in 28FFH, and that $28H = 2 * 16 + 8 = 40$ decimal.
`SAVE 4 X.Y`	Copies `0100H` through `04FFH` to `X.Y`.

The `SAVE` command can also specify a disk drive in the *ufn* portion of the command, as shown in the following example:

`SAVE 10 B:ZOT.COM`	Copies 10 pages, `0100H` through `0AFFH`, to the file `ZOT.COM` on drive `B`.

1.4.5. `TYPE` Command

Syntax:

`TYPE` *ufn*

The `TYPE` command displays the content of the ASCII source file *ufn* on the currently logged disk at the console device. The following are valid `TYPE` commands:

```
TYPE X.Y
TYPE X.PLM
TYPE XXX
```

The `TYPE` command expands tabs, CTRL-I characters, assuming tab positions are set at every eighth column. The *ufn* can also reference a drive name.

`TYPE B:X.PRN`	The file `X.PRN` from drive `B` is displayed.

1.4.6. `USER` Command

Syntax:

`USER` *n*

The `USER` command allows maintenance of separate files in the same directory. In the syntax line, *n* is an Integer value in the range 0 to 15. On cold start, the operator is automatically logged into user area number 0, which is compatible with standard CP/M 1 directories. You can issue the `USER` command at any time to move to another logical area within the same directory. Drives that are logged-in while addressing one user number are automatically active when the operator moves to another. A user number is simply a prefix that accesses particular directory entries on the active disks.

The active user number is maintained until changed by a subsequent `USER` command, or until a cold start when user 0 is again assumed.

1.5. Line Editing and Output Control

The CCP allows certain line-editing functions while typing command lines. The CTRL-key sequences are obtained by pressing the control and letter keys simultaneously. Further, CCP command lines are generally up to 255 characters in length; they are not acted upon until the carriage return key is pressed.

Character	Meaning
CTRL-C	Reboots CP/M system when pressed at start of line.
CTRL-E	Physical end of line; carriage is returned, but line is not sent until the carriage return key is pressed.
CTRL-H	Backspaces one character position.
CTRL-J	Terminates current input (line-feed).
CTRL-M	Terminates current input (carriage return).
CTRL-P	Copies all subsequent console output to the currently assigned list device (see Section 1.6.1). Output is sent to the list device and the console device until the next CTRL-P is pressed.
CTRL-R	Retypes current command line; types a clean line following character deletion with rubouts.
CTRL-S	Stops the console output temporarily. Program execution and output continue when you press any character at the console, for example another CTRL-S. This feature stops output on high speed consoles, such as CRTs, in order to view a segment of output before continuing.
CTRL-U	Deletes the entire line typed at the console.
CTRL-X	Same as CTRL-U.
CTRL-Z	Ends input from the console (used in `PIP` and `ED`).
rubout/del	Deletes and echoes the last character typed at the console.

Table 1-1: Line-editing Control Characters

1.6. Transient Commands

Transient commands are loaded from the currently logged disk and executed in the TPA. The transient commands for execution under the CCP are below. Additional functions are easily defined by the user (see Section 1.6.3).

Command	Function
STAT	Lists the number of bytes of storage remaining on the currently logged disk, provides statistical information about particular files, and displays or alters device assignment.
ASM	Loads the CP/M assembler and assembles the specified program from disk.
LOAD	Loads the file in Intel HEX machine code format and produces a file in machine executable form which can be loaded into the TPA. This loaded program becomes a new command under the CCP.
DDT	Loads the CP/M debugger into TPA and starts execution.
PIP	Loads the Peripheral Interchange Program for subsequent disk file and peripheral transfer operations.
ED	Loads and executes the CP/M text editor program.
SYSGEN	Creates a new CP/M system disk.
SUBMIT	Submits a file of commands for batch processing.
DUMP	Dumps the contents of a file in hex.
MOVCPM	Regenerates the CP/M system for a particular memory size.

Table 1-2: CP/M Transient Commands

Transient commands are specified in the same manner as built-in commands, and additional commands are easily defined by the user. For convenience, the transient command can be preceded by a drive name which causes the transient to be loaded from the specified drive into the TPA for execution. Thus, the command

```
B:STAT
```

causes CP/M to temporarily log in drive B for the source of the STAT transient, and then return to the original logged disk for subsequent processing.

1.6.1. `STAT` Command

Syntax:

`STAT`

`STAT` *command line*

Command	Function
STAT	If you type an empty command line, the STAT transient calculates the storage remaining on all active drives, and prints one of the following messages: *d*: R/W, Space: *nnn*k *d*: R/O, Space: *nnn*k for each active drive *d*:, where R/W indicates the drive can be read or written, and R/O indicates the drive is Read-Only (a drive becomes R/O by explicitly setting it to Read- Only, as shown below, or by inadvertently changing disks without performing a warm start). The space remaining on the disk in drive *d*: is given in kilobytes by *nnn*.
STAT *d*:	If a drive name is given, then the drive is selected before the storage is computed. Thus, the command STAT B: could be issued while logged into drive A, resulting in the message Bytes Remaining On B: *nnn*k
STAT *afn*	The command line can also specify a set of files to be scanned by STAT. The files that satisfy *afn* are listed in alphabetical order, with storage requirements for each file under the heading: Recs Bytes Ext Acc *rrrr* *bbb*k *ee acc d:filename.typ* where *rrrr* is the number of 128-byte records allocated to the file, *bbb* is the number of kilobytes allocated to the file $$\mathrm{bbb} = \frac{\mathrm{rrrr} * 128}{1024}$$ *ee* is the number of 16K extents $$\mathrm{ee} = \frac{\mathrm{bbb}}{16}$$ *acc* is either R/O, or R/W, *d* is the drive name containing the file (A ... P), *filename* is the eight-character primary filename, and *typ* is the three-character filetype. After listing the individual files, the storage usage is summarized.

Command	Function
STAT *d*:*afn*	The drive name can be given ahead of the *afn*. The specified drive is first selected, and the form STAT *afn* is executed.
STAT *d*:=R/O	This form sets the drive given by *d* to Read-Only, remaining in effect until the next warm or cold start takes place. When a disk is Read-Only, the message BDOS Err On *d*: R/O appears if there is an attempt to write to the Read-Only disk. CP/M waits until a key is pressed before performing an automatic warm start, at which time the disk becomes R/W.

The STAT command allows you to control the physical-to-logical device assignment. See the IOBYTE function described in Sections 5 and 6.

There are four logical peripheral devices that are, at any particular instant, each assigned one of several physical peripheral devices. The following is a list of the four logical devices:

CON: is the system console device, used by CCP for communication with the operator.

RDR: is the paper tape reader device.

PUN: is the paper tape punch device.

LST: is the output list device.

The actual devices attached to any particular computer system are driven by subroutines in the BIOS portion of CP/M. Thus, the logical RDR: device, for example, could actually be a high speed reader, teletype reader, or cassette tape. To allow some flexibility in device naming and assignment, several physical devices are defined in Table 1-3.

Device	Meaning
TTY:	Teletype device (slow speed console)
CRT:	Cathode ray tube device (high speed console)
BAT:	Batch processing (console is current RDR:, output goes to current LST: device)
UC1:	User-defined console
PTR:	Paper tape reader (high speed reader)
UR1:	User-defined reader #1
UR2:	User-defined reader #2
PTP:	Paper tape punch (high speed punch)
UP1:	User-defined punch #1
UP2:	User-defined punch #2
LPT:	Line printer
UL1:	User-defined list device #1

Table 1-3: Physical Devices

It is emphasized that the physical device names might not actually correspond to devices that the names imply. That is, you can implement the PTP: device as a cassette write operation. The exact correspondence and driving subroutine is defined in the BIOS portion of CP/M. In the standard distribution version of CP/M, these devices correspond to their names on the MDS 800 development system.

The command:

```
STAT VAL:
```

produces a summary of the available status commands, resulting in the output:

```
Temp R/O Disk d:=R/O
Set Indicator: d:filename.typ $R/O $R/W $SYS $DIR
Disk Status: DSK: d:DSK
User Status: USR:
```

which gives an instant summary of the possible STAT commands and shows the permissible logical-to-physical device assignments:

```
Iobyte Assign:
CON:=TTY:CRT:BAT:UCI:
RDR:=TTY:PTR:URI:UR2:
PUN:=TTY:PTP:UP1:UP2:
LST:=TTY:CRT:LPT:ULI:
```

The logical device to the left takes any of the four physical assignments shown to the right. The current logical-to-physical mapping is displayed by typing the command:

```
STAT DEV:
```

This command produces a list of each logical device to the left and the current corresponding physical device to the right. For example, the list might appear as follows:

```
CON:=CRT:
RDR:=URl:
PUN:=PTP:
LST:=TTY:
```

The current logical-to-physical device assignment is changed by typing a `STAT` command of the form:

`STAT` *ld1=pd1*,*ld2=pd2*,...*ldn=pdn*,

where *ld1* through *ldn* are logical device names and *pd1* through *pdn* are compatible physical device names. For example, *ld1* and *pd1* appear on the same line in the `VAL:` command shown above. The following example shows valid `STAT` commands that change the current logical-to-physical device assignments:

```
STAT CON:=CRT:
STAT PUN:=TTY:,LST:=LPT:,RDR:=TTY
```

The command form:

`STAT` *d*:*filename*.*typ* `$S`

where *d*: is an optional drive name and *filename.typ* is an unambiguous or ambiguous filename, produces the following output display format:

```
   Size      Recs      Bytes   Ext Acc

     48        48         6K     1 R/O A:ED.COM
     55        55        12K     1 R/O (A:PIP.COM)
  65536       128        16K     2 R/W A:X.DAT
Bytes Remaining On A: 168k
```

where the `$S` parameter causes the `Size` field to be displayed. Without the `$S`, the `Size` field is skipped, but the remaining fields are displayed. The `Size` field lists the virtual file size in records, while the `Recs` field sums the number of virtual records in each extent. For files constructed sequentially, the `Size` and `Recs` fields are identical. The `Bytes` field lists the actual

number of bytes allocated to the corresponding file. The minimum allocation unit is determined at configuration time; thus, the number of bytes corresponds to the record count plus the remaining unused space in the last allocated block for sequential files. Random access files are given data areas only when written, so the `Bytes` field contains the only accurate allocation figure. In the case of random access, the `Size` field gives the logical end-of-file record position and the `Recs` field counts the logical records of each extent. Each of these extents, however, can contain unallocated holes even though they are added into the record count.

The `Ext` field counts the number of physical extents allocated to the file. The `Ext` count corresponds to the number of directory entries given to the file. Depending on allocation size, there can be up to 128K bytes (8 logical extents) directly addressed by a single directory entry. In a special case, there are actually 256K bytes that can be directly addressed by a physical extent.

The `Acc` field gives the `R/O` or `R/W` file indicator, which you can change using the commands shown. The four command forms,

`STAT` *d*:*filename*.*typ* `$R/O`

`STAT` *d*:*filename*.*typ* `$R/W`

`STAT` *d*:*filename*.*typ* `$SYS`

`STAT` *d*:*filename*.*typ* `$DIR`

set or reset various permanent file indicators. The `R/O` indicator places the file, or set of files, in a Read-Only status until changed by a subsequent `STAT` command. The `R/O` status is recorded in the directory with the file so that it remains `R/O` through intervening cold start operations. The `R/W` indicator places the file in a permanent Read-Write status. The `SYS` indicator attaches the system indicator to the file, while the `DIR` command removes the system indicator. The *filename.typ* may be ambiguous or unambiguous, but files whose attributes are changed are listed at the console when the change occurs. The drive name denoted by *d*: is optional.

When a file is marked `R/O`, subsequent attempts to erase or write into the file produce the following BDOS message at your screen:

`BDOS Err On` *d*`: File R/O`

lists the drive characteristics of the disk named by *d*: that is in the range `A:`, `B:`, ..., `P:`. The drive characteristics are listed in the following format:

```
    d: Drive Characteristics
65536: 128 Byte Record Capacity
 8192: Kilobyte Drive Capacity
  128: 32 Byte Directory Entries
    0: Checked Directory Entries
 1024: Records/Extent
  128: Records/Block
   58: Sectors/Track
    2: Reserved Tracks
```

where *d*: is the selected drive, followed by the total record capacity (65536 is an eight-megabyte drive), followed by the total capacity listed in kilobytes. The directory size is listed next, followed by the checked entries. The number of checked entries is usually identical to the directory size for removable media, because this mechanism is used to detect changed media during CP/M operation without an intervening warm start. For fixed media, the number is usually zero, because the media are not changed without at least a cold or warm start.

The number of records per extent determines the addressing capacity of each directory entry (1024 times 128 bytes, or 128K in the previous example). The number of records per block shows the basic allocation size (in the example, 128 records/block times 128 bytes per record, or 16K bytes per block). The listing is then followed by the number of physical sectors per track and the number of reserved tracks.

For logical drives that share the same physical disk, the number of reserved tracks can be quite large because this mechanism is used to skip lower-numbered disk areas allocated to other logical disks. The command form:

```
STAT DSK:
```

produces a drive characteristics table for all currently active drives. The final `STAT` command form is

```
STAT USR:
```

which produces a list of the user numbers that have files on the currently addressed disk. The display format is

```
Active User : 0
Active Fi1es: 0 1 3
```

where the first line lists the currently addressed user number, as set by the last CCP `USER` command, followed by a list of user numbers scanned from the current directory. In this case, the active user number is `0` (default at cold start) with three user numbers that have active files on the current disk. The operator can subsequently examine the directories of the other user numbers by logging in with `USER 1` or `USER 3` commands, followed by a `DIR` command at the CCP level.

1.6.2. ASM Command

Syntax:

ASM *ufn*

The ASM command loads and executes the CP/M 8080 assembler. The ufn specifies a source file containing assembly language statements, where the filetype is assumed to be ASM and is not specified. The following ASM commands are valid:

```
ASM X
ASM GAMMA
```

The two-pass assembler is automatically executed. Assembly errors that occur during the second pass are printed at the console.

The assembler produces a file:

X.PRN

where *X* is the primary name specified in the ASM command. The PRN file contains a listing of the source program with embedded tab characters if present in the source program, along with the machine code generated for each statement and diagnostic error messages, if any. The PRN file is listed at the console using the TYPE command, or sent to a peripheral device using PIP (see Section 1.6.4). Note that the PRN file contains the original source program, augmented by miscellaneous assembly information in the leftmost 16 columns; for example, program addresses and hexadecimal machine code. The PRN file serves as a backup for the original source file. If the source file is accidentally removed or destroyed, the PRN file can be edited by removing the leftmost 16 characters of each line (see Section 2). This is done by issuing a single editor macro command. The resulting file is identical to the original source file and can be renamed from PRN to ASM for subsequent editing and assembly. The file

X.HEX

is also produced, which contains 8080 machine language in Intel HEX format suitable for subsequent loading and execution (see Section 1.6.3). For complete details of CP/M's assembly language program, see Section 3.

The source file for assembly is taken from an alternate disk by prefixing the assembly language filename by a disk drive name. The command

```
ASM B:ALPHA
```

loads the assembler from the currently logged drive and processes the source program ALPHA.ASM on drive B. The HEX and PRN files are also placed on drive B in this case.

1.6.3. LOAD Command

Syntax:

LOAD *ufn*

The LOAD command reads the file *ufn*, which is assumed to contain HEX format machine code, and produces a memory image file that can subsequently be executed. The filename *ufn* is assumed to be of the form:

X.HEX

and only the filename *X* need be specified in the command. The LOAD command creates a file named

X.COM

that marks it as containing machine executable code. The file is actually loaded into memory and executed when the user types the filename X immediately after the prompting character > printed by the CCP.

Generally, the CCP reads the filename X following the prompting character and looks for a built-in function name. If no function name is found, the CCP searches the system disk directory for a file by the name

X.COM

If found, the machine code is loaded into the TPA, and the program executes. Thus, the user need only LOAD a hex file once; it can be subsequently executed any number of times by typing the primary name. This way, you can invent new commands in the CCP. Initialized disks contain the transient commands as COM files, which are optionally deleted. The operation takes place on an alternate drive if the filename is prefixed by a drive name. Thus,

```
LOAD B:BETA
```

brings the LOAD program into the TPA from the currently logged disk and operates on drive B after execution begins.

Note: the BETA.HEX file must contain valid Intel format hexadecimal machine code records (as produced by the ASM program, for example) that begin at 0100H of the TPA. The addresses in the hex records must be in ascending order; gaps in unfilled memory regions are filled with zeroes by the LOAD command as the hex records are read. Thus, LOAD must be used only for creating CP/M standard COM files that operate in the TPA. Programs that occupy regions of memory other than the TPA are loaded under DDT.

1.6.4. PIP Command

Syntax:

PIP

PIP *destination*=*source#1*,*source#2*,...*source#n*

PIP is the CP/M Peripheral Interchange Program that implements the basic media conversion operations necessary to load, print, punch, copy, and combine disk files. The PIP program is initiated by typing one of the following forms:

PIP

PIP *command line*

In both cases PIP is loaded into the TPA and executed. In the first form, PIP reads command lines directly from the console, prompted with the * character, until an empty command line is typed (for example, a single carriage return is issued by the operator). Each successive command line causes some media conversion to take place according to the rules shown below.

In the second form, the PIP command is equivalent to the first, except that the single command line given with the PIP command is automatically executed, and PIP terminates immediately with no further prompting of the console for input command lines. The form of each command line is

destination=*source#1*,*source#2*,...*source#n*

where *destination* is the file or peripheral device to receive the data, and *source#1*,*source#2*,... *source#n* is a series of one or more files or devices that are copied from left to right to the destination.

When multiple files are given in the command line (for example, $n > 1$), the individual files are assumed to contain ASCII characters, with an assumed CP/M end-of-file character (CTRL-Z) at the end of each file (see the O parameter to override this assumption). Lower-case ASCII alphabetic characters are internally translated to upper-case to be consistent with CP/M file and device name conventions. Finally, the total command line length cannot exceed 255 characters. CTRL-E can be used to force a physical carriage return for lines that exceed the console width.

The destination and source elements are unambiguous references to CP/M source files with or without a preceding disk drive name. That is, any file can be referenced with a preceding drive name (A: through P:) that defines the particular drive where the file can be obtained or stored. When the drive name is not included, the currently logged disk is assumed. The destination file can also appear as one or more of the source files; in which case the source file is not altered until the entire concatenation is complete. If it already exists, the destination file is removed if the command line is properly formed. It is not removed if an error condition arises. The following command lines, with explanations to the right, are valid as input to PIP:

X=Y	Copies to file X from file Y, where X and Y are unambiguous filenames; Y remains unchanged.

`X=Y,Z`	Concatenates files `Y` and `Z` and copies to file `X`, with `Y` and `Z` unchanged.
`X.ASM=Y.ASM,Z.ASM`	Creates the file `X.ASM` from the concatenation of the `Y.ASM` and `Z.ASM` files.
`NEW.ZOT=B:OLD.ZAP`	Moves a copy of `OLD.ZAP` from drive `B` to the currently logged disk; names the file `NEW.ZOT`.
`B:A.U=B:B.V,A:C.W,D.X`	Concatenates file `B.Y` from drive `B` with `C.W` from drive `A` and `D.X` from the logged disk; creates the file `A.U` on drive `B`.

For convenience, `PIP` allows abbreviated commands for transferring files between disk drives. The abbreviated `PIP` forms are

`PIP` *d*`:=`*afn*

`PIP` *d1*`:=`*d2*`:`*afn*

`PIP` *ufn*`:=`*d2*

`PIP` *d1*`:`*ufn*`=`*d2*`:`

The first form copies all files from the currently logged disk that satisfy the *afn* to the same files on drive *d*, where *d* = `A`...`P`. The second form is equivalent to the first, where the source for the copy is drive *d2* where *d2* = `A` ... `P`. The third form is equivalent to the command `PIP` *d1*`:`*ufn*`=`*d2*`:`*ufn* which copies the file given by *ufn* from drive *d2* to the file *ufn* on drive *d1*. The fourth form is equivalent to the third, where the source disk is explicitly given by *d2*.

The source and destination disks must be different in all of these cases. If an *afn* is specified, `PIP` lists each *ufn* that satisfies the *afn* as it is being copied. If a file exists by the same name as the destination file, it is removed after successful completion of the copy and replaced by the copied file.

The following `PIP` commands give examples of valid disk-to-disk copy operations:

`B=*.COM`	Copies all files that have the secondary name `COM` to drive `B` from the current drive.
`A:=B:ZAP.*`	Copies all files that have the primary name `ZAP` to drive `A` from drive `B`.
`ZAP.ASM=B:`	Same as `ZAP.ASM=B:ZAP.ASM`

`B:ZOT.COM=A:`	Same as `B:ZOT.COM=A:ZOT.COM`
`B:=GAMMA.BAS`	Same as `B:GAMMA.BAS=GAMMA.BAS`
`B:=A:GAMMA.BAS`	Same as `B:GAMMA.BAS=A:GAMMA.BAS`

`PIP` allows reference to physical and logical devices that are attached to the CP/M system. The device names are the same as given under the `STAT` command, along with a number of specially named devices. The following is a list of logical devices given in the `STAT` command

`CON:` console
`RDR:` reader
`PUN:` punch
`LST:` list

while the physical devices are

`TTY:` console, reader, punch or list
`CRT:` console, or list
`UC1:` console
`PTR:` reader
`UR1:` reader
`UR2:` reader
`PTP:` punch
`UP1:` punch
`UP2:` punch
`LPT:` list
`UL1:` list

The `BAT:` physical device is not included, because this assignment is used only to indicate that the `RDR:` and `LST:` devices are used for console input/output.

The `RDR:`, `LST:`, `PUN:`, and `CON:` devices are all defined within the BIOS portion of CP/M, and are easily altered for any particular I/O system. The current physical device mapping is defined by IOBYTE; see Section 6 for a discussion of this function. The destination device must be capable of receiving data, for example, data cannot be sent to the punch, and the source devices must be capable of generating data, for example, the `LST:` device cannot be read.

The following list describes additional device names that can be used in `PIP` commands.

`NUL:` sends 40 nulls (ASCII 0s) to the device. This can be issued at the end of punched output.
`EOF:` sends a CP/M end-of-file (ASCII CTRL-Z) to the destination device (sent automatically at the end of all ASCII data transfers through `PIP`).

PRN: is the same as LST:, except that tabs are expanded at every eighth character position, lines are numbered, and page ejects are inserted every 60 lines with an initial eject (same as using PIP options [t8np]).

INP: is a special PIP input source that can be patched into the PIP program. PIP gets the input data character-by-character, by CALLing location 0103H, with data returned in location 0109H (parity bit must be zero).

OUT: is a special PIP output destination that can be patched into the PIP program. PIP CALLs location 0106H with data in register C for each character to transmit.

Note: The Locations 010AH through 01FFH of the PIP memory image are not used and can be replaced by special purpose drivers using DDT (See Section 4).

File and device names can be interspersed in the PIP commands. In each case, the specific device is read until end-of-file (CTRL-Z for ASCII files, and end-of-data for non-ASCII disk files). Data from each device or file are concatenated from left to right until the last data source has been read.

The destination device or file is written using the data from the source files, and an end-of-file character, CTRL-Z, is appended to the result for ASCII files. If the destination is a disk file, a temporary file is created ($$$ secondary name) that is changed to the actual filename only on successful completion of the copy. Files with the extension COM are always assumed to be non-ASCII.

The copy operation can be aborted at any time by pressing any key on the keyboard. PIP responds with the message

```
ABORTED
```

to indicate that the operation has not been completed. If any operation is aborted, or if an error occurs during processing, PIP removes any pending commands that were set up while using the SUBMIT command.

PIP performs a special function if the destination is a disk file with type HEX (an Intel hex-formatted machine code file), and the source is an external peripheral device, such as a paper tape reader. In this case, the PIP program checks to ensure that the source file contains a properly formed hex file, with legal hexadecimal values and checksum records.

When an invalid input record is found, PIP reports an error message at the console and waits for corrective action. Usually, you can open the reader and rerun a section of the tape (pull the tape back about 20 inches). When the tape is ready for the reread, a single carriage return is typed at the console, and PIP attempts another read. If the tape position cannot be properly read, continue the read by typing a return following the error message, and enter the record manually with the ED program after the disk file is constructed.

PIP allows the end-of-file to be entered from the console if the source file is an RDR: device. In this case, the PIP program reads the device and monitors the keyboard. If CTRL-Z is typed at the keyboard, the read operation is terminated normally.

The following are valid PIP commands:

`PIP LST:=X.PRN`	Copies X.PRN to the LST device and terminates the PIP program.
`PIP`	Starts PIP for a sequence of commands. PIP prompts with *. A single carriage return stops PIP.
`PIP CON:=X.ASM,Y.ASM,Z.ASM`	Concatenates three ASM files and copies to the CON device.
`PIP X.HEX=CON:,Y.HEX,PTR:`	Creates a HEX file by reading the CON until a CTRL-Z is typed, followed by data from Y.HEX and PTR until a CTRL-Z is encountered.
`PIP PUN:=NUL:,X.ASM,EOF:,NUL:`	Sends 40 nulls to the punch device; copies the X.ASM file to the punch, followed by an end-of-file, CTRL-Z, and 40 more null characters.

You can also specify one or more PIP parameters, enclosed in left and right square brackets, separated by zero or more blanks. Each parameter affects the copy operation, and the enclosed list of parameters must immediately follow the affected file or device. Generally, each parameter can be followed by an optional decimal integer value (the S and Q parameters are exceptions).

Parameter	Meaning
B	Blocks mode transfer. Data are buffered by PIP until an ASCII XOFF character, CTRL-S, is received from the source device. This allows transfer of data to a disk file from a continuous reading device, such as a cassette reader. Upon receipt of the XOFF, PIP clears the disk buffers and returns for more input data. The amount of data that can be buffered depends on the memory size of the host system. PIP issues an error message if the buffers overflow.
D*n*	Deletes characters that extend past column *n* in the transfer of data to the destination from the character source. This parameter is generally used to truncate long lines that are sent to a narrow printer or console device.

Parameter	Meaning
E	Echoes all transfer operations to the console as they are being performed.
F	Filters form-feeds from the file. All embedded form-feeds are removed. The P parameter can be used simultaneously to insert new form-feeds.
G*n*	Gets file from user number *n* (*n* in the range 0-15).
H	Transfers HEX data. All data are checked for proper Intel hex file format. Nonessential characters between hex records are removed during the copy operation. The console is prompted for corrective action in case errors occur.
I	Ignores :00 records in the transfer of Intel hex format file. The I parameter automatically sets the H parameter.
L	Translates upper-case alphabetic characters to lower-case.
N	Adds line numbers to each line transferred to the destination, starting at one and incrementing by 1. Leading zeroes are suppressed, and the number is followed by a colon. If N2 is specified, leading zeroes are included and a tab is inserted following the number. The tab is expanded if T is set.
O	Transfers non-ASCII object files. The normal CP/M end-of-file is ignored.
P*n*	Includes page ejects at every *n* lines with an initial page eject. If *n* = 1 or is excluded altogether, page ejects occur every 60 lines. If the F parameter is used, form-feed suppression takes place before the new page ejects are inserted.
Q*s*^Z	Quits copying from the source device or file when the string *s*, terminated by CTRL-Z, is encountered.
R	Reads system files.

Parameter	Meaning
Ss^Z	Start copying from the source device when the string *s*, terminated by CTRL-Z, is encountered. The S and Q parameters can be used to abstract a particular section of a file, such as a subroutine. The start and quit strings are always included in the copy operation. If you specify a command line after the PIP command keyword, the CCP translates strings following the S and Q parameters to uppercase. If you do not specify a command line, PIP does not perform the automatic upper-case translation.
T*n*	Expands tabs, CTRL-I characters, to every *n*th column during the transfer of characters to the destination from the source.
U	Translates lower-case alphabetic characters to upper-case during the copy operation.
V	Verifies that data have been copied correctly by rereading after the write operation (the destination must be a disk file).
W	Writes over R/O files without console interrogation.
Z	Zeros the parity bit on input for each ASCII character.

Table 1-4: PIP Parameters

The following examples show valid PIP commands that specify parameters in the file transfer.

`PIP X.ASM=B:[V]`	Copies X.ASM from drive B to the current drive and verifies that the data were properly copied.
`PIP LPT:=X.ASM[NT8U]`	Copies X.ASM to the LPT: device; numbers each line, expands tabs to every eighth column, and translates lower-case alphabetic characters to upper-case.
`PIP PUN:=X.HEX[I],Y.ZOT[H]`	First copies X.HEX to the PUN: device and ignores the trailing :00 record in X.HEX; continues the transfer of data by reading

	`Y.ZOT`, which contains HEX records, including any `:00` records it contains.
`PIP X.LIB=Y.ASM[sSUBR1:^zqJMP L3^z]`	Copies from the file `Y.ASM` into the file `X.LIB`. The command starts the copy when the string `SUBR1:` has been found, and quits copying after the string `JMP L3` is encountered.
`PIP PRN:=X.ASM[p50]`	Sends `X.ASM` to the `LST:` device with line numbers, expands tabs to every eighth column, and elects pages at every 50th line. The assumed parameter list for a `PRN` file is `nt8p60`; `p50` overrides the default value.

Under normal operation, `PIP` does not overwrite a file that is set to a permanent R/O status. If an attempt is made to overwrite an R/O file, the following prompt appears:

```
DESTINATION FILE IS R/O, DELETE (Y/N)?
```

If you type `Y`, the file is overwritten. Otherwise, the following response appears:

```
** NOT DELETED **
```

The file transfer is skipped, and `PIP` continues with the next operation in sequence. To avoid the prompt and response in the case of R/O file overwrite, the command line can include the `W` parameter, as shown in this example:

```
PIP A:=B:*.COM[W]
```

The `W` parameter copies all non-system files to the `A` drive from the `B` drive and overwrites any R/O files in the process. If the operation involves several concatenated files, the `W` parameter need only be included with the last file in the list, as in this example:

```
PIP A.DAT=B.DAT,F:NEW.DAT,G:OLD.DAT[W]
```

Files with the system attribute can be included in `PIP` transfers if the `R` parameter is included; otherwise, system files are not recognized. For example, the command line:

```
PIP ED.COM=B:ED.COM[R]
```

reads the `ED.COM` file from the `B` drive, even if it has been marked as an R/O and system file. The system file attributes are copied, if present.

Downward compatibility with previous versions of CP/M is only maintained if the file does not exceed one megabyte, no file attributes are set, and the file is created by user `0`. If compatibility

is required with nonstandard, for example, double-density versions of 1.4, it might be necessary to select 1.4 compatibility mode when constructing the internal disk parameter block. See Section 6 and refer to Section 6.10, which describes BIOS differences.

Note:

To copy files into another user area, PIP.COM must be located in that user area. Use the following procedure to make a copy of PIP.COM in another user area.

```
A>USER 0↵                    Log in as user 0
A>DDT PIP.COM↵               Load PIP into memory. Note PIP size S
DDT VERS 2.2
NEXT  PC
1E00 0100
-G0↵                         Return to CCP
A>USER N↵                    Log in as desired user
A>SAVE S PIP.COM↵            Save PIP.COM in new user area
A>
```

In this procedure, *S* is the integral number of memory pages, 256- byte segments, occupied by PIP.COM. The number *S* can be determined when PIP.COM is loaded under DDT, by referring to the value under the NEXT display. If, for example, the next available address is 1E00, then PIP.COM requires 1D hexadecimal pages, or 29 pages in decimal. So the value of *S* is 29 in the subsequent SAVE command. Once PIP is copied in this manner, it can be copied to another disk belonging to the same user number through normal PIP transfers.

1.6.5. ED Command

Syntax:

ED *ufn*

The ED program is the CP/M system context editor that allows creation and alteration of ASCII files in the CP/M environment. Complete details of operation are given in Section 2. ED allows the operator to create and operate upon source files that are organized as a sequence of ASCII characters, separated by end-of-line characters (a carriage return/line-feed sequence). There is no practical restriction on line length (no single line can exceed the size of the working memory) that is defined by the number of characters typed between carriage returns.

The ED program has a number of commands for character string searching, replacement, and insertion that are useful for creating and correcting programs or text files under CP/M. Although the CP/M has a limited memory work space area (approximately 5000 characters in a 20K CP/M system), the file size that can be edited is not limited, since data are easily paged through this work area.

If it does not exist, ED creates the specified source file and opens the file for access. If the source file does exist, the programmer appends data for editing (see the A command). The appended data can then be displayed, altered, and written from the work area back to the disk (see the W command). Particular points in the program can be automatically paged and located by context, allowing easy access to particular portions of a large file (see the N command).

If you type the following command line:

```
ED X.ASM
```

the ED program creates an intermediate work file with the name

```
X.$$$
```

to hold the edited data during the ED run. Upon completion of ED, the X.ASM file (original file) is renamed to X.BAK, and the edited work file is renamed to X.ASM. Thus, the X.BAK file contains the original unedited file, and the X.ASM file contains the newly edited file. The operator can always return to the previous version of a file by removing the most recent version and renaming the previous version. If the current X.ASM file has been improperly edited, the following sequence of commands reclaim the backup file.

DIR X.* Checks to see that .BAK file is available.

ERA X.ASM Erases the most recent version.

REN X.ASM=X.BAK Renames the .BAK file to .ASM

You can abort the edit at any point (reboot, power failure, CTRL-C, or CTRL-Q command) without destroying the original file. In this case, the .BAK file is not created and the original file is always intact.

The ED program allows the user to edit the source on one disk and create the back-up file on another disk. This form of the ED command is

ED *ufn d*:

where *ufn* is the name of the file to edit on the currently logged disk and *d* is the name of an alternate drive. The ED program reads and processes the source file and writes the new file to drive *d* using the name *ufn*. After processing, the original file becomes the back-up file. If the operator is addressing disk A, the following command is valid.

```
ED X.ASM B:
```

This edits the file X.ASM on drive A, creating the new file X.$$$ on drive B. After a successful edit, A:X.ASM is renamed to A:X.BAK, and B:X.$$$ is renamed to B:X.ASM. For convenience, the currently logged disk becomes drive B at the end of the edit. Note that if a file named B:X.ASM exists before the editing begins, the following message appears on the screen:

```
FILE EXISTS
```

This message is a precaution against accidentally destroying a source file. You should first erase the existing file and then restart the edit operation.

Similar to other transient commands, editing can take place on a drive different from the currently logged disk by preceding the source filename by a drive name. The following are examples of valid edit requests:

`ED A:X.ASM`	Edits the file X.ASM on drive A, with new file and back-up on drive A.
`ED B:X.ASM A:`	Edits the file X.ASM on drive B to the temporary file X.$$$ on drive A. After editing, this command changes X.ASM on drive B to X.BAK and changes X.$$$ on drive A to X.ASM

1.6.6. SYSGEN Command

Syntax:

```
SYSGEN
```

The SYSGEN transient command allows generation of an initialized disk containing the CP/M operating system. The SYSGEN program prompts the console for commands by interacting as shown.

`SYSGEN`	Initiates the SYSGEN program.
`SYSGEN VERSION 2.0`	SYSGEN sign-on message.
`SOURCE DRIVE NAME (OR RETURN TO SKIP)`	Respond with the drive name (one of the letters A, B, C, or D) of the disk containing a CP/M system, usually A. If a copy of CP/M already exists in memory due to a MOVCPM command, press only a carriage return. Typing a drive name *d* causes the response:

SOURCE ON *d* THEN TYPE RETURN	Place a disk containing the CP/M operating system on drive *d* (*d* is one of A, B, C, or D). Answer by pressing a carriage return when ready.
FUNCTION COMPLETE	System is copied to memory. SYSGEN then prompts with the following:
DESTINATION DRIVE NAME (OR RETURN TO REBOOT)	If a disk is being initialized, place the new disk into a drive and answer with the drive name. Otherwise, press a carriage return and the system reboots from drive A. Typing drive name *d* causes SYSGEN to prompt with the following message:
DESTINATION ON *d*, THEN TYPE RETURN	Place the new disk on drive *d* and press the RETURN key when ready.
FUNCTION COMPLETE	New disk is initialized in drive *d*.

The DESTINATION prompt is repeated until a single carriage return is pressed at the console, so that more than one disk can be initialized.

Upon completion of a successful system generation, the new disk contains the operating system, and only the built-in commands are available. An IBM-compatible disk appears to CP/M as a disk with an empty directory; therefore, the operator must copy the appropriate COM files from an existing CP/M disk to the newly constructed disk using the PIP transient.

You can copy all files from an existing disk by typing the following PIP command:

```
PIP B:=A:*.*[v]
```

This command copies all files from disk drive A to disk drive B and verifies that each file has been copied correctly. The name of each file is displayed at the console as the copy operation proceeds.

Note that a SYSGEN does not destroy the files that already exist on a disk; it only constructs a new operating system. If a disk is being used only on drives B through P and will never be the source of a bootstrap operation on drive A, the SYSGEN need not take place.

1.6.7. SUBMIT Command

Syntax:

SUBMIT *ufn parm#1 ... parm#n*

The SUBMIT command allows CP/M commands to be batched for automatic processing. The *ufn* given in the SUBMIT command must be the filename of a file that exists on the currently logged disk, with an assumed file type of SUB. The .SUB file contains CP/M prototype commands with possible parameter substitution. The actual parameters *parm#1 ... parm#n* are substituted into the prototype commands, and, if no errors occur, the file of substituted commands are processed sequentially by CP/M.

The prototype command file is created using the ED program, with interspersed $ parameters of the form:

```
$1 $2 $3 … $n
```

corresponding to the number of actual parameters that will be included when the file is submitted for execution. When the SUBMIT transient is executed, the actual parameters *parm#1 ... parm#n* are paired with the formal parameters $1 ... $n in the prototype commands. If the numbers of formal and actual parameters do not correspond, the SUBMIT function is aborted with an error message at the console. The SUBMIT function creates a file of substituted commands with the name

```
$$$.SUB
```

on the logged disk. When the system reboots, at the termination of the SUBMIT, this command file is read by the CCP as a source of input rather than the console. If the SUBMIT function is performed on any disk other than drive A, the commands are not processed until the disk is inserted into drive A and the system reboots. You can abort command processing at any time by pressing the rubout key when the command is read and echoed. In this case, the $$$.SUB file is removed and the subsequent commands come from the console. Command processing is also aborted if the CCP detects an error in any of the commands. Programs that execute under CP/M can abort processing of command files when error conditions occur by erasing any existing $$$.SUB file.

To introduce dollar signs into a SUBMIT file, you can type a $$ which reduces to a single $ within the command file. A caret, ^, precedes an alphabetic character *X*, which produces a single CTRL-*X* character within the file.

The last command in a SUB file can initiate another SUB file, allowing chained batch commands.

Suppose the file ASMBL.SUB exists on disk and contains the prototype commands:

```
ASM $1
DIR $1.*
ERA *.BAK
PIP $2:=$1.PRN
ERA $1.PRN
```

then, you issue the following command:

```
SUBMIT ASMBL X PRN
```

The SUBMIT program reads the ASMBL.SUB file, substituting X for all occurrences of $1 and PRN for all occurrences of $2. This results in a $$$.SUB file containing the commands:

```
ASM X
DIR X.*
ERA *.BAK
PIP PRN:=X.PRN
ERA X.PRN
```

which are executed in sequence by the CCP.

The SUBMIT function can access a SUB file on an alternate drive by preceding the filename by a drive name. Submitted files are only acted upon when they appear on drive A. Thus, it is possible to create a submitted file on drive B that is executed at a later time when inserted in drive A.

An additional utility program called XSUB extends the power of the SUBMIT facility to include line input to programs as well as the CCP. The XSUB command is included as the first line of the SUBMIT file. When it is executed, XSUB self-relocates directly below the CCP. All subsequent SUBMIT command lines are processed by XSUB so that programs that read buffered console input, BDOS Function 10, receive their input directly from the SUBMIT file. For example, the file SAVER.SUB can contain the following SUBMIT lines:

```
XSUB
DDT
I $1.COM
R
G0
SAVE 1 $2.COM
```

a subsequent SUBMIT command, such as

```
A:SUBMIT SAVER PIP Y
```

substitutes PIP for $1 and Y for $2 in the command stream. The XSUB program loads, followed by DDT, which is sent to the command lines PIP.COM, R, and G0, thus returning to the CCP. The final command SAVE 1 Y.COM is processed by the CCP.

The XSUB program remains in memory and prints the message

```
(xsub active)
```

on each warm start operation to indicate its presence. Subsequent SUBMIT command streams do not require the XSUB, unless an intervening cold start occurs. Note that XSUB must be loaded after the optional CP/M DESPOOL utility, if both are to run simultaneously.

1.6.8. DUMP Command

Syntax:

DUMP *ufn*

The DUMP program types the contents of the disk file (*ufn*) at the console in hexadecimal form. The file contents are listed sixteen bytes at a time, with the absolute byte address listed to the left of each line in hexadecimal. Long type-outs can be aborted by pressing the rubout key during printout. The source listing of the DUMP program is given in Section 5 as an example of a program written for the CP/M environment.

1.6.9. MOVCPM Command

Syntax:

MOVCPM

MOVCPM *n*

MOVCPM *n* *

MOVCPM * *

The MOVCPM program allows you to reconfigure the CP/M system for any particular memory size. Two optional parameters can be used to indicate the desired size of the new system and the disposition of the new system at program termination. If the first parameter is omitted or an * is given, the MOVCPM program reconfigures the system to its maximum size, based upon the kilobytes of contiguous RAM in the host system (starting at 0000H). If the second parameter is omitted, the system is executed, but not permanently recorded; if * is given, the system is left in memory, ready for a SYSGEN operation. The MOVCPM program relocates a memory image of CP/M and places this image in memory in preparation for a system generation operation. The following is a list of MOVCPM command forms:

MOVCPM	Relocates and executes CP/M for management of the current memory configuration (memory is examined for contiguous RAM, starting at 100H). On completion of the relocation, the new system is executed but not permanently recorded on the disk. The system that is constructed contains a BIOS for the Intel MDS 800.
MOVCPM *n*	Creates a relocated CP/M system for management of an *n* kilobyte system (*n* must be in the range of 20 to 64), and executes the system as described.

`MOVCPM * *`	Constructs a relocated memory image for the current memory configuration, but leaves the memory image in memory in preparation for a SYSGEN operation.
`MOVCPM` *n* `*`	Constructs a relocated memory image for an *n* kilobyte memory system, and leaves the memory image in preparation for a SYSGEN operation.

For example, the command,

```
MOVCPM * *
```

constructs a new version of the CP/M system and leaves it in memory, ready for a SYSGEN operation. The message

```
READY FOR "SYSGEN" OR
"SAVE 34 CPMxx.COM"
```

appears at the console upon completion, where xx is the current memory size in kilobytes. You can then type the following sequence:

`SYSGEN`	This starts the system generation.
`SOURCE DRIVE NAME (OR RETURN TO SKIP)`	Respond with a carriage return to skip the CP/M read operation, because the system is already in memory as a result of the previous MOVCPM operation.
`DESTINATION DRIVE NAME (OR RETURN TO REBOOT)`	Respond with B to write new system to the disk in drive B. SYSGEN prompts with the following message:
`DESTINATION ON B, THEN TYPE RETURN`	Place the new disk on drive B and press the RETURN key when ready.

If you respond with A rather than B above, the system is written to drive A rather than B. SYSGEN continues to print this prompt:

```
DESTINATION DRIVE NAME (OR RETURN TO REBOOT)
```

until you respond with a single carriage return, which stops the SYSGEN program with a system reboot.

You can then go through the reboot process with the old or new disk. Instead of performing the SYSGEN operation, you can type a command of the form:

```
SAVE 34 CPMxx.COM
```

at the completion of the MOVCPM function, where *xx* is the value indicated in the SYSGEN message. The CP/M memory image on the currently logged disk is in a form that can be patched. This is necessary when operating in a nonstandard environment where the BIOS must be altered for a particular peripheral device configuration, as described in Section 6.

The following are valid MOVCPM commands:

`MOVCPM 48`	Constructs a 48K version of CP/M and starts execution.

`MOVCPM 48 *`	Constructs a 48K version of CP/M in preparation for permanent recording; the response is:
	`READY FOR "SYSGEN" OR`
	`"SAVE 34 CPM48.COM"`

`MOVCPM`	Constructs a maximum memory version of CP/M and starts execution.

The newly created system is serialized with the number attached to the original disk and is subject to the conditions of the Digital Research Software Licensing Agreement.

1.7. BDOS Error Messages

There are three error situations that the Basic Disk Operating System intercepts during file processing. When one of these conditions is detected, the BDOS prints the message:

`BDOS Err On` *d*`:` *error*

where *d* is the drive name and *error* is one of the three error messages:

- `Bad Sector`
- `Select`
- `R/O`

The `Bad Sector` message indicates that the disk controller electronics has detected an error condition in reading or writing the disk. This condition is generally caused by a malfunctioning disk controller or an extremely worn disk. If you find that CP/M reports this error more than once a month, the state of the controller electronics and the condition of the media should be checked.

You can also encounter this condition in reading files generated by a controller produced by a different manufacturer. Even though controllers claim to be IBM compatible, one often finds

small differences in recording formats. The MDS-800 controller, for example, requires two bytes of ones following the data CRC byte, which is not required in the IBM format. As a result, disks generated by the Intel MDS can be read by almost all other IBM- compatible systems, while disk files generated on other manufacturers' equipment produce the Bad Sector message when read by the MDS. To recover from this condition, press a CTRL-C to reboot (the safest course), or a return, which ignores the bad sector in the file operation.

Note:
pressing a return might destroy disk integrity if the operation is a directory write. Be sure you have adequate back-ups in this case.

The Select error occurs when there is an attempt to address a drive beyond the range supported by the BIOS. In this case, the value of d in the error message gives the selected drive. The system reboots following any input from the console.

The R/O (Read-Only) message occurs when there is an attempt to write to a disk or file that has been designated as Read-Only in a STAT command or has been set to Read-Only by the BDOS. Reboot CP/M by using the warm start procedure, CTRL-C, or by performing a cold start whenever the disks are changed. If a changed disk is to be read but not written, BDOS allows the disk to be changed without the warm or cold start, but internally marks the drive as Read-Only. The status of the drive is subsequently changed to Read- Write if a warm or cold start occurs. On issuing this message, CP/M waits for input from the console. An automatic warm start takes place following any input.

1.8. Operation of CP/M on the MDS

This section gives operating procedures for using CP/M on the Intel MDS microcomputer development system. Basic knowledge of the MDS hardware and software systems is assumed.

CP/M is initiated in essentially the same manner as the Intel ISIS operating system. The disk drives are labeled 0 through 3 on the MDS, corresponding to CP/M drives A through D, respectively. The CP/M system disk is inserted into drive 0, and the BOOT and RESET switches are pressed in sequence. The interrupt 2 light should go on at this point. The space bar is then pressed on the system console, and the light should go out. If it does not, the user should check connections and baud rates. The BOOT switch is turned off, and the CP/M sign-on message should appear at the selected console device, followed by the A> system prompt. You can then issue the various resident and transient commands.

The CP/M system can be restarted (warm start) at any time by pushing the INT 0 switch on the front panel. The built-in Intel ROM monitor can be initiated by pushing the INT 7 switch, which

generates an `RST 7`, except when operating under `DDT`, in which case the `DDT` program gets control instead.

Diskettes can be removed from the drives at any time, and the system can be shut down during operation without affecting data integrity. Do not remove a disk and replace it with another without rebooting the system (cold or warm start) unless the inserted disk is Read-Only.

As a result of hardware hang-ups or malfunctions, CP/M might print the following message:

`BDOS Err On` *d*`: Bad Sector`

where *d* is the drive that has a permanent error. This error can occur when drive doors are opened and closed randomly, followed by disk operations, or can be caused by a disk, drive, or controller failure. You can optionally elect to ignore the error by pressing a single return at the console. The error might produce a bad data record, requiring re-initialization of up to 128 bytes of data. You can reboot the CP/M system and try the operation again.

Termination of a CP/M session requires no special action, except that it is necessary to remove the disks before turning the power off to avoid random transients that often make their way to the drive electronics.

You should use IBM-compatible disks rather than disks that have previously been used with any ISIS version. In particular, the ISIS FORMAT operation produces nonstandard sector numbering throughout the disk. This nonstandard numbering seriously degrades the performance of CP/M, and causes CP/M to operate noticeably slower than the distribution version. If it becomes necessary to reformat a disk, which should not be the case for standard disks, a program can be written under CP/M that causes the MDS 800 controller to reformat with sequential sector numbering (1-26) on each track.

Generally, IBM-compatible 8-inch disks do not need to be formatted. However, 5 1/4-inch disks need to be formatted.

2. The CP/M Editor

2.1. Introduction to ED

ED is the context editor for CP/M, and is used to create and alter CP/M source files. To start ED, type a command of the following form:

ED *filename*

or

ED *filename.typ*

Generally, ED reads segments of the source file given by *filename* or *filename.typ* into the central memory, where you edit the file and it is subsequently written back to disk after alterations. If the source file does not exist before editing, it is created by ED and initialized to empty. The overall operation of ED is shown in Figure 2-1.

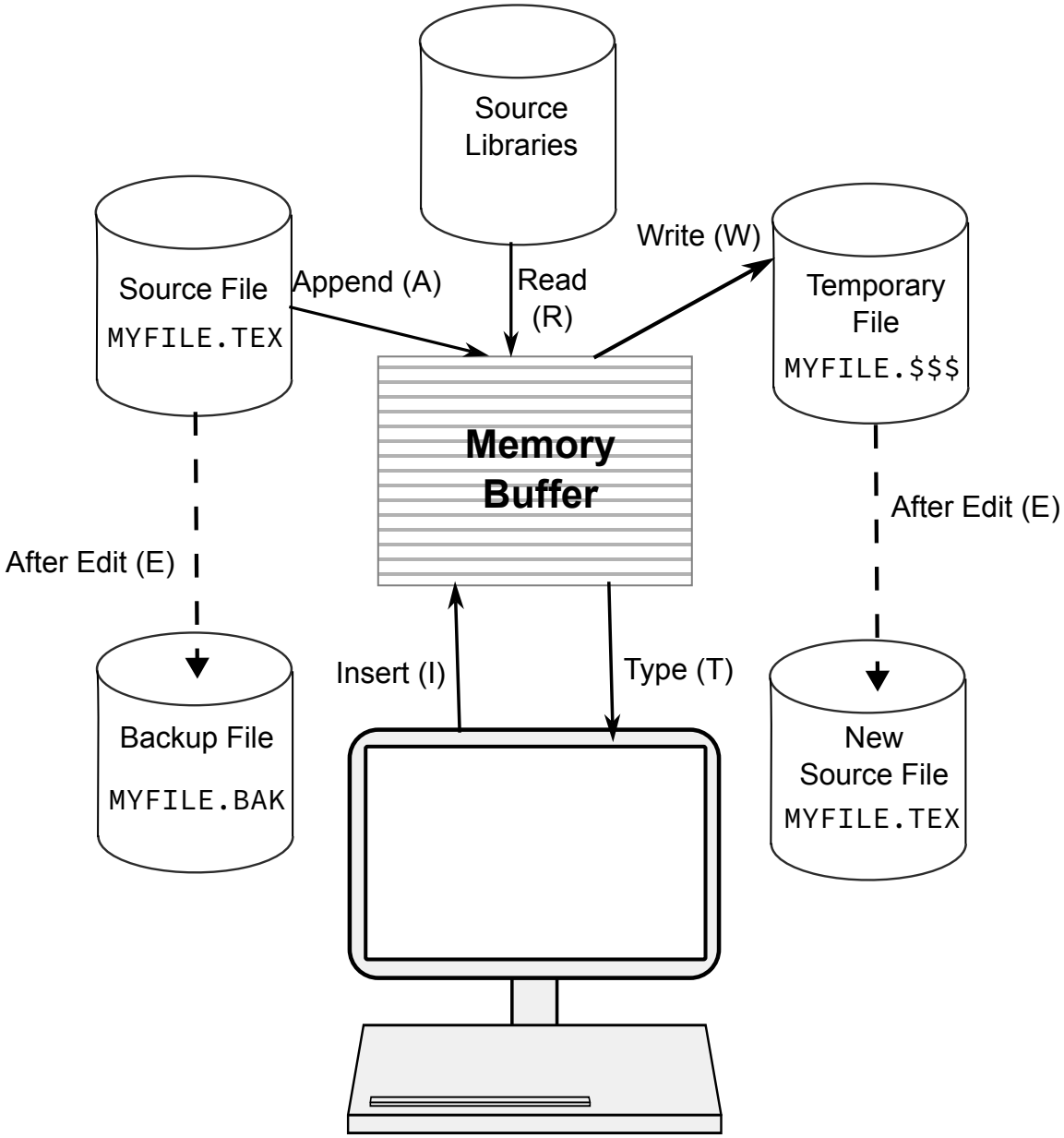


Figure 2-1: Overall ED Operation

2.1.1. ED Operation

ED operates upon the source file, shown in Figure 2-1 by MYFILE.TEX, and passes all text through a memory buffer where the text can be viewed or altered. The number of lines that can be maintained in the memory buffer varies with the line length, but has a total capacity of about 5000 characters in a 20K CP/M system.

Edited text material is written into a temporary work file under your command. Upon termination of the edit, the memory buffer is written to the temporary file, followed by any remaining (unread) text in the source file. The name of the original file is changed from MYFILE.TEX to MYFILE.BAK so that the most recent edited source file can be reclaimed if necessary. See the

CP/M commands ERASE and RENAME. The temporary file is then changed from `MYFILE.$$$` to `MYFILE.TEX`, which becomes the resulting edited file.

The memory buffer is logically between the source file and working file, as shown in Figure 2-2

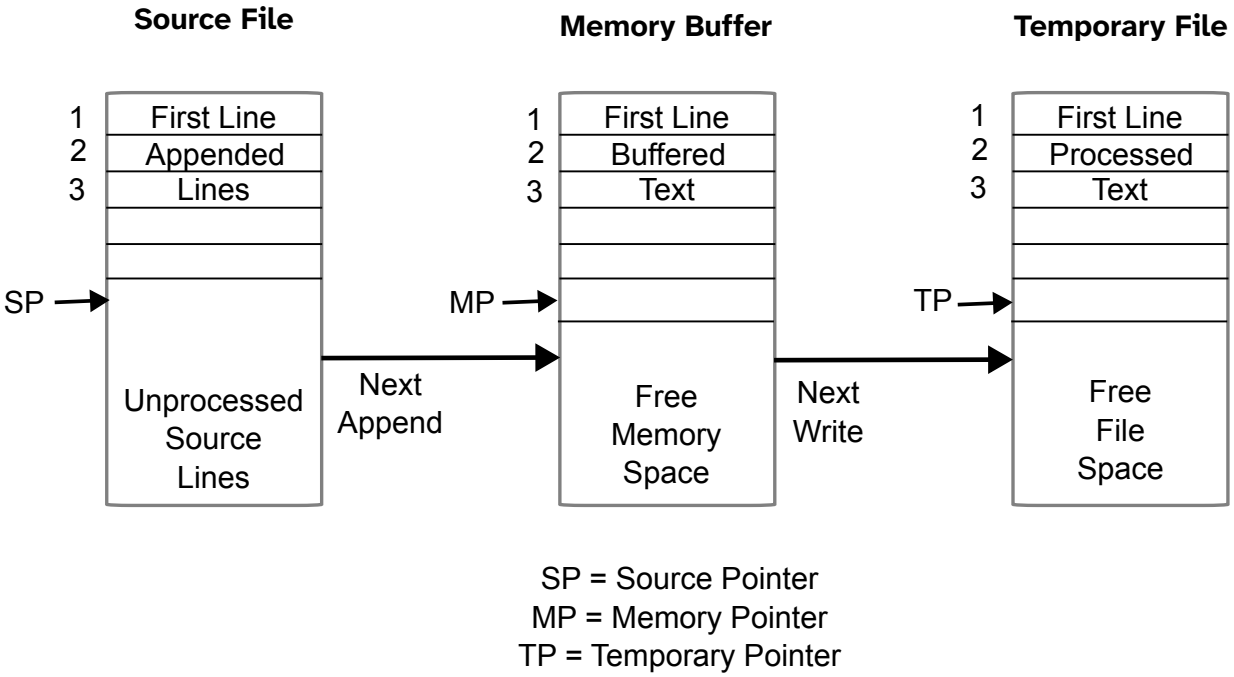


Figure 2-2: Memory Buffer Organization

2.1.2. Text Transfer Commands

Command	Result
*n*A	Appends the next *n* unprocessed source lines from the source file at **SP** to the end of the memory buffer at **MP**. Increment **SP** and **MP** by *n*. If upper-case translation is set (see the U command) and the A command is typed in upper-case, all input lines will automatically be translated to upper-case.
*n*W	Writes the first *n*) lines of the memory buffer to the temporary file free space. Shift the remaining lines $n+1$ through **MP** to the top of the memory buffer. Increment **TP** by *n*.
E	Ends the edit. Copy all buffered text to temporary file and copy all unprocessed source lines to temporary file. Rename files.
H	Moves to head of new file by performing automatic E command. The temporary file becomes the new source file, the memory buffer is emptied, and a new temporary file is created. The effect is equivalent to issuing an E

Command	Result
	command, followed by a re-invocation of ED, using MYFILE.TEX as the file to edit.
O	Returns to original file. The memory buffer is emptied, the temporary file is deleted, and the **SP** is returned to position 1 of the source file. The effects of the previous editing commands are thus nullified.
Q	Quits edit with no file alterations, returns to CP/M.

Table 2-5: ED Text Transfer Commands

There are a number of special cases to consider. If the integer *n* is omitted in any ED command where an integer is allowed, then 1 is assumed. Thus, the commands A and W append one line and write one line, respectively. In addition, if a pound sign # is given in the place of *n*, then the integer 65535 is assumed (the largest value for *n* that is allowed). Because most source files can be contained entirely in the memory buffer, the command #A is often issued at the beginning of the edit to read the entire source file to memory. Similarly, the command #W writes the entire buffer to the temporary file.

Two special forms of the A and W commands are provided as a convenience. The command 0A fills the current memory buffer at least half full, while 0W writes lines until the buffer is at least half empty. An error is issued if the memory buffer size is exceeded. You can then enter any command, such as W, that does not increase memory requirements. The remainder of any partial line read during the overflow will be brought into memory on the next successful append.

2.1.3. Memory Buffer Organization

The memory buffer can be considered a sequence of source lines brought in with the A command from a source file. The memory buffer has an imaginary character pointer (**CP**) that moves throughout the memory buffer under command of the operator. The memory buffer appears logically as shown in Figure 2-3, where the dashes represent characters of the source line of indefinite length, terminated by carriage return (<cr>) and line-feed (<lf>) characters, and **CP** represents the imaginary character pointer. Note that the **CP** is always located ahead of the first character of the first line, behind the last character of the last line, or between two characters. The current line **CL** is the source line that contains the **CP**.

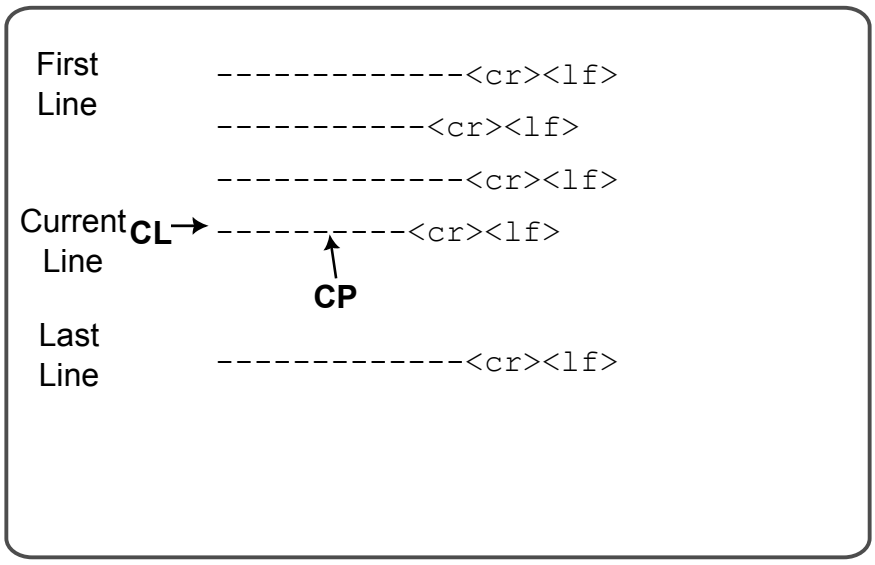


Figure 2-3: Logical Organization of Memory Buffer

2.1.4. Line Numbers and ED Start-up

ED produces absolute line number prefixes that are used to reference a line or range of lines. The absolute line number is displayed at the beginning of each line when ED is in insert mode (see the I command in Section 2.1.5). Each line number takes the form

```
nnnnn:
```

where *nnnnn* s an absolute line number in the range of 1 to 65535. If the memory buffer is empty or if the current line is at the end of the memory buffer, *nnnnn* appears as 5 blanks.

You can reference an absolute line number by preceding any command by a number followed by a colon, in the same format as the line number display. In this case, the ED program moves the current line reference to the absolute line number, if the line exists in the current memory buffer. The line denoted by the absolute line number must be in the memory buffer (see the A command). Thus, the command

```
345:T
```

is interpreted as move to absolute 345, and type the line. Absolute line numbers are produced only during the editing process and are not recorded with the file. In particular, the line numbers will change following a deleted or expanded section of text.

You can also reference an absolute line number as a backward or forward distance from the current line by preceding the absolute number by a colon. Thus, the command

```
:400T
```

is interpreted as type from the current line number through the line whose absolute number is 400. Combining the two line reference forms, the command

```
345::400T
```

is interpreted as move to absolute line 345, then type through absolute line 400. Absolute line references of this sort can precede any of the standard ED commands.

Line numbering is controlled by the V (Verify Line Numbers) command. Line numbering can be turned off by typing the -V command.

If the file to edit does not exist, ED displays the following message:

```
NEW FILE
```

To move text into the memory buffer, you must enter an i command before typing input lines and terminate each line with a carriage return. A single CTRL-Z character returns ED to command mode.

2.1.5. Memory Buffer Operation

When ED begins, the memory buffer is empty. You can either append lines from the source file with the A command, or enter the lines directly from the console with the insert command. The insert command takes the following form:

```
I
```

ED then accepts any number of input lines. You must terminate each line with a <cr> (also rendered as ↵ or $^{C}_{R}$) (the <lf>, also rendered as $^{L}_{F}$ is supplied automatically). A single CTRL-Z, denoted by an up arrow ^Z, returns ED to command mode. The **CP** is positioned after the last character entered. The following sequence:

```
I↵
NOW IS THE↵
TIME FOR↵
ALL GOOD MEN↵
^Z
```

leaves the memory buffer as

NOW IS THE$^{C}_{R}{}^{L}_{F}$

TIME FOR$^{C}_{R}{}^{L}_{F}$

ALL GOOD MEN$^{C}_{R}{}^{L}_{F}$

Generally, ED accepts command letters in upper- or lower-case. If the command is upper-case, all input values associated with the command are translated to upper-case. If the I command is typed, all input lines are automatically translated internally to upper-case. The lower-case form of the i command is most often used to allow both upper- and lower-case letters to be entered.

Various commands can be issued that control the **CP** or display source text in the vicinity of the **CP**. The commands shown below with a preceding *n* indicate that an optional unsigned value can be specified. When preceded by +-, the command can be unsigned, or have an optional preceding plus or minus sign. As before, the pound sign # is replaced by 65535. If an integer *n* is optional, but not supplied, then $n = 1$ is assumed. Finally, if a plus sign is optional, but none is specified, then + is assumed.

Command	Result
+B *or* -B	Move **CP** to beginning of memory buffer if +B and to bottom if -B
+*n*C *or* -*n*C	Move **CP** by *n* characters (moving ahead if +), counting the ^R^F as two characters.
+*n*D *or* -*n*D	Delete *n* characters ahead of **CP** if plus and behind **CP** if minus.
+*n*K *or* -*n*K	Kill (remove) *n* lines of source text using **CP** as the current reference. If **CP** is not at the beginning of the current line when K is issued, the characters before **CP** remain if + is specified, while the characters after **CP** remain if - is given in the command.
+*n*L *or* -*n*L	If $n = 0$, move **CP** to the beginning of the current line, if it is not already there. If $n \neq 0$, first move the **CP** to the beginning of the current line and then move it to the beginning of the line that is *n* lines down (if +) or up (if -). The **CP** will stop at the top or bottom of the memory buffer if too large a value of *n* is specified.
+*n*T *or* -*n*T	If $n = 0$, type the contents of the current line up to **CP**. If $n = 1$, type the contents of the current line from **CP** to the end of the line. If $n > 1$, type the current line along with $n + 1$ lines that follow, if + is specified. Similarly, if $n > 1$ and - is given, type the previous n lines up to the **CP**. Any key can be depressed to abort long type-outs.
+*n* *or* -*n*	Equivalent to +-*n*LT, which moves up or down and types a single line.

Table 2-6: Editing Commands

2.1.6. Command Strings

Any number of commands can be typed contiguously (up to the capacity of the console buffer) and are executed only after you press the <cr>. Table 2-7 summarizes the CP/M console line-editing commands used to control the input command line.

Character	Meaning
CTRL-C	Reboots CP/M system when pressed at start of line.
CTRL-E	Physical end of line; carriage is returned, but line is not sent until the carriage return key is pressed.
CTRL-H	Backspaces one character position.
CTRL-J	Terminates current input (line-feed).
CTRL-M	Terminates current input (carriage return).
CTRL-R	Retypes current command line; types a clean line following character deletion with rubouts.
CTRL-U	Deletes the entire line typed at the console.
CTRL-X	Same as CTRL-U.
CTRL-Z	Ends input from the console (used in PIP and ED).
Rubout/DEL	Deletes and echoes the last character typed at the console.

Table 2-7: ED Line-editing Controls

Suppose the memory buffer contains the characters shown in the previous section, with the **CP** "↑" following the last character of the buffer. In the following example, the command strings on the left produce the results shown to the right. Use lower-case command letters to avoid automatic translation of strings to upper-case.

Command String	Effect
B2T↵	Move to the beginning of the buffer and type two lines: NOW IS THE↵ TIME FOR↵ The result in the memory buffer is ↑NOW IS THE$^{C_R}{}_{L_F}$ TIME FOR$^{C_R}{}_{L_F}$ ALL GOOD MEN$^{C_R}{}_{L_F}$

Command String	Effect
`5C0T↲`	Move **CP** five characters and type the beginning of the line `NOW I`, The result in the memory buffer is `NOW I↑S THE`$^{C}_{R}{}^{L}_{F}$ `TIME FOR`$^{C}_{R}{}^{L}_{F}$ `ALL GOOD MEN`$^{C}_{R}{}^{L}_{F}$
`2L-T↲`	Move two lines down and type the previous line `TIME FOR`. The result in the memory buffer is `NOW IS THE`$^{C}_{R}{}^{L}_{F}$ `TIME FOR`$^{C}_{R}{}^{L}_{F}$ `↑ALL GOOD MEN`$^{C}_{R}{}^{L}_{F}$
`-L#K↲`	Move up one line, delete `65535` lines that follow. The result in the memory buffer is `NOW IS THE`$^{C}_{R}{}^{L}_{F}$`↑`

`I↲`
`TIME TO↲`
`INSERT↲`
`^Z`
	Insert two lines of text with automatic translation to upper-case. The result in the memory buffer is
	`NOW IS THE`$^{C}_{R}{}^{L}_{F}$
	`TIME TO`$^{C}_{R}{}^{L}_{F}$
	`INSERT`$^{C}_{R}{}^{L}_{F}$`↑`

`-2L#T↲`	Move up two lines and type 65535 lines ahead of **CP** `NOW IS THE`. The result in the memory `NOW IS THE`$^{C}_{R}{}^{L}_{F}$ `↑TIME TO`$^{C}_{R}{}^{L}_{F}$ `INSERT`$^{C}_{R}{}^{L}_{F}$
`↲`	Move down one line and type one line `INSERT`. The result in the memory buffer is

Command String	Effect
	NOW IS THE$^{C}_{R}{}^{L}_{F}$ TIME TO$^{C}_{R}{}^{L}_{F}$ ↑INSERT$^{C}_{R}{}^{L}_{F}$

2.1.7. Text Search and Alteration

ED has a command that locates strings within the memory buffer. The command takes the form

*n*F*s*^Z

where s represents the string to match, followed by either a <cr> ↲ or CTRL-Z, denoted by ^Z. ED starts at the current position of **CP** and attempts to match the string. The match is attempted *n* times and, if successful, the **CP** is moved directly after the string. If the *n* matches are not successful, the **CP** is not moved from its initial position. Search strings can include CTRL-L, which is replaced by the pair of symbols $^{C}_{R}{}^{L}_{F}$.

The following commands illustrate the use of the F command:

Command String	Effect
B#T↲	Move to the beginning and type the entire buffer. The result in the memory buffer is ↑NOW IS THE$^{C}_{R}{}^{L}_{F}$ TIME FOR$^{C}_{R}{}^{L}_{F}$ ALL GOOD MEN$^{C}_{R}{}^{L}_{F}$
FS T↲	Find the end of the string S T. The result in the memory buffer is NOW IS T↑HE$^{C}_{R}{}^{L}_{F}$ TIME FOR$^{C}_{R}{}^{L}_{F}$ ALL GOOD MEN$^{C}_{R}{}^{L}_{F}$
FIs^Z0TT	Find the next I and type to the **CP**; then type the remainder of the current line ME FOR. The result in the memory buffer is

Command String	Effect
	NOW IS THE$^{C}_{R}{}^{L}_{F}$ TI↑ME FOR$^{C}_{R}{}^{L}_{F}$ ALL GOOD MEN$^{C}_{R}{}^{L}_{F}$

An abbreviated form of the insert command is also allowed, which is often used in conjunction with the F command to make simple textual changes. The form is

Is^Z

or

Is↵

where s is the string to insert. If the insertion string is terminated by a CTRL-Z, the string is inserted directly following the **CP**, and the **CP** is positioned directly after the string. The action is the same if the command is followed by a <cr> ↵ except that a $^{C}_{R}{}^{L}_{F}$ is automatically inserted into the text following the string. The following command sequences are examples of the F and I commands:

Command String	Effect
BITHIS IS ^Z↵	Insert THIS IS at the beginning of the text. The result in the memory buffer is THIS IS ↑NOW IS THE$^{C}_{R}{}^{L}_{F}$ TIME FOR$^{C}_{R}{}^{L}_{F}$ ALL GOOD MEN$^{C}_{R}{}^{L}_{F}$
FTIME^Z-DIPLACE^Z↵	Find TIME and delete it; then insert PLACE. The result in the memory buffer THIS IS NOW IS THE$^{C}_{R}{}^{L}_{F}$ PLACE ↑FOR$^{C}_{R}{}^{L}_{F}$ ALL GOOD MEN$^{C}_{R}{}^{L}_{F}$
3FO^Z-3D5DICHANGES^Z↵	Find third occurrence of O (that is, the second O in GOOD), delete previous 3 characters and the subsequent 5 characters; then insert CHANGES. The result in the memory buffer is

Command String	Effect
	THIS IS NOW IS THE$^{C_R L_F}$ PLACE FOR$^{C_R L_F}$ ALL CHANGES↑$^{C_R L_F}$
-8CISOURCE↵	Move back 8 characters and insert the line SOURCE$^{C_R L_F}$. The result in the memory buffer is THIS IS NOW IS THE$^{C_R L_F}$ PLACE FOR$^{C_R L_F}$ ALL SOURCE$^{C_R L_F}$ ↑CHANGES$^{C_R L_F}$

ED also provides a single command that combines the F and I commands to perform simple string substitutions. The command takes the following form:

*n*S*S1*^Z*S2*↵

or

*n*S*S1*^Z*S2*^Z

and has exactly the same effect as applying the following command string a total of *n* times:

F*S1*^Z-*k*DI*S2*↵

or

F*S1*^Z-*k*DI*S2*^Z

where *k* is the length of the string. ED searches the memory buffer starting at the current position of **CP** and successively substitutes the second string for the first string until the end of buffer, or until the substitution has been performed *n* times.

As a convenience, a command similar to F is provided by ED that automatically appends and writes lines as the search proceeds. The form is

*n*N*S*↵

or

*n*N*S*^Z

which searches the entire source file for the *n*th occurrence of the strings (you should recall that F fails if the string cannot be found in the current buffer). The operation of the N command is precisely the same as F except in the case that the string cannot be found within the current

memory buffer. In this case, the entire memory content is written (that is, an automatic #W is issued). Input lines are then read until the buffer is at least half full, or the entire source file is exhausted. The search continues in this manner until the string has been found *n* times, or until the source file has been completely transferred to the temporary file.

A final line editing function, called the Juxtaposition command, takes the form

*n*J*S1*^Z*S2*^Z*S3*↵

or

*n*J*S1*^Z*S2*^Z*S3*^Z

with the following action applied *n* times to the memory buffer: search from the current **CP** for the next occurrence of the string *S1*. If found, insert the string *S2*, and move **CP** to follow *S2*. Then delete all characters following **CP** up to, but not including, the string *S3*, leaving **CP** directly after *S2*. If *S3* cannot be found, then no deletion is made. If the current line is

NOW IS THE TIME$^{C}_{R}{}^{L}_{F}$

the command

JW^ZWHAT^Z^L↵

results in

NOW WHAT$^{C}_{R}{}^{L}_{F}$

You should recall that a ^L (CTRL-L) represents the pair $^{C}_{R}{}^{L}_{F}$ in search and substitute strings. The number of characters ED allows in the F, S, N, and J commands is limited to 100 symbols.

2.1.8. Source Libraries

ED also allows the inclusion of source libraries during the editing process with the R command. The form of this command is

R*filename*↵

or

R*filename*^Z

where filename is the primary filename of a source file on the disk with an assumed filetype of LIB. ED reads the specified file, and places the characters into the memory buffer after **CP**, in a manner similar to the I command. Thus, if the command

RMACRO↵

is issued by the operator, ED reads from the file MACRO.LIB until the end-of-file and automatically inserts the characters into the memory buffer.

ED also includes a block move facility implemented through the X (Transfer) command. The form

*n*X

transfers the next n lines from the current line to a temporary file called

X$$$$$$.LIB

which is active only during the editing process. You can reposition the current line reference to any portion of the source file and transfer lines to the temporary file. The transferred lines accumulate one after another in this file and can be retrieved by simply typing

R

which is the trivial case of the library read command. In this case, the entire transferred set of lines is read into the memory buffer. Note that the X command does not remove the transferred lines from the memory buffer, although a K command can be used directly after the X, and the R command does not empty the transferred LIB file. That is, given that a set of lines has been transferred with the X command, they can be reread any number of times back into the source file. The command

0X

is provided to empty the transferred line file.

Note that upon normal completion of the ED program through Q or E, the temporary LIB file is removed. If ED is aborted with a CTRL-C, the LIB file will exist if lines have been transferred, but will generally be empty (a subsequent ED invocation will erase the temporary file).

2.1.9. Repetitive Command Execution

The macro command M allows you to group ED commands together for repeated evaluation. The M command takes the following form:

*n*M*CS*↵

or

*n*M*CS*^Z

where *CS* represents a string of ED commands, not including another M command. ED executes the command string *n* times if $n > 1$. If $n = 0$ or $n = 1$, the command string is executed repetitively until an error condition is encountered (for example, the end of the memory buffer is reached with an F command).

As an example, the following macro changes all occurrences of GAMMA to DELTA within the current buffer, and types each line that is changed:

MFGAMMA^Z-5DIDELTA^Z0TT↵

or equivalently

MSGAMMA^ZDELTA^Z0TT↵

2.2. ED Error Conditions

On error conditions, ED prints the message BREAK *X* AT *C* where *X* is one of the error indicators shown in Table 2-8

Symbol	Meaning
?	Unrecognized command.
>	Memory buffer full (use one of the commands D, K, N, S, or W to remove characters); F, N, or S strings too long.
#	Cannot apply command the number of times specified (for example, in F command).
O	Cannot open LIB file in R command.

Table 2-8: Error Message Symbols

If there is a disk error, CP/M displays the following message:

```
BDOS Err On d: Bad Sector
```

You can choose to ignore the error by pressing RETURN at the console (in this case, the memory buffer data should be examined to see if they were incorrectly read), or you can reset the system with a CTRL-C and reclaim the backup file if it exists. The file can be reclaimed by first typing the contents of the BAK file to ensure that it contains the proper information. For example, type the following:

```
TYPE x.BAK
```

where *x* is the file being edited. Then remove the primary file

```
ERA x.y
```

and rename the BAK file

```
REN x.y=x.BAK
```

The file can then be reedited, starting with the previous version.

ED also takes file attributes into account. If you attempt to edit a Read-Only file, the message

```
** FILE IS READ/ONLY **
```

appears at the console. The file can be loaded and examined, but cannot be altered. You must end the edit session and use STAT to change the file attribute to R/W. If the edited file has the system attribute set, the following message:

```
"SYSTEM" FILE NOT ACCESSIBLE
```

is displayed and the edit session is aborted. Again, the STAT program can be used to change the system attribute, if desired.

2.3. Control Characters and Commands

Table 2-9 summarizes the control characters and commands available in ED.

Control Character	Function
CTRL-C	System reboot
CTRL-E	Physical <cr><lf> (not actually entered in command)
CTRL-H	Backspaces
CTRL-I	Logical tab (cols 1, 8, 16, ...)
CTRL-R	Repeat line
CTRL-U	Line delete
CTRL-X	Line delete
CTRL-Z	String terminator
rubout/del	Character delete

Table 2-9: ED Control Characters

Table 2-10 summarizes the commands used in ED.

Command	**Result**
*n*A	Append lines
+B *or* -B	Begin or bottom of buffer
+*n*C *or* -*n*C	Move character positions
+*n*D *or* -*n*D	Delete characters
E	End edit and close files (normal end)
*n*F	Find string
H	End edit, close and reopen files

Command	Result
I or i	Insert characters, use i if both upper- and lower-case characters are to be entered.
*n*J	Place strings in juxtaposition
+*n*K *or* -*n*K	Kill lines
+*n*L *or* -*n*L	Move down/up lines
*n*M	Macro definition
*n*N	Find next occurrence with autoscan
O	Return to original file
+*n*P *or* -*n*P	Move and print pages
Q	Quit with no file changes
R	Read library file
*n*S	Substitute strings
+*n*T *or* -*n*T	Type lines
U or -U	Translate lower- to upper-case if U, no translation if -U
V	Verify line numbers, or show remaining free character space
0V	A special case of the V command, 0V, prints the memory buffer statistics in the form *free/total* where *free* is the number of free bytes in the memory buffer (in decimal) and *total* is the size of the memory buffer
*n*W	Write lines
*n*Z	Wait (sleep) for approximately n seconds
+*n or* -*n*	Move and type

Table 2-10: ED Commands

Because of common typographical errors, `ED` requires several potentially disastrous commands to be typed as single letters, rather than in composite commands. The following commands:

`E` end

`H` head

`O` original

`Q` quit

must be typed as single letter commands.

The commands `I`, `J`, `M`, `N`, `R`, and `S` should be typed as `i`, `j`, `m`, `n`, `r`, and `s` if both upper- and lower-case characters are used in the operation, otherwise all characters are converted to upper-case. When a command is entered in upper-case, `ED` automatically converts the associated string to upper-case, and vice versa.

3. CP/M Assembler

3.1. Introduction

The CP/M assembler reads assembly-language source files from the disk and produces 8080 machine language in Intel hex format. To start the CP/M assembler, type a command in one of the following forms:

ASM *filename*

ASM *filename.parms*

In both cases, the assembler assumes there is a file on the disk with the name:

filename.ASM

which contains an 8080 assembly-language source file. The first and second forms shown above differ only in that the second form allows parameters to be passed to the assembler to control source file access and hex and print file destinations.

In either case, the CP/M assembler loads and prints the message:

CP/M ASSEMBLER VER *n.n*

where *n.n* is the current version number. In the case of the first command, the assembler reads the source file with assumed filetype ASM and creates two output files

filename.HEX

filename.PRN

The HEX file contains the machine code corresponding to the original program in Intel hex format, and the PRN file contains an annotated listing showing generated machine code, error flags, and source lines. If errors occur during translation, they are listed in the PRN file and at the console.

The form ASM *filename.parms* is used to redirect input and output files from their defaults. In this case, the *parms* portion of the command is a three-letter group that defines the origin of the source file, the destination of the hex file, and the destination of the print file. The form is

filename.p1p2p3

where *p1*, *p2* and *p3* are single letters.

p1 can be A, B, ..., P which designates the disk name that contains the source file.

p2 can be A, B, ..., P, which designates the disk name that will receive the hex file; or, *p2* can be Z which skips the generation of the hex file.

p3 can be A, B, ..., P which designates the disk name that will receive the print file. *p3* can also be specified as X which places the listing at the console; or Z which skips generation of the print file.

Thus, the command

```
ASM X.AAA
```

indicates that the source, `X.HEX` and print, `X.PRN` files are also to be created on disk `A`. This form of the command is implied if the assembler is run from disk `A`. Given that you are currently addressing disk `A`, the above command is the same as

```
ASM X
```

The command

```
ASM X.ABX
```

indicates that the source file is to be taken from disk `A`, the hex file is to be placed on disk `B`, and the listing file is to be sent to the console.

The command

```
ASM X.BZZ
```

takes the source file from disk `B` and skips the generation of the hex and print files. This command is useful for fast execution of the assembler to check program syntax.

The source program format is compatible with the Intel 8080 assembler. Macros are not implemented in `ASM`; see the optional `MAC` macro assembler. There are certain extensions in the CP/M assembler that make it somewhat easier to use. These extensions are described below.

3.2. Program Format

An assembly-language program acceptable as input to the assembler consists of a sequence of statements of the form

line# label operation operand ;comment

where any or all of the fields may be present in a particular instance. Each assembly language statement is terminated with a carriage return and line-feed (the line-feed is inserted automatically by the `ED` program), or with the character `!`, which is treated as an end-of-line by the assembler. Thus, multiple assembly-language statements can be written on the same physical line if separated by exclamation point symbols.

The *line#* is an optional decimal integer value representing the source program line number, and `ASM` ignores this field if present.

The *label* field takes either of the following forms:

identifier

identifier:

The label field is optional, except where noted in particular statement types. The identifier is a sequence of alphanumeric characters where the first character is alphabetic. Identifiers can be

freely used by the programmer to label elements such as program steps and assembler directives, but cannot exceed 16 characters in length. All characters are significant in an identifier, except for the embedded dollar symbol `$`, which can be used to improve readability of the name. Further, all lower-case alphabetic characters are treated as upper-case. The following are all valid instances of labels:

- `x`
- `xy`
- `long$name`
- `x:`
- `yxl:`
- `longer$named$data:`
- `X1Y2`
- `X1x2`
- `x234$5678$9012#456:`

The *operation* field contains either an **assembler directive** or **pseudo operation**, or an **8080 machine operation code**. The pseudo operations and machine operation codes are described in Section 3.3.

Generally, the *operand* field of the statement contains an expression formed out of constants and labels, along with arithmetic and logical operations on these elements. Again, the complete details of properly formed expressions are given in Section 3.3.

The *comment* field contains arbitrary characters following the semicolon symbol until the next real or logical end-of-line. These characters are read, listed, and otherwise ignored by the assembler. The CP/M assembler also treats statements that begin with an `*` in column one as comment statements that are listed and ignored in the assembly process.

The assembly-language program is formulated as a sequence of statements of the above form, terminated by an optional `END` statement. All statements following the `END` are ignored by the assembler.

3.3. Forming the Operand

To describe the operation codes and pseudo operations completely, it is necessary first to present the form of the operand field, since it is used in nearly all statements. Expressions in the operand field consist of simple operands, labels, constants, and reserved words, combined in properly formed sub-expressions by arithmetic and logical operators. The expression computation is carried out by the assembler as the assembly proceeds. Each expression must produce a 16-bit value during the assembly. Further, the number of significant digits in the result must not exceed the intended use. If an expression is to be used in a byte move immediate

instruction, the most significant 8 bits of the expression must be zero. The restriction on the expression significance is given with the individual instructions.

3.3.1. Labels

A label is an identifier that occurs on a particular statement. In general, the label is given a value determined by the type of statement that it precedes. If the label occurs on a statement that generates machine code or reserves memory space (for example, a `MOV` instruction or a `DS` pseudo operation), the label is given the value of the program address that it labels. If the label precedes an `EQU` or `SET`, the label is given the value that results from evaluating the operand field. Except for the `SET` statement, an identifier can label only one statement.

When a label appears in the operand field, its value is substituted by the assembler. This value can then be combined with other operands and operators to form the operand field for a particular instruction.

3.3.2. Numeric Constants

A numeric constant is a 16-bit value in one of several bases. The base, called the radix of the constant, is denoted by a trailing radix indicator. The following are radix indicators:

B is a binary constant (base 2).

O is a octal constant (base 8).

Q is a octal constant (base 8).

D is a decimal constant (base 10).

H is a hexadecimal constant (base 16).

`Q` is an alternate radix indicator for octal numbers because the letter `O` is easily confused with the digit `0`. Any numeric constant that does not terminate with a radix indicator is a decimal constant.

A constant is composed as a sequence of digits, followed by an optional radix indicator, where the digits are in the appropriate range for the radix. Binary constants must be composed of `0` and `1` digits, octal constants can contain digits in the range `0-7`, while decimal constants contain decimal digits. Hexadecimal constants contain decimal digits as well as hexadecimal digits `A`(`10D`), `B`(`11D`), `C`(`12D`), `D`(`13D`), `E`(`14D`), and `F`(`15D`). Note that the leading digit of a hexadecimal constant must be a decimal digit to avoid confusing a hexadecimal constant with an identifier. A leading `0` will always suffice. A constant composed in this manner must evaluate to a binary number that can be contained within a 16-bit counter, otherwise it is truncated on the right by the assembler.

Similar to identifiers, embedded `$` signs are allowed within constants to improve their readability. Finally, the radix indicator is translated to upper-case if a lower-case letter is encountered. The following are all valid instances of numeric constants:

- 1234
- 1234D
- 1100B
- 1111$0000$1111$0000B
- 1234H
- 0FFEH
- 33770
- 33$77$22Q
- 3377o
- 0fe3h
- 1234d
- 0ffffh

3.3.3. Reserved Words

There are several reserved character sequences that have predefined meanings in the operand field of a statement. The names of 8080 registers are given below. When they are encountered, they produce the values shown to the right.

Character	Value
A	7
B	0
C	1
D	2
E	3
H	4
L	5
M	6
SP	6
PSW	6

Table 3-11: Reserved Characters

Again, lower-case names have the same values as their upper-case equivalents. Machine instructions can also be used in the operand field; they evaluate to their internal codes. In the

case of instructions that require operands, where the specific operand becomes a part of the binary bit pattern of the instruction, for example, `MOV A,B`, the value of the instruction, in this case `MOV`, is the bit pattern of the instruction with zeros in the optional fields, for example, `MOV` produces 40H.

When the symbol `$` occurs in the operand field, not embedded within identifiers and numeric constants, its value becomes the address of the next instruction to generate, not including the instruction contained within the current logical line.

3.3.4. String Constants

String constants represent sequences of ASCII characters and are represented by enclosing the characters within apostrophe symbols. All strings must be fully contained within the current physical line (thus allowing exclamation point symbols within strings) and must not exceed 64 characters in length. The apostrophe character itself can be included within a string by representing it as a double apostrophe (the two keystrokes `''`), which becomes a single apostrophe when read by the assembler. In most cases, the string length is restricted to either one or two characters (the `DB` pseudo operation is an exception), in which case the string becomes an 8- or 16-bit value, respectively. Two-character strings become a 16-bit constant, with the second character as the low-order byte, and the first character as the high-order byte.

The value of a character is its corresponding ASCII code. There is no case translation within strings; both upper- and lower-case characters can be represented. You should note that only graphic printing ASCII characters are allowed within strings.

Valid String	How the Assembler Reads String
`'A' 'AB' 'ab' 'c'`	`A AB ab c`
`'''' 'a''' '''''' '''"'`	`' a' '' '"`
`'Walla Walla Wash.'`	`Walla Walla Wash.`
`'She said "Hello" to me.'`	`She said "Hello" to me.`
`'I said "Hello" to her.'`	`I said "Hello" to her.`

3.3.5. Arithmetic and Logical Operators

The operands described in Section 3.3 can be combined in normal algebraic notation using any combination of properly formed operands, operators, and parenthesized expressions. The operators recognized in the operand field are described below in Table 3-12.

Operators	Meaning
`a + b`	unsigned arithmetic sum of *a* and *b*
`a - b`	unsigned arithmetic difference between *a* and *b*
`+ b`	unary plus (produces *b*)
`- b`	unary minus (identical to `0 - b`)
`a * b`	unsigned magnitude multiplication of *a* and *b*
`a / b`	unsigned magnitude division of *a* by *b*
`a MOD b`	remainder after *a* / *b*
`NOT b`	logical inverse of *b* (all 0s become 1s, 1s become 0s), where *b* is considered a 16-bit value
`a AND b`	bit-by-bit logical and of *a* and *b*
`a OR b`	bit-by-bit logical or of *a* and *b*
`a XOR b`	bit-by-bit logical exclusive or of *a* and *b*
`a SHL b`	the value that results from shifting *a* to the left by an amount *b*, with zero fill
`a SHR b`	the value that results from shifting *a* to the right by an amount *b*, with zero fill

Table 3-12: Arithmetic and Logical Operators

In each case, a and b represent simple operands (labels, numeric constants, reserved words, and one- or two-character strings) or fully enclosed parenthesized sub-expressions, like those shown in the following examples:

- `10+20`
- `10h+37Q`
- `L1/3`
- `(L2+4) SHR 3`
- `('a' and 5fh)+'O'`
- `('B'+B)OR(PSW+M)`
- `(1+(2+c))shr(A-(B+1))`

3.3.6. Precedence of Operators

As a convenience to the programmer, ASM assumes that operators have a relative precedence of application that allows the programmer to write expressions without nested levels of parentheses. The resulting expression has assumed parentheses that are defined by the relative precedence. The order of application of operators in unparenthesized expressions is listed below. Operators listed first have highest precedence (they are applied first in an unparenthesized expression), while operators listed last have lowest precedence. Operators listed on the same line have equal precedence, and are applied from left to right as they are encountered in an expression.

```
* / MOD SHL SHR
- +
NOT
AND
OR XOR
```

Thus, the expressions shown to the left below are interpreted by the assembler as the fully parenthesized expressions shown to the right.

Expression	How the Assembler Interprets It
`a*b+c`	`(a*b)+c`
`a+b*c`	`a+(b*c)`
`a MOD b*c SHL d`	`((a MOD b) * c) SHL d`
`a OR b  AND NOT c+d SHL e`	`a OR (b AND (NOT (c + (d SHL e))))`

Balanced, parenthesized sub-expressions can always be used to override the assumed parentheses; thus, the last expression above could be rewritten to force application of operators in a different order, as shown:

```
( a OR b ) AND ( NOT c ) + d SHL e
```

This results in these assumed parentheses:

```
(a OR b ) AND ( (NOT c ) + ( d SHL e ) )
```

An unparenthesized expression is well-formed only if the expression that results from inserting the assumed parentheses is well-formed.

3.4. Assembler Directives

Assembler directives are used to set labels to specific values during the assembly, perform conditional assembly, define storage areas, and specify starting addresses in the program. Each

assembler directive is denoted by a pseudo operation that appears in the operation field of the line. The acceptable pseudo operations are shown below in Table 3-13.

Directive	Meaning
ORG	set the program or data origin
END	end program, optional start address
EQU	numeric equate
SET	numeric set
IF	begin conditional assembly
ENDIF	end of conditional assembly
DB	define data bytes
DW	define data words
DS	define data storage area

Table 3-13: Assembler Directives

3.4.1. The ORG Directive

The ORG statement takes the form:

label ORG *expression*

where *label* is an optional program identifier and *expression* is a 16-bit expression, consisting of operands that are defined before the ORG statement. The assembler begins machine code generation at the location specified in the expression. There can be any number of ORG statements within a particular program, and there are no checks to ensure that the programmer is not defining overlapping memory areas. Note that most programs written for the CP/M system begin with an ORG statement of the form:

```
ORG 100H
```

which causes machine code generation to begin at the base of the CP/M transient program area. If a label is specified in the ORG statement, the label is given the value of the expression. This label can then be used in the operand field of other statements to represent this expression.

3.4.2. The END Directive

The END statement is optional in an assembly-language program, but if it is present it must be the last statement. All subsequent statements are ignored in the assembly. The END statement takes the following two forms:

label END

label END *expression*

where the *label* is again optional. If the first form is used, the assembly process stops, and the default starting address of the program is taken as 0000. Otherwise, the *expression* is evaluated, and becomes the program starting address. This starting address is included in the last record of the Intel-formatted machine code hex file that results from the assembly. Thus, most CP/M assembly-language programs end with the statement:

```
END 100H
```

resulting in the default starting address of 100H (beginning of the transient program area).

3.4.3. The EQU Directive

The EQU (equate) statement is used to set up synonyms for particular numeric values. The EQU statement takes the form:

label EQU *expression*

where the *label* must be present and must not label any other statement. The assembler evaluates the *expression* and assigns this value to the identifier given in the *label* field. The identifier is usually a name that describes the value in a more human-oriented manner. Further, this name is used throughout the program to place parameters on certain functions. Suppose data received from a teletype appears on a particular input port, and data is sent to the teletype through the next output port in sequence. For example, you can use this series of equate statements to define these ports for a particular hardware environment:

```
TTYBASE   EQU 10H        ;BASE PORT NUMBER FOR TTY
TTYIN     EQU TTYBASE    ;TTY DATA IN
TTYOUT    EQU TTYBASE+1  ;TTY DATA OUT
```

At a later point in the program, the statements that access the teletype can appear as follows:

```
IN  TTYIN      ;READ TTY DATA TO REG-A
   ....

OUT TTYOUT     ;WRITE DATA TO TTY FROM REG-A
```

making the program more readable than if the absolute I/O ports are used. Further, if the hardware environment is redefined to start the teletype communications ports at 7FH instead of 10H, the first statement need only be changed to

```
TTYBASE   EQU 7FH        ;BASE PORT NUMBER FOR TTY
```

and the program can be reassembled without changing any other statements.

3.4.4. The SET Directive

The SET statement is similar to the EQU, taking the form:

label SET *expression*

except that the label can occur on other SET statements within the program. The expression is evaluated and becomes the current value associated with the label. Thus, the EQU statement defines a label with a single value, while the SET statement defines a value that is valid from the current SET statement to the point where the label occurs on the next SET statement. The use of the SET is similar to the EQU statement, but is used most often in controlling conditional assembly.

3.4.5. The IF and ENDIF Directive

The IF and ENDIF statements define a range of assembly-language statements that are to be included or excluded during the assembly process. These statements take on the form:

```
IF expression
statement#1
statement#2
   ...
statement#n
ENDIF
```

When encountering the IF statement, the assembler evaluates the expression following the IF. All operands in the expression must be defined ahead of the IF statement. If the expression evaluates to a nonzero value, then statement#1 through statement#n are assembled. If the expression evaluates to zero, the statements are listed but not assembled. Conditional assembly is often used write a single generic program that includes a number of possible run-time environments, with only a few specific portions of the program selected for any particular assembly. The following program segments, for example, might be part of a program that communicates with either a teletype or a CRT console (but not both) by selecting a particular value for TTY before the assembly begins.

```
TRUE    EQU  0FFFFH      ;DEFINE VALUE OF TRUE
FALSE   EQU  NOT TRUE    ;DEFINE VALUE OF FALSE
;
TTY     EQU  TRUE        ;TRUE IF TTY, FALSE IF CRT
;
TTYBASE EQU  10H         ;BASE OF TTY I/O PORTS
CRTBASE EQU  20H         ;BASE OF CRT I/O PORTS
        IF   TTY         ;ASSEMBLE RELATIVE TO
                         ;TTYBASE
CONIN   EQU  TTYBASE     ;CONSOLE INPUT
```

```
CONOUT  EQU  TTYBASE+1   ;CONSOLE OUTPUT
        ENDIF
;
        IF   NOT TTY     ;ASSEMBLE RELATIVE TO
                         ;CRTBASE
CONIN   EQU  CRTBASE     ;CONSOLE INPUT
CONOUT  EQU  CRTBASE+1   ;CONSOLE OUTPUT
        ENDIF
        ...
        IN   CONIN       ;READ CONSOLE DATA
        OUT  CONTOUT     ;WRITE CONSOLE DATA
```

In this case, the program assembles for an environment where a teletype is connected, based at port 10H. The statement defining TTY can be changed to

```
TTY EQU FALSE
```

and, in this case, the program assembles for a CRT based at port 20H.

3.4.6. The DB Directive

The DB directive allows the programmer to define initialized storage areas in single precision byte format. The DB statement takes the form:

label DB *e#1*, *e#2*, ..., *e#n*

where *e#1* through *e#n* are either expressions that evaluate to 8-bit values (the high-order bit must be zero) or are ASCII strings of length no greater than 64 characters. There is no practical restriction on the number of expressions included on a single source line. The expressions are evaluated and placed sequentially into the machine code file following the last program address generated by the assembler. String characters are similarly placed into memory starting with the first character and ending with the last character. Strings of length greater than two characters cannot be used as operands in more complicated expressions.

Note: ASCII characters are always placed in memory with the parity bit reset (0)

Note Also, there is no translation from lower- to upper-case within strings.

The optional label can be used to reference the data area throughout the remainder of the program. The following are examples of valid DB statements:

```
data:     DB  0,1,2,3,4,5
          DB  data and 0ffh,5,377Q,1+2+3+4
sign-on:  DB  'please type your name',CR,LF,0
          DB  'AB' SHR 8,'C','DE',AND 7FH
```

3.4.7. The DW Directive

The DW statement is similar to the DB statement except double- precision two-byte words of storage are initialized. The DW statement takes the form:

label DW *e#1*, *e#2*, ..., *e#n*

where *e#1* through *e#n* are expressions that evaluate to 16-bit results. Note that ASCII strings of one or two characters are allowed, but strings longer than two characters are disallowed. In all cases, the data storage is consistent with the 8080 processor; the least significant byte of the expression is stored first in memory, followed by the most significant byte. The following are examples of DW statements:

```
doub: DW 0ffefh,doub+4,signon-$,255+255
      DW 'a',5,'ab','CD',6 shl 8 or llb.
```

3.4.8. The DS Directive

The DS statement is used to reserve an area of uninitialized memory, and takes the form:

label DS *expression*

where the *label* is optional. The assembler begins subsequent code generation after the area reserved by the DS. Thus, the DS statement given above has exactly the same effect as the following statement:

```
label: EQU $            ;LABEL VALUE IS CURRENT CODE LOCATION
       ORG $+expression ;MOVE PAST RESERVED AREA
```

3.5. Operation Codes

Assembly-language operation codes form the principal part of assembly-language programs and form the operation field of the instruction. In general, ASM accepts all the standard mnemonics for the Intel 8080 microcomputer, which are given in detail in the Intel 8080 Assembly Language Programming Manual. Labels are optional on each input line. The individual operators are listed briefly in the following sections for completeness, although the Intel manuals should be referenced for exact operator details. In Table 3-14 through Table 3-18, bit values have the following meaning:

e3 represents a 3-bit value in the range 0-7 that can be one of the predefined registers A, B, C, D, E, H, L, M, SP, or PSW.

e8 represents an 8-bit value in the range 0-255.

e16 represents a 16-bit value in the range 0-65535.

These expressions can be formed from an arbitrary combination of operands and operators. In some cases, the operands are restricted to particular values within the allowable range, such as the PUSH instruction. These cases are noted as they are encountered.

3.5.1. Jumps, Calls, and Returns

The Jump, Call, and Return instructions allow several different forms that test the condition flags set in the 8080 microcomputer CPU. The forms are shown in Table 3-14.

Form	Bit Value	Example	Meaning
JMP	*e16*	JMP L1	Jump unconditionally to label
JNZ	*e16*	JNZ L2	Jump on nonzero condition to label
JZ	*e16*	JZ 100H	Jump on zero condition to label
JNC	*e16*	JNC L1+4	Jump no carry to label
JC	*e16*	JC L3	Jump on carry to label
JPO	*e16*	JPO $+8	Jump on parity odd to label
JPE	*e16*	JPE L4	Jump on even parity to label
JP	*e16*	JP GAMMA	Jump on positive result to label
JM	*e16*	JM A1	Jump on minus to label
CALL	*e16*	CALL S1	Call subroutine unconditionally
CNZ	*e16*	CNZ S2	Call subroutine on nonzero condition
CZ	*e16*	CZ 100H	Call subroutine on zero condition
CNC	*e16*	CNC SI+4	Call subroutine if no carry set
CC	*e16*	CC S3	Call subroutine if carry set
CPO	*e16*	CPO $+8$	Call subroutine if parity odd
CPE	*e16*	CPE $4	Call subroutine if parity even
CP	*e16*	CP GAMMA	Call subroutine if positive result
CM	*e16*	CM b1$c2	Call subroutine if minus flag
RST	*e3*	RST 0	Programmed restart, equivalent to CALL $8 * e3$, except one byte call
RET	-	RET	Return from subroutine
RNZ	-	RNZ	Return if nonzero flag set

Form	Bit Value	Example	Meaning
RZ	-	RZ	Return if zero flag set
RNC	-	RNC	Return if no carry
RC	-	RC	Return if carry flag set
RPO	-	RPO	Return if parity is odd
RPE	-	RPE	Return if parity is even
RP	-	RP	Return if positive result
RM	-	RM	Return if minus flag is set

Table 3-14: Jumps, Calls, and Returns

3.5.2. Immediate Operand Instructions

Several instructions are available that load single- or double-precision registers or single-precision memory cells with constant values, along with instructions that perform immediate arithmetic or logical operations on the accumulator (register A). Table 3-15 describes the immediate operand instructions.

Form and Bit Value	Example	Meaning
MVI *e3,e8*	MVI B,255	Move immediate data to register A, B, C, D, E, H, L, or M (memory)
ADI *e8*	ADI 1	Add immediate operand to A without carry
ACI *e8*	ACI 0FFH	Add immediate operand to A with carry
SUI *e8*	SUI L + 3	Subtract from A without borrow (carry)
SBI *e8*	SBI L AND 11B	Subtract from A with borrow (carry)
ANI *e8*	ANI $ AND 7FH	Logical and A with immediate data
XRI *e8*	XRI 1111$0000B	Exclusive-or A with immediate data
ORI *e8*	ORI L AND 1+1	Logical-or A with immediate data

Form and Bit Value	Example	Meaning
CPI *e8*	CPI 'a'	Compare A with immediate data, same as SUI except register A not changed.
LXI *e3,e16*	LXI B, 100H	Load extended immediate to register pair. *e3* must be equivalent to B, D, H,or SP.

Table 3-15: Immediate Operand Instructions

3.5.3. Increment and Decrement Instructions

The 8080 provides instructions for incrementing or decrementing single- and double precision registers. The instructions are described in Table 3-16

Form and Bit Value	Example	Meaning
INR *e3*	INR E	Single-precision increment register. *e3* produces one of A, B, C, D, E, H, L, or M (memory)
DCR *e3*	DCR A	Single-precision decrement register. *e3* produces one of A, B, C, D, E, H, L, or M (memory)
INX *e3*	INX SP	Double-precision increment register pair. *e3* must be equivalent to B, D, H, or SP.
DCX *e3*	DCX B	Double-precision decrement register pair. *e3* must be equivalent to B, D, H, or SP.

Table 3-16: Increment and Decrement Instructions

3.5.4. Data Movement Instructions

Instructions that move data from memory to the CPU and from CPU to memory are given in Table 3-17.

Form and Bit Value	Example	Meaning
MOV *e3,e3*	MOV A,B	Move data to leftmost element from rightmost element. *e3* produces one of A, B, C, D, E, H, L, or M (memory). **Note:** MOV M,M is disallowed
LDAX *e3*	LDAX B	Load register A from computed address. *e3* must produce either B or D.

Form and Bit Value	Example	Meaning
STAX *e3*	`STAX D`	Store register A to computed address. *e3* must produce either B or D.
LHLD *e16*	`LHLD L1`	Load HL direct from location *e16*. Double-precision load to H and L.
SHLD *e16*	`SHLD L5+x`	Store HL direct to location *e16*. Double-precision store from H and L to memory.
LDA *e16*	`LDA GAMMA`	Load register A from address *e16*.
STA *e16*	`STA X3-5`	Store register A into memory at *e16*.
POP *e3*	`POP PSW`	Load register pair from stack, set SP. *e3* must produce one of B, D, H, or PSW.
PUSH *e3*	`PUSH B`	Store register pair into stack, set SP. *e3* must produce on of B, D, H, or PSW.
IN *e8*	`IN 0`	Load register A with data from port *e8*.
OUT *e8*	`OUT 255`	Send data from register A to port *e8*.
XTHL	`XTHL`	Exchange data from top of stack with HL.
PCHL	`PCHL`	Fill program counter with data from HL.
SPHL	`SPHL`	Fill stack pointer with data from HL.
XCHG	`XCHG`	Exchange DE pair with HL pair.

Table 3-17: Data Movement Instructions

3.5.5. Arithmetic Logic Unit

Instructions that act upon the single-precision accumulator to perform arithmetic and logic operations are given in Table 3-18

Form and Bit Value	Example	Meaning
ADD *e3*	`ADD B`	Add register given by *e3* to accumulator without `carry`. *e3* must produce one of A, B, C, D, E, H, or L.

Form and Bit Value	Example	Meaning
ADC *e3*	`ADC L`	Subtract register *e3* from `A` with carry, *e3* defined as above.
SUB *e3*	`SUB H`	Subtract reg *e3* from `A` without `carry`, *e3* is defined as above.
SBB *e3*	`SBB 2`	Subtract register *e3* from `A` with `carry`, *e3* defined as above.
ANA *e3*	`ANA 1+1`	Logical-and reg with `A`, *e3* as above.
XRA *e3*	`XRA A`	Exclusive-or with `A`, *e3* as above.
ORA *e3*	`ORA B`	Logical-or with `A`, *e3* defined as above.
CMP *e3*	`CMP H`	Compare register with `A`, *e3* as above.
DAA	`DAA`	Decimal adjust register `A` based upon last arithmetic logic unit operation.
CMA	`CMA`	Complement the bits in register `A`.
STC	`STC`	Set the `carry` flag to 1.
CMC	`CMC`	Complement the `carry` flag.
RLC	`RLC`	Rotate bits left, (re)set carry as a side effect. High-order `A` bit becomes carry.
RRC	`RRC`	Rotate bits right, (re)set `carry` as side effect. Low-order `A` bit becomes carry.
RAL	`RAL`	Rotate `carry`/`A` register to left. Carry is involved in the rotate.
RAR	`RAR`	Rotate `carry`/`A` register to right. Carry is involved in the rotate.
DAD *e3*	`DAD B`	Double-precision add register pair *e3* to HL. *e3* must produce `B`, `D`, `H`, or `SP`.

Table 3-18: Arithmetic Logic Unit Operations

3.5.6. Control Instructions

The four remaining instructions, categorized as control instructions, are the following:

HLT halts the 8080 processor.

DI disables the interrupt system.

EI enables the interrupt system.

NOP means no operation.

3.6. Error Messages

When errors occur within the assembly-language program, they are listed as single character flags in the leftmost position of the source listing. The line in error is also echoed at the console so that the source listing need not be examined to determine if errors are present. The error codes are listed in Table 3-19.

Error Code	Meaning
D	Data error: element in data statement cannot be placed in the specified data area.
E	Expression error: expression is ill-formed and cannot be computed at assembly time.
L	Label error: label cannot appear in this context; might be duplicate label.
N	Not implemented: features that will appear in future ASM versions. For example, macros are recognized, but flagged in this version.
O	Overflow: expression is too complicated (too many pending operators) to be computed and should be simplified.
P	Phase error: label does not have the same value on two subsequent passes through the program.
R	Register error: the value specified as a register is not compatible with the operation code.
S	Syntax error: statement is not properly formed.
V	Value error: operand encountered in expression is improperly formed.

Table 3-19: ASM Error Codes

Table 3-20 lists the error messages that are due to terminal error conditions.

Message	Meaning
NO SOURCE FILE PRESENT	The file specified in the ASM command does not exist on disk.
NO DIRECTORY SPACE	The disk directory is full; erase files that are not needed and retry.
SOURCE FILE NAME ERROR	Improperly formed ASM filename, for example, It is specified with ? characters.
SOURCE FILE READ ERROR	Source file cannot be read properly by the assembler; execute a TYPE to determine the point of error.
OUTPUT FILE WRITE ERROR	Output files cannot be written properly; most likely cause is a full disk, erase and retry.
CANNOT CLOSE FILE	The file specified in the ASM command does not exist on disk.

Table 3-20: ASM Error Messages

3.7. A Sample Session

The following sample session shows interaction with the assembler and debugger in the development of a simple assembly-language program.

In the sample:

output text `is in monospace`

input text `is in purple`

carriage-return is displayed as ↲

rubout/DEL key is displayed as ⌫.

comments *displayed in 'hand-written' text.*

```
A>ASM SORT↲

CP/M ASSEMBLER - VER 2.0
0015C                          next free address
003H USE FACTOR                percent of table used 00 to ff (hexadecimal)
END OF ASSEMBLY

A>DIR SORT.*↲

SORT  ASM                      Source file
SORT  BAK                      Back-up from last edit
SORT  PRN                      Print file (contains tab characters)
SORT  HEX                      Machine code file
```

```
A>TYPE SORT.PRN↵
                                    ↙Source Program↘
            ;       SORT PROGRAM IN CP/M ASSEMBLY LANGUAGE
            ;       START AT THE BEGINNING OF THE TRANSIENT PROGRAM AREA

 ↙Machine code location
0100                ORG             100H

            ↙generated code
0100 214601 SORT:   LXI H,SW  ;ADDRESS SWITCH TOGGLE
0103 3601           MVI M,1   ;SET TO 1 FOR FIRST ITERATION
0105 214701         LXI H,I   ;ADDRESS INDEX
0108 3600           MVI M,0   ;I=0
            ;
            ;       COMPARE I WITH ARRAY SIZE
010A 7E     COMPL:  MOV A,M   ;A REGISTER = I
010B FE09           CPI N-1   ;CY SET IF I<(N-1)
010D D21901         JNC CONT  ;CONTINUE IF I<=(N-2)
            ;
            ;       END OF ONE PASS THROUGH DATA
0110 214601         LXI H,SW  ;CHECK FOR ZERO SWITCHES
0113 7EB7C200001    MOV A, M! ORA A! JNZ SORT ;END OF SORT IF SW=0
            ;
0118 FF             RST 7     ;GO TO THE DEBUGGER INSTEAD OF REB
            ;
            ;       CONTINUE THIS PASS
            ;       ADDRESSING I, SO LOAD AV(I) INTO REGISTERS
0119 5F16002148CONT:    MOV E, A! MVI D, 0! LXI H, AV! DAD D! DAD D
 ↙truncated↘
0121 4E792346       MOV C, M! MOV A, C! INX H! MOV B, M
            ;       LOW ORDER BYTE IN A AND C, HIGH ORDER BYTE IN B
            ;
            ;       MOV H AND L TO ADDRESS AV(I+1)
0125 23             INX H
            ;
            ;       COMPARE VALUE WITH REGS CONTAINING AV (I)
0126 965778239E     SUB M! MOV D, A! MOV A, B! INX H! SBB M  ;SUBTRACT
            ;
            ;       BORROW SET IF AV(I+1)>AV(I)
012B DA3F01         JC  INCI  ;SKIP IF IN PROPER ORDER
```

```
                ;                   (continued)
                ;       CHECK FOR EQUAL VALUES
012E B2CA3F01           ORA D! JZ INCI ;SKIP IF AV(I) = AV(I+1)
0132 56702B5E           MOV D, M! MOV M, B! DCX H! MOV E, M
0136 712B722B73         MOV M, C! DCX H! MOV M, D! DCX H! MOV M, E
                ;
                ;       INCREMENT SWITCH COUNT
013B 21460134           LXI H,SW! INR M
                ;
                ;       INCREMENT I
013F 21470134C3INCI:    LXI H,I! INR M! JMP COMP
                ;
                ;       DATA DEFINITION SECTION
0146 00         SW:     DB 0        ;RESERVE SPACE FOR SWITCH COUNT
0147            I:      DS 1        ;SPACE FOR INDEX
0148 050064001EAV:      DW 5, 100, 30, 50, 20, 7, 1000, 300, 100, -32767
000A =          N       EQU($-AV)/2     ;COMPUTE N INSTEAD OF PRE
015C                    END
```

↖equate value

```
A>TYPE SORT.HEX↵
:10010000214601360121470136007EFE09D2190140
:100110002146017EB7C20001FF5F16002148011988
:10012000194E79234623965778239EDA3F01B2CAA7
:100130003F0156702B5E712B722B732146013421C7
:07014000470134C30A01006E
:1001480005006400 1E00320014000700E8032C01BB
:040158006400 0180BE
:0000000000
```

Machine Code
in Intel HEX format

```
A>DDT SORT.HEX↵                    Start debug run
DDT VER 2.2
NEXT PC
015C 0000                          Default address (no address on END statement)
-XP↵
P=0000 100↵                        Change PC to 100
-UFFFF↵                            Untrace for 65535 Steps
⌫                                  Abort trace with Rubout/DEL
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0100 LXI H,0146*0100
-T10↵                              Trace 10 hex/16 decimal steps
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0146 S=0100 P=0100 LXI H, 0146
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0146 S=0100 P=0103 MVI M, 01
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0146 S=0100 P=0105 LXI H, 0147
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0147 S=0100 P=0108 MVI M, 00
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0147 S=0100 P=010A MOV A, M
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=010B CPI 09
C1Z0M1E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=010D JNC 0119
C1Z0M1E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=0110 LXI H, 0146
C1Z0M1E0I0 A=00 B=0000 D=0000 H=0146 S=0100 P=0113 MOV A, M
C1Z0M1E0I0 A=01 B=0000 D=0000 H=0146 S=0100 P=0114 ORA A
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0146 S=0100 P=0115 JNZ 0100
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0146 S=0100 P=0100 LXI H, 0146
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0146 S=0100 P=0103 MVI M, 01
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0146 S=0100 P=0105 LXI H, 0147
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0147 S=0100 P=0108 MVI M, 00
C0Z0M0E0I0 A=01 B=0000 D=0000 H=0147 S=0100 P=010A MOV A, M*010B
                                                   ↙Stopped at 10BH
-A10D↵

010D JC 119↵
0110 ↵                             Change to Jump on Carry

-XP↵
P=010B 100↵                        Reset the Program Counter
                                   Back to beginning of program
```

```
-T10↵                                  Trace execution for 10h steps
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=0100 LXI H,0146
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0146 S=0100 P=0103 MVI M,01
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0146 S=0100 P=0105 LXI H,0147
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=0108 MVI M,00
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=010A MOV A,M
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=010B CPI 09
C1Z0M1E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=010D JC 0119   ← Altered instruction
C1Z0M1E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=0119 MOV E,A
C1Z0M1E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=011A MVI D,00
C1Z0M1E0I0 A=00 B=0000 D=0000 H=0147 S=0100 P=011C LXI H,0148
C1Z0M1E0I0 A=00 B=0000 D=0000 H=0148 S=0100 P=011F DAD D
C0Z0M1E0I0 A=00 B=0000 D=0000 H=0148 S=0100 P=0120 DAD D
C0Z0M1E0I0 A=00 B=0000 D=0000 H=0148 S=0100 P=0121 MOV C,M
C0Z0M1E0I0 A=00 B=0005 D=0000 H=0148 S=0100 P=0122 MOV A,C
C0Z0M1E0I0 A=05 B=0005 D=0000 H=0148 S=0100 P=0123 INX H
C0Z0M1E0I0 A=05 B=0005 D=0000 H=0149 S=0100 P=0124 MOV B,M*0125
                                                  ← Automatic breakpoint

-L100↵
                                       List some code from 100H
 0100   LXI H,0146
 0103   MVI M,01
 0105   LXI H,0147
 0108   MVI M,00
 010A   MOV A,M
 010B   CPI 09
 010D   JC  0119
 0110   LXI H,0146
 0113   MOV A,M
 0114   ORA A
 0115   JNZ 0100

-L↵

                                       List some more
 0118   RST 07
 0119   MOV E,A
 011A   MVI D,00
 011C   LXI H,0148

⌫

                                       Abort list with Rubout/DEL
```

```
-G,11B↵                                  Start program from current PC (0125H)
                                         and run in real time to 11BH
*0127                                    Front-panel interrupt (infinite loop)
-T4↵                                     Look at looping program in trace mode

C0Z0M0E0I0 A=38 B=0064 D=0006 H=0156 S=0100 P=0127 MOV D,A
C0Z0M0E0I0 A=38 B=0064 D=3806 H=0156 S=0100 P=0128 MOV A,B
C0Z0M0E0I0 A=00 B=0064 D=3806 H=0156 S=0100 P=0129 INX H
C0Z0M0E0I0 A=00 B=0064 D=3806 H=0157 S=0100 P=012A SBB M*012B

-D148↵                                   Examine memory
0148 05 00 07 00 14 00 1E 00 ..... Data are sorted, but program does not stop
0150 32 00 64 00 64 00 2C 01 E8 03 01 80 00 00 00 00 2.D.D.,........
0160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................

-G0↵                                     Return to CP/M
A>DDT SORT.HEX↵                          Reload the memory image
DDT VER 2.2
NEXT PC
015C 0000
-XP↵
P=0000 100↵                              Set PC to beginning of program
-L10D↵                                   List Bad OPCODE

010D   JNC 0119
0110   LXI H,0146

⌫
-A10D↵                                   Abort list with Rubout/DEL
                                         Assemble new OPCODE
010D JC 119↵
0110 ↵
```

```
-L100↲                                  List starting section of program

0100   LXI H,0146
0103   MVI M,01
0105   LXI H,0147
0108   MVI M,00

⌫                                       Abort list with Rubout/DEL
-A103↲                                  Change switch initialization to 00

0103 MVI M,0↲
0115 ↲

-^C                                     Return to CP/M with CTRL-C
A>SAVE 1 SORT.COM↲                      Save 1 page (256 bytes, from 100H to 1FFH)
                                        on disk in case there is a need to reload later
A>DDT SORT.COM↲                         Restart with saved memory image

DDT VER 2.2
NEXT PC
0200 0100
                                        COM file always starts with address 100H
-G↲                                     Run the program from PC=100H
*0118                                   Program stop (RST 1) encountered
-D148↲

0148 05 00 07 00 14 00 1E 00 ........
0150 32 00 64 00 64 00 2C 01 E8 03 01 00 00 00 00 00 2.D.D.....
0160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
0170 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
                                        ✓ Data properly sorted

-G0↲                                    Return to CP/M
```

```
A>ED SORT.ASM↲              Make changes to original program
*N,0^Z0TT↲                  find next ",0" (^Z is CTRL-Z)
        MVI     M,0     ;I = 0
*-↲                         up one line in text
        LXI     H,I     ;ADDRESS INDEX
*-↲                         up another line
        MVI     M,1     ;SET TO 1 FOR FIRST ITERATION
*KT↲                        kill line and type next line
        LXI     H,I     ;ADDRESS INDEX
*I↲                         insert new line
        MVI     M,0     ;ZERO SW↲
*T↲
        LXI     H,I     ;ADDRESS INDEX
*NJNC^Z0T↲
        JNC* T↲
        CONT    ;CONTINUE IF I <= (N-2)
*-2DIC^Z0LT↲
        JC      CONT    ;CONTINUE IF I <= (N-2)
*E↲
```

```
A>ASM SORT.AAZ↵                    Source from disk A, Hex to disk A, Skip PRN

CP/M ASSEMBLER - VER 2.0
015C                               next address to assemble
003H USE FACTOR
END OF ASSEMBLY

A>DDT SORT.HEX↵                    test program changes

DDT VER 2.2
NEXT PC
015C 0000

-G100↵
*0118
-D148↵
                                   data sorted

0148 05 00 07 00 14 00 1E 00 ........
0150 32 00 64 00 64 00 2C 01 E8 03 01 80 00 00 00 00 2.D.D...........
0160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
0170 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................

⌫
-G0↵                               abort with Rubout/DEL
                                   Return to CP/M - program checks OK.
```

4. CP/M Dynamic Debugging Tool

4.1. Introduction

The DDT program allows dynamic interactive testing and debugging of programs generated in the CP/M environment. Invoke the debugger with a command of one of the following forms:

```
DDT
DDT filename.HEX
DDT filename.COM
```

where *filename* is the name of the program to be loaded and tested. In both cases, the DDT program is brought into main memory in place of the Console Command Processor (CCP) and resides directly below the Basic Disk Operating System (BDOS) portion of CP/M. Refer to Section 5 for standard memory organization. The BDOS starting address, located in the address field of the JMP instruction at location 0005H, is altered to reflect the reduced Transient Program Area (TPA) size.

The second and third forms of the DDT command perform the same actions as the first, except there is a subsequent automatic load of the specified HEX or COM file. The action is identical to the following sequence of commands:

```
DDT
Ifilename.HEX or Ifilename.COM
R
```

where the I and R commands set up and read the specified program to test. See the explanation of the I and R commands below for exact details.

Upon initiation, DDT prints a sign-on message in the form:

```
DDT VER m.m
```

where *m.m* is the revision number.

Following the sign-on message, DDT prompts you with the hyphen character, -, and waits for input commands from the console. You can type any of several single character commands, followed by a carriage return to execute the command. Each line of input can be line-edited using the following standard CP/M controls:

Character	Meaning
CTRL-C	Reboots CP/M system.
CTRL-H	Backspaces one character position.
CTRL-U	Deletes the entire line typed at the console.

Character	Meaning
CTRL-X	Same as CTRL-U.
rubout/del	Deletes and echoes the last character typed at the console.

Table 4-21: DDT Line-editing Controls

Any command can be up to 32 characters in length. An automatic carriage return is inserted as character 33, where the first character determines the command type. Table 4-22 describes DDT commands.

Character	Result
A	enters assembly-language mnemonics with operands.
D	displays memory in hexadecimal and ASCII.
F	fills memory with constant data.
G	begins execution with optional breakpoints.
I	sets up a standard input File Control Block.
L	lists memory using assembler mnemonics.
M	moves a memory segment from source to destination.
R	reads a program for subsequent testing.
S	substitutes memory values.
T	traces program execution.
U	untraced program monitoring.
X	examines and optionally alters the CPU state.

Table 4-22: DDT Commands

The command character, in some cases, is followed by zero, one, two, or three hexadecimal values, which are separated by commas or single blank characters. All DDT numeric output is in hexadecimal form. The commands are not executed until the carriage return is typed at the end of the command.

At any point in the debug run, you can stop execution of DDT by using either a CTRL-C or G0 (jump to location 0000H) and save the current memory image by using a SAVE command of the form:

SAVE *n filename*.COM↵

where n is the number of pages (256 byte blocks) to be saved on disk. The number of blocks is determined by taking the high-order byte of the address in the TPA and converting this number to decimal. For example, if the highest address in the TPA is 134H, the number of pages is 12H or 18 in decimal. You could type a CTRL-C during the debug run, returning to the CCP level, followed by

```
SAVE 18 X.COM↵
```

The memory image is saved as X.COM on the disk and can be directly executed by typing the name X. If further testing is required, the memory image can be recalled by typing

```
DDT X.COM↵
```

which reloads the previously saved program from location 100H through page 18, 23FFH. The CPU state is not a part of the COM file; thus, the program must be restarted from the beginning to test it properly.

4.2. DDT Commands

The individual commands are detailed below. In each case, the operator must wait for the hyphen prompt character before entering the command. If control is passed to a program under test, and the program has not reached a breakpoint, control can be returned to DDT by executing a RST 7 from the front panel. In the explanation of each command, the command letter is shown in some cases with numbers separated by commas, the numbers are represented by lower-case letters. These numbers are always assumed to be in a hexadecimal radix and from one to four digits in length. Longer numbers are automatically truncated on the right.

Many of the commands operate upon a CPU state that corresponds to the program under test. The CPU state holds the registers of the program being debugged and initially contains zeros for all registers and flags except for the program counter, P, and stack pointer, S, which default to 100H. The program counter is subsequently set to the starting address given in the last record of a HEX file if a file of this form is loaded, see the I and R commands.

4.2.1. The A (Assembly) Command

DDT allows in-line assembly language to be inserted into the current memory image using the A command, which takes the form:

A*s*

where *s* is the hexadecimal starting address for the in-line assembly. DDT prompts the console with the address of the next instruction to fill and reads the console, looking for assembly-

language mnemonics followed by register references and operands in absolute hexadecimal form. See the Intel 8080 Assembly Language Reference Card for a list of mnemonics. Each successive load address is printed before reading the console. The A command terminates when the first empty line is input from the console.

Upon completion of assembly language input, you can review the memory segment using the DDT disassembler (see the L command).

Note that the assembler/disassembler portion of DDT can be overlaid by the transient program being tested, in which case the DDT program responds with an error condition when the A and L commands are used.

4.2.2. The D (Display) Command

The D command allows you to view the contents of memory in hexadecimal and ASCII formats. The D command takes the forms:

D

D*s*

D*s*,*f*

In the first form, memory is displayed from the current display address, initially 100H, and continues for 16 display lines. Each display line takes the following form:

```
aaaa bb bb bb bb bb bb bb bb bb bb bb bb bb bb bb bb cccccccccccccccc
```

where *aaaa* is the display address in hexadecimal and *bb* represents data present in memory starting at *aaaa*. The ASCII characters starting at *aaaa* are to the right (represented by the sequence of character *c*) where non-graphic characters are printed as a period. You should note that both upper- and lower-case alphabetic characters are displayed, and will appear as upper-case symbols on a console device that supports only upper-case. Each display line gives the values of 16 bytes of data, with the first line truncated so that the next line begins at an address that is a multiple of 16.

The second form of the D command is similar to the first, except that the display address is first set to address *s*.

The third form causes the display to continue from address *s* through address *f*. In all cases, the display address is set to the first address not displayed in this command, so that a continuing display can be accomplished by issuing successive D commands with no explicit addresses.

Excessively long displays can be aborted by pressing the return key.

4.2.3. The F (Fill) Command

The F command takes the form:

F*s*,*f*,*c*

where *s* is the starting address, *f* is the final address, and *c* is a hexadecimal byte constant. DDT stores the constant *c* at address *s*, increments the value of *s* and test against *f*. If *s* exceeds *f*, the operation terminates, otherwise the operation is repeated. Thus, the fill command can be used to set a memory block to a specific constant value.

4.2.4. The G (Go) Command

A program is executed using the G command, with up to two optional breakpoint addresses. The G command takes the forms:

G

G*s*

G*s*,*b*

G*s*,*b*,*c*

G,*b*

G,*b*,*c*

The first form executes the program at the current value of the program counter in the current machine state, with no breakpoints set. The only way to regain control in DDT is through a RST 7 execution. The current program counter can be viewed by typing an X or XP command.

The second form is similar to the first, except that the program counter in the current machine state is set to address *s* before execution begins.

The third form is the same as the second, except that program execution stops when address *b* is encountered (*b* must be in the area of the program under test). The instruction at location *b* is not executed when the breakpoint is encountered.

The fourth form is identical to the third, except that two breakpoints are specified, one at *b* and the other at *c*. Encountering either breakpoint causes execution to stop and both breakpoints are cleared. The last two forms take the program counter from the current machine state and set one and two breakpoints, respectively.

Execution continues from the starting address in real-time to the next breakpoint. There is no intervention between the starting address and the break address by DDT. If the program under test does not reach a breakpoint, control cannot return to DDT without executing a RST 7 instruction. Upon encountering a breakpoint, DDT stops execution and types

**d*

where *d* is the stop address. The machine state can be examined at this point using the X (Examine) command. You must specify breakpoints that differ from the program counter address at the beginning of the G command. Thus, if the current program counter is 1234H, then the following commands:

```
G,1234
G400,400
```

both produce an immediate breakpoint without executing any instructions.

4.2.5. The I (Input) Command

The `I` command allows you to insert a filename into the default File Control Block (FCB) at 5CH. The FCB created by CP/M for transient programs is placed at this location (see Section 5). The default FCB can be used by the program under test as if it had been passed by the CP/M Console Processor. Note that this filename is also used by `DDT` for reading additional `HEX` and `COM` files. The `I` command takes the forms:

`I`*filename*

`I`*filename*`.`*typ*

If the second form is used and the filetype is either `HEX` or `COM`, subsequent `R` commands can be used to read the pure binary or hex format machine code. Section 4.2.8 gives further details.

4.2.6. The L (List) Command

The `L` command is used to list assembly-language mnemonics in a particular program region. The `L` command takes the forms:

`L`

`L`*s*

`L`*s*`,`*f*

The first form lists twelve lines of disassembled machine code from the current list address. The second form sets the list address to *s* and then lists twelve lines of code. The last form lists disassembled code from *s* through address *f*. In all three cases, the list address is set to the next unlisted location in preparation for a subsequent `L` command. Upon encountering an execution breakpoint, the list address is set to the current value of the program counter (`G` and `T` commands). Again, long type-outs can be aborted by pressing RETURN during the list process.

4.2.7. The M (Move) Command

The `M` command allows block movement of program or data areas from one location to another in memory. The `M` command takes the form:

`M`*s*`,`*f*`,`*d*

where *s* is the start address of the move, *f* is the final address, and *d* is the destination address. Data is first removed from *s* to *d*, and both addresses are incremented. If *s* exceeds *f*, the move operation stops; otherwise, the move operation is repeated.

4.2.8. The R (Read) Command

The R command is used in conjunction with the I command to read COM and HEX files from the disk into the transient program area in preparation for the debug run. The R command takes the forms:

R

R*b*

where *b* is an optional bias address that is added to each program or data address as it is loaded. The load operation must not overwrite any of the system parameters from 000H through 0FFH (that is, the first page of memory). If *b* is omitted, then $b = 0000$ is assumed. The R command requires a previous I command, specifying the name of a HEX or COM file. The load address for each record is obtained from each individual HEX record, while an assumed load address of 100H is used for COM files. Note that any number of R commands can be issued following the I command to reread the program under test, assuming the tested program does not destroy the default area at 5CH. Any file specified with the filetype COM is assumed to contain machine code in pure binary form (created with the LOAD or SAVE command), and all others are assumed to contain machine code in Intel hex format (produced, for example, with the ASM command).

Recall that the command,

DDT *filename.typ*↵

which initiates the DDT program, equals to the following commands:

DDT↵

-*filename.typ*↵ -R↵

Whenever the R command is issued, DDT responds with either the error indicator ? (file cannot be opened, or a checksum error occurred in a HEX file) or with a load message. The load message takes the form:

NEXT PC

nnnn pppp

where *nnnn* is the next address following the loaded program and *pppp* is the assumed program counter (100H for COM files, or taken from the last record if a HEX file is specified).

4.2.9. The S (Set) Command

The S command allows memory locations to be examined and optionally altered. The S command takes the form:

S*s*

where s is the hexadecimal starting address for examination and alteration of memory. DDT responds with a numeric prompt, giving the memory location, along with the data currently held in memory. If you type a carriage return, the data is not altered. If a byte value is typed, the value is stored at the prompted address. In either case, DDT continues to prompt with successive addresses and values until you type either a period or an invalid input value is detected.

4.2.10. The T (Trace) Command

The T command allows selective tracing of program execution for 1 to 65535 program steps. The T command takes the forms:

T

T*n*

In the first form, the CPU state is displayed and the next program step is executed. The program terminates immediately, with the termination address displayed as

**hhhh*

where *hhhh* is the next address to execute. The display address (used in the D command) is set to the value of H and L, and the list address (used in the L command) is set to *hhhh*. The CPU state at program termination can then be examined using the X command.

The second form of the T command is similar to the first, except that execution is traced for *n* steps (*n* is a hexadecimal value) before a program breakpoint occurs. A breakpoint can be forced in the trace mode by typing a rubout character. The CPU state is displayed before each program step is taken in trace mode. The format of the display is the same as described in the X command.

You should note that program tracing is discontinued at the CP/M interface and resumes after return from CP/M to the program under test. Thus, CP/M functions that access I/O devices, such as the disk drive, run in real-time, avoiding I/O timing problems. Programs running in trace mode execute approximately 500 times slower than real-time because DDT gets control after each user instruction is executed. Interrupt processing routines can be traced, but commands that use the breakpoint facility (G, T, and U) accomplish the break using an RST 7 instruction, which means that the tested program cannot use this interrupt location. Further, the trace mode always runs the tested program with interrupts enabled, which may cause problems if asynchronous interrupts are received during tracing.

To get control back to DDT during trace, press RETURN rather than executing an RST 7. This ensures that the trace for current instruction is completed before interruption.

4.2.11. The U (Untrace) Command

The U command is identical to the T command, except that intermediate program steps are not displayed. The untrace mode allows from 1 to 65535 (0FFFFH) steps to be executed in monitored

mode and is used principally to retain control of an executing program while it reaches steady state conditions. All conditions of the T command apply to the U command.

4.2.12. The X (Examine) Command

The X command allows selective display and alteration of the current CPU state for the program under test. The X command takes the forms:

X

X*r*

where *r* is one of the 8080 CPU registers listed in Table 4-23.

Register	Meaning	Value
C	Carry flag	0 or 1
Z	Zero flag	0 or 1
M	Minus flag	0 or 1
E	Even parity flag	0 or 1
I	Half-carry	0 or 1
A	Accumulator	0 ... FF
B	BC register pair	0 ... FFFF
D	DE register pair	0 ... FFFF
H	HL register pair	0 ... FFFF
S	Stack pointer	0 ... FFFF
P	Program counter	0 ... FFFF

Table 4-23: CPU Registers

In the first case, the CPU register state is displayed in the format:

C*f* **Z***f* **M***f* **E***f* **I***f* **A=***bb* **B=***dddd* **D=***dddd* **H=***dddd* **S=***dddd* **P=***dddd* *inst*

where *f* is a 0 or 1 flag value, *bb* is a byte value, and *dddd* is a double-byte quantity corresponding to the register pair. The *inst* field contains the disassembled instruction, that occurs at the location addressed by the CPU state's program counter.

The second form allows display and optional alteration of register values, where r is one of the registers given above (C, Z, M, E, I, A, B, D, H, S, or P). In each case, the flag or register value is first

displayed at the console. The DDT program then accepts input from the console. If a carriage return is typed, the flag or register value is not altered. If a value in the proper range is typed, the flag or register value is altered. You should note that BC, DE, and HL are displayed as register pairs. Thus, you must type the entire register pair when B, C, or the BC pair is altered.

4.3. Implementation Notes

The organization of DDT allows certain nonessential portions to be overlaid to gain a larger transient program area for debugging large programs. The DDT program consists of two parts: the DDT nucleus and the assembler/disassembler module. The DDT nucleus is loaded over the CCP and, although loaded with the DDT nucleus, the assembler/disassembler can be overlaid unless used to assemble or disassemble.

In particular, the BDOS address at location 0006H (address field of the JMP instruction at location 0005H) is modified by DDT to address the base location of the DDT nucleus, which, in turn, contains a JMP instruction to the BDOS. Thus, programs that use this address field to size memory see the logical end of memory at the base of the DDT nucleus rather than the base of the BDOS.

The assembler/disassembler module resides directly below the DDT nucleus in the transient program area. If the A, L, T, or X commands are used during the debugging process, the DDT program again alters the address field at 0006H to include this module, further reducing the logical end of memory. If a program loads beyond the beginning of the assembler/disassembler module, the A and L commands are lost (their use produces a ? in response) and the trace and display (T and X) commands list the inst field of the display in hexadecimal, rather than as a decoded instruction.

4.4. A Sample Program

The following example shows an edit, assemble, and debug for a simple program that reads a set of data values and determines the largest value in the set. The largest value is taken from the vector and stored into LARGE at the termination of the program.

In the example:

output text `is in a monospace font`

input text `is in purple`

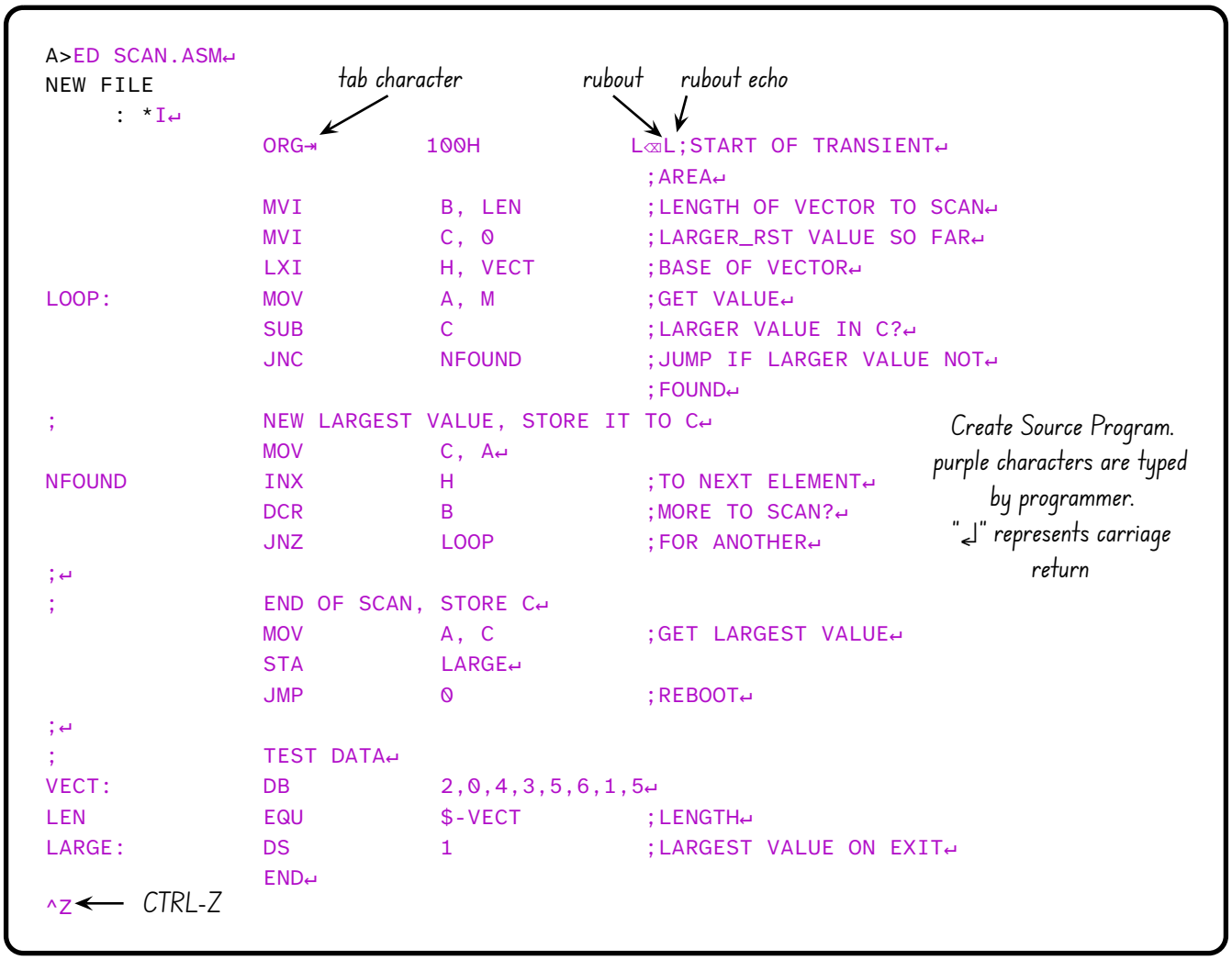
carriage-return is displayed as ↲

rubout/DEL key is displayed as ⌫

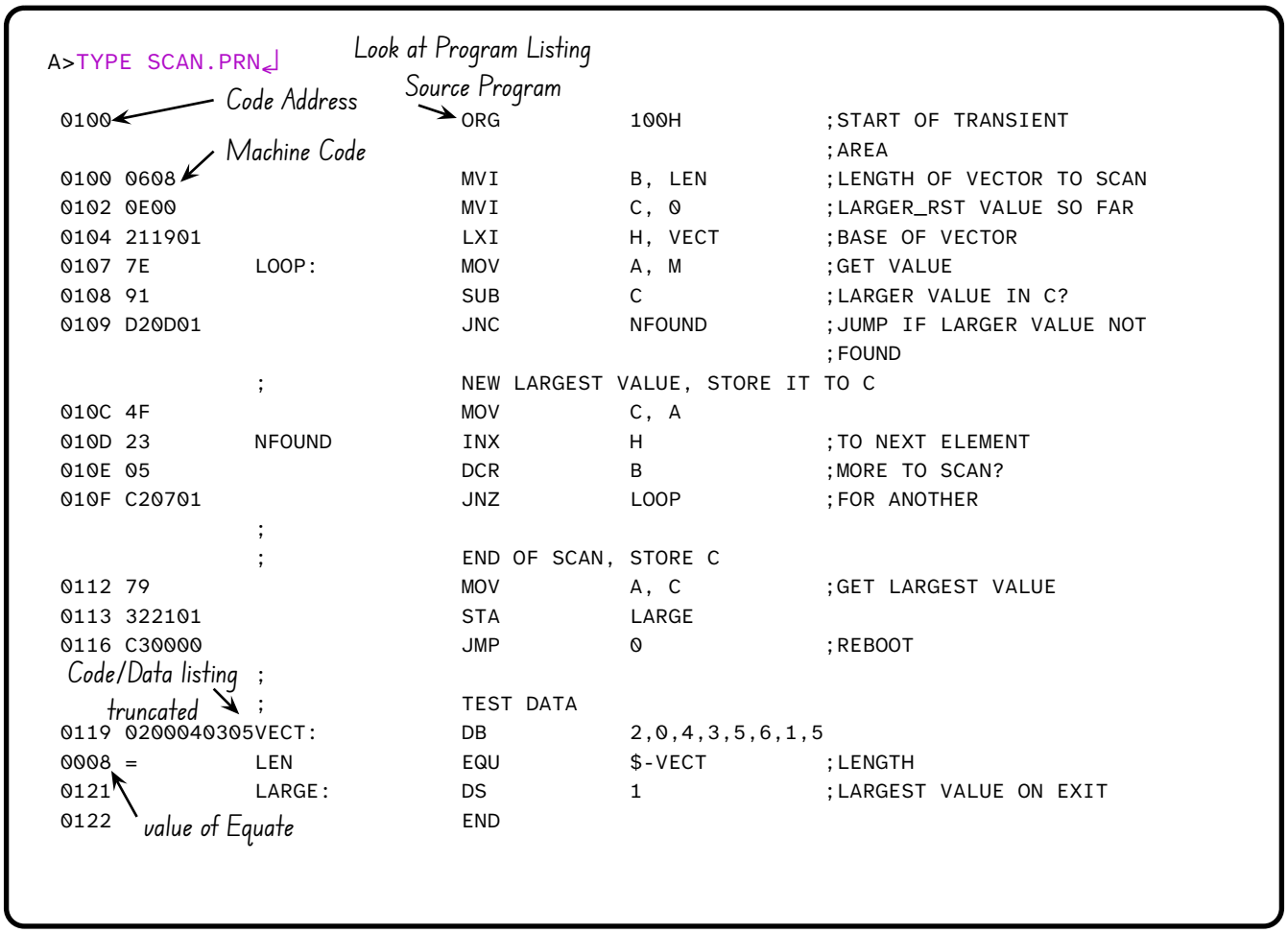
tab key is displayed as →|

CTRL-Z ^Z

comments *are displayed in 'hand-written' text.*


A>ED SCAN.ASM↵
NEW FILE
: *I↵
tab character
rubout
rubout echo
ORG⇥ 100H L⌫L;START OF TRANSIENT↵
;AREA↵
MVI B, LEN ;LENGTH OF VECTOR TO SCAN↵
MVI C, 0 ;LARGER_RST VALUE SO FAR↵
LXI H, VECT ;BASE OF VECTOR↵
LOOP: MOV A, M ;GET VALUE↵
SUB C ;LARGER VALUE IN C?↵
JNC NFOUND ;JUMP IF LARGER VALUE NOT↵
;FOUND↵
; NEW LARGEST VALUE, STORE IT TO C↵
MOV C, A↵
NFOUND INX H ;TO NEXT ELEMENT↵
DCR B ;MORE TO SCAN?↵
JNZ LOOP ;FOR ANOTHER↵
;↵
; END OF SCAN, STORE C↵
MOV A, C ;GET LARGEST VALUE↵
STA LARGE↵
JMP 0 ;REBOOT↵
;↵
; TEST DATA↵
VECT: DB 2,0,4,3,5,6,1,5↵
LEN EQU $-VECT ;LENGTH↵
LARGE: DS 1 ;LARGEST VALUE ON EXIT↵
END↵
^Z ← CTRL-Z
Create Source Program.
purple characters are typed
by programmer.
"↵" represents carriage
return

```
    : *B0P↵
   1:                   ORG          100H           ;START OF TRANSIENT
   2:                                               ;AREA
   3:                   MVI          B, LEN         ;LENGTH OF VECTOR TO SCAN
   4:                   MVI          C, 0           ;LARGER_RST VALUE SO FAR
   5:                   LXI          H, VECT        ;BASE OF VECTOR
   6:   LOOP:           MOV          A, M           ;GET VALUE
   7:                   SUB          C              ;LARGER VALUE IN C?
   8:                   JNC          NFOUND         ;JUMP IF LARGER VALUE NOT
   9:                                               ;FOUND
  10:   ;               NEW LARGEST VALUE, STORE IT TO C
  11:                   MOV          C, A
  12:   NFOUND          INX          H              ;TO NEXT ELEMENT
  13:                   DCR          B              ;MORE TO SCAN?
  14:                   JNZ          LOOP           ;FOR ANOTHER
  15:   ;
  16:   ;               END OF SCAN, STORE C
  17:                   MOV          A, C           ;GET LARGEST VALUE
  18:                   STA          LARGE
  19:                   JMP          0              ;REBOOT
  20:   ;
  21:   ;               TEST DATA
  22:   VECT:           DB           2,0,4,3,5,6,1,5
  23:   LEN             EQU          $-VECT         ;LENGTH
   1: *E↵  ← End of Edit

A>ASM SCAN↵        Start Assembler
CP/M ASSEMBLER - VER 1.0

0122
002H USE FACTOR
END OF ASSEMBLY       Assembly Complete - Look at Program Listing
```


A>TYPE SCAN.PRN↵
Look at Program Listing
Source Program
Code Address
Machine Code
0100
0100 0608
0102 0E00
0104 211901
0107 7E
0108 91
0109 D20D01
010C 4F
010D 23
010E 05
010F C20701
0112 79
0113 322101
0116 C30000
0119 0200040305
0008 =
0121
0122
LOOP:
NFOUND
VECT:
LEN
LARGE:
ORG 100H ;START OF TRANSIENT
;AREA
MVI B, LEN ;LENGTH OF VECTOR TO SCAN
MVI C, 0 ;LARGER_RST VALUE SO FAR
LXI H, VECT ;BASE OF VECTOR
MOV A, M ;GET VALUE
SUB C ;LARGER VALUE IN C?
JNC NFOUND ;JUMP IF LARGER VALUE NOT
;FOUND
; NEW LARGEST VALUE, STORE IT TO C
MOV C, A
INX H ;TO NEXT ELEMENT
DCR B ;MORE TO SCAN?
JNZ LOOP ;FOR ANOTHER
;
; END OF SCAN, STORE C
MOV A, C ;GET LARGEST VALUE
STA LARGE
JMP 0 ;REBOOT
Code/Data listing truncated
; TEST DATA
DB 2,0,4,3,5,6,1,5
EQU $-VECT ;LENGTH
DS 1 ;LARGEST VALUE ON EXIT
END
value of Equate

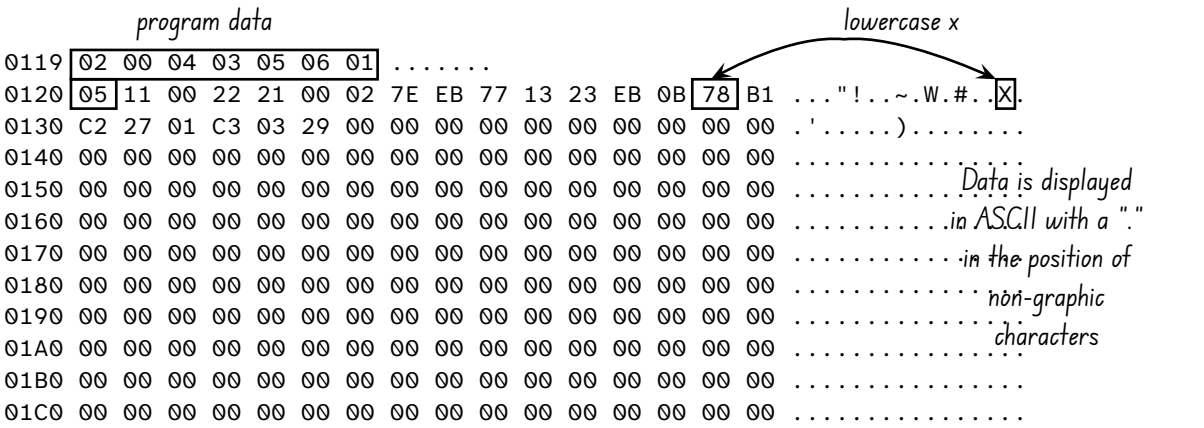
```
A>DDT SCAN.HEX↵          Start Debugger with hex format machine code

16K DDT VERS 1.0
NEXT PC
0121 0000
-X↵          last load address + 1                    next instruction to execute
                                                          at PC = 0
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0000 JMP   FA03
        examine registers before debug run
-XP↵
P=0000 100↵      change PC to 100
-X↵          look at registers again

                                          PC changed
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0100 MVI B,08
                                                   next instruction
-L100↵                                             to execute at PC=100

  0100  MVI  B,08
  0102  MVI  C,00
  0104  LXI  H,0119
  0107  MOV  A,M
  0108  SUB  C
  0109  JNC  010D
  010C  MOV  C,A
  010D  INX  H
  010E  DCR  B
  010F  JNZ  0107
  0112  MOV  A,C
-L↵

  0113  STA  0121
  0116  JMP  0000
  0119  STAX B
  011A  NOP
  011B  INR  B
  011C  INX  B
  011D  DCR  B
  011E  MVI  B,01
  0120  DCR  B
  0121  ??=  20
  0122  MOV  D,D
```

Diassembled machine code at 100H (See source listing for comparison)

A little more machine code (note that program ends at location 116 with a JMP to 0000)

```
-A116↵                enter inline assembly mode to change the JMP to 0000 into a RST 7, which will
0116 RST 7↵           cause the program under test to return to DDT if 116H is ever executed
0117 ↵                (single carriage return stops assemble mode)
-L113↵
                                  list code at 113H to check that RST 7 was properly inserted
  0113  STA  0121                 in place of JMP
  0116  RST  07
  0117  NOP
  0118  NOP
  0119  STAX B
  011A  NOP
  011B  INR  B
  011C  INX  B
  011D  DCR  B
  011E  MVI  B,01
  0120  DCR  B
-X↵             look at registers

C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0100 MVI B,08

-T↵             execute program for one step          initial CPU state, before instruction is executed

C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0100 MVI B,08*0102

-T↵             trace one step again (note 08H in B)  automatic breakpoint

C0Z0M0E0I0 A=00 B=0800 D=0000 H=0000 S=0100 P=0102 MVI C,00*0104

-T↵             trace again (register C is cleared)

C0Z0M0E0I0 A=00 B=0800 D=0000 H=0000 S=0100 P=0104 LXI H,0119*0107

-T3↵            trace three steps

C0Z0M0E0I0 A=00 B=0800 D=0000 H=0119 S=0100 P=0107 MOV A,M
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0108 SUB C
C0Z0M0E0I1 A=02 B=0800 D=0000 H=0119 S=0100 P=0109 JNC 010D*010D
                                        automatic breakpoint at 10DH
```

```
-D119↵      display memory starting at 119H
```




```
-X↵         current CPU state

C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=010D INX  H

-T5↵        trace 5 steps from current CPU state

C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=010D INX  H
C0Z0M0E0I0 A=02 B=0800 D=0000 H=011A S=0100 P=010E DCR  B
C0Z0M0E0I0 A=02 B=0700 D=0000 H=011A S=0100 P=010F JNZ  0107
C0Z0M0E0I0 A=02 B=0700 D=0000 H=011A S=0100 P=0107 MOV  A,M
C0Z0M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=0108 SUB  C*0109
-U5↵        trace without listing intermediate states
```

automatic breakpoint

```
C0Z1M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=0109 JNC  010D*0108
-X↵         CPU state at end of U5

C0Z0M0E0I0 A=04 B=0600 D=0000 H=011B S=0100 P=0108 SUB  C

-G↵         run program from current PC until completion (in real-time)
*0116       breakpoint at 116H, caused by executing RST 7 in machine code
-X↵         CPU state at end of program

C0Z1M0E0I0 A=00 B=0000 D=0000 H=0121 S=0100 P=0116 RST  07

-XP↵        examine and change program counter
P=0116 100↵
-X↵

C0Z1M0E0I0 A=00 B=0000 D=0000 H=0121 S=0100 P=0100 MVI  B,08
```

```
-T10↵
C0Z1M0E0I0 A=00 B=0000 D=0000 H=0121 S=0100 P=0100 MVI  B,08
C0Z1M0E0I0 A=00 B=0800 D=0000 H=0121 S=0100 P=0102 MVI  C,00
C0Z1M0E0I0 A=00 B=0800 D=0000 H=0121 S=0100 P=0104 LXI  H,0119
C0Z1M0E0I0 A=00 B=0800 D=0000 H=0119 S=0100 P=0107 MOV  A,M
C0Z1M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0108 SUB  C
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0109 JNC  010D
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=010D INX  H
C0Z0M0E0I0 A=02 B=0800 D=0000 H=011A S=0100 P=010E DCR  B
C0Z0M0E0I0 A=02 B=0700 D=0000 H=011A S=0100 P=010F JNZ  0107
C0Z0M0E0I0 A=02 B=0700 D=0000 H=011A S=0100 P=0107 MOV  A,M
C0Z0M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=0108 SUB  C
C0Z1M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=0109 JNC  010D
C0Z1M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=010D INX  H
C0Z1M0E0I0 A=00 B=0700 D=0000 H=011B S=0100 P=010E DCR  B
C0Z0M0E0I0 A=00 B=0600 D=0000 H=011B S=0100 P=010F JNZ  0107
C0Z0M0E0I0 A=00 B=0600 D=0000 H=011B S=0100 P=0107 MOV  A,M*0108

-A109↵
-JC 10D↵
010C↵
-G0↵
A>SAVE 1 SCAN.COM↵
A>DDT SCAN.COM↵

DDT VERS 2.2
NEXT PC
0200 0100

-L100↵

  0100  MVI  B,08
  0102  MVI  C,00
  0104  LXI  H,0119
  0107  MOV  A,M
  0108  SUB  C
  0109  JC   010D
  010C  MOV  C,A
  010D  INX  H
  010E  DCR  B
  010F  JNZ  0107
  0112  MOV  A,C

-XP↵
P=0100 ↵
```

trace 10 (hexadecimal) steps

first data element

current largest value

subtract for comparison

A < C

Insert a "hot patch" into the machine code to change the JNC to JC

stop DDT so that a version of the patched program can be saved

program resides on first page, so save 1 page

restart DDT with the saved memory image to continue testing

list some code

previous patch is present in SCAN.COM

Program should have moved the value from A into C since A > C. Since this case was not executed, it appears that the JNC should have been a JC instruction

```
-T10↲      trace to see how patched version operates
                                        data is moved from A to C
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0100 MVI B,08
C0Z0M0E0I0 A=00 B=0800 D=0000 H=0000 S=0100 P=0102 MVI C,00
C0Z0M0E0I0 A=00 B=0800 D=0000 H=0000 S=0100 P=0104 LXI H,0119
C0Z0M0E0I0 A=00 B=0800 D=0000 H=0119 S=0100 P=0107 MOV A,M
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0108 SUB C
C0Z0M0E0I1 A=02 B=0800 D=0000 H=0119 S=0100 P=0109 JC  010D
C0Z0M0E0I1 A=02 B=0800 D=0000 H=0119 S=0100 P=010C MOV C,A
C0Z0M0E0I1 A=02 B=0802 D=0000 H=0119 S=0100 P=010D INX H
C0Z0M0E0I1 A=02 B=0802 D=0000 H=011A S=0100 P=010E DCR B
C0Z0M0E0I1 A=02 B=0702 D=0000 H=011A S=0100 P=010F JNZ 0107
C0Z0M0E0I1 A=02 B=0702 D=0000 H=011A S=0100 P=0107 MOV A,M
C0Z0M0E0I1 A=00 B=0702 D=0000 H=011A S=0100 P=0108 SUB C
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011A S=0100 P=0109 JC  010D
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011A S=0100 P=010D INX H
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011B S=0100 P=010E DCR B
C1Z0M0E1I1 A=FE B=0602 D=0000 H=011B S=0100 P=010F JNZ 0107*0107
                                        breakpoint after 16 steps
-X↲
C1Z0M0E1I1 A=FE B=0602 D=0000 H=011B S=0100 P=0107 MOV A,M

-G,108↲  run from current PC and breakpoint at 108H
*0108
-X↲          next data item
C1Z0M0E1I1 A=04 B=0602 D=0000 H=011B S=0100 P=0108 SUB C

-T↲                  single step for a few cycles
C1Z0M0E1I1 A=04 B=0602 D=0000 H=011B S=0100 P=0108 SUB C*0109

-T↲
C0Z0M0E0I1 A=02 B=0602 D=0000 H=011B S=0100 P=0109 JC 010D*010C

-X↲
C0Z0M0E0I1 A=02 B=0602 D=0000 H=011B S=0100 P=010C MOV C,A

-G↲    run to completion
*0116
-X↲
C0Z1M0E1I1 A=03 B=0003 D=0000 H=0121 S=0100 P=0116 RST 07

-S121↲     look at the value of "LARGE"
 0121   03  ↲      Wrong value!
 0122   52  .↲
```

```
-L100↵

  0100  MVI   B,08
  0102  MVI   C,00
  0104  LXI   H,0119
  0107  MOV   A,M
  0108  SUB   C
  0109  JC    010D
  010C  MOV   C,A
  010D  INX   H
  010E  DCR   B
  010F  JNZ   0107
  0112  MOV   A,C
-L↵

  0113  STA   0121
  0116  RST   07
  0117  NOP
  0118  NOP
  0119  STAX  B
  011A  NOP
  011B  INR   B
  011C  INX   B
  011D  DCR   B
  011E  MVI   B,01
  0120  DCR   B
-XP↵
```

Review the code

reset the PC

```
P=0116 100↵
-T↵
```

single step, and watch data values

```
C0Z1M0E1I1 A=03 B=0003 D=0000 H=0121 S=0100 P=0100 MVI B,08*0102

-T↵
```

count set

```
C0Z1M0E1I1 A=03 B=0803 D=0000 H=0121 S=0100 P=0102 MVI C,00*0104
-T↵
```

"largest" set

```
C0Z1M0E1I1 A=03 B=0800 D=0000 H=0121 S=0100 P=0104 LXI H,0119*0107
-T↵
```

base address of data set

```
C0Z1M0E1I1 A=03 B=0800 D=0000 H=0119 S=0100 P=0107 MOV A,M*0108
-T↵
```

first data item brought to A

```
C0Z1M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0108 SUB  C*0109
```

```
-T↲
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0109 JC   010D*010C
-T↲
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=010C MOV  C,A*010D
-T↲                                  first data item moved to C correctly
C0Z0M0E0I1 A=02 B=0802 D=0000 H=0119 S=0100 P=010D INX H*010E
-T↲
C0Z0M0E0I1 A=02 B=0802 D=0000 H=011A S=0100 P=010E DCR B*010F
-T↲
C0Z0M0E0I1 A=02 B=0702 D=0000 H=011A S=0100 P=010F JNZ 0107*0107
-T↲
C0Z0M0E0I1 A=02 B=0702 D=0000 H=011A S=0100 P=0107 MOV A,M*0108
-T↲                                  second data item brought to A
C0Z0M0E0I1 A=00 B=0702 D=0000 H=011A S=0100 P=0108 SUB C*0109
-T↲                                  subtract destroys data value which was loaded!
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011A S=0100 P=0109 JC 010D*010D
-T↲
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011A S=0100 P=010D INX H*010E
-L100↲
 0100   MVI         B,08
 0102   MVI         C,00
 0104   LXI         H,0119
 0107   MOV         A,M
 0108   SUB         C          <-- This should have been a CMP so that register A
 0109   JC          010D           would not be changed
 010C   MOV         C,A
 010D   INX         H
 010E   DCR         B
 010F   JNZ         0107
 0112   MOV         A,C
-A108↲
0108 CMP C↲        hot patch at 108H changes SUB to CMP
0109 ↲
-G0↲               stop DDT for Save
A>SAVE 1 SCAN.COM↲     save memory image
```

```
A>DDT SCAN.COM↲    restart DDT

DDT VERS 2.2
NEXT PC
0200 0100

-XP↲
P=0100 ↲
-L116↲
 0116   RST         07
 0117   NOP
 0118   NOP
 0119   STAX        B
 011A   NOP
```

look at code to see if it was properly loaded (long typeout aborted with rubout)

```
⌫
-G,116↲          run from 100H to completion


*0116

-XC↲             look at Carry (accidental typo)
C1 ↲
-X↲              look at CPU state
C1Z1M0E1I1 A=06 B=0006 D=0000 H=0121 S=0100 P=0116 RST 07

-S121↲             look at "LARGE" - it appears to be correct
 0121   06  ↲
 0122   00  ↲
 0123   22  .↲
-G0↲               stop DDT
A>ED SCAN.ASM↲     Re-edit the source program, and make both changes
     : *NSUB↲
    7: *0LT↲        CTRL-Z
    7:                  SUB     C           ;LARGER VALUE IN C?
    7: *SSUB^ZCMP^Z0LT↲
    7:                  CMP     C           ;LARGER VALUE IN C?
    7: *↲
    8:                  JNC     NFOUND      ;JUMP IF LARGER VALUE NOT FOUND
    8: *SNC^ZC^Z0LT↲
    8:                  JC      NFOUND      ;JUMP IF LARGER VALUE NOT FOUND
    8: *E↲
```

```
A>ASM SCAN.AAZ↵                    Re-assemble. Selection source from disk A
                                                hex to disk A
CP/M ASSEMBLER - VER 2.0                        print to Z (selects no file)
0122
002H USE FACTOR
END OF ASSEMBLY

A>DDT SCAN.HEX↵                    Re-run debugger to check changes


DDT VERS 2.2
NEXT PC
0121 0000

-L116↵              check to ensure end is still at 116H

 0116   JMP         0000
 0119   STAX        B
 011A   NOP
 011B   INR         B

⌫  (rubout)
-G100,116↵          Go from beginning with breakpoint at end

*0116               breakpoint reached

-D121↵              look at "LARGE"
                    correct value computed
0121 06 00 22 21 00 02 7E EB 77 13 23 EB 0B 78 B1 .. '!... W .#..X.
0130 C2 27 01 C3 03 29 00 00 00 00 00 00 00 00 00 .'...).........
0140 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...............

⌫           (rubout) abort long typeout
-G0↵        stop DDT, debug session complete
```

5. CP/M 2 System Interface

5.1. Introduction

This chapter describes CP/M (release 2) system organization including the structure of memory and system entry points. This section provides the information you need to write programs that operate under CP/M and that use the peripheral and disk I/O facilities of the system.

CP/M is logically divided into four parts, called the Basic Input/Output System (BIOS), the Basic Disk Operating System (BDOS), the Console Command Processor (CCP), and the Transient Program Area (TPA). The BIOS is a hardware-dependent module that defines the exact low level interface with a particular computer system that is necessary for peripheral device I/O. Although a standard BIOS is supplied by Digital Research, explicit instructions are provided for field reconfiguration of the BIOS to match nearly any hardware environment, see Section 6.

The BIOS and BDOS are logically combined into a single module with a common entry point and referred to as the FDOS. The CCP is a distinct program that uses the FDOS to provide a human-oriented interface with the information that is cataloged on the back-up storage device. The TPA is an area of memory, not used by the FDOS and CCP, where various nonresident operating system commands and user programs are executed. The lower portion of memory is reserved for system information and is detailed in later sections. Memory organization of the CP/M system is shown in Table 5-24

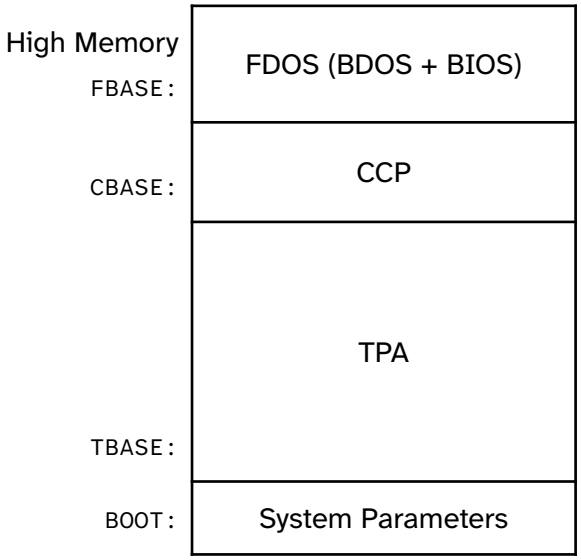


Table 5-24: CP/M Memory Organization

The exact memory addresses corresponding to BOOT, TBASE, CBASE, and FBASE vary from version to version and are described fully in Section 6.

All standard CP/M versions assume BOOT=0000H, which is the base of random access memory. The machine code found at location BOOT performs a system warm start, which loads and

initializes the programs and variables necessary to return control to the CCP. Thus, transient programs need only jump to location BOOT to return control to CP/M at the command level. further, the standard versions assume TBASE=BOOT+0100H, which is normally location 0100H. The principal entry point to the FDOS is at location BOOT+0005H (normally 0005H) where a jump to FBASE is found. The address field at BOOT+0006H (normally 0006H) contains the value of FBASE and can be used to determine the size of available memory, assuming that the CCP is being overlaid by a transient program.

Transient programs are loaded into the TPA and executed as follows. The operator communicates with the CCP by typing command lines following each prompt. Each command line takes one of the following forms:

command
command file1
command file file2

where *command* is either a built-in function, such as DIR or TYPE, or the name of a transient command or program. If the *command* is a built-in function of CP/M, it is executed immediately. Otherwise, the CCP searches the currently addressed disk for a file by the name

command.COM

If the file is found, it is assumed to be a memory image of a program that executes in the TPA and thus implicitly originates at TBASE in memory. The CCP loads the COM file from the disk into memory starting at TBASE and can extend up to CBASE.

If the command is followed by one or two file specifications, the CCP prepares one or two File Control Block (FCB) names in the system parameter area. These optional FCBs are in the form necessary to access files through the FDOS and are described in Section 5.2.

The transient program receives control from the CCP and begins execution, using the I/O facilities of the FDOS. The transient program is called from the CCP. Thus, it can simply return to the CCP upon completion of its processing, or can Jump to BOOT to pass control back to CP/M. In the first case, the transient program must not use memory above CBASE, while in the latter case, memory up through FBASE-1 can be used.

The transient program can use the CP/M I/O facilities to communicate with the operator's console and peripheral devices, including the disk subsystem. The I/O system is accessed by passing a function number and an information address to CP/M through the FDOS entry point at BOOT+0005H. In the case of a disk read, for example, the transient program sends the number corresponding to a disk read, along with the address of an FCB to the CP/M FDOS. The FDOS, in turn, performs the operation and returns with either a disk read completion indication or an error number indicating that the disk read was unsuccessful.

5.2. Operating System Call Conventions

This section provides detailed information for performing direct operating system calls from user programs. Many of the functions listed below, however, are accessed more simply through the I/O macro library provided with the MAC macro assembler and listed in the Digital Research manual entitled *Programmer's Utilities Guide for the CP/M Family of Operating Systems*.

CP/M facilities that are available for access by transient programs fall into two general categories: simple device I/O and disk file I/O. The simple device operations are

- read a console character
- write a console character
- read a sequential character
- write a sequential character
- get or set I/O status
- print console buffer
- interrogate console ready

The following FDOS operations perform disk I/O:

- disk system reset
- drive selection
- file creation
- file close
- directory search
- file delete
- file rename
- random or sequential read
- random or sequential write
- interrogate available disks
- interrogate selected disk
- set DMA address
- set/reset file indicators.

As mentioned above, access to the FDOS functions is accomplished by passing a function number and information address through the primary point at location BOOT+0005H. In general, the function number is passed in register C with the information address in the double byte pair DE. Single byte values are returned in register A, with double byte values returned in HL, a zero value is returned when the function number is out of range. For reasons of compatibility, register A = L and register B = H upon return in all cases. Note that the register passing conventions of CP/M agree with those of the Intel PL/M systems programming language. CP/M functions and their numbers are listed below.

0	System Reset	21	Write Sequential
1	Console Input	22	Make File
2	Console Output	23	Rename File
3	Reader Input	24	Return Login Vector
4	Punch Output	25	Return Current Disk
5	List Output	26	Set DMA Address
6	Direct Console I/O	27	Get Addr(ALLOC)
7	Get IOBYTE	28	Write Protect Disk
8	Set IOBYTE	29	Get R/O Vector
9	Print String	30	Set File Attributes
10	Read Console Buffer	31	Get Addr(Disk Parameter Block)
11	Get Console Status	32	Set/Get User Code
12	Version Number	33	Read Random
13	Reset Disk System	34	Write Random
14	Select Disk	35	Compute File Size
15	Open File	36	Set Random Record
16	Close File	37	Reset Drives
17	Search for First	38	Access Drive
18	Search for Next	39	Free Drive
19	Delete File	40	Write Random with Zero Fill
20	Read Sequential		

CP/M 2.2 BDOS functions

Note Function 28 and Function 32 should be avoided in application programs to maintain upward compatibility with CP/M.

Upon entry to a transient program, the CCP leaves the stack pointer set to an eight-level stack area with the CCP return address pushed onto the stack, leaving seven levels before overflow occurs. Although this stack is usually not used by a transient program (most transients return to the CCP through a jump to location 0000H) it is large enough to make CP/M system calls because the FDOS switches to a local stack at system entry. For example, the assembly-

language program segment below reads characters continuously until an asterisk is encountered, at which time control returns to the CCP, assuming a standard CP/M system with BOOT=0000H.

```
BDOS   EQU 0005H    ;STANDARD CP/M ENTRY
CONIN  EQU 1        ;CONSOLE INPUT FUNCTION
;
       ORG 0100H    ;BASE OF TPA
NEXTC: MVI C,CONIN  ;READ NEXT CHARACTER
       CALL BDOS    ;RETURN CHARACTER IN <A>
       CPI  '*'     ;END OF PROCESSING?
       JNZ  NEXTC   ;LOOP IF NOT
       RET          ;RETURN TO CCP
       END
```

CP/M implements a named file structure on each disk, providing a logical organization that allows any particular file to contain any number of records from completely empty to the full capacity of the drive. Each drive is logically distinct with a disk directory and file data area. The disk filenames are in three parts: the drive select code, the filename (consisting of one to eight non-blank characters), and the filetype (consisting of zero to three non-blank characters). The filetype names the generic category of a particular file, while the filename distinguishes individual files in each category. The filetypes listed in Table 5-25 name a few generic categories that have been established, although they are somewhat arbitrary.

Filetype	Meaning
ASM	Assembler Source
PRN	Printer Listing
HEX	Hex Machine Code
BAS	Basic Source File
INT	Intermediate Code
COM	Command File
PLI	PL/I Source File
REL	Relocatable Module
TEX	TEX Formatter Source

Filetype	Meaning
BAK	ED Source Backup
SYM	SID Symbol File
$$$	Temporary File

Table 5-25: CP/M Filetypes

Source files are treated as a sequence of ASCII characters, where each line of the source file is followed by a carriage return, and line-feed sequence (0DH followed by 0AH). Thus, one 128-byte CP/M record can contain several lines of source text. The end of an ASCII file is denoted by a CTRL-Z character (1AH) or a real end-of-file returned by the CP/M read operation. CTRL-Z characters embedded within machine code files (for example, COM files) are ignored and the end-of-file condition returned by CP/M is used to terminate read operations.

Files in CP/M can be thought of as a sequence of up to 65536 records of 128 bytes each, numbered from 0 through 65535, thus allowing a maximum of 8 megabytes per file. Note, however, that although the records may be considered logically contiguous, they may not be physically contiguous in the disk data area. Internally, all files are divided into 16K byte segments called logical extents, so that counters are easily maintained as 8-bit values. The division into extents is discussed in the paragraphs that follow: however, they are not particularly significant for the programmer, because each extent is automatically accessed in both sequential and random access modes.

In the file operations starting with Function 15, DE usually addresses a FCB. Transient programs often use the default FCB area reserved by CP/M at location BOOT+005CH (normally 005CH) for simple file operations. The basic unit of file information is a 128-byte record used for all file operations. Thus, a default location for disk I/O is provided by CP/M at location BOOT+0080H (normally 0080H) which is the initial default DMA address. See Function 26.

All directory operations take place in a reserved area that does not affect write buffers as was the case in release 1, with the exception of **Search for First** and **Search for Next**, where compatibility is required.

The FCB data area consists of a sequence of 33 bytes for sequential access and a series of 36 bytes in the case when the file is accessed randomly. The default FCB, normally located at 005CH, can be used for random access files, because the three bytes starting at BOOT+007DH are available for this purpose.

Field	DR	F1 … F8	T1	T2	T3	EX	S1	S2	RC	D0 … D15	CR	R0	R1	R2
Decimal	00	01 … 08	09	10	11	12	13	14	15	16 … 31	32	33	34	35
Hex	00	01 … 08	09	0A	0B	0C	0D	0E	0F	10 … 1F	20	21	22	23

Table 5-26: File Control Block Format

Field	Definition
DR	drive code (0-16). 0=default, 1=Drive A:, 2=B:, … 16=P:
F1 … **F8**	contain the filename in ASCII upper-case, with high bits = `f1'` … `f8'` normally zero
T1	contains the first character of the filetype in ASCII upper-case, with high bit = `t1'`. Set for Read/Only file.
T2	contains the second character of the filetype in ASCII upper-case with high bit = `t2'`. Set for SYS file, no DIR list.
T3	contains the third character of the filetype in ASCII upper-case with high bit = `t3'` normally zero.
EX	contains the current extent number, normally set to 00 by the user, but in range 0-31 during file I/O
S1	reserved for internal system use
S2	reserved for internal system use. Set to zero on calls to **Open**, **Make** and **Search for First**
RC	record count for extent **EX**; takes on values from 0-127
D0 … **D15**	filled in by CP/M; reserved for system use
CR	current record to read or write in a sequential file operation; normally set to zero by user
R0	Optional Random Record Number low byte
R1	Optional Random Record Number high byte
R2	Optional Random Record Number overflow

Table 5-27: File Control Block Fields

Each file being accessed through CP/M must have a corresponding FCB, which provides the name and allocation information for all subsequent file operations. When accessing files, it is the programmer's responsibility to fill the lower 16 bytes of the FCB and initialize the **CR** field. Normally, bytes 1 through 11 are set to the ASCII character values for the filename and filetype, while all other fields are zero.

FCBs are stored in a directory area of the disk, and are brought into central memory before the programmer proceeds with file operations (see the **Open** and **Make** functions). The memory copy of the FCB is updated as file operations take place and later recorded permanently on disk at the termination of the file operation, (see the **Close** command).

The CCP constructs the first 16 bytes of two optional FCBs for a transient by scanning the remainder of the line following the transient name, denoted by *file1* and *file2* in the prototype command line described above, with unspecified fields set to ASCII blanks. The first FCB is constructed at location BOOT+005CH and can be used as is for subsequent file operations. The second FCB occupies the **D0** ... **Dn** portion of the first FCB and must be moved to another area of memory before use. If, for example, the following command line is typed:

```
PROGNAME B:X.ZOT Y.ZAP
```

the file PROGNAME.COM is loaded into the TPA, and the default FCB at BOOT+005CH is initialized to drive code 2, filename X, and filetype ZOT. The second drive code takes the default value 0, which is placed at BOOT+006CH, with the filename Y placed into location BOOT+006DH and filetype ZAP located 8 bytes later at BOOT+0075H. All remaining fields through **CR** are set to zero. Note again that it is the programmer's responsibility to move this second filename and filetype to another area, usually a separate file control block, before opening the file that begins at BOOT+005CH, because the open operation overwrites the second name and type.

If no filenames are specified in the original command, the fields beginning at BOOT+005DH and BOOT+006DH contain blanks. In all cases, the CCP translates lower-case alphabetic characters to upper-case to be consistent with the CP/M file naming conventions

As an added convenience, the default buffer area at location BOOT+0080H is initialized to the command line tail typed by the operator following the program name. The first position contains the number of characters, with the characters themselves following the character count. Given the above command line, the area beginning at BOOT+0080H is initialized as follows:

Offset	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	90 ... FF
Content	0EH	*sp*	'B'	':'	'X'	'.'	'Z'	'O'	'T'	*sp*	'Y'	'.'	'Z'	'A'	'P'	00H	???

Table 5-28: Command-line Layout

where the characters are translated to upper-case ASCII with uninitialized memory following the last valid character. Again, it is the responsibility of the programmer to extract the information

from this buffer before any file operations are performed, unless the default DMA address is explicitly changed.

Individual functions are described in detail in the pages that follow.

5.2.1. System Reset

Function 0: System Reset

Entry Parameters:	
Register C:	0 = 00H
Returned Value:	**Does not return**

The **System Reset** function returns control to the CP/M operating system at the CCP level. The CCP re-initializes the disk subsystem by selecting and logging-in disk drive A. This function has exactly the same effect as a jump to location 0000H (BOOT).

5.2.2. Console Input

Function 1: Console Input

Entry Parameters:	
Register C:	1 = 01H
Returned Value:	
Register A:	ASCII Character

The **Console Input** function reads the next console character to register A. Graphic characters, along with carriage return, line-feed, and back space (CTRL-H) are echoed to the console. Tab characters, CTRL-I, move the cursor to the next tab stop. A check is made for start/stop scroll, CTRL-S, and start/stop printer echo, CTRL-P. The FDOS does not return to the calling program until a character has been typed, thus suspending execution if a character is not ready.

5.2.3. Console Output

Function 2: Console Output

Entry Parameters:	
Register C:	2 = 02H
Register E:	ASCII character to output
Returned Value:	**none**

The **Console Output** function sends the ASCII character from register E to the console device. As in Function 1, tabs are expanded and checks are made for start/stop scroll and printer echo.

5.2.4. Reader Input

Function 3: Reader Input

Entry Parameters:	
Register C:	3 = 03H

Returned Value:	
Register A:	ASCII Character

The **Reader Input** function reads the next character from the logical reader into register A. Control does not return until the character has been read.

5.2.5. Punch Output

Function 4: Punch Output

Entry Parameters:	
Register C:	4 = 04H
Register E:	ASCII Character
Returned Value:	**none**

The **Punch Output** function sends the character from register E to the logical punch device.

5.2.6. List Output

Function 5: List Output

Entry Parameters:	
Register C:	5 = 05H
Register E:	ASCII Character
Returned Value:	**none**

The **List Output** function sends the ASCII character in register E to the logical listing device.

5.2.7. Direct Console I/O

Function 6: Direct Console I/O

Entry Parameters:	
Register C:	6 = 06H
Register E:	0FFH (input) or char (output)
Returned Value:	
Register A:	character or status

Direct Console I/O is supported under CP/M for those specialized applications where basic console input and output are required. Use of this function should, in general, be avoided since it bypasses all of the CP/M normal control character functions (for example, CTRL-S and CTRL-P). Programs that perform direct I/O through the BIOS under previous releases of CP/M, however, should be changed to use direct I/O under BDOS so that they can be fully supported under future releases of MP/M and CP/M.

Upon entry to Function 6, register E either contains hexadecimal 0FFH, denoting a console input request, or an ASCII character. If the input value is 0FFH, Function 6 returns A = 00 if no character is ready, otherwise A contains the next console input character.

If the input value in E is not 0FFH, Function 6 assumes that E contains a valid ASCII character that is sent to the console.

5.2.8. Get IOBYTE

Function 7: Get IOBYTE

Entry Parameters:	
Register C:	7 = 07H
Returned Value:	
Register A:	IOBYTE value

The **Get IOBYTE** function returns the current value of IOBYTE in register A.

5.2.9. Set IOBYTE

Function 8: Set IOBYTE

Entry Parameters:	
Register C:	8 = 08H
Register E:	IOBYTE value
Returned Value:	**none**

The **Set IOBYTE** function changes the IOBYTE value to that given in register E.

5.2.10. Print String

Function 9: Print String

Entry Parameters:	
Register C:	9 = 09H
Registers DE:	String Address
Returned Value:	**none**

The **Print String** function sends the character string stored in memory at the location given by DE to the console device, until a '$' (24H) is encountered in the string. Tabs are expanded as in Function 2, and checks are made for start/stop scroll and printer echo.

5.2.11. Read Console Buffer

Function 10: Read Console Buffer

Entry Parameters:	
Register C:	10 = 0AH
Registers DE:	Buffer Address
Returned Value:	Characters input are in the Buffer

The **Read Buffer** functions reads a line of edited console input into a buffer addressed by registers DE. Console input is terminated when either input buffer overflows or a carriage return or line-feed is typed. The Read Buffer takes the form:

DE:	+0	+1	+2	+3	+4	+5	+6	+7	+8	...	+n
	mx	**nc**	*c1*	*c2*	*c3*	*c4*	*c5*	*c6*	*c7*	...	??

where **m** is the maximum number of characters that the buffer will hold, 1 to 255, and **nc** is the number of characters read (set by FDOS upon return) followed by the characters read from the console. If $\mathbf{nc} < \mathbf{mx}$, then uninitialized positions follow the last character, denoted by ?? in the above figure. A number of control functions, summarized in Table 5-29, are recognized during line editing.

Table 5-29: Edit Control Characters

Character	**Meaning**
CTRL-C	reboots when at beginning of line
CTRL-E	causes physical end of line
CTRL-H	backspaces one character position
CTRL-J	(line-feed) terminates input line
CTRL-M	(carriage return) terminates input line
CTRL-R	retypes the current line after new line
CTRL-U	removes current line
CTRL-X	Same as CTRL-U.
rubout/del	Deletes and echoes the last character typed at the console.

The user should also note that certain functions that return the carriage to the leftmost position (for example, CTRL-X) do so only to the column position where the prompt ended. In earlier releases, the carriage returned to the extreme left margin. This convention makes operator data input and line correction more legible.

5.2.12. Get Console Status

Function 11: Get Console Status

Entry Parameters:	
Register C:	11 = 0BH
Returned Value:	
Register A:	0FFH if a character is ready, 0 if not

The **Console Status** function checks to see if a character has been typed at the console. If a character is ready, the value 0FFH is returned in register A. Otherwise a 00H value is returned.

5.2.13. Version Number

Function 12: Version Number

Entry Parameters:	
Register C:	12 = 0CH
Returned Value:	
Registers HL:	Version Number

Function 12 provides information that allows version independent programming. A two-byte value is returned, with H = 00H designating the CP/M release (H = 01 for MP/M) and L = 00 for all releases previous to 2.0. CP/M 2.0 returns a hexadecimal 20 in register L, with subsequent version 2 releases in the hexadecimal range 21, 22, through 2F. Using Function 12, for example, the user can write application programs that provide both sequential and random access functions.

5.2.14. Reset Disk System

Function 13: Reset Disk System

Entry Parameters:	
Register C:	13 = 0DH
Returned Value:	
Register A:	0FFH if $ file present, 0 if not

The **Reset Disk** function is used to programmatically restore the file system to a reset state where all disks are set to Read-Write. See functions 28 and 29, only disk drive A is selected, and the default DMA address is reset to BOOT+0080H. This function can be used, for example, by an application program that requires a disk change without a system reboot.

This function is normally called by the CCP after the system reset and before any other activity. If the CCP is not executed, this function must be called by what replaced it.

In 1.x and 2.x version the return value depends upon the presence of a file with a name starting with a dollar-sign ($) on drive A. If such a file is present, a 0FFH value is returned, otherwise a zero is returned.

5.2.15. Select Disk

Function 14: Select Disk

Entry Parameters:	
Register C:	14 = 0EH

Register E:	Drive number: 0 for A, 1 for B, ...
Returned Value:	
Register A:	0 if successful, 0FFH on error

The **Select Disk** function designates the disk drive named in register E as the default disk for subsequent file operations, with E = 0 for drive A, 1 for drive B, and so on through 15, corresponding to drive P in a full 16 drive system. The drive is placed in an on-line status, which activates its directory until the next cold start, warm start, or disk system reset operation. If the disk medium is changed while it is on-line, the drive automatically goes to a Read-Only status in a standard CP/M environment, see Function 28. FCBs that specify drive code zero (**DR** = 00H) automatically reference the currently selected default drive. Drive code values between 1 and 16 ignore the selected default drive and directly reference drives A through P.

5.2.16. Open File

Function 15: Open File

Entry Parameters:	
Register C:	15 = 0FH
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Open** File operation is used to activate a file that currently exists in the disk directory for the currently active user number. The FDOS scans the referenced disk directory for a match in positions 1 through 14 of the FCB referenced by DE (byte **S1** is automatically zeroed) where an ASCII question mark '?' (3FH) matches any directory character in any of these positions. Normally, no question marks are included, and bytes **EX** and **S2** of the FCB are zero.

If a directory element is matched, the relevant directory information is copied into bytes **D0** through **Dn** of FCB, thus allowing access to the files through subsequent read and write operations. The user should note that an existing file must not be accessed until a successful open operation is completed. Upon return, the open function returns a directory code with the value 0 through 3 if the open was successful or 0FFH (255 decimal) if the file cannot be found. If question marks occur in the FCB, the first matching FCB is activated. Note that the current record, (**CR**) must be zeroed by the program if the file is to be accessed sequentially from the first record.

5.2.17. Close File

Function 16: Close File

Entry Parameters:	

Register C:	16 = 10H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Close** File function performs the inverse of the **Open** File function. Given that the FCB addressed by DE has been previously activated through an **Open** or **Make** function, the **Close** function permanently records the new FCB in the reference disk directory see functions 15 and 22. The FCB matching process for the close is identical to the open function. The directory code returned for a successful close operation is 0, 1, 2, or 3, while a 0FFH (255 decimal) is returned if the filename cannot be found in the directory. A file need not be closed if only read operations have taken place. If write operations have occurred, the close operation is necessary to record the new directory information permanently.

5.2.18. Search for First

Function 17: Search for First

Entry Parameters:	
Register C:	17 = 11H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

Search for First scans the directory for a match with the file given by the FCB addressed by DE. The value 255 (hexadecimal 0FFH) is returned if the file is not found; otherwise, 0, 1, 2, or 3 is returned indicating the file is present. When the file is found, the current DMA address is filled with the record containing the directory entry, and the relative starting position is $A * 32$ (that is, rotate the A register left 5 bits, or ADD A five times). Although not normally required for application programs, the directory information can be extracted from the buffer at this position.

An ASCII question mark '?' (63 decimal, 3F hexadecimal) in any position from **F1** through **EX** matches the corresponding field of any directory entry on the default or auto-selected disk drive. If the **DR** field contains an ASCII question mark, the auto disk select function is disabled and the default disk is searched, with the search function returning any matched entry, allocated or free, belonging to any user number. This latter function is not normally used by application programs, but it allows complete flexibility to scan all current directory values. If the **DR** field is not a question mark, the **S2** byte is automatically zeroed.

5.2.19. Search for Next

Function 18: Search for Next

Entry Parameters:	
Register C:	18 = 12H
Returned Value:	
Register A:	Directory Code

The **Search for Next** function is similar to the **Search for First** function, except that the directory scan continues from the last matched entry. Similar to Function 17, Function 18 returns the decimal value 255 in A when no more directory items match.

5.2.20. Delete File

Function 19: Delete File

Entry Parameters:	
Register C:	19 = 13H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Delete** File function removes files that match the FCB addressed by DE. The filename and type may contain ambiguous references (that is, question marks in various positions), but the drive select code (**DR**) cannot be ambiguous, as in the **Search for First** and **Search for Next** functions.

Function 19 returns a decimal 255 if the referenced file or files cannot be found; otherwise, a value in the range 0 to 3 returned.

5.2.21. Read Sequential

Function 20: Read Sequential

Entry Parameters:	
Register C:	20 = 14H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

Given that the FCB addressed by DE has been activated through an **Open** or **Make** function, the **Read Sequential** function reads the next 128-byte record from the file into memory at the current DMA address. The record is read from position **CR** of the extent, and the **CR** field is automatically incremented to the next record position. If the **CR** field overflows, the next logical extent is automatically opened and the **CR** field is reset to zero in preparation for the next read operation. The value 00H is returned in the A register if the read operation was successful, while

a nonzero value is returned if no data exist at the next record position (for example, end-of-file occurs).

5.2.22. Write Sequential

Function 21: Write Sequential

Entry Parameters:	
Register C:	21 = 15H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

Given that the FCB addressed by DE has been activated through an **Open** or **Make** function, the **Write Sequential** function writes the 128-byte data record at the current DMA address to the file named by the FCB. The record is placed at position **CR** of the file, and the **CR** field is automatically incremented to the next record position. If the **CR** field overflows, the next logical extent is automatically opened and the **CR** field is reset to zero in preparation for the next write operation. Write operations can take place into an existing file, in which case, newly written records overlay those that already exist in the file. Register A = 00H upon return from a successful write operation, while a nonzero value indicates an unsuccessful write caused by a full disk.

5.2.23. Make File

Function 22: Make File

Entry Parameters:	
Register C:	22 = 16H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Make** File operation is similar to the **Open** File operation except that the FCB must name a file that does not exist in the currently referenced disk directory (that is, the one named explicitly by a nonzero **DR** code or the default disk if **DR** is zero). The FDOS creates the file and initializes both the directory and main memory value to an empty file. The programmer must ensure that no duplicate filenames occur, and a preceding delete operation is sufficient if there is any possibility of duplication. Upon return, register A = 0, 1, 2, or 3 if the operation was successful and 0FFH (255 decimal) if no more directory space is available. The Make function has the side effect of activating the FCB and thus a subsequent open is not necessary.

5.2.24. Rename File

Function 23: Rename File

Entry Parameters:	
Register C:	23 = 17H
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The **Rename** function uses the FCB addressed by DE to change all occurrences of the file named in the first 16 bytes to the file named in the second 16 bytes. The drive code **DR** at position 0 is used to select the drive, while the drive code for the new filename at position 16 of the FCB is assumed to be zero. Upon return, register A is set to a value between 0 and 3 if the rename was successful and 0FFH (255 decimal) if the first filename could not be found in the directory scan.

5.2.25. Return Login Vector

Function 24: Return Login Vector

Entry Parameters:	
Register C:	24 = 18H
Returned Value:	
Registers HL:	Log-in Vector

The log-in vector value returned by CP/M is a 16-bit value in HL, where the least significant bit of L corresponds to the first drive A and the high-order bit of H corresponds to the sixteenth drive, labeled P. A 0 bit indicates that the drive is not on-line, while a 1 bit marks a drive that is actively on-line as a result of an explicit disk drive selection or an implicit drive select caused by a file operation that specified a nonzero **DR** field. The user should note that compatibility is maintained with earlier releases, because registers A and L contain the same values upon return.

5.2.26. Return Current Disk

Function 25: Return Current Disk

Entry Parameters:	
Register C:	25 = 19H
Returned Value:	
Register A:	Current Disk. 0 for A, ... 15 for P

Function 25 returns the currently selected default disk number in register A. The disk numbers range from 0 through 15 corresponding to drives A through P.

5.2.27. Set DMA Address

Function 26: Set DMA Address

Entry Parameters:	

Register C:	26 = 1AH
Registers DE:	DMA Address
Returned Value:	**none**

DMA is an acronym for Direct Memory Address, which is often used in connection with disk controllers that directly access the memory of the mainframe computer to transfer data to and from the disk subsystem. Although many computer systems use non-DMA access (that is, the data is transferred through programmed I/O operations), the DMA address has, in CP/M, come to mean the address at which the 128-byte data record resides before a disk write and after a disk read. Upon cold start, warm start, or disk system reset, the DMA address is automatically set to BOOT+0080H. The **Set DMA** function can be used to change this default value to address another area of memory where the data records reside. Thus, the DMA address becomes the value specified by DE until it is changed by a subsequent **Set DMA** function, cold start, warm start, or disk system reset.

5.2.28. Get Addr(ALLOC)

Function 27: Get Addr(ALLOC)

Entry Parameters:	
Register C:	27 = 1BH
Returned Value:	
Registers HL:	ALLOC Address

An allocation vector (ALLOC) is maintained in main memory for each on-line disk drive. Various system programs use the information provided by the allocation vector to determine the amount of remaining storage (see the STAT program). Function 27 returns the base address of the allocation vector for the currently selected disk drive. However, the allocation information might be invalid if the selected disk has been marked Read-Only. This function is not normally used by application programs.

5.2.29. Write Protect Disk

Function 28: Write Protect Disk

Entry Parameters:	
Register C:	28 = 1CH
Returned Value:	**none**

The Write Protect Disk function provides temporary write protection for the currently selected disk. Any attempt to write to the disk before the next cold or warm start operation produces the message:

BDOS Err On *d*:R/O

5.2.30. Get R/O Vector

Function 29: Get R/O Vector

Entry Parameters:	
Register C:	29 = 1DH
Returned Value:	
Registers HL:	R/O Vector Value

Function 29 returns a bit vector in register pair HL, which indicates drives that have the temporary Read-Only bit set. As in Function 24, the least significant bit corresponds to drive A, while the most significant bit corresponds to drive P. The R/O bit is set either by an explicit call to Function 28 or by the automatic software mechanisms within CP/M that detect changed disks.

5.2.31. Set File Attributes

Function 30: Set File Attributes

Entry Parameters:	
Register C:	30 = 1EH
Registers DE:	FCB Address
Returned Value:	
Register A:	Directory Code

The Set File Attributes function allows programmatic manipulation of permanent indicators attached to files. In particular, the R/O and System attributes (t1' and t2') can be set or reset. The DE pair addresses an unambiguous filename with the appropriate attributes set or reset. Function 30 searches for a match and changes the matched directory entry to contain the selected indicators. Indicators f1' through f4' are not currently used, but may be useful for applications programs, since they are not involved in the matching process during file open and close operations. Indicators f5' through f8' and t3' are reserved for future system expansion.

5.2.32. Get Addr(Disk Parameter Block)

Function 31: Get Addr(Disk Parameter Block)

Entry Parameters:	
Register C:	31 = 1FH
Returned Value:	
Registers HL:	**DPB** Address

The address of the BIOS resident disk parameter block (**DPB**) is returned in HL as a result of this function call. This address can be used for either of two purposes. First, the disk parameter values can be extracted for display and space computation purposes, or transient programs can dynamically change the values of current disk parameters when the disk environment changes, if required. Normally, application programs will not require this facility.

5.2.33. Set/Get User Code

Function 32: Set/Get User Code

Entry Parameters:	
Register C:	32 = 20H
Register E:	0FFH (get) or *User Code* (set)
Returned Value:	
Register A:	Current code (get) or none (set)

An application program can change or interrogate the currently active user number by calling Function 32. If register E = 0FFH, the value of the current user number is returned in register A, where the value is in the range of 0 to 15. If register E is not 0FFH, the current user number is changed to the value of E, modulo 16.

5.2.34. Read Random

Function 33: Read Random

Entry Parameters:	
Register C:	33 = 21H
Registers DE:	FCB Address
Returned Value:	
Register A:	Return code (see details)

The **Read Random** function is similar to the sequential file read operation of previous releases, except that the read operation takes place at a particular record number, selected by the 24-bit value constructed from the 3-byte field following the FCB (byte positions **R0** at 33, **R1** at 34, and **R3** at 35). The user should note that the sequence of 24 bits is stored with least significant byte first (**R0**), middle byte next (**R1**), and high byte last (**R2**). CP/M does not reference byte **R2**, except in computing the size of a file (Function 35). Byte **R2** must be zero, however, since a nonzero value indicates overflow past the end of file.

Thus, the **R0**, **R1** byte pair is treated as a double-byte, or word value, that contains the record to read. This value ranges from 0 to 65535, providing access to any particular record of the 8-megabyte file. To process a file using random access, the base extent (extent 0) must first be opened. Although the base extent might or might not contain any allocated data, this ensures that the file is properly recorded in the directory and is visible in DIR requests. The selected

record number is then stored in the random record field (**R0**, **R1**), and the BDOS is called to read the record.

Upon return from the call, register A either contains an error code, as listed below, or the value 00, indicating the operation was successful. In the latter case, the current DMA address contains the randomly accessed record. Note that contrary to the sequential read operation, the record number is not advanced. Thus, subsequent random read operations continue to read the same record.

Upon each random read operation, the logical extent and current record values are automatically set. Thus, the file can be sequentially read or written, starting from the current randomly accessed position. However, note that, in this case, the last randomly read record will be reread as one switches from random mode to sequential read and the last record will be rewritten as one switches to a sequential write operation. The user can simply advance the random record position following each random read or write to obtain the effect of sequential I/O operation.

Error codes returned in register A following a random read are listed below.

01 reading unwritten data

02 (not returned in random mode)

03 cannot close current extent

04 seek to unwritten extent

05 (not returned in read mode)

06 seek past physical end of disk

09 invalid FCB

10 media changed

0FFH hardware error (on 3.x)

Error codes 01 and 04 occur when a random read operation accesses a data block that has not been previously written or an extent that has not been created, which are equivalent conditions. Error code 03 does not normally occur under proper system operation. If it does, it can be cleared by simply rereading or reopening extent zero as long as the disk is not physically write protected. Error code 06 occurs whenever byte **R2** is nonzero under the current 2.0 release. Normally, nonzero return codes can be treated as missing data, with zero return codes indicating operation complete.

5.2.35. Write Random

Function 34: Write Random

Entry Parameters:	

Register C:	34 = 22H
Registers DE:	FCB Address
Returned Value:	
Register A:	Return code (see details)

The **Write Random** operation is initiated similarly to the Read Random call, except that data is written to the disk from the current DMA address. Further, if the disk extent or data block that is the target of the write has not yet been allocated, the allocation is performed before the write operation continues. As in the Read Random operation, the random record number is not changed as a result of the write. The logical extent number and current record positions of the FCB are set to correspond to the random record that is being written. Again, sequential read or write operations can begin following a random write, with the notation that the currently addressed record is either read or rewritten again as the sequential operation begins. You can also simply advance the random record position following each write to get the effect of a sequential write operation. Note that reading or writing the last record of an extent in random mode does not cause an automatic extent switch as it does in sequential mode.

The error codes returned by a random write are identical to the random read operation with the addition of error code 05, which indicates that a new extent cannot be created as a result of directory overflow.

5.2.36. Compute File Size

Function 35: Compute File Size

Entry Parameters:	
Register C:	35 = 23H
Registers DE:	FCB Address
Returned Value:	Random Record Field Set in FCB

When computing the size of a file, the DE register pair addresses an FCB in random mode format (bytes **R0**, **R1**, and **R2** are present). The FCB contains an unambiguous filename that is used in the directory scan. Upon return, the random record bytes contain the virtual file size, which is, in effect, the record address of the record following the end of the file. Following a call to Function 35, if the high record byte **R2** is 01, the file contains the maximum record count 65536. Otherwise, bytes **R0** and **R1** constitute a 16-bit value as before (**R0** is the least significant byte), which is the file size.

Data can be appended to the end of an existing file by simply calling Function 35 to set the random record position to the end of file and then performing a sequence of random writes starting at the preset record address.

The virtual size of a file corresponds to the physical size when the file is written sequentially. If the file was created in random mode and holes exist in the allocation, the file might contain fewer records than the size indicates. For example, if only the last record of an 8-megabyte file is written in random mode (that is, record number 65535), the virtual size is 65536 records, although only one block of data is actually allocated.

5.2.37. Set Random Record

Function 36: Set Random Record

Entry Parameters:	
Register C:	36 = 24H
Registers DE:	FCB Address
Returned Value:	Random Record Field Set in FCB

The Set Random Record function causes the BDOS automatically to produce the random record position from a file that has been read or written sequentially to a particular point. The function can be useful in two ways.

First, it is often necessary initially to read and scan a sequential file to extract the positions of various key fields. As each key is encountered, Function 36 is called to compute the random record position for the data corresponding to this key. If the data unit size is 128 bytes, the resulting record position is placed into a table with the key for later retrieval. After scanning the entire file and tabulating the keys and their record numbers, the user can move instantly to a particular keyed record by performing a random read, using the corresponding random record number that was saved earlier. The scheme is easily generalized for variable record lengths, because the program need only store the buffer-relative byte position along with the key and record number to find the exact starting position of the keyed data at a later time.

A second use of Function 36 occurs when switching from a sequential read or write over to random read or write. A file is sequentially accessed to a particular point in the file, Function 36 is called, which sets the record number, and subsequent random read and write operations continue from the selected point in the file.

5.2.38. Reset Drives

Function 37: Reset Drives

Entry Parameters:	
Register C:	37 = 25H
Registers DE:	Drive Vector
Returned Value:	
Register A:	00H

The Reset Drive function allows resetting of specified drives. The passed parameter is a 16-bit vector of drives to be reset; the least significant bit is drive A.

To maintain compatibility with MP/M, CP/M returns a zero value.

5.2.39. Access Drive

Function 38: Access Drive

Entry Parameters:	
Register C:	38 = 26H
Returned Value:	
Register A:	00H

This is an MP/M function that is not supported under CP/M. If called, the file system returns a zero In register A indicating that the access request is successful.

5.2.40. Free Drive

Function 39: Free Drive

Entry Parameters:	
Register C:	39 = 27H
Returned Value:	
Register A:	00H

This is an MP/M function that is not supported under CP/M. If called, the file system returns a zero In register A indicating that the free request is successful.

5.2.41. Write Random with Zero Fill

Function 40: Write Random with Zero Fill

Entry Parameters:	
Register C:	40 = 28H
Registers DE:	FCB Address
Returned Value:	
Register A:	Return code

The Write With Zero Fill operation is similar to Function 34, with the exception that a previously unallocated block is filled with zeros before the data is written.

5.3. A Sample File-to-File Copy Program

The following program provides a relatively simple example of file operations. The program source file is created as COPY.ASM using the CP/M ED program and then assembled using ASM or MAC, resulting in a HEX file. The LOAD program is used to produce a COPY.COM file that executes directly under the CCP. The program begins by setting the stack pointer to a local area and proceeds to move the second name from the default area at 006CH to a 33-byte File Control

Block called DFCB. The DFCB is then prepared for file operations by clearing the current record field. At this point, the source and destination FCBs are ready for processing, because the SFCB at 005CH is properly set up by the CCP upon entry to the COPY program. That is, the first name is placed into the default FCB, with the proper fields zeroed, including the current record field at 007CH. The program continues by opening the source file, deleting any existing destination file, and creating the destination file. If all this is successful, the program loops at the label COPY until each record is read from the source file and placed into the destination file. Upon completion of the data transfer, the destination file is closed and the program returns to the CCP command level by jumping to BOOT.

```
                ;       sample file-to-file copy program
                ;
                ;       at the ccp level, the command
                ;
                ;            copy a:x.y b:u.v
                ;
                ;       copies the file named x.y from drive
                ;       a to a file named u.v. on drive b.
                ;
0000 =          boot    equ 0000h       ;system reboot
0005 =          bdos    equ 0005h       ;bdos entry point
005C =          fcbl    equ 005ch       ;first file name
005C =          sfcb    equ fcbl        ;source fcb
006C =          fcb2    equ 006ch       ;second file name
0080 =          dbuff   equ 0080h       ;default buffer
0100 =          tpa     equ 0100h       ;beginning of tpa
                ;
0009 =          printf  equ 9           ;print buffer func#
000F =          openf   equ 15          ;open file func#
0010 =          closef  equ 16          ;close file func#
0013 =          deletef equ 19          ;delete file func#
0014 =          readf   equ 20          ;sequential read
0015 =          writef  equ 21          ;sequential write
0016 =          makef   equ 22          ;make file func#
                ;
0100                    org tpa         ;beginning of tpa
0100 311A02             lxi sp,stack    ;local stack
                ;
                ;       move second file name to dfcb
0103 0E10               mvi c,16        ;half an fcb
0105 116C00             lxi d,fcb2      ;source of move
0108 21D901             lxi h,dfcb      ;destination fcb
010B 1A         mfcb:   ldax d          ;source fcb
010C 13                 inx  d          ;ready next
```

```
010D 77                 mov  m,a       ;dest fcb
010E 23                 inx  h         ;ready next
010F 0D                 dcr  c         ;count 16...0
0110 C20B01             jnz  mfcb      ;loop 16 times
                ;
                ;       name has been removed, zero cr
0113 AF                 xra  a         ;a = 00h
0114 32F901             sta  dfcbcr    ;current rec = 0
                ;
                ;       source and destination fcbs ready
                ;
0117 115C00             lxi  d,sfcb    ;source file
011A CD6901             call open      ;error if 255
011D 118701             lxi  d,nofile  ;ready message
0120 3C                 inr  a         ;255 becomes 0
0121 CC6101             cz   finis     ;done if no file
                ;
                ;       source file open, prep destination
0124 11D901             lxi  d,dfcb    ;destination
0127 CD7301             call delete    ;remove if present
                ;
012A 11D901             lxi  d,dfcb    ;destination
012D CD8201             call make      ;create the file
0130 119601             lxi  d,nodir   ;ready message
0133 3C                 inr  a         ;255 becomes 0
0134 CC6101             cz   finis     ;done if no dir space
                ;
                ;       source file open, dest file open
                ;       copy until end of file on source
                ;
0137 115C00     copy:   lxi  d,sfcb    ;source
013A CD7801             call read      ;read next record
013D B7                 ora  a         ;end of file?
013E C25101             jnz  eofile    ;skip write if so
                ;
                ;       not end of file, write the record
0141 11D901             lxi  d,dfcb    ;destination
0144 CD7D01             call write     ;write record
0147 11A901             lxi  d,space   ;ready message
014A B7                 ora  a         ;00 if write ok
014B C46101             cnz  finis     ;end if so
014E C33701             jmp  copy      ;loop until eof
                ;
                eofile: ;end of file, close destination
0151 11D901             lxi  d,dfcb    ;destination
```

```
0154 CD6E01             call close     ;255 if error
0157 21BA01             lxi  h,wrprot  ;ready message
015A 3C                 inr  a         ;255 becomes 00
015B CC6101             cz   finis     ;should not happen
                ;
                ;       copy operation complete, end
015E 11CB01             lxi  d,normal  ;ready message
                ;
                finis   ;write message given by de, reboot
0161 0E09               mvi  c,printf
0163 CD0500             call bdos      ;write message
0166 C30000             jmp  boot      ;reboot system
                ;
                ;       system interface subroutines
                ;       (all return directly from bdos)
                ;
0169 0E0F       open:   mvi  c,openf
016B C30500             jmp  bdos
                ;
016E 0E10       close:  mvi  c,closef
0170 C30500             jmp  bdos
                ;
0173 0E13       delete  mvi  c,deletef
0175 C30500             jmp  bdos
                ;
0178 0E14       read:   mvi  c,readf
017A C30500             jmp  bdos
                ;
017D 0E15       write:  mvi  c,writef
017F C30500             jmp  bdos
                ;
0182 0E16       make:   mvi  c,makef
0184 C30500             jmp  bdos
                ;
                ;       console messages
0187 6E6F20736Fnofile:  db   'no source file$'
0196 6E6F206469nodir:   db   'no directory space$'
01A9 6F7574206Fspace:   db   'out of dat space$'
01BA 7772697465wrprot:  db   'write protected?$'
01CB 636F707920normal:  db   'copy complete$'
                ;
                ;       data areas
01D9            dfcb:   ds   33        ;destination fcb
01F9 =          dfcbcr  equ  dfcb+32   ;current record
                ;
```

```
01FA                    ds   32        ;16 level stack
                stack:
021A                    end
```

Note that there are several simplifications in this particular program. First, there are no checks for invalid filenames that could contain ambiguous references. This situation could be detected by scanning the 32-byte default area starting at location 005CH for ASCII question marks. A check should also be make to ensure that the filenames have been included (check locations 005DH and 006DH for non-blank ASCII characters). Finally, a check should be made to ensure that the source and destination filenames are different. An improvement in speed could be obtained by buffering more data on each read operation. One could, for example, determine the size of memory by fetching FBASE from location 0006H and using the entire remaining portion of memory for a data buffer. In this case, the programmer simply resets the DMA address to the next successive 128-byte area before each read. Upon writing to the destination file, the DMA address is reset to the beginning of the buffer and incremented by 128 bytes to the end as each record is transferred to the destination file.

5.4. A Sample File Dump Utility

The following file dump program is slightly more complex than the simple copy program given in the previous section. The dump program reads an input file, specified in the CCP command line, and displays the content of each record in hexadecimal format at the console. Note that the dump program saves the CCP's stack upon entry, resets the stack to a local area, and restores the CCP's stack before returning directly to the CCP. Thus, the dump program does not perform a warm start at the end of processing.

```
                ;       FILE DUMP PROGRAM, READS AN INPUT FILE AND PRINTS IN HEX
                ;
                ;       COPYRIGHT (C) 1975, 1976, 1977, 1978
                ;       DIGITAL RESEARCH
                ;       BOX 579, PACIFIC GROVE
                ;       CALIFORNIA, 93950
                ;
0100                    ORG     100H
0005 =          BDOS    EQU     0005H   ;DOS ENTRY POINT
0001 =          CONS    EQU     1       ;READ CONSOLE
0002 =          TYPEF   EQU     2       ;TYPE FUNCTION
0009 =          PRINTF  EQU     9       ;BUFFER PRINT ENTRY
000B =          BRKF    EQU     11      ;BREAK KEY FUNCTION (TRUE IF CHAR READY)
000F =          OPENF   EQU     15      ;FILE OPEN
0014 =          READF   EQU     20      ;READ FUNCTION
                ;
005C =          FCB     EQU     5CH     ;FILE CONTROL BLOCK ADDRESS
```

```
0080 =          BUFF    EQU     80H     ;INPUT DISK BUFFER ADDRESS
                ;
                ;       NON GRAPHIC CHARACTERS
000D =          CR      EQU     0DH     ;CARRIAGE RETURN
000A =          LF      EQU     0AH     ;LINE FEED
                ;
                ;       FILE CONTROL BLOCK DEFINITIONS
005C =          FCBDN   EQU     FCB+0   ;DISK NAME
005D =          FCBFN   EQU     FCB+1   ;FILE NAME
0065 =          FCBFT   EQU     FCB+9   ;DISK FILE TYPE (3 CHARACTERS)
0068 =          FCBRL   EQU     FCB+12  ;FILE'S CURRENT REEL NUMBER
006B =          FCBRC   EQU     FCB+15  ;FILE'S RECORD COUNT (0 TO 128)
007C =          FCBCR   EQU     FCB+32  ;CURRENT (NEXT) RECORD NUMBER (0 TO 127)
007D =          FCBLN   EQU     FCB+33  ;FCB LENGTH
                ;
                ;       SET UP STACK
0100 210000             LXI     H,0
0103 39                 DAD     SP
                ;       ENTRY STACK POINTER IN HL FROM THE CCP
0104 221502             SHLD    OLDSP
                ;       SET SP TO LOCAL STACK AREA (RESTORED AT FINIS)
0107 315702             LXI     SP,STKTOP
                ;       READ AND PRINT SUCCESSIVE BUFFERS
010A CDC101             CALL    SETUP   ;SET UP INPUT FILE
010D FEFF               CPI     255     ;255 IF FILE NOT PRESENT
010F C21B01             JNZ     OPENOK  ;SKIP IF OPEN IS OK
                ;
                ;       FILE NOT THERE, GIVE ERROR MESSAGE AND RETURN
0112 11F301             LXI     D,OPNMSG
0115 CD9C01             CALL    ERR
0118 C35101             JMP     FINIS   ;TO RETURN
                ;
                OPENOK: ;OPEN OPERATION OK, SET BUFFER INDEX TO END
011B 3E80               MVI     A,80H
011D 321302             STA     IBP     ;SET BUFFER POINTER TO 80H
                ;       HL CONTAINS NEXT ADDRESS TO PRINT
0120 210000             LXI     H,0     ;START WITH 0000
                ;
                GLOOP:
0123 E5                 PUSH    H       ;SAVE LINE POSITION
0124 CDA201             CALL    GNB
0127 E1                 POP     H       ;RECALL LINE POSITION
0128 DA5101             JC      FINIS   ;CARRY SET BY GNB IF END FILE
012B 47                 MOV     B,A
                ;       PRINT HEX VALUES
```

```
                ;       CHECK FOR LINE FOLD
012C 7D                 MOV     A,L
012D E60F               ANI     0FH     ;CHECK LOW 4 BITS
012F C24401             JNZ     NONUM
                ;       PRINT LINE NUMBER
0132 CD7201             CALL    CRLF
                ;
                ;       CHECK FOR BREAK KEY
0135 CD5901             CALL    BREAK
                ;       ACCUM LSB = 1 IF CHARACTER READY
0138 0F                 RRC             ;INTO CARRY
0139 DA5101             JC      FINIS   ;DON'T PRINT ANY MORE
                ;
013C 7C                 MOV     A,H
013D CD8F01             CALL    PHEX
0140 7D                 MOV     A,L
0141 CD8F01             CALL    PHEX
                NONUM:
0144 23                 INX     H       ;TO NEXT LINE NUMBER
0145 3E20               MVI     A,' '
0147 CD6501             CALL    PCHAR
014A 78                 MOV     A,B
014B CD8F01             CALL    PHEX
014E C32301             JMP     GLOOP
                ;
                FINIS:
                ;       END OF DUMP, RETURN TO CCP
                ;       (NOTE THAT A JMP TO 0000H REBOOTS)
0151 CD7201             CALL    CRLF
0154 2A1502             LHLD    OLDSP
0157 F9                 SPHL
                ;       STACK POINTER CONTAINS CCP'S STACK LOCATION
0158 C9                 RET             ;TO THE CCP
                ;
                ;
                ;       SUBROUTINES
                ;
                BREAK:  ;CHECK BREAK KEY (ACTUALLY ANY KEY WILL DO)
0159 E5D5C5             PUSH H! PUSH D! PUSH B; ENVIRONMENT SAVED
015C 0E0B               MVI     C,BRKF
015E CD0500             CALL    BDOS
0161 C1D1E1             POP B! POP D! POP H; ENVIRONMENT RESTORED
0164 C9                 RET
                ;
                PCHAR:  ;PRINT A CHARACTER
```

```
0165 E5D5C5            PUSH H! PUSH D! PUSH B; SAVED
0168 0E02              MVI     C,TYPEF
016A 5F                MOV     E,A
016B CD0500            CALL    BDOS
016E C1D1E1            POP B! POP D! POP H; RESTORED
0171 C9                RET
                ;
                CRLF:
0172 3E0D              MVI     A,CR
0174 CD6501            CALL    PCHAR
0177 3E0A              MVI     A,LF
0179 CD6501            CALL    PCHAR
017C C9                RET
                ;
                ;
                PNIB:   ;PRINT NIBBLE IN REG A
017D E60F              ANI     0FH     ;LOW 4 BITS
017F FE0A              CPI     10
0181 D28901            JNC     P10
                ;       LESS THAN OR EQUAL TO 9
0184 C630              ADI     '0'
0186 C38B01            JMP     PRN
                ;
                ;       GREATER OR EQUAL TO 10
0189 C637       P10:    ADI     'A' - 10
018B CD6501     PRN:    CALL    PCHAR
018E C9                RET
                ;
                PHEX:   ;PRINT HEX CHAR IN REG A
018F F5                PUSH    PSW
0190 0F                RRC
0191 0F                RRC
0192 0F                RRC
0193 0F                RRC
0194 CD7D01            CALL    PNIB    ;PRINT NIBBLE
0197 F1                POP     PSW
0198 CD7D01            CALL    PNIB
019B C9                RET
                ;
                ERR:    ;PRINT ERROR MESSAGE
                ;       D,E ADDRESSES MESSAGE ENDING WITH "$"
019C 0E09              MVI     C,PRINTF        ;PRINT BUFFER FUNCTION
019E CD0500            CALL    BDOS
01A1 C9                RET
                ;
```

```
                ;
                GNB:    ;GET NEXT BYTE
01A2 3A1302             LDA     IBP
01A5 FE80               CPI     80H
01A7 C2B301             JNZ     G0
                ;       READ ANOTHER BUFFER
                ;
                ;
01AA CDCE01             CALL    DISKR
01AD B7                 ORA     A       ;ZERO VALUE IF READ OK
01AE CAB301             JZ      G0      ;FOR ANOTHER BYTE
                ;       END OF DATA, RETURN WITH CARRY SET FOR EOF
01B1 37                 STC
01B2 C9                 RET
                ;
                G0:     ;READ THE BYTE AT BUFF+REG A
01B3 5F                 MOV     E,A     ;LS BYTE OF BUFFER INDEX
01B4 1600               MVI     D,0     ;DOUBLE PRECISION INDEX TO DE
01B6 3C                 INR     A       ;INDEX=INDEX+1
01B7 321302             STA     IBP     ;BACK TO MEMORY
                ;       POINTER IS INCREMENTED
                ;       SAVE THE CURRENT FILE ADDRESS
01BA 218000             LXI     H,BUFF
01BD 19                 DAD     D
                ;       ABSOLUTE CHARACTER ADDRESS IS IN HL
01BE 7E                 MOV     A,M
                ;       BYTE IS IN THE ACCUMULATOR
01BF B7                 ORA     A       ;RESET CARRY BIT
01C0 C9                 RET
                ;
                SETUP:  ;SET UP FILE
                ;       OPEN THE FILE FOR INPUT
01C1 AF                 XRA     A       ;ZERO TO ACCUM
01C2 327C00             STA     FCBCR   ;CLEAR CURRENT RECORD
                ;
01C5 115C00             LXI     D,FCB
01C8 0E0F               MVI     C,OPENF
01CA CD0500             CALL    BDOS
                ;       255 IN ACCUM IF OPEN ERROR
01CD C9                 RET
                ;
                DISKR:  ;READ DISK FILE RECORD
01CE E5D5C5             PUSH H! PUSH D! PUSH B
01D1 115C00             LXI     D,FCB
01D4 0E14               MVI     C,READF
```

```
01D6 CD0500            CALL    BDOS
01D9 C1D1E1            POP B! POP D! POP H
01DC C9                RET
                ;
                ;       FIXED MESSAGE AREA
01DD 46494C4520SIGNON: DB      'FILE DUMP VERSION 1.4$'
01F3 0D0A4E4F20OPNMSG: DB      CR,LF,'NO INPUT FILE PRESENT ON DISK$'

                ;       VARIABLE AREA
0213            IBP:    DS      2       ;INPUT BUFFER POINTER
0215            OLDSP:  DS      2       ;ENTRY SP VALUE FROM CCP
                ;
                ;       STACK AREA
0217                    DS      64      ;RESERVE 32 LEVEL STACK
                STKTOP:
                ;
0257                    END
```

5.5. A Sample Random Access Program

This chapter concludes with an extensive example of random access operation. The program listed below performs the simple function of reading or writing random records upon command from the terminal. When a program has been created, assembled, and placed into a file labeled RANDOM.COM, the CCP level command

```
RANDOM X.DAT
```

starts the test program. The program looks for a file by the name X.DAT and, if found, proceeds to prompt the console for input. If not found, the file is created before the prompt is given. Each prompt takes the form

```
next command?
```

and is followed by operator input, followed by a carriage return. The input commands take the form

*n*W *n*R Q

where *n* is an integer value in the range 0 to 65535, and W, R, and Q are simple command characters corresponding to random write, random read, and quit processing, respectively. If the W command is issued, the RANDOM program issues the prompt

```
type data:
```

The operator then responds by typing up to 127 characters, followed by a carriage return. RANDOM then writes the character string into the X.DAT file at record *n*. If the R command is issued, RANDOM reads record number *n* and displays the string value at the console, If the Q command is

issued, the X.DAT file is closed, and the program returns to the CCP. In the interest of brevity, the only error message is

```
error, try again.
```

The program begins with an initialization section where the input file is opened or created, followed by a continuous loop at the label ready where the individual commands are interpreted. The DFBC at 005CH and the default buffer at 0080H are used in all disk operations. The utility subroutines then follow, which contain the principal input line processor, called readc. This particular program shows the elements of random access processing, and can be used as the basis for further program development.

Sample Random Access Program for CP/M 2.0

```
0100                    org     100h        ;base of tpa
                ;
0000 =          reboot  equ     0000h       ;system reboot
0005 =          bdos    equ     0005h       ;bdos entry point
                ;
0001 =          coninp  equ     1           ;console input function
0002 =          conout  equ     2           ;console output function
0009 =          pstring equ     9           ;print string until '$'
000A =          rstring equ     10          ;read console buffer
000C =          version equ     12          ;return version number
000F =          openf   equ     15          ;file open function
0010 =          closef  equ     16          ;close function
0016 =          makef   equ     22          ;make file function
0021 =          readr   equ     33          ;read random
0022 =          writer  equ     34          ;write random
                ;
005C =          fcb     equ     005ch       ;default file control
                                            ;block
007D =          ranrec  equ     fcb+33      ;random record position
007F =          ranovf  equ     fcb+35      ;high order (overflow)
                                            ;byte
0080 =          buff    equ     0080h       ;buffer address
                ;
000D =          cr      equ     0dh         ;carriage return
000A =          lf      equ     0ah         ;line feed
                ;
                ;       Load SP, Set-Up File for Random Access
                ;
0100 31BC02             lxi     sp,stack
                ;
                ;       version 2.0
0103 0E0C               mvi     c,version
```

```
0105 CD0500             call   bdos
0108 FE20               cpi    20h       ;version 2.0 or better?
010A D21601             jnc    versok
               ;        bad version, message and go back
010D 111B02             lxi    d,badver
0110 CDDA01             call   print
0113 C30000             jmp    reboot
               ;
               versok:
               ;        correct versionm for random access
0116 0E0F               mvi    c,openf   ;open default fcb
0118 115C00             lxi    d,fcb
011B CD0500             call   bdos
011E 3C                 inr    a         ;err 255 becomes zero
011F C23701             jnz    ready
               ;
               ;        connot open file, so create it
0122 0E16               mvi    c,makef
0124 115C00             lxi    d,fcb
0127 CD0500             call   bdos
012A 3C                 inr    a         ;err 255 becomes zero
012B C23701             jnz    ready
               ;
               ;        cannot create file, directory full
012E 113A02             lxi    d,nospace
0131 CDDA01             call   print
0134 C30000             jmp    reboot    ;back to ccp
               ;
               ;        Loop Back to Ready After Each Command
               ;
               ready:
               ;        file is ready for processing
               ;
0137 CDE501             call   readcom   ;read next command
013A 227D00             shld   ranrec    ;store input record#
013D 217F00             lxi    h,ranovf
0140 3600               mvi    m,0       ;clear high byte if set
0142 FE51               cpi    'Q'       ;quit?
0144 C25601             jnz    notq
               ;
               ;        quit processing, close file
0147 0E10               mvi    c,closef
0149 115C00             lxi    d,fcb
014C CD0500             call   bdos
014F 3C                 inr    a         ;err 255 becomes 0
```

```
0150 CAB901             jz      error     ;error message, retry
0153 C30000             jmp     reboot    ;back to ccp
                ;
                ;
                ;       End of Quit Command, Process Write
                ;
                notq:
                ;       not the quit command, random write?
0156 FE57               cpi     'W'
0158 C28901             jnz     notw
                ;
                ;       this is a random write, fill buffer untill cr
015B 114D02             lxi     d,datmsg
015E CDDA01             call    print     ;data prompt
0161 0E7F               mvi     c,127     ;up to 127 characters
0163 218000             lxi     h,buff    ;destination
                rloop:  ;read next character to buff
0166 C5                 push    b         ;save counter
0167 E5                 push    h         ;next destination
0168 CDC201             call    getchr    ;character to a
016B E1                 pop     h         ;restore counter
016C C1                 pop     b         ;restore next to fill
016D FE0D               cpi     cr        ;end of line?
016F CA7801             jz      erloop
                ;       not end, store character
0172 77                 mov     m,a
0173 23                 inx     h         ;next to fill
0174 0D                 dcr     c         ;counter goes down
0175 C26601             jnz     rloop     ;end of buffer?
                erloop:
                ;       end of read loop, store 00
0178 3600               mvi     m,0
                ;
                ;       write the record to selected record number
017A 0E22               mvi     c,writer
017C 115C00             lxi     d,fcb
017F CD0500             call    bdos
0182 B7                 ora     a         ;erro code zero?
0183 C2B901             jnz     error     ;message if not
0186 C33701             jmp     ready     ;for another record
                ;
                ;       End of Write Command, Process Read
                ;
                notw:
                ;       not a write command, read record?
```

```
0189 FE52               cpi     'R'
018B C2B901             jnz     error      ;skip if not
                ;
                ;       read random record
018E 0E21               mvi     c,readr
0190 115C00             lxi     d,fcb
0193 CD0500             call    bdos
0196 B7                 ora     a          ;return code 00?
0197 C2B901             jnz     error
                ;
                ;       read was successful, write to console
019A CDCF01             call    crlf       ;new line
019D 0E80               mvi     c,128      ;max 128 characters
019F 218000             lxi     h,buff     ;next to get
                wloop:
01A2 7E                 mov     a,m        ;next character
01A3 23                 inx     h          ;next to get
01A4 E67F               ani     7fh        ;mask parity
01A6 CA3701             jz      ready      ;for another command
                                           ;if 00
01A9 C5                 push    b          ;save counter
01AA E5                 push    h          ;save next to get
01AB FE20               cpi     ' '        ;graphic?
01AD D4C801             cnc     putchr     ;skip output if not
01B0 E1                 pop     h
01B1 C1                 pop     b
01B2 0D                 dcr     c          ;count=count-1
01B3 C2A201             jnz     wloop
01B6 C33701             jmp     ready
                ;
                ;       End of Read Command, All Errors End Up Here
                ;
                error:
01B9 115902             lxi     d,errmsg
01BC CDDA01             call    print
01BF C33701             jmp     ready
                ;

                getchr:
                        ;read next console character to a
01C2 0E01               mvi     c,coninp
01C4 CD0500             call    bdos
01C7 C9                 ret
                ;
                putchr:
```

```
                        ;write character from a to console
01C8 0E02               mvi    c,conout
01CA 5F                 mov    e,a       ;character to send
01CB CD0500             call   bdos      ;send character
01CE C9                 ret
                ;
                crlf:
                        ;send carriage return line feed
01CF 3E0D               mvi    a,cr      ;carriage return
01D1 CDC801             call   putchr
01D4 3E0A               mvi    a,lf      ;line feed
01D6 CDC801             call   putchr
01D9 C9                 ret
                ;
                print:
                        ;print the buffer addressed by de untill $
01DA D5                 push   d
01DB CDCF01             call   crlf
01DE D1                 pop    d         ;new line
01DF 0E09               mvi    c,pstring
01E1 CD0500             call   bdos      ;print the string
01E4 C9                 ret
                ;
                readcom:
                        ;read the next command line to the conbuf
01E5 116B02             lxi    d,prompt
01E8 CDDA01             call   print     ;command?
01EB 0E0A               mvi    c,rstring
01ED 117A02             lxi    d,conbuf
01F0 CD0500             call   bdos      ;read command line
                ;       command line is present, scan it
01F3 210000             lxi    h,0       ;start with 0000
01F6 117C02             lxi    d,conlin  ;command line
01F9 1A         readc:  ldax   d         ;next command
                                         ;character
01FA 13                 inx    d         ;to next command
                                         ;position
01FB B7                 ora    a         ;cannot be end of
                                         ;command
01FC C8                 rz
                ;       not zero, numeric?
01FD D630               sui    '0'
01FF FE0A               cpi    10        ;carry if numeric
0201 D21302             jnc    endrd
                ;       add-in next digit
```

```
0204 29                 dad     h           ;*2
0205 4D                 mov     c,l
0206 44                 mov     b,h         ;bc = value * 2
0207 29                 dad     h           ;*4
0208 29                 dad     h           ;*8
0209 09                 dad     b           ;*2 + *8 = *10
020A 85                 add     l           ;*digit
020B 6F                 mov     l,a
020C D2F901             jnc     readc       ;for another char
020F 24                 inr     h           ;overflow
0210 C3F901             jmp     readc       ;for another char
                endrd:
                ;       end of read, restore value in a
0213 C630               adi     '0'         ;command
0215 FE61               cpi     'a'         ;translate case?
0217 D8                 rc
                ;       lower case, mask lower case bits
0218 E65F               ani     101$1111b
021A C9                 ret
                ;
                ;       String Data Area for Console Messages
                ;
                badver:
021B 736F727279         db      'sorry, you need cp/m version 2$'
                nospace:
023A 6E6F206469         db      'no directory space$'
                datmsg:
024D 7479706520         db      'type data: $'
                errmsg:
0259 6572726F72         db      'error, try again.$'
                prompt:
026B 6E65787420         db      'next command? $'
                ;
                ;       Fixed and Variable Data Area
                ;
027A 21         conbuf: db      conlen      ;length of console buffer
027B            consiz: ds      1           ;resulting size after read
027C            conlin: ds      32          ;length 32 buffer
0021 =          conlen  equ     $-consiz
                ;
029C                    ds      32          ;16 level stack
                stack:
02BC                    end
```

Major improvements could be made to this particular program to enhance its operation. In fact, with some work, this program could evolve into a simple data base management system. One

could, for example, assume a standard record size of 128 bytes, consisting to arbitrary fields within the record. A program, called GETKEY, could be developed that first reads a sequential file and extracts a specific field defined by the operator. For example, the command

```
GETKEY NAMES.DAT LASTNAME 10 20
```

would cause GETKEY to read the data base file NAMES.DAT and extract the LASTNAME field from each record, starting in position 10 and ending at character 20. GETKEY builds a table in memory consisting of each particular LASTNAME field, along with its 16-bit record number location within the file. The GETKEY program then sorts this list and writes a new file, called LASTNAME.KEY, which is an alphabetical list of LASTNAME fields with their corresponding record numbers. This list is called an inverted index in information retrieval parlance.

If the programmer were to rename the program shown above as QUERY and modify it so that it reads a sorted key file into memory, the command line might appear as

```
QUERY NAMES.DAT LASTNAME.KEY
```

Instead of reading a number, the QUERY program reads an alphanumeric string that is a particular key to find in the NAMES.DAT data base. Because the LASTNAME.KEY list is sorted, one can find a particular entry rapidly by performing a binary search, similar to looking up a name in the telephone book. Starting at both ends of the list, one examines the entry halfway in between and, if not matched, splits either the upper half or the lower half for the next search. You will quickly reach the item you are looking for and find the corresponding record number. You should fetch and display this record at the console, just as was done in the program shown above.

With some more work, you can allow a fixed grouping size that differs from the 128-byte record shown above. This is accomplished by keeping track of the record number and the byte offset within the record. Knowing the group size, you randomly access the record containing the proper group, offset to the beginning of the group within the record read sequentially until the group size has been exhausted.

Finally, you can improve QUERY considerably by allowing boolean expressions, which compute the set of records that satisfy several relationships, such as a LASTNAME between HARDY and LAUREL and an AGE lower than 45. Display all the records that fit this description. Finally, if your lists are getting too big to fit into memory, randomly access key files from the disk as well.

5.6. System Function Summary

#	Name	Parameters	Results
0	System Reset	**none**	**Does not return**
1	Console Input	**none**	**A**: ASCII Character
2	Console Output	**E**: ASCII character to output	**none**
3	Reader Input	**none**	**A**: ASCII Character

#	Name	Parameters	Results
4	Punch Output	**E**: ASCII Character	**none**
5	List Output	**E**: ASCII Character	**none**
6	Direct Console I/O	**E**: 0FFH (input) or char (output)	**A**: character or status
7	Get IOBYTE	**none**	**A**: IOBYTE value
8	Set IOBYTE	**E**: IOBYTE value	**none**
9	Print String	**DE**: String Address	**none**
10	Read Console Buffer	**DE**: Buffer Address	Characters input are in the Buffer
11	Get Console Status	**none**	**A**: 0FFH if a character is ready, 0 if not
12	Version Number	**none**	**HL**: Version Number
13	Reset Disk System	**none**	**A**: 0FFH if $ file present, 0 if not
14	Select Disk	**E**: Drive number: 0 for A, 1 for B, ...	**A**: 0 if successful, 0FFH on error
15	Open File	**DE**: FCB Address	**A**: Directory Code
16	Close File	**DE**: FCB Address	**A**: Directory Code
17	Search for First	**DE**: FCB Address	**A**: Directory Code
18	Search for Next	**none**	**A**: Directory Code
19	Delete File	**DE**: FCB Address	**A**: Directory Code
20	Read Sequential	**DE**: FCB Address	**A**: Directory Code
21	Write Sequential	**DE**: FCB Address	**A**: Directory Code
22	Make File	**DE**: FCB Address	**A**: Directory Code
23	Rename File	**DE**: FCB Address	**A**: Directory Code
24	Return Login Vector	**none**	**HL**: Log-in Vector
25	Return Current Disk	**none**	**A**: Current Disk. 0 for A, ... 15 for P
26	Set DMA Address	**DE**: DMA Address	**none**
27	Get Addr(ALLOC)	**none**	**HL**: ALLOC Address
28	Write Protect Disk	**none**	**none**
29	Get R/O Vector	**none**	**HL**: R/O Vector Value
30	Set File Attributes	**DE**: FCB Address	**A**: Directory Code

#	Name	Parameters	Results
31	Get Addr(Disk Parameter Block)	**none**	**HL**: **DPB** Address
32	Set/Get User Code	**E**: 0FFH (get) or *User Code* (set)	**A**: Current code (get) or none (set)
33	Read Random	**DE**: FCB Address	**A**: Return code (see details)
34	Write Random	**DE**: FCB Address	**A**: Return code (see details)
35	Compute File Size	**DE**: FCB Address	Random Record Field Set in FCB
36	Set Random Record	**DE**: FCB Address	Random Record Field Set in FCB
37	Reset Drives	**DE**: Drive Vector	**A**: 00H
38	Access Drive	**none**	**A**: 00H
39	Free Drive	**none**	**A**: 00H
40	Write Random with Zero Fill	**DE**: FCB Address	**A**: Return code

BDOS System Function Summary

6. CP/M Alteration

6.1. Introduction

The standard CP/M system assumes operation on an Intel MDS-800 microcomputer development system, but is designed so you can alter a specific set of subroutines that define the hardware operating environment.

Although standard CP/M 2 is configured for single-density floppy disks, field alteration features allow adaptation to a wide variety of disk subsystems from single drive minidisks to high-capacity, hard disk systems. To simplify the following adaptation process, it is assumed that CP/M 2 is first configured for single-density floppy disks where minimal editing and debugging tools are available. If an earlier version of CP/M is available, the customizing process is eased considerably. In this latter case, you might want to review the system generation process and skip to later sections that discuss system alteration for nonstandard disk systems.

To achieve device independence, CP/M is separated into three distinct modules:

BIOS is the Basic I/O System, which is environment dependent.

BDOS is the Basic Disk Operating System, which is not dependent upon the hardware configuration.

CCP is the Console Command Processor, which uses the BDOS.

Of these modules, only the BIOS is dependent upon the particular hardware. You can patch the distribution version of CP/M to provide a new BIOS that provides a customized interface between the remaining CP/M modules and the hardware system. This document provides a step-by-step procedure for patching a new BIOS into CP/M.

All disk-dependent portions of CP/M 2 are placed into a BIOS, a resident disk parameter block, which is either hand coded or produced automatically using the disk definition macro library provided with CP/M 2. The end user need only specify the maximum number of active disks, the starting and ending sector numbers, the data allocation size, the maximum extent of the logical disk, directory size information, and reserved track values. The macros use this information to generate the appropriate tables and table references for use during CP/M 2 operation. Deblocking information is provided, which aids in assembly or disassembly of sector sizes that are multiples of the fundamental 128-byte data unit, and the system alteration manual includes general purpose subroutines that use the deblocking information to take advantage of larger sector sizes. Use of these subroutines, together with the table-drive data access algorithms, makes CP/M 2 a universal data management system.

File expansion is achieved by providing up to 512 logical file extents, where each logical extent contains 16K bytes of data. CP/M 2 is structured, however, so that as much as 128K bytes of data are addressed by a single physical extent, corresponding to a single directory entry, maintaining

compatibility with previous versions while taking advantage of directory space. If CP/M is being tailored to a computer system for the first time, the new BIOS requires some simple software development and testing. The standard BIOS is listed in Appendix A and can be used as a model for the customized package. A skeletal version of the BIOS given in Appendix B can serve as the basis for a modified BIOS.

In addition to the BIOS, you must write a simple memory loader, called GETSYS, which brings the operating system into memory. To patch the new BIOS into CP/M, you must write the reverse of GETSYS, called PUTSYS, which places an altered version of CP/M back onto the disk. PUTSYS can be derived from GETSYS by changing the disk read commands into disk write commands. Sample skeletal GETSYS and PUTSYS programs are described in Section 6.4 and listed in Appendix C.

To make the CP/M system load automatically, you must also supply a cold start loader, similar to the one provided with CP/M, listed in Appendices A and D. A skeletal form of a cold start loader is given in Appendix E, which serves as a model for the loader.

6.2. First-level System Regeneration

The procedure to patch the CP/M system is given below. Address references in each step are shown with H denoting the hexadecimal radix, and are given for a 20K CP/M system. For larger CP/M systems, a **bias** is added to each address that is shown with a '+*b*' following it, where *b* is equal to the $\text{memory size} - 20\text{K}$. Values for *b* in various standard memory sizes are listed in Table 6-30

Memory Size	Value
24K	*b* = 24K - 20K = 4K = 1000H
32K	*b* = 32K - 20K = 12K = 3000H
40K	*b* = 40K - 20K = 20K = 5000H
48K	*b* = 48K - 20K = 28K = 7000H
56K	*b* = 56K - 20K = 36K = 9000H
62K	*b* = 62K - 20K = 42K = A800H
64K	*b* = 64K - 20K = 44K = B000H

Table 6-30: Standard Memory Size Values

Note that the standard distribution version of CP/M is set for operation within a 20K CP/M system. Therefore, you must first bring up the 20K CP/M system, then configure it for actual memory size (see Section 6.3).

Follow these steps to patch your CP/M system:

1. Read Section 6.4 and write a GETSYS program that reads the first two tracks of a disk into memory. The program from the disk must be loaded starting at location 3380H. GETSYS is coded to start at location 100H (base of the TPA) as shown in Appendix C.

2. Test the GETSYS program by reading a blank disk into memory, and check to see that the data has been read properly and that the disk has not been altered in any way by the GETSYS program.
3. Run the GETSYS program using an initialized CP/M disk to see if GETSYS loads CP/M starting at 3380H (the operating system actually starts 128 bytes later at 3400H).
4. Read Section 6.4 and write the PUTSYS program. This writes memory starting at 3380H back onto the first two tracks of the disk. The PUTSYS program should be located at 200H, as shown in Appendix C.
5. Test the PUTSYS program using a blank, uninitialized disk by writing a portion of memory to the first two tracks; clear memory and read it back using GETSYS. Test PUTSYS completely, because this program will be used to alter CP/M on disk.
6. Study Section 6.5, Section 6.6, and Section 6.7 along with the distribution version of the BIOS given in Appendix A and write a simple version that performs a similar function for the customized environment. Use the program given in Appendix B as a model. Call this new BIOS by name CBIOS (customized BIOS). Implement only the primitive disk operations on a single drive and simple console input/output functions in this phase.
7. Test CBIOS completely to ensure that it properly performs console character I/O and disk reads and writes. Be careful to ensure that no disk write operations occur during read operations and check that the proper track and sectors are addressed on all reads and writes. Failure to make these checks might cause destruction of the initialized CP/M system after it is patched.
8. Referring to Table 6-32 in Section 6.5, note that the BIOS is placed between locations 4A00H and 4FFFH. Read the CP/M system using GETSYS and replace the BIOS segment by the CBIOS developed in step 6 and tested in step 7. This replacement is done in memory.
9. Use PUTSYS to place the patched memory image of CP/M onto the first two tracks of a blank disk for testing.
10. Use GETSYS to bring the copied memory image from the test disk back into memory at 3380H and check to ensure that it has loaded back properly (clear memory, 1 if possible, before the load). Upon successful load, branch to the cold start code at location 4A00H. The cold start routine initializes page zero, then jumps to the CCP at location 3400H, which calls the BDOS, which calls the CBIOS. The CCP asks the CBIOS to read sixteen sectors on track 2, and CP/M types A>, the system prompt.
    If difficulties are encountered, use whatever debug facilities are available to trace and breakpoint the CBIOS.
11. Upon completion of step 10, CP/M has prompted the console for a command input. To test the disk write operation, type

```
SAVE 1 X.COM
```

All commands must be followed by a carriage return. CP/M responds with another prompt after several disk accesses:

```
A>
```

If it does not, debug the disk write functions and retry.

12. Test the directory command by typing

    ```
    DIR
    ```

    CP/M responds with

    ```
    A:X       COM
    ```

13. Test the erase command by typing

    ```
    ERA X.COM
    ```

    CP/M responds with the A prompt. This is now an operational system that only requires a bootstrap loader to function completely.

14. Write a bootstrap loader that is similar to GETSYS and place it on track 00, sector 01, using PUTSYS (again using the test disk, not the distribution disk). See Section 6.5, and Section 6.8 for more information on the bootstrap operation.
15. Retest the new test disk with the bootstrap loader installed by executing steps 11, 12, and 13. Upon completion of these tests, type a CTRL-C. The system executes a warm start, which reboots the system, and types the A prompt.
16. At this point, there is probably a good version of the customized CP/M system on the test disk. Use GETSYS to load CP/M from the test disk. Remove the test disk, place the distribution disk, or a legal copy, into the drive, and use PUTSYS to replace the distribution version with the customized version. Do not make this replacement if you are unsure of the patch because this step destroys the system that was obtained from Digital Research.
17. Load the modified CP/M system and test it by typing

    ```
    DIR
    ```

    CP/M responds with a list of files that are provided on the initialized disk. The file DDT.COM is the memory image for the debugger. Note that from now on, you must always reboot the CP/M system (CTRL-C is sufficient) when the disk is removed and replaced by another disk, unless the new disk is to be Read-Only.

18. Load and test the debugger by typing

    ```
    DDT
    ```

    See Section 4 for operating procedures.

19. Before making further CBIOS modifications, practice using the editor (Section 2), and assembler (see Section 3). Recode and test the GETSYS, PUTSYS, and CBIOS programs using ED, ASM, and DDT. Code and test a COPY program that does a sector-to-sector copy from one disk to another to obtain back-up copies of the original disk. Read the CP/M Licensing

Agreement specifying legal responsibilities when copying the CP/M system. Place the following copyright notice:

```
Copyright (c), 1983
Digital Research
```

on each copy that is made with the COPY program.

20. Modify the CBIOS to include the extra functions for punches, readers, and sign-on messages, and add the facilities for additional disk drives, if desired. These changes can be made with the GETSYS and PUTSYS programs or by referring to the regeneration process in Section 6.3.

You should now have a good copy of the customized CP/M system. Although the CBIOS portion of CP/M belongs to the user, the modified version cannot be legally copied.

It should be noted that the system remains file-compatible with all other CP/M systems (assuming media compatibility) which allows transfer of nonproprietary software between CP/M users.

6.3. Second-level System Regeneration

Once the system is running, the next step is to configure CP/M for the desired memory size. Usually, a memory image is first produced with the MOVCPM program (system relocator) and then placed into a named disk file. The disk file can then be loaded, examined, patched, and replaced using the debugger and the system generation program (refer to Section 1).

The CBIOS and BOOT are modified using ED and assembled using ASM, producing files called CBIOS.HEX and BOOT.HEX, which contain the code for CBIOS and BOOT in Intel hex format.

To get the memory image of CP/M into the TPA configured for the desired memory size, type the command:

```
MOVCPM xx *
```

where *xx* is the memory size in decimal K bytes, for example, 32 for 32K. The response is as follows:

```
CONSTRUCTING xxK CP/M VERS 2.0

READY FOR "SYSGEN" OR

"SAVE 34 CPMxx.COM"
```

An image of CP/M in the TPA is configured for the requested memory size. The memory image is at location 0900H through 227FH, that is, the BOOT is at 0900H, the CCP is at 0980H, the BDOS starts at 1180H, and the BIOS is at 1F80H. Note that the memory image has the standard Model 800 BIOS and BOOT on it. It is now necessary to save the memory image in a file so that you can patch the CBIOS and CBOOT into it:

```
SAVE 34 CPMxx.COM
```

The memory image created by the MOVCPM program is offset by a negative bias so that it loads into the free area of the TPA, and thus does not interfere with the operation of CP/M in higher memory. This memory image can be subsequently loaded under DDT and examined or changed in preparation for a new generation of the system. DDT is loaded with the memory image by typing:

```
DDT CPMxx.COM
```

Which loads DDT, then reads the CP/M image.

DDT should respond with the following:

```
DDT VERS 2.2
NEXT PC
2300 0100
-
```

Where - is the DDT prompt.

You can then give the display and disassembly commands to examine portions of the memory image between 900H and 227FH. Note, however, that to find any particular address within the memory image, you must apply the negative bias to the CP/M address to find the actual address. Track 00, sector 01, is loaded to location 900H (the user should find the cold start loader at 900H to 97FH); track 00, sector 02, is loaded into 980H (this is the base of the CCP); and so on through the entire CP/M system load. In a 20K system, for example, the CCP resides at the CP/M address 3400H, but is placed into memory at 980H by the SYSGEN program. Thus, the negative bias, denoted by *n*, satisfies

3400H + *n* = 980H, or *n*=980H - 3400H

Assuming two's complement arithmetic, *n* = D580H, which can be checked by

3400H + D580H = 10980H = 0980H

when ignoring high-order overflows.

Note that for larger systems, *n* satisfies:

(3400H + *b*) + *n* = 0980H, or

n = 0980H - (3400H + *b*), or

n = D580H - *b*

The value for *n* for common CP/M systems is given in Table 6-31.

Memory Size	Bias *b*	Negative Offset *n*
20K	1000H	*n* = D580H - 0000H = D580H
24K	1000H	*n* = D580H - 1000H = C580H

Memory Size	Bias *b*	Negative Offset *n*
32K	3000H	*n* = D580H - 3000H = A580H
40K	5000H	*n* = D580H - 5000H = 8580H
48K	7000H	*n* = D580H - 7000H = 6580H
56K	9000H	*n* = D580H - 9000H = 4580H
62K	A800H	*n* = D580H - A800H = 2D80H
64K	B000H	*n* = D580H - B000H = 2580H

Table 6-31: Common Value for CP/M Systems

If you want to locate the address *x* within the memory image loaded under DDT in a 20K system, first type

H*x*,*n*

for the Hexadecimal sum and difference command.

and DDT responds with the value of $x + n$ (sum) and $x - n$ (difference). The first number printed by DDT is the actual memory address in the image where the data or code is located. For example, the following DDT command:

```
H3400,D580
```

produces 980H as the sum, which is where the CCP is located in the memory image under DDT.

Type the L command to disassemble portions of the BIOS located at (4A00H+*b*)-*n*, which, when one uses the H command, produces an actual address of 1F80H. The disassembly command would thus be as follows:

```
L1F80
```

It is now necessary to patch in the CBOOT and CBIOS routines. The BOOT resides at location 0900H in the memory image. If the actual load address is *n*, then to calculate the bias (*m*), type the command:

H900,*n*

The second number typed by DDT in response to the command is the desired bias (*m*). For example, if the BOOT executes at 0080H, the command

```
H900,80
```

produces the sum and difference in hex

```
0980 0880
```

Therefore, the bias *m* would be 0880H. To read-in the BOOT, give the command:

```
ICBOOT.HEX
```

R*m*

to read CBOOT.HEX with a bias of *m* (=900H-*n*)

Examine the CBOOT with

```
L900
```

You are now ready to replace the CBIOS by examining the area at 1F80H, where the original version of the CBIOS resides, and then typing

ICBIOS.HEX Ready the hex file for loading

Assume that the CBIOS is being integrated into a 20K CP/M system and thus originates at location 4A00H. To locate the CBIOS properly in the memory image under DDT, you must apply the negative bias n for a 20K system when loading the hex file. This is accomplished by typing

RD580 Read the file with bias of D580H

Upon completion of the read, reexamine the area where the CBIOS has been loaded (use an L1F80 command) to ensure that it is properly loaded. When you are satisfied that the change has been made, return from DDT using a CTRL-C or, G0 command.

6.4. Sample GETSYS and PUTSYS Program

The following program provides a framework for the GETSYS and PUTSYS programs referenced in Sections 6.1 and 6.2. To read and write the specific sectors, you must insert the READSEC and WRITESEC subroutines.

```
;  GETSYS PROGRAM -- READ TRACKS 0 AND 1 TO MEMORY AT 3380H
;  REGISTER                    USE

;         A               (SCRATCH REGISTER)

;         B               TRACK COUNT (0, 1)

;         C               SECTOR COUNT (1,2,...,26)

;         DE              (SCRATCH REGISTER PAIR)

;         HL              LOAD ADDRESS

;         SP              SET TO STACK ADDRESS

;
START:    LXI  SP,3380H   ;SET STACK POINTER TO SCRATCH
                          ;AREA
          LXI  H,3380H    ;SET BASE LOAD ADDRESS
          MVI  B,0        ;START WITH TRACK 0
RDTRK:                    ;READ NEXT TRACK (INITIALLY 0)
          MVI  C,1        ;READ STARTING WITH SECTOR 1

RDSEC:                    ;READ NEXT SECTOR
```

```
          CALL READSEC    ;USER-SUPPLIED SUBROUTINE
          LXI  D,128      ;MOVE LOAD ADDRESS TO NEXT 1/2
                          ;PAGE
          DAD  D          ;HL = HL + 128
          INR  C          ;SECTOR = SECTOR + 1
          MOV  A,C        ;CHECK FOR END OF TRACK
          CPI  27
          JC   RDSEC      ;CARRY GENERATED IF SECTOR <27

;
;  ARRIVE HERE AT END OF TRACK, MOVE TO NEXT TRACK
          INR  B
          MOV  A,B        ;TEST FOR LAST TRACK
          CPI  2
          JC   RDTRK      ;CARRY GENERATED IF TRACK <2

;
;  USER-SUPPLIED SUBROUTINE TO READ THE DISK
READSEC:
;         ENTER WITH TRACK NUMBER IN REGISTER B,
;         SECTOR NUMBER IN REGISTER C, AND
;         ADDRESS TO FILL IN HL

;
          PUSH B          ;SAVE B AND C REGISTERS
          PUSH H          ;SAVE HL REGISTERS

;         ************************************************
;         perform disk read at this point, branch to
;         label START if an error occurs
;         ************************************************
          POP H           ;RECOVER HL
          POP B           ;RECOVER B AND C REGISTERS
          RET             ;BACK TO MAIN PROGRAM

          END START
```

This program is assembled and listed in Appendix B for reference purposes, with an assumed origin of 100H. The hexadecimal operation codes that are listed on the left might be useful if the program has to be entered through the panel switches.

The PUTSYS program can be constructed from GETSYS by changing only a few operations in the GETSYS program given above, as shown in Appendix C. The register pair HL becomes the dump address, next address to write, and operations on these registers do not change within the program. The READSEC subroutine is replaced by a WRITESEC subroutine, which performs the

opposite function; data from address HL is written to the track given by register B and sector given by register C. It is often useful to combine GETSYS and PUTSYS into a single program during the test and development phase, as shown in Appendix C.

6.5. Disk Organization

The sector allocation for the standard distribution version of CP/M is given here for reference purposes. The first sector contains an optional software boot section (see the table on the following page. Disk controllers are often set up to bring track 0, sector 1, into memory at a specific location, often location 0000H. The program in this sector, called BOOT, has the responsibility of bringing the remaining sectors into memory starting at location 3400H+*b*. If the controller does not have a built-in sector load, the program in track 00, sector 01 can be ignored. In this case, load the program from track 00, sector 02, to location 3400H+*b*.

As an example, the Intel Model 800 hardware cold start loader brings track 00, sector 01, into absolute address 3000H. Upon loading this sector, control transfers to location 3000H, where the bootstrap operation commences by loading the remainder of track 00 and all of track 01 into memory, starting at 3400H+b. Note that this bootstrap loader is of little use in a non-microcomputer development system environment, although it is useful to examine it because some of the boot actions will have to be duplicated in the user's cold start loader.

Track #	Sector #	Page #	Memory Address	CP/M Module name
00	01		(boot address)	Cold Start Loader
00	02	00	3400H + *b*	CCP
00	03	00	3480H + *b*	CCP
00	04	01	3500H + *b*	CCP
00	05	01	3580H + *b*	CCP
00	06	02	3600H + *b*	CCP
00	07	02	3680H + *b*	CCP
00	08	03	3700H + *b*	CCP
00	09	03	3780H + *b*	CCP
00	10	04	3800H + *b*	CCP
00	11	04	3880H + *b*	CCP
00	12	05	3900H + *b*	CCP
00	13	05	3980H + *b*	CCP
00	14	06	3A00H + *b*	CCP
00	15	06	3A80H + *b*	CCP
00	16	07	3B00H + *b*	CCP
00	17	07	3B80H + *b*	CCP
00	18	08	3C00H + *b*	BDOS

Track #	Sector #	Page #	Memory Address	CP/M Module name
00	19	08	3C80H + *b*	BDOS
00	20	09	3D00H + *b*	BDOS
00	21	09	3D80H + *b*	BDOS
00	22	10	3E00H + *b*	BDOS
00	23	10	3E80H + *b*	BDOS
00	24	11	3F00H + *b*	BDOS
00	25	11	3F80H + *b*	BDOS
00	26	12	4000H + *b*	BDOS
01	01	12	4080H + *b*	BDOS
01	02	13	4100H + *b*	BDOS
01	03	13	4180H + *b*	BDOS
01	04	14	4200H + *b*	BDOS
01	05	14	4280H + *b*	BDOS
01	06	15	4300H + *b*	BDOS
01	07	15	4380H + *b*	BDOS
01	08	16	4400H + *b*	BDOS
01	09	16	4480H + *b*	BDOS
01	10	17	4500H + *b*	BDOS
01	11	17	4580H + *b*	BDOS
01	12	18	4600H + *b*	BDOS
01	13	18	4680H + *b*	BDOS
01	14	19	4700H + *b*	BDOS
01	15	19	4780H + *b*	BDOS
01	16	20	4800H + *b*	BDOS
01	17	20	4880H + *b*	BDOS
01	18	21	4900H + *b*	BDOS
01	19	21	4980H + *b*	BDOS
01	20	22	4A00H + *b*	BIOS
01	21	22	4A80H + *b*	BIOS
01	22	23	4B00H + *b*	BIOS
01	23	23	4B80H + *b*	BIOS
01	24	24	4C00H + *b*	BIOS
01	25	24	4C80H + *b*	BIOS
01	26	25	4D00H + *b*	BIOS

Table 6-32: CP/M Disk Sector Allocation

6.6. The BIOS Entry Points

The entry points into the BIOS from the cold start loader and BDOS are detailed below. Entry to the BIOS is through a jump vector located at 4A00H + *b*, as shown below. See Appendixes A and B. The jump vector is a sequence of 17 jump instructions that send program control to the individual BIOS subroutines. The BIOS subroutines might be empty for certain functions (they might contain a single RET operation) during reconfiguration of CP/M, but the entries must be present in the jump vector.

The jump vector at 4A00H + *b* takes the form shown below, where the individual jump addresses are given to the left:

```
4A00H+b      JMP BOOT       ;ARRIVE HERE FROM COLD START LOAD

4A03H+b      JMP WBOOT      ;ARRIVE HERE FOR WARM START

4A06H+b      JMP CONST      ;CHECK FOR CONSOLE CHAR READY

4A09H+b      JMP CONIN      ;READ CONSOLE CHARACTER IN

4A0CH+b      JMP CONOUT     ;WRITE CONSOLE CHARACTER OUT

4A0FH+b      JMP LIST       ;WRITE LISTING CHARACTER OUT

4A12H+b      JMP PUNCH      ;WRITE CHARACTER TO PUNCH DEVICE

4A15H+b      JMP READER     ;READ READER DEVICE

4A18H+b      JMP HOME       ;MOVE TO TRACK 00 ON SELECTED DISK

4A1BH+b      JMP SELDSK     ;SELECT DISK DRIVE

4A1EH+b      JMP SETTRK     ;SET TRACK NUMBER

4A21H+b      JMP SETSEC     ;SET SECTOR NUMBER

4A24H+b      JMP SETDMA     ;SET DMA ADDRESS

4A27H+b      JMP READ       ;READ SELECTED SECTOR

4A2AH+b      JMP WRITE      ;WRITE SELECTED SECTOR

4A2DH+b      JMP LISTST     ;RETURN LIST STATUS

4A30H+b      JMP SECTRAN    ;SECTOR TRANSLATE SUBROUTINE
```

Each jump address corresponds to a particular subroutine that performs the specific function, as outlined below. There are three major divisions in the jump table: the system re-initialization, which results from calls on BOOT and WBOOT; simple character I/O, performed by calls on CONST, CONIN, CONOUT, LIST, PUNCH, READER, and LISTST; and disk I/O, performed by calls on HOME, SELDSK, SETTRK, SETSEC, SETDMA, READ, WRITE, and SECTRAN.

All simple character I/O operations are assumed to be performed in ASCII, upper- and lower-case, with high-order (parity bit) set to zero. An end-of-file condition for an input device is given by an ASCII CTRL-Z (1AH). Peripheral devices are seen by CP/M as logical devices and are assigned to physical devices within the BIOS.

To operate, the BDOS needs only the CONST, CONIN, and CONOUT subroutines. LIST, PUNCH, and READER can be used by PIP, but not the BDOS. Further, the LISTST entry is currently used only by DESPOOL, the print spooling utility. Thus, the initial version of CBIOS can have empty subroutines for the remaining ASCII devices.

The following list describes the characteristics of each device.

CONSOLE is the principal interactive console that communicates with the operator and it is accessed through CONST, CONIN, and CONOUT. Typically, the CONSOLE is a device such as a CRT or teletype.

LIST is the principal listing device. If it exists on the user's system, it is usually a hard-copy device, such as a printer or teletype.

PUNCH is the principal tape punching device. If it exists, it is normally a high-speed paper tape punch or teletype.

READER is the principal tape reading device, such as a simple optical reader or teletype.

A single peripheral can be assigned as the LIST, PUNCH, and READER device simultaneously. If no peripheral device is assigned as the LIST, PUNCH, or READER device, the CBIOS gives an appropriate error message so that the system does not hang if the device is accessed by PIP or some other user program. Alternately, the PUNCH and LIST routines can just simply return, and the READER routine can return with a 1AH (CTRL-Z) in register A to indicate immediate end-of-file.

For added flexibility, you can optionally implement the IOBYTE function, which allows reassignment of physical devices. The IOBYTE function creates a mapping of logical-to-physical devices that can be altered during CP/M processing, see the STAT command in Section 1.6.1.

The definition of the IOBYTE function corresponds to the Intel standard as follows: a single location in memory, currently location 0003H, is maintained, called IOBYTE, which defines the logical-to-physical device mapping that is in effect at a particular time. The mapping is performed by splitting the IOBYTE into four distinct fields of two bits each, called the CONSOLE, READER, PUNCH, and LIST fields, as shown in Table 6-33

Most Significant				Bits			Least Significant
List		Punch		Reader		Console	
7	6	5	4	3	2	1	0

Table 6-33: IOBYTE Fields

Value	Bits	Meaning
	1 0	**Console** field
0	0 0	CONSOLE is assigned to the console printer device (TTY:)
1	0 1	CONSOLE is assigned to the CRT device (CRT:)
2	1 0	Batch mode: use the READER as the CONSOLE input, and the LIST device as the CONSOLE output (BAT:)
3	1 1	CONSOLE is assigned to the user-defined console device (UC1:)
	3 2	**Reader** field
0	0 0	READER is the teletype device (TTY:)
1	0 1	READER is the high speed reader device (PTR:)
2	1 0	READER is the user-defined reader #1 (UR1:)
3	1 1	READER is the user-defined reader #2 (UR2:)
	5 4	**Punch** field
0	0 0	PUNCH is the teletype device (TTY:)
1	0 1	PUNCH is the high speed punch device (PTP:)
2	1 0	PUNCH is the user-defined punch #1 (UP1:)
3	1 1	PUNCH is the user-defined punch #2 (UP2:)
	7 6	**List** field
0	0 0	LIST is the teletype device (TTY:)
1	0 1	LIST is the CRT device (CRT:)
2	1 0	LIST is the line printer device (LPT:)
3	1 1	LIST is the user-defined list device (UL1:)

Table 6-34: IOBYTE Field Values

The implementation of the IOBYTE is optional and effects only the organization of the CBIOS. No CP/M systems use the IOBYTE (although they tolerate the existence of the IOBYTE at location 0003H) except for PIP, which allows access to the physical devices, and STAT, which allows logical-physical assignments to be make or displayed. For more information see Section 1. In any case the IOBYTE implementation should be omitted until the basic CBIOS is fully implemented and tested; then you should add the IOBYTE to increase the facilities.

Disk I/O is always performed through a sequence of calls on the various disk access subroutines that set up the disk number to access, the track and sector on a particular disk, and the Direct

Memory Access (DMA) address involved in the I/O operation. After all these parameters have been set up, a call is made to the READ or WRITE function to perform the actual I/O operation.

There is often a single call to SELDSK to select a disk drive, followed by a number of read or write operations to the selected disk before selecting another drive for subsequent operations. Similarly, there might be a single call to set the DMA address, followed by several calls that read or write from the selected DMA address before the DMA address is changed. The track and sector subroutines are always called before the READ or WRITE operations are performed.

The READ and WRITE routines should perform several retries (10 is standard) before reporting the error condition to the BDOS. If the error condition is returned to the BDOS, it reports the error to the user. The HOME subroutine might or might not actually perform the track 00 seek, depending upon controller characteristics; the important point is that track 00 has been selected for the next operation and is often treated in exactly the same manner as SETTRK with a parameter of 00.

The following table describes the exact responsibilities of each BIOS entry point subroutine.

Entry Point	Function
BOOT	The BOOT entry point gets control from the cold start loader and is responsible for basic system initialization, including sending a sign-on message, which can be omitted in the first version. If the IOBYTE function is implemented, it must be set at this point. The various system parameters that are set by the WBOOT entry point must be initialized, and control is transferred to the CCP at 3400H + *b* for further processing. Note that register C must be set to zero to select drive A.

Entry Point	Function
WBOOT	The WBOOT entry point gets control when a warm start occurs. A warm start is performed whenever a user program branches to location 0000H, or when the CPU is reset from the front panel. The CP/M system must be loaded from the first two tracks of drive A up to, but not including, the BIOS, or CBIOS, if the user has completed the patch. System parameters must be initialized as follows: **locations 0, 1, and 2** Set to JMP WBOOT for warm starts (000H: JMP 4A03H+*b*) **location 3** Set initial value of IOBYTE, if implemented in the CBIOS. **location 4** High nibble = current user number; low nibble = current drive **locations 5, 6, and 7** Set to JMP BDOS, which is the primary entry point to CP/M for transient programs. (0005H: JMP 3C06H+*b*) Refer to Section 6.9 for complete details of page zero use. Upon completion of the initialization, the WBOOT program must branch to the CCP at 3400H+*b* to restart the system. Upon entry to the CCP, register C is set to the drive to select after system initialization. The WBOOT routine should read location 4 in memory, verify that is a legal drive, and pass it to the CCP in register C.
CONST	You should sample the status of the currently assigned console device and return 0FFH in register A if a character is ready to read and 00H in register A if no console characters are ready.
CONIN	The next console character is read into register A, and the parity bit is set, high-order bit, to zero. If no console character is ready, wait until a character is typed before returning.
CONOUT	The character is sent from register C to the console output device. The character is in ASCII, with high-order parity bit set to zero. You might want to include a time-out on a line-feed or carriage return, if the console device requires some time interval at the end of the line (such as a TI Silent 700 terminal). You can filter out control characters that cause the console device to react in a strange way (CTRL-Z causes the Lear-Siegler terminal to clear the screen, for example).
LIST	The character is sent from register C to the currently assigned listing device. The character is in ASCII with zero parity bit.

Entry Point	Function
PUNCH	The character is sent from register C to the currently assigned punch device. The character is in ASCII with zero parity.
READER	The next character is read from the currently assigned reader device into register A with zero parity (high-order bit must be zero); an end-of-file condition is reported by returning an ASCII CTRL-Z(1AH).
HOME	The disk head of the currently selected disk (initially disk A) is moved to the track 00 position. If the controller allows access to the track 0 flag from the drive, the head is stepped until the track 0 flag is detected. If the controller does not support this feature, the HOME call is translated into a call to SETTRK with a parameter of 0.
SELDSK	The disk drive given by register C is selected for further operations, where register C contains 0 for drive A, 1 for drive B, and so on up to 15 for drive P (the standard CP/M distribution version supports four drives). On each disk select, SELDSK must return in HL the base address of a 16-byte area, called the **Disk Parameter Header**, described in Section 6.10. For standard floppy disk drives, the contents of the header and associated tables do not change; thus, the program segment included in the sample CBIOS performs this operation automatically. If there is an attempt to select a nonexistent drive, SELDSK returns HL=0000H as an error indicator. Although SELDSK must return the header address on each call, it is advisable to postpone the physical disk select operation until an I/O function (seek, read, or write) is actually performed, because disk selects often occur without ultimately performing any disk I/O, and many controllers unload the head of the current disk before selecting the new drive. This causes an excessive amount of noise and disk wear. The least significant bit of register E is zero if this is the first occurrence of the drive select since the last cold or warm start.

Entry Point	Function
SETTRK	Register BC contains the track number for subsequent disk accesses on the currently selected drive. The sector number in BC is the same as the number returned from the SECTRAN entry point. You can choose to seek the selected track at this time or delay the seek until the next read or write actually occurs. Register BC can take on values in the range 0-76 corresponding to valid track numbers for standard floppy disk drives and 0-65535 for nonstandard disk subsystems.
SETSEC	Register BC contains the sector number, 01 through 26, for subsequent disk accesses on the currently selected drive. The sector number in BC is the same as the number returned from the SECTRAN entry point. You can choose to send this information to the controller at this point or delay sector selection until a read or write operation occurs.
SETDMA	Register BC contains the DMA (Disk Memory Access) address for subsequent read or write operations. For example, if B = 00H and C = 80H when SETDMA is called, all subsequent read operations read their data into 80H through 0FFH and all subsequent write operations get their data from 80H through 0FFH, until the next call to SETDMA occurs. The initial DMA address is assumed to be 80H. The controller need not actually support Direct Memory Access. If, for example, all data transfers are through I/O ports, the CBIOS that is constructed uses the 128-byte area starting at the selected DMA address for the memory buffer during the subsequent read or write operations.

Entry Point	Function
READ	Assuming the drive has been selected, the track has been set, and the DMA address has been specified, the READ subroutine attempts to read one sector based upon these parameters and returns the following error codes in register A: **0** no errors occurred **1** nonrecoverable error condition occurred Currently, CP/M responds only to a zero or nonzero value as the return code. That is, if the value in register A is 0, CP/M assumes that the disk operation was completed properly. If an error occurs the CBIOS should attempt at least 10 retries to see if the error is recoverable. When an error is reported the BDOS prints the message `BDOS Err On X:  Bad Sector` The operator then has the option of pressing a carriage return to ignore the error, or CTRL-C to abort.
WRITE	Data is written from the currently selected DMA address to the currently selected drive, track, and sector. For floppy disks, the data should be marked as non-deleted data to maintain compatibility with other CP/M systems. The error codes given in the READ command are returned in register A, with error recovery attempts as described above.
LISTST	You return the ready status of the list device used by the DESPOOL program to improve console response during its operation. The value 00 is returned in A if the list device is not ready to accept a character and 0FFH if a character can be sent to the printer. A 00 value should be returned if LIST status is not implemented.

Entry Point	Function
SECTRAN	Logical-to-physical sector translation is performed to improve the overall response of CP/M. Standard CP/M systems are shipped with a skew factor of 6, where six physical sectors are skipped between each logical read operation. This skew factor allows enough time between sectors for most programs to load their buffers without missing the next sector. In particular computer systems that use fast processors, memory, and disk subsystems, the skew factor might be changed to improve overall response. However, the user should maintain a single-density IBM-compatible version of CP/M for information transfer into and out of the computer system, using a skew factor of 6. In general, SECTRAN receives a logical sector number relative to zero in BC and a translate table address in DE. The sector number is used as an index into the translate table, with the resulting physical sector number in HL. For standard systems, the table and indexing code is provided in the CBIOS and need not be changed.

Table 6-35: BIOS Entry Points

6.7. A Sample BIOS

The program shown in Appendix B can serve as a basis for your first BIOS. The simplest functions are assumed in this BIOS, so that you can enter it through a front panel, if absolutely necessary. You must alter and insert code into the subroutines for CONST, CONIN, CONOUT, READ, WRITE, and WAITIO subroutines. Storage is reserved for user-supplied code in these regions. The scratch area reserved in page zero (see Section 6.9) for the BIOS is used in this program, so that it could be implemented in ROM, if desired.

Once operational, this skeletal version can be enhanced to print the initial sign-on message and perform better error recovery. The subroutines for LIST, PUNCH, and READER can be filled out and the IOBYTE function can be implemented.

6.8. A Sample Cold Start Loader

The program shown in Appendix E can serve as a basis for a cold start loader. The disk read function must be supplied by the user, and the program must be loaded somehow starting at location 0000. Space is reserved for the patch code so that the total amount of storage required for the cold start loader is 128 bytes.

Eventually, you might want to get this loader onto the first disk sector (track 00, sector 01) and cause the controller to load it into memory automatically upon system start up. Alternatively, the cold start loader can be placed into ROM, and above the CP/M system. In this case, it is

necessary to originate the program at a higher address and key in a jump instruction at system start up that branches to the loader. Subsequent warm starts do not require this key-in operation, because the entry point WBOOT gets control, thus bringing the system in from disk automatically. The skeletal cold start loader has minimal error recovery, which might be enhanced in later versions.

6.9. Reserved Locations in Page Zero

Locations	Contents
0000H ... 0002H	Contains a jump instruction to the warm start entry location 4A03H + *b*. This allows a simple programmed restart (JMP 0000H) or manual restart from the front panel.
0003H	Contains the Intel standard IOBYTE is optionally included in the user's CBIOS (refer to Section 6.6)
0004H	High nibble is user number (0 ... 15). Low nibble is current default drive number (0=A, 1=B, ... 15=P)
0005H ... 0007H	Contains a jump instruction to the BDOS and serves two purposes: JMP 0005H provides the primary entry point to the BDOS, as described in Chapter 5, and LHLD 0006H brings the address field of the instruction to the HL register pair. This value is the lowest address in memory used by CP/M, assuming the CCP is being overlaid. The DDT program changes the address field to reflect the reduced memory size in debug mode.
0008H ... 0027H	Interrupt locations 1 through 5 not used.
0030H ... 0037H	Interrupt location 6 (not currently used) is reserved.
0038H ... 003AH	Restart 7; contains a jump instruction into the DDT or SID program when running in debug mode for programmed breakpoints, but is not otherwise used by CP/M.
003BH ... 003FH	Not currently used; reserved.
0040H ... 004FH	A 16-byte area reserved for scratch by CBIOS, but is not used for any purpose in the distribution version of CP/M.
0050H ... 005BH	Not currently used; reserved.
005CH ... 007CH	Default File Control Block produced for a transient program by the CCP.
007DH ... 007FH	Optional default random record position.
0080H ... 00FFH	Default 128-byte disk buffer, also filled with the command line when a transient is loaded under the CCP.

Table 6-36: Reserved Locations in Page Zero

This information is set up for normal operation under the CP/M system, but can be overwritten by a transient program if the BDOS facilities are not required by the transient.

If, for example, a particular program performs only simple I/O and must begin execution at location 0, it can first be loaded into the TPA, using normal CP/M facilities, with a small memory move program that gets control when loaded. The memory move program must get control from location 0100H, which is the assumed beginning of all transient programs. The move program can then proceed to the entire memory image down to location 0 and pass control to the starting address of the memory load.

If the BIOS is overwritten or if location 0, containing the warm start entry point, is overwritten, the operator must bring the CP/M system back into memory with a cold start sequence.

6.10. Disk Parameter Tables

Tables are included in the BIOS that describe the particular characteristics of the disk subsystem used with CP/M. These tables can be either hand-coded, as shown in the sample CBIOS in Appendix B, or automatically generated using the DISKDEF macro library, as shown in Appendix F. The purpose here is to describe the elements of these tables.

In general, each disk drive has an associated (16-byte) disk parameter header that contains information about the disk drive and provides a scratch pad area for certain BDOS operations. The format of the disk parameter header for each drive is shown in Table 6-37, where each element is a word (16-bit) value.

XLT	0000	0000	0000	**DIRBUF**	**DPB**	**CSV**	**ALV**
16b	16b	16b	16b	16b	16b	16b	16b

Table 6-37: Disk Parameter Header Format

The meaning of each Disk Parameter Header (**DPH**) element is detailed in Table 6-38.

Parameter	Meaning
XLT	Address of the logical-to-physical translation vector, if used for this particular drive, or the value 0000H if no sector translation takes place (that is, the physical and logical sector numbers are the same). Disk drives with identical sector skew factors share the same translate tables.
0000	Scratch pad values for use within the BDOS, initial value is unimportant.
DIRBUF	Address of a 128-byte scratch pad area for directory operations within BDOS. All **DPH**s address the same scratch pad area.

Parameter	Meaning
DPB	Address of a disk parameter block for this drive. Drives with identical disk characteristics address the same disk parameter block.
CSV	Address of a scratch pad area used for software check for changed disks. This address is different for each DPH.
ALV	Address of a scratch pad area used by the BDOS to keep disk storage allocation information. This address is different for each DPH.

Table 6-38: Disk Parameter Headers

Given *n* disk drives, the **DPH**'s are arranged in a table whose first row of 16 bytes corresponds to drive 0, with the last row corresponding to drive $n-1$. In the following figure the label DPBASE defines the base address of the **DPH** table.

DPBASE:								
+00	XLT00	0000	0000	0000	DIRBUF	DPB00	CSV00	ALV00
+01	XLT00	0000	0000	0000	DIRBUF	DPB00	CSV00	ALV00
				(and so on through)				
$n-1$	XLT*n-1*	0000	0000	0000	DIRBUF	DPB*n-1*	CSV*n-1*	ALV*n-1*

Table 6-39: Disk Parameter Header Table

A responsibility of the SELDSK subroutine is to return the base address of the **DPH** for the selected drive. The following sequence of operations returns the table address, with a 0000H returned if the selected drive does not exist.

```
NDISKS      EQU     4           ;NUMBER OF DISK DRIVES
.........
SELDSK:     ;SELECT DISK GIVEN BY BC
            LSI     H,0000H     ;ERROR CODE
            MOV     A,C         ;DRIVE OK?
            CPI     NDISKS      ;CY IF SO
            RNC                 ;RET IF ERROR
            ;NO ERROR, CONTINUE
            MOV     L,C         ;LOW(DISK)
            MOV     H,B         ;HIGH(DISK)
            DAD     H           ;*2
            DAD     H           ;*4
            DAD     H           ;*8
            DAD     H           ;*16
            LXI     D,DPBASE    ;FIRST DPH
```

```
        DAD     D           ;DPH(DISK)
        RET
```

The translation vectors, XLT00 through XLT$n-1$, are located elsewhere in the BIOS, and simply correspond one-for-one with the logical sector numbers zero through the $\text{sector count} - 1$. The Disk Parameter Block (**DPB**) for each drive is more complex. As shown in Table 6-40, a particular **DPB**, that is addressed by one or more **DPH**s, takes the general form:

SPT	BSH	BLM	EXM	DSM	DRM	AL0	AL1	CHK	OFF
16b	8b	8b	8b	16b	16b	8b	8b	16b	16b

Table 6-40: Disk Parameter Block Format

where each is a byte or word value, as shown by the 8b or 16b indicator below the field.

The following field abbreviations are used in Table 6-40:

SPT is the total number of sectors per track.

BSH is the data allocation block shift factor, determined by the data block allocation size.

BLM is the data allocation block mask ($2^{\text{BSH}} - 1$)

EXM is the extent mask, determined by the data block allocation size and the number of disk blocks.

DSM determines the total storage capacity of the disk drive.

DRM determines the total number of directory entries that can be stored on this drive.

AL0 and AL1 determine reserved directory blocks.

CKS is the size of the directory check vector.

OFF is the number of reserved tracks at the beginning of the (logical) disk.

The values of **BSH** and **BLM** determine the data allocation size **BLS**, which is not an entry in the **DPB**. Given that the designer has selected a value for **BLS**, the values of **BSH** and **BLM** are shown in Table 6-41.

BLS	BSH	BLM
1024	3	7
2048	4	15
4096	5	31
8192	6	63
16384	7	127

Table 6-41: **BSH** and **BLM** Values

where all values are in decimal. The value of **EXM** depends upon both the **BLS** and whether the **DSM** value is less than `256` or greater than `255`, as shown in Table 6-42.

BLS	$\mathrm{DSM} < 256$	$\mathrm{DSM} > 255$
1024	0	n/a
2048	1	0
4096	3	1
8192	7	3
16384	15	7

Table 6-42: **EXM** Values

The value of **DSM** is the maximum data block number supported by this particular drive, measured in **BLS** units. The product **($\mathrm{DSM} + 1$)** is the total number of bytes held by the drive and must be within the capacity of the physical disk, not counting the reserved operating system tracks.

The **DRM** entry is the one less than the total number of directory entries that can take on a 16-bit value. The values of **AL0** and **AL1**, however, are determined by **DRM**. The values **AL0** and **AL1** can together be considered a string of 16-bits, as shown in Table 6-43.

AL0								**AL1**							
07	06	05	04	03	02	01	00	07	06	05	04	03	02	01	00
00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15

Table 6-43: **AL0** and **AL1**

Position 00 corresponds to the high-order bit of the byte labeled **AL0** and 15 corresponds to the low-order bit of the byte labeled **AL1**. Each bit position reserves a data block for number of directory entries, thus allowing a total of 16 data blocks to be assigned for directory entries (bits are assigned starting at 00 and filled to the right until position 15). Each directory entry occupies 32 bytes, resulting in Table 6-44:

BLS	Directory Entries
1024	$32 * \text{number of bits}$
2048	$64 * \text{number of bits}$
4096	$128 * \text{number of bits}$
8192	$256 * \text{number of bits}$
16384	$512 * \text{number of bits}$

Table 6-44: **BLS** Tabulation

Thus, if **DRM** = 127 (128 directory entries) and **BLS** = 1024, there are 32 directory entries per block, requiring 4 reserved blocks. In this case, the 4 high-order bits of **AL0** are set, resulting in the values **AL0** = 0F0H and **AL1** = 00H.

The **CKS** value is determined as follows: if the disk drive media is removable, then $\mathbf{CKS} = \frac{\mathbf{DRM}+1}{4}$, where **DRM** is the last directory entry number. If the media are fixed, then set **CKS** = 0 (no directory records are checked in this case).

Finally, the **OFF** field determines the number of tracks that are skipped at the beginning of the physical disk. This value is automatically added whenever SETTRK is called and can be used as a mechanism for skipping reserved operating system tracks or for partitioning a large disk into smaller segmented sections.

To complete the discussion of the **DPB**, several **DPH**s can address the same **DPB** if their drive characteristics are identical. Further, the **DPB** can be dynamically changed when a new drive is addressed by simply changing the pointer in the **DPH**; because the BDOS copies the **DPB** values to a local area whenever the SELDSK function is invoked.

Returning back to **DPH** for a particular drive, the two address values **CSV** and **ALV** remain. Both addresses reference an area of uninitialized memory following the BIOS. The areas must be unique for each drive, and the size of each area is determined by the values in the **DPB**.

The size of the area addressed by **CSV** is **CKS** bytes, which is sufficient to hold the directory check information for this particular drive, If $\mathbf{CKS} = \frac{\mathbf{DRM}+1}{4}$, you must reserve $\frac{\mathbf{DRM}+1}{4}$ bytes for directory check use. If **CKS** = 0, no storage is reserved.

The size of the area addressed by **ALV** is determined by the maximum number of data blocks allowed for this particular disk and is computed as $\left(\frac{\mathrm{DSM}}{8}\right)+1$.

The CBIOS shown in Appendix B demonstrates an instance of these tables for standard 8-inch, single-density drives. It might be useful to examine this program and compare the tabular values with the definitions given above.

6.11. The DISKDEF Macro Library

A macro library called DISKDEF (shown in Appendix F), greatly simplifies the table construction process. You must have access to the MAC macro assembler, of course, to use the DISKDEF facility, while the macro library is included with all CP/M 2 distribution disks.

A BIOS disk definition consists of the following sequence of macro statements:

```
MACLIB          DISKDEF
.....
DISKS           n
DISKDEF         0,...
DISKDEF         1,...
.....
DISKDEF         n-1
.....
ENDEF
```

where the MACLIB statement loads the DISKDEF.LIB file, on the same disk as the BIOS, into MAC's internal tables. The DISKS macro call follows, which specifies the number of drives to be configured with the user's system, where *n* is an integer in the range 1 to 16. A series of DISKDEF macro calls then follow that define the characteristics of each logical disk, 0 through $n-1$, corresponding to logical drives A through P. The DISKS and DISKDEF macros generate the in-line fixed data tables described in the previous section and thus must be placed in a non-executable portion of the BIOS, typically directly following the BIOS jump vector.

The remaining portion of the BIOS is defined following the DISKDEF macros, with the ENDEF macro call immediately preceding the END statement. The ENDEF (End of DISKDEF) macro generates the necessary uninitialized RAM areas that are located in memory above the BIOS.

The DISKDEF macro call takes the form:

DISKDEF *dn*,*fsc*,*lsc*,[*skf*],*bls dks*,*dir*,*cks*,*ofs*,[0]

where

dn is the logical disk number, 0 to $n-1$.

fsc is the first physical sector number (0 or 1).

lsc is the last sector number.

skf optional sector skew factor

bls is the data allocation block size.

dks is the number of blocks on the disk.

dir is the number of directory entries.

cks is the number of checked directory entries.

ofs is the track offset to logical track 00.

[0] is an optional 1.4 compatibility flag.

The value *dn* is the drive number being defined with this DISKDEF macro invocation. The *fsc* parameter accounts for differing sector numbering systems and is usually 0 to 1. The *lsc* is the last numbered sector on a track. When present, the *skf* parameter defines the sector skew factor, which is used to create a sector translation table according to the skew.

If the number of sectors is less than 256, a single-byte table is created, otherwise each translation table element occupies two bytes. No translation table is created if the *skf* parameter is omitted, or equal to 0.

The *bls* parameter specifies the number of bytes allocated to each data block, and takes on the values 1024, 2048, 4096, 8192, or 16384. Generally, performance increases with larger data block sizes because there are fewer directory references, and logically connected data records are physically close on the disk. Further, each directory entry addresses more data and the BIOS-resident RAM space is reduced.

The *dks* parameter specifies the total disk size in *bls* units. That is, if the *bls* = 2048 and *dks* = 1000, the total disk capacity is 2,048,000 bytes. If *dks* is greater than 255, the block size parameter *bls* must be greater than 1024. The value of *dir* is the total number of directory entries that might exceed 255, if desired.

The *cks* parameter determines the number of directory items to check on each directory scan and is used internally to detect changed disks during system operation, where an intervening cold or warm start has not occurred. When this situation is detected, CP/M automatically marks the disk Read-Only so that data is not subsequently destroyed.

As stated in the previous section, the value of *cks* = *dir* when the medium is easily changed, as is the case with a floppy disk subsystem. If the disk is permanently mounted, the value of *cks* is typically 0, because the probability of changing disks without a restart is low.

The *ofs* value determines the number of tracks to skip when this particular drive is addressed, which can be used to reserve additional operating system space or to simulate several logical drives on a single large capacity physical drive.

Finally, the [0] parameter is included when file compatibility is required with versions of 1.4 that have been modified for higher density disks. This parameter ensures that only 16K is allocated

for each directory record, as was the case for previous versions. Normally, this parameter is not included.

For convenience and economy of table space, the special form:

DISKDEF *i,j*

gives disk *i* the same characteristics as a previously defined drive *j*. A standard four-drive, single-density system, which is compatible with version 1.4, is defined using the following macro invocations:

```
DISKS       4
DISKDEF     0,1,26,6,1024,243,64,2
DISKDEF     1,0
DISKDEF     2,0
DISKDEF     3,0
....
ENDEF
```

with all disks having the same parameter values of 26 sectors per track, numbered 01 through 26, with 6 sectors skipped between each access, 1024 bytes per data block, 243 data blocks for a total of 243K-byte disk capacity, 64 checked directory entries, and two operating system tracks.

The DISKS macro generates *n* **DPH**s, starting at the **DPH** table address DPBASE generated by the macro. Each disk header block contains sixteen bytes, as described above, and correspond one-for-one to each of the defined drives. In the four-drive standard system, for example, the DISKS macro generates a table of the form:

```
DPBASE     EQU  $
DPE0:      DW XLT0,0000H,0000H,0000H,DIRBUF,DPB0,CSV0,ALV0
DPE1:      DW XLT0,0000H,0000H,0000H,DIRBUF,DPB0,CSV1,ALV1
DPE2:      DW XLT0,0000H,0000H,0000H,DIRBUF,DPB0,CSV2,ALV2
DPE3:      DW XLT0,0000H,0000H,0000H,DIRBUF,DPB0,CSV3,ALV3
```

where the **DPH** labels are included for reference purposes to show the beginning table addresses for each drive 0 through 3. The values contained within the **DPH** are described in detail in the previous section. The check and allocation vector addresses are generated by the ENDEF macro in the ram area following the BIOS code and tables.

Note that if the *skf* (skew factor) parameter is omitted, or equal to 0, the translation table is omitted and a 0000H value is inserted in the **XLT** position of the **DPH** for the disk. In a subsequent call to perform the logical-to-physical translation, SECTRAN receives a translation table address of DE = 0000H and simply returns the original logical sector from BC in the HL register pair.

A translate table is constructed when the *skf* parameter is present, and the (nonzero) table address is placed into the corresponding **DPH**s. The following for example, is constructed when the standard skew factor *skf* = 6 is specified in the DISKDEF macro call:

```
XLT0:    DB    1,7,13,19,25,5,11,17,23,3,9,15,21
         DB    2,8,14,20,26,6,12,18,24,4,10,16,22
```

Following the ENDEF macro call, a number of uninitialized data areas are defined. These data areas need not be a part of the BIOS that is loaded upon cold start, but must be available between the BIOS and the end of memory. The size of the uninitialized RAM area is determined by EQU statements generated by the ENDEF macro. For a standard four-drive system, the ENDEF macro might produce the following EQU statement:

```
4C72 =        BEGDAT EQU $
              (data areas)

4DB0 =        ENDDAT EQU $

013C =        DATSIZ EQU $-BEGDAT
```

which indicates that uninitialized RAM begins at location 4C72H, ends at 4DB0H-1, and occupies 013CH bytes. You must ensure that these addresses are free for use after the system is loaded.

After modification, you can use the STAT program to check drive characteristics, because STAT uses the disk parameter block to decode the drive information. A STAT command of the form:

STAT *d*:DSK:

decodes the disk parameter block for drive *d* (*d*=A,...,P) and displays the following values:

```
r:  128-byte record capacity
k:  kilobyte drive capacity
d:  32-byte directory entries
c:  checked directory entries
e:  records/extent
b:  records/block
s:  sectors/track
t:  reserved tracks
```

Three examples of DISKDEF macro invocations are shown below with corresponding STAT parameter values. The last example produces a full 8-megabyte system.

```
DISKDEF 0,1,58,,2048,256,128,128,2
```

- r=4096, k=512, d=128, c=128, e=256, b=16, s=58, t=2

```
DISKDEF 0,1,58,,2048,1024,300,0,2
```

- r=16348, k=2048, d=300, c=0, e=128, b=16, s=58, t=2

```
DISKDEF 0,1,58,,16348,512,128,128,2
```

- r=65536, k=8192, d=128, c=128, e=1024, b=128, s=58, t=2

6.12. Sector Blocking and Deblocking

Upon each call to BIOS `WRITE` entry point, the CP/M BDOS includes information that allows effective sector blocking and deblocking where the host disk subsystem has a sector size that is a multiple of the basic 128-byte unit. The purpose here is to present a general-purpose algorithm that can be included within the BIOS and that uses the BDOS information to perform the operations automatically.

On each call to `WRITE`, the BDOS provides the following information in register `C`:

0 normal sector write

1 write to directory sector

2 write to the first sector of a new block

Condition `0` occurs whenever the next write operation is into a previously written area, such as a random mode record update; when the write is to other than the first sector of an unallocated block; or when the write is not into the directory area. Condition `1` occurs when a write into the directory area is performed. Condition `2` occurs when the first record (only) of a newly allocated data block is written. In most cases, application programs read or write multiple 128-byte sectors in sequence; thus, there is little overhead involved in either operation when blocking and deblocking records, because pre-read operations can be avoided when writing records.

Appendix G lists the blocking and deblocking algorithms in skeletal form; this file is included on your CP/M disk. Generally, the algorithms map all CP/M sector read operations onto the host disk through an intermediate buffer that is the size of the host disk sector. Throughout the program, values and variables that relate to the CP/M sector involved in a seek operation are prefixed by `sek`, while those related to the host disk system are prefixed by `hst`. The equate statements beginning on line 29 of Appendix G define the mapping between CP/M and the host system, and must be changed if other than the sample host system is involved.

The entry points `BOOT` and `WBOOT` must contain the initialization code starting on line 57, while the `SELDSK` entry point must be augmented by the code starting on line 65. Note that although the `SELDSK` entry point computes and returns the Disk Parameter Header address, it does not physically select the host disk at this point (it is selected later at `READHST` or `WRITEHST`). Further, `SETTRK`, `SETSEC`, and `SETDMA` simply store the values, but do not take any other action at this point. `SECTRAN` performs a trivial function of returning the physical sector number.

The principal entry points are `READ` and `WRITE`, starting on lines 110 and 125, respectively. These subroutines take the place of your previous `READ` and `WRITE` operations.

The actual physical read or write takes place at either `WRITEHST` or `READHST`, where all values have been prepared: `hstdsk` is the host disk number, `hsttrk` is the host track number, and

`hstsec` is the host sector number, which may require translation to physical sector number. You must insert code at this point that performs the full sector read or write into or out of the buffer at `hstbuf` of length `hstsiz`. All other mapping functions are performed by the algorithms.

This particular algorithm was tested using an 80-megabyte hard disk unit that was originally configured for 128-byte sectors, producing approximately 35 megabytes of formatted storage. When configured for 512-byte host sectors, usable storage increased to 57 megabytes, with a corresponding 400% improvement in overall response. In this situation, there is no apparent overhead involved in deblocking sectors, with the advantage that user programs still maintain 128-byte sectors. This is primarily because of the information provided by the BDOS, which eliminates the necessity for pre-read operations.

A. The MDS Basic I/O System (BIOS)

Note

This appendix consists of a cross-reference listing generated by the XREF utility from the results of assembly with MAC.

```
 1                ; MDS-800 I/O DRIVERS FOR CP/M 2.2
 2                ; (FOUR DRIVE SINGLE DENSITY VERSION)
 3                ;
 4                ; VERSION 2.2 FEBRUARY, 1980
 5                ;
 6   0016 =       VERS  EQU 22  ;VERSION 2.2
 7                ;
 8                ; COPYRIGHT (C) 1980
 9                ; DIGITAL RESEARCH
10                ; BOX 579, PACIFIC GROVE
11                ; CALIFORNIA, 93950
12                ;
13                ;
14   FFFF =       TRUE  EQU 0FFFFH  ;VALUE OF "TRUE"
15   0000 =       FALSE EQU NOT TRUE  ;"FALSE"
16   0000 =       TEST  EQU FALSE ;TRUE IF TEST BIOS
17                ;
18                  IF  TEST
19                BIAS  EQU 03400H  ;BASE OF CCP IN TEST SYSTEM
20                  ENDIF
21                  IF  NOT TEST
22   0000 =       BIAS  EQU 0000H ;GENERATE RELOCATABLE CP/M SYSTEM
23                  ENDIF
24                ;
25   1600 =       PATCH EQU 1600H
26                ;
27   1600           ORG PATCH
28   0000 =       CPMB  EQU $-PATCH ;BASE OF CPM CONSOLE PROCESSOR
29   0806 =       BDOS  EQU 806H+CPMB ;BASIC DOS (RESIDENT PORTION)
30   1600 =       CPML  EQU $-CPMB  ;LENGTH (IN BYTES) OF CPM SYSTEM
31   002C =       NSECTS  EQU CPML/128  ;NUMBER OF SECTORS TO LOAD
32   0002 =       OFFSET  EQU 2 ;NUMBER OF DISK TRACKS USED BY CP/M
33   0004 =       CDISK EQU 0004H ;ADDRESS OF LAST LOGGED DISK ON WARM START
34   0080 =       BUFF  EQU 0080H ;DEFAULT BUFFER ADDRESS
35   000A =       RETRY EQU 10  ;MAX RETRIES ON DISK I/O BEFORE ERROR
36                ;
37                ; PERFORM FOLLOWING FUNCTIONS
38                ; BOOT  COLD START
39                ; WBOOT WARM START (SAVE I/O BYTE)
40                ; (BOOT AND WBOOT ARE THE SAME FOR MDS)
41                ; CONST CONSOLE STATUS
42                ;   REG-A = 00 IF NO CHARACTER READY
43                ;   REG-A = FF IF CHARACTER READY
44                ; CONIN CONSOLE CHARACTER IN (RESULT IN REG-A)
```

```
 45                 ; CONOUT  CONSOLE CHARACTER OUT (CHAR IN REG-C)
 46                 ; LIST  LIST OUT (CHAR IN REG-C)
 47                 ; PUNCH PUNCH OUT (CHAR IN REG-C)
 48                 ; READER  PAPER TAPE READER IN (RESULT TO REG-A)
 49                 ; HOME  MOVE TO TRACK 00
 50                 ;
 51                 ; (THE FOLLOWING CALLS SET-UP THE IO PARAMETER BLOCK FOR THE
 52                 ; MDS, WHICH IS USED TO PERFORM SUBSEQUENT READS AND WRITES)
 53                 ; SELDSK  SELECT DISK GIVEN BY REG-C (0,1,2...)
 54                 ; SETTRK  SET TRACK ADDRESS (0,...76) FOR SUBSEQUENT READ/WRITE
 55                 ; SETSEC  SET SECTOR ADDRESS (1,...,26) FOR SUBSEQUENT READ/WRITE
 56                 ; SETDMA  SET SUBSEQUENT DMA ADDRESS (INITIALLY 80H)
 57                 ;
 58                 ; (READ AND WRITE ASSUME PREVIOUS CALLS TO SET UP THE IO PARAMETERS)
 59                 ; READ  READ TRACK/SECTOR TO PRESET DMA ADDRESS
 60                 ; WRITE WRITE TRACK/SECTOR FROM PRESET DMA ADDRESS
 61                 ;
 62                 ; JUMP VECTOR FOR INDIVIUAL ROUTINES
 63   1600 C3B316     JMP BOOT
 64   1603 C3C316   WBOOTE: JMP WBOOT
 65   1606 C36117     JMP CONST
 66   1609 C36417     JMP CONIN
 67   160C C36A17     JMP CONOUT
 68   160F C36D17     JMP LIST
 69   1612 C37217     JMP PUNCH
 70   1615 C37517     JMP READER
 71   1618 C37817     JMP HOME
 72   161B C37D17     JMP SELDSK
 73   161E C3A717     JMP SETTRK
 74   1621 C3AC17     JMP SETSEC
 75   1624 C3BB17     JMP SETDMA
 76   1627 C3C117     JMP READ
 77   162A C3CA17     JMP WRITE
 78   162D C37017     JMP LISTST  ;LIST STATUS
 79   1630 C3B117     JMP SECTRAN
 80                 ;
 81                   MACLIB  DISKDEF ;LOAD THE DISK DEFINITION LIBRARY
 82                   DISKS 4 ;FOUR DISKS
 83   1633+=        DPBASE  EQU $ ;BASE OF DISK PARAMETER BLOCKS
 84   1633+82160000 DPE0: DW  XLT0,0000H  ;TRANSLATE TABLE
 85   1637+00000000   DW  0000H,0000H ;SCRATCH AREA
 86   163B+6E187316   DW  DIRBUF,DPB0 ;DIR BUFF,PARM BLOCK
 87   163F+0D19EE18   DW  CSV0,ALV0 ;CHECK, ALLOC VECTORS
 88   1643+82160000 DPE1: DW  XLT1,0000H  ;TRANSLATE TABLE
 89   1647+00000000   DW  0000H,0000H ;SCRATCH AREA
 90   164B+6E187316   DW  DIRBUF,DPB1 ;DIR BUFF,PARM BLOCK
 91   164F+3C191D19   DW  CSV1,ALV1 ;CHECK, ALLOC VECTORS
 92   1653+82160000 DPE2: DW  XLT2,0000H  ;TRANSLATE TABLE
 93   1657+00000000   DW  0000H,0000H ;SCRATCH AREA
 94   165B+6E187316   DW  DIRBUF,DPB2 ;DIR BUFF,PARM BLOCK
 95   165F+6B194C19   DW  CSV2,ALV2 ;CHECK, ALLOC VECTORS
 96   1663+82160000 DPE3: DW  XLT3,0000H  ;TRANSLATE TABLE
 97   1667+00000000   DW  0000H,0000H ;SCRATCH AREA
 98   166B+6E187316   DW  DIRBUF,DPB3 ;DIR BUFF,PARM BLOCK
 99   166F+9A197B19   DW  CSV3,ALV3 ;CHECK, ALLOC VECTORS
100                   DISKDEF 0,1,26,6,1024,243,64,64,OFFSET
101   1673+=        DPB0  EQU $   ;DISK PARM BLOCK
```

```
102   1673+1A00        DW  26    ;SEC PER TRACK
103   1675+03          DB  3   ;BLOCK SHIFT
104   1676+07          DB  7   ;BLOCK MASK
105   1677+00          DB  0   ;EXTNT MASK
106   1678+F200        DW  242   ;DISK SIZE-1
107   167A+3F00        DW  63    ;DIRECTORY MAX
108   167C+C0          DB  192   ;ALLOC0
109   167D+00          DB  0   ;ALLOC1
110   167E+1000        DW  16    ;CHECK SIZE
111   1680+0200        DW  2   ;OFFSET
112   1682+=         XLT0  EQU $   ;TRANSLATE TABLE
113   1682+01          DB  1
114   1683+07          DB  7
115   1684+0D          DB  13
116   1685+13          DB  19
117   1686+19          DB  25
118   1687+05          DB  5
119   1688+0B          DB  11
120   1689+11          DB  17
121   168A+17          DB  23
122   168B+03          DB  3
123   168C+09          DB  9
124   168D+0F          DB  15
125   168E+15          DB  21
126   168F+02          DB  2
127   1690+08          DB  8
128   1691+0E          DB  14
129   1692+14          DB  20
130   1693+1A          DB  26
131   1694+06          DB  6
132   1695+0C          DB  12
133   1696+12          DB  18
134   1697+18          DB  24
135   1698+04          DB  4
136   1699+0A          DB  10
137   169A+10          DB  16
138   169B+16          DB  22
139                    DISKDEF 1,0
140   1673+=         DPB1  EQU DPB0  ;EQUIVALENT PARAMETERS
141   001F+=         ALS1  EQU ALS0  ;SAME ALLOCATION VECTOR SIZE
142   0010+=         CSS1  EQU CSS0  ;SAME CHECKSUM VECTOR SIZE
143   1682+=         XLT1  EQU XLT0  ;SAME TRANSLATE TABLE
144                    DISKDEF 2,0
145   1673+=         DPB2  EQU DPB0  ;EQUIVALENT PARAMETERS
146   001F+=         ALS2  EQU ALS0  ;SAME ALLOCATION VECTOR SIZE
147   0010+=         CSS2  EQU CSS0  ;SAME CHECKSUM VECTOR SIZE
148   1682+=         XLT2  EQU XLT0  ;SAME TRANSLATE TABLE
149                    DISKDEF 3,0
150   1673+=         DPB3  EQU DPB0  ;EQUIVALENT PARAMETERS
151   001F+=         ALS3  EQU ALS0  ;SAME ALLOCATION VECTOR SIZE
152   0010+=         CSS3  EQU CSS0  ;SAME CHECKSUM VECTOR SIZE
153   1682+=         XLT3  EQU XLT0  ;SAME TRANSLATE TABLE
154                  ; ENDEF OCCURS AT END OF ASSEMBLY
155                  ;
156                  ; END OF CONTROLLER - INDEPENDENT CODE, THE REMAINING SUBROUTINES
157                  ; ARE TAILORED TO THE PARTICULAR OPERATING ENVIRONMENT, AND MUST
158                  ; BE ALTERED FOR ANY SYSTEM WHICH DIFFERS FROM THE INTEL MDS.
```

```
159                     ;
160                     ; THE FOLLOWING CODE ASSUMES THE MDS MONITOR EXISTS AT 0F800H
161                     ; AND USES THE I/O SUBROUTINES WITHIN THE MONITOR
162                     ;
163                     ; WE ALSO ASSUME THE MDS SYSTEM HAS FOUR DISK DRIVES
164   00FD =            REVRT EQU 0FDH  ;INTERRUPT REVERT PORT
165   00FC =            INTC  EQU 0FCH  ;INTERRUPT MASK PORT
166   00F3 =            ICON  EQU 0F3H  ;INTERRUPT CONTROL PORT
167   007E =            INTE  EQU 0111$1110B  ;ENABLE RST 0(WARM BOOT), RST 7 (MONITOR)
168                     ;
169                     ; MDS MONITOR EQUATES
170   F800 =            MON80 EQU 0F800H  ;MDS MONITOR
171   FF0F =            RMON80  EQU 0FF0FH  ;RESTART MON80 (BOOT ERROR)
172   F803 =            CI  EQU 0F803H  ;CONSOLE CHARACTER TO REG-A
173   F806 =            RI  EQU 0F806H  ;READER IN TO REG-A
174   F809 =            CO  EQU 0F809H  ;CONSOLE CHAR FROM C TO CONSOLE OUT
175   F80C =            PO  EQU 0F80CH  ;PUNCH CHAR FROM C TO PUNCH DEVICE
176   F80F =            LO  EQU 0F80FH  ;LIST FROM C TO LIST DEVICE
177   F812 =            CSTS  EQU 0F812H  ;CONSOLE STATUS 00/FF TO REGISTER A
178                     ;
179                     ; DISK PORTS AND COMMANDS
180   0078 =            BASE  EQU 78H ;BASE OF DISK COMMAND IO PORTS
181   0078 =            DSTAT EQU BASE  ;DISK STATUS (INPUT)
182   0079 =            RTYPE EQU BASE+1  ;RESULT TYPE (INPUT)
183   007B =            RBYTE EQU BASE+3  ;RESULT BYTE (INPUT)
184                     ;
185   0079 =            ILOW  EQU BASE+1  ;IOPB LOW ADDRESS (OUTPUT)
186   007A =            IHIGH EQU BASE+2  ;IOPB HIGH ADDRESS (OUTPUT)
187                     ;
188   0004 =            READF EQU 4H  ;READ FUNCTION
189   0006 =            WRITF EQU 6H  ;WRITE FUNCTION
190   0003 =            RECAL EQU 3H  ;RECALIBRATE DRIVE
191   0004 =            IORDY EQU 4H  ;I/O FINISHED MASK
192   000D =            CR  EQU 0DH ;CARRIAGE RETURN
193   000A =            LF  EQU 0AH ;LINE FEED
194                     ;
195                     SIGNON: ;SIGNON MESSAGE: XXK CP/M VERS Y.Y
196   169C 0D0A0A         DB  CR,LF,LF
197                       IF  TEST
198                       DB  '32'  ;32K EXAMPLE BIOS
199                       ENDIF
200                       IF  NOT TEST
201   169F 3030           DB  '00'  ;MEMORY SIZE FILLED BY RELOCATOR
202                       ENDIF
203   16A1 6B2043502F     DB  'k CP/M vers '
204   16AD 322E32         DB  VERS/10+'0','.',VERS MOD 10+'0'
205   16B0 0D0A00         DB  CR,LF,0
206                     ;
207                     BOOT: ;PRINT SIGNON MESSAGE AND GO TO CCP
208                     ; (NOTE: MDS BOOT INITIALIZED IOBYTE AT 0003H)
209   16B3 310001         LXI SP,BUFF+80H
210   16B6 219C16         LXI H,SIGNON
211   16B9 CDD317         CALL  PRMSG ;PRINT MESSAGE
212   16BC AF             XRA A ;CLEAR ACCUMULATOR
213   16BD 320400         STA CDISK ;SET INITIALLY TO DISK A
214   16C0 C30F17         JMP GOCPM ;GO TO CP/M
215                     ;
```

```
216                 ;
217                 WBOOT:; LOADER ON TRACK 0, SECTOR 1, WHICH WILL BE SKIPPED FOR WARM
218                 ; READ CP/M FROM DISK - ASSUMING THERE IS A 128 BYTE COLD START
219                 ; START.
220                 ;
221   16C3 318000     LXI SP,BUFF ;USING DMA - THUS 80 THRU FF AVAILABLE FOR STACK
222                 ;
223   16C6 0E0A       MVI C,RETRY ;MAX RETRIES
224   16C8 C5         PUSH  B
225                 WBOOT0: ;ENTER HERE ON ERROR RETRIES
226   16C9 010000     LXI B,CPMB  ;SET DMA ADDRESS TO START OF DISK SYSTEM
227   16CC CDBB17     CALL  SETDMA
228   16CF 0E00       MVI C,0 ;BOOT FROM DRIVE 0
229   16D1 CD7D17     CALL  SELDSK
230   16D4 0E00       MVI C,0
231   16D6 CDA717     CALL  SETTRK  ;START WITH TRACK 0
232   16D9 0E02       MVI C,2 ;START READING SECTOR 2
233   16DB CDAC17     CALL  SETSEC
234                 ;
235                 ; READ SECTORS, COUNT NSECTS TO ZERO
236   16DE C1         POP B ;10-ERROR COUNT
237   16DF 062C       MVI B,NSECTS
238                 RDSEC:  ;READ NEXT SECTOR
239   16E1 C5         PUSH  B ;SAVE SECTOR COUNT
240   16E2 CDC117     CALL  READ
241   16E5 C24917     JNZ BOOTERR ;RETRY IF ERRORS OCCUR
242   16E8 2A6C18     LHLD  IOD ;INCREMENT DMA ADDRESS
243   16EB 118000     LXI D,128 ;SECTOR SIZE
244   16EE 19         DAD D ;INCREMENTED DMA ADDRESS IN HL
245   16EF 44         MOV B,H
246   16F0 4D         MOV C,L ;READY FOR CALL TO SET DMA
247   16F1 CDBB17     CALL  SETDMA
248   16F4 3A6B18     LDA IOS ;SECTOR NUMBER JUST READ
249   16F7 FE1A       CPI 26  ;READ LAST SECTOR?
250   16F9 DA0517     JC  RD1
251                 ; MUST BE SECTOR 26, ZERO AND GO TO NEXT TRACK
252   16FC 3A6A18     LDA IOT ;GET TRACK TO REGISTER A
253   16FF 3C         INR A
254   1700 4F         MOV C,A ;READY FOR CALL
255   1701 CDA717     CALL  SETTRK
256   1704 AF         XRA A ;CLEAR SECTOR NUMBER
257   1705 3C       RD1:  INR A ;TO NEXT SECTOR
258   1706 4F         MOV C,A ;READY FOR CALL
259   1707 CDAC17     CALL  SETSEC
260   170A C1         POP B ;RECALL SECTOR COUNT
261   170B 05         DCR B ;DONE?
262   170C C2E116     JNZ RDSEC
263                 ;
264                 ; DONE WITH THE LOAD, RESET DEFAULT BUFFER ADDRESS
265                 GOCPM:  ;(ENTER HERE FROM COLD START BOOT)
266                 ; ENABLE RST0 AND RST7
267   170F F3         DI
268   1710 3E12       MVI A,12H ;INITIALIZE COMMAND
269   1712 D3FD       OUT REVRT
270   1714 AF         XRA A
271   1715 D3FC       OUT INTC  ;CLEARED
272   1717 3E7E       MVI A,INTE  ;RST0 AND RST7 BITS ON
```

```
273   1719 D3FC        OUT INTC
274   171B AF          XRA A
275   171C D3F3        OUT ICON  ;INTERRUPT CONTROL
276                  ;
277                  ; SET DEFAULT BUFFER ADDRESS TO 80H
278   171E 018000      LXI B,BUFF
279   1721 CDBB17      CALL  SETDMA
280                  ;
281                  ; RESET MONITOR ENTRY POINTS
282   1724 3EC3        MVI A,JMP
283   1726 320000      STA 0
284   1729 210316      LXI H,WBOOTE
285   172C 220100      SHLD  1 ;JMP WBOOT AT LOCATION 00
286   172F 320500      STA 5
287   1732 210608      LXI H,BDOS
288   1735 220600      SHLD  6 ;JMP BDOS AT LOCATION 5
289                    IF  NOT TEST
290   1738 323800      STA 7*8 ;JMP TO MON80 (MAY HAVE BEEN CHANGED BY DDT)
291   173B 2100F8      LXI H,MON80
292   173E 223900      SHLD  7*8+1
293                    ENDIF
294                  ; LEAVE IOBYTE SET
295                  ; PREVIOUSLY SELECTED DISK WAS B, SEND PARAMETER TO CPM
296   1741 3A0400      LDA CDISK ;LAST LOGGED DISK NUMBER
297   1744 4F          MOV C,A ;SEND TO CCP TO LOG IT IN
298   1745 FB          EI
299   1746 C30000      JMP CPMB
300                  ;
301                  ; ERROR CONDITION OCCURRED, PRINT MESSAGE AND RETRY
302                  BOOTERR:
303   1749 C1          POP B ;RECALL COUNTS
304   174A 0D          DCR C
305   174B CA5217      JZ  BOOTER0
306                  ; TRY AGAIN
307   174E C5          PUSH  B
308   174F C3C916      JMP WBOOT0
309                  ;
310                  BOOTER0:
311                  ; OTHERWISE TOO MANY RETRIES
312   1752 215B17      LXI H,BOOTMSG
313   1755 CDD317      CALL  PRMSG
314   1758 C30FFF      JMP RMON80  ;MDS HARDWARE MONITOR
315                  ;
316                  BOOTMSG:
317   175B 3F626F6F74  DB  '?boot',0
318                  ;
319                  ;
320                  CONST:  ;CONSOLE STATUS TO REG-A
321                  ; (EXACTLY THE SAME AS MDS CALL)
322   1761 C312F8      JMP CSTS
323                  ;
324                  CONIN:  ;CONSOLE CHARACTER TO REG-A
325   1764 CD03F8      CALL  CI
326   1767 E67F        ANI 7FH ;REMOVE PARITY BIT
327   1769 C9          RET
328                  ;
329                  CONOUT: ;CONSOLE CHARACTER FROM C TO CONSOLE OUT
```

```
330   176A C309F8      JMP CO
331                    ;
332                    LIST: ;LIST DEVICE OUT
333                    ; (EXACTLY THE SAME AS MDS CALL)
334   176D C30FF8      JMP LO
335                    ;
336                    LISTST:
337                      ;RETURN LIST STATUS
338   1770 AF          XRA A
339   1771 C9          RET   ;ALWAYS NOT READY
340                    ;
341                    PUNCH:  ;PUNCH DEVICE OUT
342                    ; (EXACTLY THE SAME AS MDS CALL)
343   1772 C30CF8      JMP PO
344                    ;
345                    READER: ;READER CHARACTER IN TO REG-A
346                    ; (EXACTLY THE SAME AS MDS CALL)
347   1775 C306F8      JMP RI
348                    ;
349                    HOME: ;MOVE TO HOME POSITION
350                    ; TREAT AS TRACK 00 SEEK
351   1778 0E00        MVI C,0
352   177A C3A717      JMP SETTRK
353                    ;
354                    SELDSK: ;SELECT DISK GIVEN BY REGISTER C
355   177D 210000      LXI H,0000H ;RETURN 0000 IF ERROR
356   1780 79          MOV A,C
357   1781 FE04        CPI NDISKS  ;TOO LARGE?
358   1783 D0          RNC   ;LEAVE HL = 0000
359                    ;
360   1784 E602        ANI 10B ;00 00 FOR DRIVE 0,1 AND 10 10 FOR DRIVE 2,3
361   1786 326618      STA DBANK ;TO SELECT DRIVE BANK
362   1789 79          MOV A,C ;00, 01, 10, 11
363   178A E601        ANI 1B  ;MDS HAS 0,1 AT 78, 2,3 AT 88
364   178C B7          ORA A ;RESULT 00?
365   178D CA9217      JZ  SETDRIVE
366   1790 3E30        MVI A,00110000B ;SELECTS DRIVE 1 IN BANK
367                    SETDRIVE:
368   1792 47          MOV B,A ;SAVE THE FUNCTION
369   1793 216818      LXI H,IOF ;IO FUNCTION
370   1796 7E          MOV A,M
371   1797 E6CF        ANI 11001111B ;MASK OUT DISK NUMBER
372   1799 B0          ORA B ;MASK IN NEW DISK NUMBER
373   179A 77          MOV M,A ;SAVE IT IN IOPB
374   179B 69          MOV L,C
375   179C 2600        MVI H,0 ;HL=DISK NUMBER
376   179E 29          DAD H ;*2
377   179F 29          DAD H ;*4
378   17A0 29          DAD H ;*8
379   17A1 29          DAD H ;*16
380   17A2 113316      LXI D,DPBASE
381   17A5 19          DAD D ;HL=DISK HEADER TABLE ADDRESS
382   17A6 C9          RET
383                    ;
384                    ;
385                    SETTRK: ;SET TRACK ADDRESS GIVEN BY C
386   17A7 216A18      LXI H,IOT
```

```
387   17AA 71          MOV M,C
388   17AB C9          RET
389                    ;
390                    SETSEC: ;SET SECTOR NUMBER GIVEN BY C
391   17AC 216B18      LXI H,IOS
392   17AF 71          MOV M,C
393   17B0 C9          RET
394                    SECTRAN:
395                        ;TRANSLATE SECTOR BC USING TABLE AT DE
396   17B1 0600        MVI B,0 ;DOUBLE PRECISION SECTOR NUMBER IN BC
397   17B3 EB          XCHG    ;TRANSLATE TABLE ADDRESS TO HL
398   17B4 09          DAD B ;TRANSLATE(SECTOR) ADDRESS
399   17B5 7E          MOV A,M ;TRANSLATED SECTOR NUMBER TO A
400   17B6 326B18      STA IOS
401   17B9 6F          MOV L,A ;RETURN SECTOR NUMBER IN L
402   17BA C9          RET
403                    ;
404                    SETDMA: ;SET DMA ADDRESS GIVEN BY REGS B,C
405   17BB 69          MOV L,C
406   17BC 60          MOV H,B
407   17BD 226C18      SHLD  IOD
408   17C0 C9          RET
409                    ;
410                    READ: ;READ NEXT DISK RECORD (ASSUMING DISK/TRK/SEC/DMA SET)
411   17C1 0E04        MVI C,READF ;SET TO READ FUNCTION
412   17C3 CDE017      CALL  SETFUNC
413   17C6 CDF017      CALL  WAITIO  ;PERFORM READ FUNCTION
414   17C9 C9          RET   ;MAY HAVE ERROR SET IN REG-A
415                    ;
416                    ;
417                    WRITE:  ;DISK WRITE FUNCTION
418   17CA 0E06        MVI C,WRITF
419   17CC CDE017      CALL  SETFUNC ;SET TO WRITE FUNCTION
420   17CF CDF017      CALL  WAITIO
421   17D2 C9          RET   ;MAY HAVE ERROR SET
422                    ;
423                    ;
424                    ; UTILITY SUBROUTINES
425                    PRMSG:  ;PRINT MESSAGE AT H,L TO 0
426   17D3 7E          MOV A,M
427   17D4 B7          ORA A ;ZERO?
428   17D5 C8          RZ
429                    ; MORE TO PRINT
430   17D6 E5          PUSH  H
431   17D7 4F          MOV C,A
432   17D8 CD6A17      CALL  CONOUT
433   17DB E1          POP H
434   17DC 23          INX H
435   17DD C3D317      JMP PRMSG
436                    ;
437                    SETFUNC:
438                    ; SET FUNCTION FOR NEXT I/O (COMMAND IN REG-C)
439   17E0 216818      LXI H,IOF ;IO FUNCTION ADDRESS
440   17E3 7E          MOV A,M ;GET IT TO ACCUMULATOR FOR MASKING
441   17E4 E6F8        ANI 11111000B ;REMOVE PREVIOUS COMMAND
442   17E6 B1          ORA C ;SET TO NEW COMMAND
443   17E7 77          MOV M,A ;REPLACED IN IOPB
```

```
                ; THE MDS-800 CONTROLLER REQUIRES DISK BANK BIT IN SECTOR BYTE
                ; MASK THE BIT FROM THE CURRENT I/O FUNCTION
17E8 E620         ANI 00100000B ;MASK THE DISK SELECT BIT
17EA 216B18       LXI H,IOS   ;ADDRESS THE SECTOR SELECT BYTE
17ED B6           ORA M   ;SELECT PROPER DISK BANK
17EE 77           MOV M,A   ;SET DISK SELECT BIT ON/OFF
17EF C9           RET
                ;
                WAITIO:
17F0 0E0A         MVI C,RETRY ;MAX RETRIES BEFORE PERM ERROR
                REWAIT:
                ; START THE I/O FUNCTION AND WAIT FOR COMPLETION
17F2 CD3F18       CALL  INTYPE  ;IN RTYPE
17F5 CD4C18       CALL  INBYTE  ;CLEARS THE CONTROLLER
                ;
17F8 3A6618       LDA DBANK   ;SET BANK FLAGS
17FB B7           ORA A   ;ZERO IF DRIVE 0,1 AND NZ IF 2,3
17FC 3E67         MVI A,IOPB AND 0FFH ;LOW ADDRESS FOR IOPB
17FE 0618         MVI B,IOPB SHR 8  ;HIGH ADDRESS FOR IOPB
1800 C20B18       JNZ IODR1 ;DRIVE BANK 1?
1803 D379         OUT ILOW    ;LOW ADDRESS TO CONTROLLER
1805 78           MOV A,B
1806 D37A         OUT IHIGH ;HIGH ADDRESS
1808 C31018       JMP WAIT0   ;TO WAIT FOR COMPLETE
                ;
                IODR1:  ;DRIVE BANK 1
180B D389         OUT ILOW+10H  ;88 FOR DRIVE BANK 10
180D 78           MOV A,B
180E D38A         OUT IHIGH+10H
                ;
1810 CD5918     WAIT0:  CALL  INSTAT    ;WAIT FOR COMPLETION
1813 E604         ANI IORDY   ;READY?
1815 CA1018       JZ  WAIT0
                ;
                ; CHECK IO COMPLETION OK
1818 CD3F18       CALL  INTYPE    ;MUST BE IO COMPLETE (00) UNLINKED
                ; 00 UNLINKED I/O COMPLETE,    01 LINKED I/O COMPLETE (NOT USED)
                ; 10 DISK STATUS CHANGED       11 (NOT USED)
181B FE02         CPI 10B   ;READY STATUS CHANGE?
181D CA3218       JZ  WREADY
                ;
                ; MUST BE 00 IN THE ACCUMULATOR
1820 B7           ORA A
1821 C23818       JNZ WERROR    ;SOME OTHER CONDITION, RETRY
                ;
                ; CHECK I/O ERROR BITS
1824 CD4C18       CALL  INBYTE
1827 17           RAL
1828 DA3218       JC  WREADY    ;UNIT NOT READY
182B 1F           RAR
182C E6FE         ANI 11111110B ;ANY OTHER ERRORS?  (DELETED DATA OK)
182E C23818       JNZ WERROR
                ;
                ; READ OR WRITE IS OK, ACCUMULATOR CONTAINS ZERO
1831 C9           RET
                ;
                WREADY: ;NOT READY, TREAT AS ERROR FOR NOW
```

```
501   1832 CD4C18      CALL  INBYTE    ;CLEAR RESULT BYTE
502   1835 C33818      JMP TRYCOUNT
503                    ;
504                    WERROR: ;RETURN HARDWARE MALFUNCTION (CRC, TRACK, SEEK, ETC.)
505                    ; THE MDS CONTROLLER HAS RETURNED A BIT IN EACH POSITION
506                    ; OF THE ACCUMULATOR, CORRESPONDING TO THE CONDITIONS:
507                    ; 0 - DELETED DATA (ACCEPTED AS OK ABOVE)
508                    ; 1 - CRC ERROR
509                    ; 2 - SEEK ERROR
510                    ; 3 - ADDRESS ERROR (HARDWARE MALFUNCTION)
511                    ; 4 - DATA OVER/UNDER FLOW (HARDWARE MALFUNCTION)
512                    ; 5 - WRITE PROTECT (TREATED AS NOT READY)
513                    ; 6 - WRITE ERROR (HARDWARE MALFUNCTION)
514                    ; 7 - NOT READY
515                    ; (ACCUMULATOR BITS ARE NUMBERED 7 6 5 4 3 2 1 0)
516                    ;
517                    ; IT MAY BE USEFUL TO FILTER OUT THE VARIOUS CONDITIONS,
518                    ; BUT WE WILL GET A PERMANENT ERROR MESSAGE IF IT IS NOT
519                    ; RECOVERABLE.  IN ANY CASE, THE NOT READY CONDITION IS
520                    ; TREATED AS A SEPARATE CONDITION FOR LATER IMPROVEMENT
521                    TRYCOUNT:
522                    ; REGISTER C CONTAINS RETRY COUNT, DECREMENT 'TIL ZERO
523   1838 0D          DCR C
524   1839 C2F217      JNZ REWAIT  ;FOR ANOTHER TRY
525                    ;
526                    ; CANNOT RECOVER FROM ERROR
527   183C 3E01        MVI A,1 ;ERROR CODE
528   183E C9          RET
529                    ;
530                    ; INTYPE, INBYTE, INSTAT READ DRIVE BANK 00 OR 10
531   183F 3A6618      INTYPE: LDA DBANK
532   1842 B7          ORA A
533   1843 C24918      JNZ INTYP1  ;SKIP TO BANK 10
534   1846 DB79        IN  RTYPE
535   1848 C9          RET
536   1849 DB89        INTYP1: IN  RTYPE+10H ;78 FOR 0,1  88 FOR 2,3
537   184B C9          RET
538                    ;
539   184C 3A6618      INBYTE: LDA DBANK
540   184F B7          ORA A
541   1850 C25618      JNZ INBYT1
542   1853 DB7B        IN  RBYTE
543   1855 C9          RET
544   1856 DB8B        INBYT1: IN  RBYTE+10H
545   1858 C9          RET
546                    ;
547   1859 3A6618      INSTAT: LDA DBANK
548   185C B7          ORA A
549   185D C26318      JNZ INSTA1
550   1860 DB78        IN  DSTAT
551   1862 C9          RET
552   1863 DB88        INSTA1: IN  DSTAT+10H
553   1865 C9          RET
554                    ;
555                    ;
556                    ;
557                    ; DATA AREAS (MUST BE IN RAM)
```

```
   558   1866 00        DBANK:  DB  0 ;DISK BANK 00 IF DRIVE 0,1
   559                          ;    10 IF DRIVE 2,3
   560                  IOPB: ;IO PARAMETER BLOCK
   561   1867 80          DB  80H ;NORMAL I/O OPERATION
   562   1868 04        IOF:  DB  READF ;IO FUNCTION, INITIAL READ
   563   1869 01        ION:  DB  1 ;NUMBER OF SECTORS TO READ
   564   186A 02        IOT:  DB  OFFSET  ;TRACK NUMBER
   565   186B 01        IOS:  DB  1 ;SECTOR NUMBER
   566   186C 8000      IOD:  DW  BUFF  ;IO ADDRESS
   567                  ;
   568                  ;
   569                  ; DEFINE RAM AREAS FOR BDOS OPERATION
   570                    ENDEF
   571   186E+=         BEGDAT  EQU $
   572   186E+          DIRBUF: DS  128 ;DIRECTORY ACCESS BUFFER
   573   18EE+          ALV0: DS  31
   574   190D+          CSV0: DS  16
   575   191D+          ALV1: DS  31
   576   193C+          CSV1: DS  16
   577   194C+          ALV2: DS  31
   578   196B+          CSV2: DS  16
   579   197B+          ALV3: DS  31
   580   199A+          CSV3: DS  16
   581   19AA+=         ENDDAT  EQU $
   582   013C+=         DATSIZ  EQU $-BEGDAT
   583   19AA             END
ALS1            001F   141#
ALS2            001F   146#
ALS3            001F   151#
ALV0            18EE    87   573#
ALV1            191D    91   575#
ALV2            194C    95   577#
ALV3            197B    99   579#
BASE            0078   180#  181   182   183   185   186
BDOS            0806    29#  287
BEGDAT          186E   571#  582
BIAS            0000    19#   22#
BOOT            16B3    63   207#
BOOTER0         1752   305   310#
BOOTERR         1749   241   302#
BOOTMSG         175B   312   316#
BUFF            0080    34#  209   221   278   566
CDISK           0004    33#  213   296
CI              F803   172#  325
CO              F809   174#  330
CONIN           1764    66   324#
CONOUT          176A    67   329#  432
CONST           1761    65   320#
CPMB            0000    28#   29    30   226   299
CPML            1600    30#   31
CR              000D   192#  196   205
CSS1            0010   142#
CSS2            0010   147#
CSS3            0010   152#
CSTS            F812   177#  322
CSV0            190D    87   574#
CSV1            193C    91   576#
```

```
CSV2            196B    95   578#
CSV3            199A    99   580#
DATSIZ          013C   582#
DBANK           1866   361   459   531   539   547   558#
DIRBUF          186E    86    90    94    98   572#
DPB0            1673    86   101#  140   145   150
DPB1            1673    90   140#
DPB2            1673    94   145#
DPB3            1673    98   150#
DPBASE          1633    83#  380
DPE0            1633    84#
DPE1            1643    88#
DPE2            1653    92#
DPE3            1663    96#
DSTAT           0078   181#  550   552
ENDDAT          19AA   581#
FALSE           0000    15#   16
GOCPM           170F   214   265#
HOME            1778    71   349#
ICON            00F3   166#  275
IHIGH           007A   186#  466   472
ILOW            0079   185#  464   470
INBYT1          1856   541   544#
INBYTE          184C   457   490   501   539#
INSTA1          1863   549   552#
INSTAT          1859   474   547#
INTC            00FC   165#  271   273
INTE            007E   167#  272
INTYP1          1849   533   536#
INTYPE          183F   456   479   531#
IOD             186C   242   407   566#
IODR1           180B   463   469#
IOF             1868   369   439   562#
ION             1869   563#
IOPB            1867   461   462   560#
IORDY           0004   191#  475
IOS             186B   248   391   400   447   565#
IOT             186A   252   386   564#
LF              000A   193#  196   196   205
LIST            176D    68   332#
LISTST          1770    78   336#
LO              F80F   176#  334
MON80           F800   170#  291
NSECTS          002C    31#  237
OFFSET          0002    32#  100   564
PATCH           1600    25#   27    28
PO              F80C   175#  343
PRMSG           17D3   211   313   425#  435
PUNCH           1772    69   341#
RBYTE           007B   183#  542   544
RD1             1705   250   257#
RDSEC           16E1   238#  262
READ            17C1    76   240   410#
READER          1775    70   345#
READF           0004   188#  411   562
RECAL           0003   190#
RETRY           000A    35#  223   453
```

```
REVRT           00FD   164#  269
REWAIT          17F2   454#  524
RI              F806   173#  347
RMON80          FF0F   171#  314
RTYPE           0079   182#  534   536
SECTRAN         17B1    79   394#
SELDSK          177D    72   229   354#
SETDMA          17BB    75   227   247   279   404#
SETDRIVE        1792   365   367#
SETFUNC         17E0   412   419   437#
SETSEC          17AC    74   233   259   390#
SETTRK          17A7    73   231   255   352   385#
SIGNON          169C   195#  210
TEST            0000    16#   18    21   197   200   289
TRUE            FFFF    14#   15
TRYCOUNT        1838   502   521#
VERS            0016     6#  204   204
WAIT0           1810   467   474#  476
WAITIO          17F0   413   420   452#
WBOOT           16C3    64   217#
WBOOT0          16C9   225#  308
WBOOTE          1603    64#  284
WERROR          1838   487   495   504#
WREADY          1832   483   492   500#
WRITE           17CA    77   417#
WRITF           0006   189#  418
XLT0            1682    84   112#  143   148   153
XLT1            1682    88   143#
XLT2            1682    92   148#
XLT3            1682    96   153#
```

B. A Skeletal CBIOS

Note

This appendix consists of a cross-reference listing generated by the XREF utility from the results of assembly with MAC.

```
 1                  ; SKELETAL CBIOS FOR FIRST LEVEL OF CP/M 2.0 ALTERATION
 2                  ;
 3   0014 =         MSIZE EQU 20    ;CP/M VERSION MEMORY SIZE IN KILOBYTES
 4                  ;
 5                  ; "BIAS" IS ADDRESS OFFSET FROM 3400H FOR MEMORY SYSTEMS
 6                  ; THAN 16K (REFERRED TO AS"B" THROUGHOUT THE TEXT)
 7                  ;
 8   0000 =         BIAS  EQU (MSIZE-20)*1024
 9   3400 =         CCP EQU 3400H+BIAS  ;BASE OF CCP
10   3C06 =         BDOS  EQU CCP+806H  ;BASE OF BDOS
11   4A00 =         BIOS  EQU CCP+1600H ;BASE OF BIOS
12   0004 =         CDISK EQU 0004H   ;CURRENT DISK NUMBER 0=A,... L5=P
13   0003 =         IOBYTE  EQU 0003H   ;INTEL I/O BYTE
14                  ;
15   4A00             ORG BIOS    ;ORIGIN OF THIS PROGRAM
16   002C =         NSECTS  EQU ($-CCP)/128 ;WARM START SECTOR COUNT
17                  ;
18                  ; JUMP VECTOR FOR INDIVIDUAL SUBROUTINES
19                  ;
20   4A00 C39C4A      JMP BOOT  ;COLD START
21   4A03 C3A64A    WBOOTE: JMP WBOOT ;WARM START
22   4A06 C3114B      JMP CONST ;CONSOLE STATUS
23   4A09 C3244B      JMP CONIN ;CONSOLE CHARACTER IN
24   4A0C C3374B      JMP CONOUT  ;CONSOLE CHARACTER OUT
25   4A0F C3494B      JMP LIST  ;LIST CHARACTER OUT
26   4A12 C34D4B      JMP PUNCH ;PUNCH CHARACTER OUT
27   4A15 C34F4B      JMP READER  ;READER CHARACTER OUT
28   4A18 C3544B      JMP HOME  ;MOVE HEAD TO HOME POSITION
29   4A1B C35A4B      JMP SELDSK  ;SELECT DISK
30   4A1E C37D4B      JMP SETTRK  ;SET TRACK NUMBER
31   4A21 C3924B      JMP SETSEC  ;SET SECTOR NUMBER
32   4A24 C3AD4B      JMP SETDMA  ;SET DMA ADDRESS
33   4A27 C3C34B      JMP READ  ;READ DISK
34   4A2A C3D64B      JMP WRITE ;WRITE DISK
35   4A2D C34B4B      JMP LISTST  ;RETURN LIST STATUS
36   4A30 C3A74B      JMP SECTRAN ;SECTOR TRANSLATE
37                  ;
38                  ; FIXED DATA TABLES FOR FOUR-DRIVE STANDARD
39                  ; IBM-COMPATIBLE 8" DISKS
40                  ;
41                  ; DISK PARAMETER HEADER FOR DISK 00
42   4A33 734A0000  DPBASE: DW  TRANS, 0000H
43   4A37 00000000    DW  0000H, 0000H
44   4A3B F04C8D4A    DW  DIRBF, DPBLK
45   4A3F EC4D704D    DW  CHK00, ALL00
```

```
                ; DISK PARAMETER HEADER FOR DISK 01
4A43 734A0000     DW  TRANS, 0000H
4A47 00000000     DW  0000H, 0000H
4A4B F04C8D4A     DW  DIRBF, DPBLK
4A4F FC4D8F4D     DW  CHK01, ALL01
                ; DISK PARAMETER HEADER FOR DISK 02
4A53 734A0000     DW  TRANS, 0000H
4A57 00000000     DW  0000H, 0000H
4A5B F04C8D4A     DW  DIRBF, DPBLK
4A5F 0C4EAE4D     DW  CHK02, ALL02
                ; DISK PARAMETER HEADER FOR DISK 03
4A63 734A0000     DW  TRANS, 0000H
4A67 00000000     DW  0000H, 0000H
4A6B F04C8D4A     DW  DIRBF, DPBLK
4A6F 1C4ECD4D     DW  CHK03, ALL03
                ;
                ; SECTOR TRANSLATE VECTOR
4A73 01070D13   TRANS:  DB   1,  7, 13, 19  ;SECTORS  1,  2,  3,  4
4A77 19050B11     DB  25,  5, 11, 17  ;SECTORS  5,  6,  7,  6
4A7B 1703090F     DB  23,  3,  9, 15  ;SECTORS  9, 10, 11, 12
4A7F 1502080E     DB  21,  2,  8, 14  ;SECTORS 13, 14, 15, 16
4A83 141A060C     DB  20, 26,  6, 12  ;SECTORS 17, 18, 19, 20
4A87 1218040A     DB  18, 24,  4, 10  ;SECTORS 21, 22, 23, 24
4A8B 1016         DB  16, 22    ;SECTORS 25, 26
                ;
                DPBLK:  ;DISK PARAMETER BLOCK, COMMON TO ALL DISKS
4A8D 1A00         DW  26    ;SECTORS PER TRACK
4A8F 03           DB  3   ;BLOCK SHIFT FACTOR
4A90 07           DB  7   ;BLOCK MASK
4A91 00           DB  0   ;NULL MASK
4A92 F200         DW  242   ;DISK SIZE-1
4A94 3F00         DW  63    ;DIRECTORY MAX
4A96 C0           DB  192   ;ALLOC 0
4A97 00           DB  0   ;ALLOC 1
4A98 1000         DW  16    ;CHECK SIZE
4A9A 0200         DW  2   ;TRACK OFFSET
                ;
                ; END OF FIXED TABLES
                ;
                ; INDIVIDUAL SUBROUTINES TO PERFORM EACH FUNCTION
                BOOT: ;SIMPLEST CASE IS TO JUST PERFORM PARAMETER INITIALIZATION
4A9C AF           XRA A   ;ZERO IN THE ACCUM
4A9D 320300       STA IOBYTE    ;CLEAR THE IOBYTE
4AA0 320400       STA CDISK   ;SELECT DISK ZERO
4AA3 C3EF4A       JMP GOCPM   ;INITIALIZE AND GO TO CP/M
                ;
                WBOOT:  ;SIMPLEST CASE IS TO READ THE DISK UNTIL ALL SECTORS LOADED
4AA6 318000       LXI SP, 80H   ;USE SPACE BELOW BUFFER FOR STACK
4AA9 0E00         MVI C, 0    ;SELECT DISK 0
4AAB CD5A4B       CALL  SELDSK
4AAE CD544B       CALL  HOME    ;GO TO TRACK 00
                ;
4AB1 062C         MVI B, NSECTS ;B COUNTS * OF SECTORS TO LOAD
4AB3 0E00         MVI C, 0    ;C HAS THE CURRENT TRACK NUMBER
4AB5 1602         MVI D, 2    ;D HAS THE NEXT SECTOR TO READ
                ; NOTE THAT WE BEGIN BY READING TRACK 0, SECTOR 2 SINCE SECTOR 1
                ; CONTAINS THE COLD START LOADER, WHICH IS SKIPPED IN A WARM START
```

```
103   4AB7 210034      LXI H, CCP    ;BASE OF CP/M (INITIAL LOAD POINT)
104                    LOAD1:  ;LOAD ONE MORE SECTOR
105   4ABA C5          PUSH  B   ;SAVE SECTOR COUNT, CURRENT TRACK
106   4ABB D5          PUSH  D   ;SAVE NEXT SECTOR TO READ
107   4ABC E5          PUSH  H   ;SAVE DMA ADDRESS
108   4ABD 4A          MOV C, D    ;GET SECTOR ADDRESS TO REGISTER C
109   4ABE CD924B      CALL  SETSEC    ;SET SECTOR ADDRESS FROM REGISTER C
110   4AC1 C1          POP B   ;RECALL DMA ADDRESS TO B, C
111   4AC2 C5          PUSH  B   ;REPLACE ON STACK FOR LATER RECALL
112   4AC3 CDAD4B      CALL  SETDMA    ;SET DMA ADDRESS FROM B, C
113                    ;
114                    ; DRIVE SET TO 0, TRACK SET, SECTOR SET, DMA ADDRESS SET
115   4AC6 CDC34B      CALL  READ
116   4AC9 FE00        CPI 00H   ;ANY ERRORS?
117   4ACB C2A64A      JNZ WBOOT   ;RETRY THE ENTIRE BOOT IF AN ERROR OCCURS
118                    ;
119                    ; NO ERROR, MOVE TO NEXT SECTOR
120   4ACE E1          POP H   ;RECALL DMA ADDRESS
121   4ACF 118000      LXI D, 128    ;DMA=DMA+128
122   4AD2 19          DAD D   ;NEW DMA ADDRESS IS IN H, L
123   4AD3 D1          POP D   ;RECALL SECTOR ADDRESS
124   4AD4 C1          POP B ;RECALL NUMBER OF SECTORS REMAINING, AND CURRENT TRK
125   4AD5 05          DCR B   ;SECTORS=SECTORS-1
126   4AD6 CAEF4A      JZ  GOCPM   ;TRANSFER TO CP/M IF ALL HAVE BEEN LOADED
127                    ;
128                    ; MORE  SECTORS REMAIN TO LOAD, CHECK FOR TRACK CHANGE
129   4AD9 14          INR D
130   4ADA 7A          MOV A,D   ;SECTOR=27?, IF SO, CHANGE TRACKS
131   4ADB FE1B        CPI 27
132   4ADD DABA4A      JC  LOAD1   ;CARRY GENERATED IF SECTOR&lt;27
133                    ;
134                    ; END OF  CURRENT TRACK,  GO TO NEXT TRACK
135   4AE0 1601        MVI D, 1    ;BEGIN WITH FIRST SECTOR OF NEXT TRACK
136   4AE2 0C          INR C   ;TRACK=TRACK+1
137                    ;
138                    ; SAVE  REGISTER STATE, AND CHANGE TRACKS
139   4AE3 C5          PUSH  B
140   4AE4 D5          PUSH  D
141   4AE5 E5          PUSH  H
142   4AE6 CD7D4B      CALL  SETTRK    ;TRACK ADDRESS SET FROM REGISTER C
143   4AE9 E1          POP H
144   4AEA D1          POP D
145   4AEB C1          POP B
146   4AEC C3BA4A      JMP LOAD1   ;FOR ANOTHER SECTOR
147                    ;
148                    ; END OF  LOAD OPERATION, SET PARAMETERS AND GO TO CP/M
149                    GOCPM:
150   4AEF 3EC3        MVI A, 0C3H   ;C3 IS A JMP INSTRUCTION
151   4AF1 320000      STA 0   ;FOR JMP TO WBOOT
152   4AF4 21034A      LXI H, WBOOTE ;WBOOT ENTRY POINT
153   4AF7 220100      SHLD  1   ;SET ADDRESS FIELD FOR JMP AT 0
154                    ;
155   4AFA 320500      STA 5   ;FOR JMP TO BDOS
156   4AFD 21063C      LXI H, BDOS   ;BDOS ENTRY POINT
157   4B00 220600      SHLD  6   ;ADDRESS FIELD OF JUMP AT 5 TO BDOS
158                    ;
159   4B03 018000      LXI B, 80H    ;DEFAULT DMA ADDRESS IS 80H
```

```
160   4B06 CDAD4B      CALL  SETDMA
161                    ;
162   4B09 FB          EI      ;ENABLE THE INTERRUPT SYSTEM
163   4B0A 3A0400      LDA CDISK   ;GET CURRENT DISK NUMBER
164   4B0D 4F          MOV C, A    ;SEND TO THE CCP
165   4B0E C30034      JMP CCP   ;GO TO CP/M FOR FURTHER PROCESSING
166                    ;
167                    ;
168                    ; SIMPLE I/O HANDLERS (MUST BE FILLED IN BY USER)
169                    ; IN EACH CASE, THE ENTRY POINT IS PROVIDED, WITH SPACE RESERVED
170                    ; TO INSERT YOUR OWN CODE
171                    ;
172                    CONST:  ;CONSOLE STATUS, RETURN 0FFH IF CHARACTER READY, 00H IF NOT
173   4B11               DS     10H    ;SPACE FOR STATUS SUBROUTINE
174   4B21 3E00          MVI    A, 00H
175   4B23 C9            RET
176                    ;
177                    CONIN:  ;CONSOLE CHARACTER INTO REGISTER A
178   4B24               DS  10H   ;SPACE FOR INPUT ROUTINE
179   4B34 E67F          ANI 7FH   ;STRIP PARITY BIT
180   4B36 C9            RET
181                    ;
182                    CONOUT: ;CONSOLE CHARACTER OUTPUT FROM REGISTER C
183   4B37 79            MOV A, C    ;GET TO ACCUMULATOR
184   4B38               DS  10H   ;SPACE FOR OUTPUT ROUTINE
185   4B48 C9            RET
186                    ;
187                    LIST: ;LIST CHARACTER FROM REGISTER C
188   4B49 79            MOV A, C      ;CHARACTER TO REGISTER A
189   4B4A C9            RET       ;NULL SUBROUTINE
190                    ;
191                    LISTST: ;RETURN LIST STATUS (0 IF NOT READY, 1 IF READY)
192   4B4B AF            XRA A   ;0 IS ALWAYS OK TO RETURN
193   4B4C C9            RET
194                    ;
195                    PUNCH:  ;PUNCH  CHARACTER FROM  REGISTER C
196   4B4D 79            MOV A, C    ;CHARACTER TO REGISTER A
197   4B4E C9            RET     ;NULL SUBROUTINE
198                    ;
199                    ;
200                    READER: ;READER CHARACTER INTO REGISTER A FROM READER DEVICE
201   4B4F 3E1A          MVI    A, 1AH   ;ENTER END OF FILE FOR NOW (REPLACE LATER)
202   4B51 E67F          ANI    7FH    ;REMEMBER TO STRIP PARITY BIT
203   4B53 C9            RET
204                    ;
205                    ;
206                    ; I/O DRIVERS FOR THE DISK FOLLOW
207                    ; FOR NOW, WE WILL SIMPLY STORE THE PARAMETERS AWAY FOR USE
208                    ; IN THE READ AND WRITE SUBROUTINES
209                    ;
210                    HOME: ;MOVE TO THE TRACK 00 POSITION OF CURRENT DRIVE
211                    ; TRANSLATE THIS CALL INTO A SETTRK CALL WITH PARAMETER 00
212   4B54 0E00          MVI    C, 0   ;SELECT TRACK 0
213   4B56 CD7D4B        CALL   SETTRK
214   4B59 C9            RET     ;WE WILL MOVE TO 00 ON FIRST READ/WRITE
215                    ;
216                    SELDSK: ;SELECT DISK GIVEN BY REGISTER C
```

```
217   4B5A 210000      LXI H, 0000H  ;ERROR RETURN CODE
218   4B5D 79          MOV A, C
219   4B5E 32EF4C      STA DISKNO
220   4B61 FE04        CPI 4   ;MUST BE BETWEEN 0 AND 3
221   4B63 D0          RNC     ;NO CARRY IF 4, 5,...
222                  ; DISK NUMBER IS IN THE PROPER RANGE
223   4B64             DS  10    ;SPACE FOR DISK SELECT
224                  ; COMPUTE PROPER DISK PARAMETER HEADER ADDRESS
225   4B6E 3AEF4C      LDA DISKNO
226   4B71 6F          MOV L, A    ;L=DISK NUMBER 0, 1, 2, 3
227   4B72 2600        MVI H, 0    ;HIGH ORDER ZERO
228   4B74 29          DAD H   ;*2
229   4B75 29          DAD H   ;*4
230   4B76 29          DAD H   ;*8
231   4B77 29          DAD H   ;*16 (SIZE OF EACH HEADER)
232   4B78 11334A      LXI D, DPBASE
233   4B7B 09          DAD 0   ;HL=,DPBASE (DISKNO*16)
234   4B7C C9          RET
235                  ;
236                  SETTRK: ;SET TRACK GIVEN BY REGISTER C
237   4B7D 79          MOV A, C
238   4B7E 32E94C      STA TRACK
239   4B81             DS  10H   ;SPACE FOR TRACK SELECT
240   4B91 C9          RET
241                  ;
242                  SETSEC: ;SET SECTOR GIVEN BY REGISTER C
243   4B92 79          MOV A, C
244   4B93 32EB4C      STA SECTOR
245   4B96             DS  10H   ;SPACE FOR SECTOR SELECT
246   4BA6 C9          RET
247                  ;
248                  ;
249                  SECTRAN:
250                    ;TRANSLATE THE SECTOR GIVEN BY BC USING THE
251                    ;TRANSLATE TABLE GIVEN BY DE
252   4BA7 EB          XCHG      ;HL=.TRANS
253   4BA8 09          DAD B   ;HL=.TRANS (SECTOR)
254   4BA9 6E          MOV L, M    ;L=TRANS (SECTOR)
255   4BAA 2600        MVI H, 0    ;HL=TRANS (SECTOR)
256   4BAC C9          RET     ;WITH VALUE IN HL
257                  ;
258                  SETDMA: ;SET  DMA ADDRESS GIVEN BY REGISTERS B AND C
259   4BAD 69          MOV L, C    ;LOW ORDER ADDRESS
260   4BAE 60          MOV H, B    ;HIGH ORDER ADDRESS
261   4BAF 22ED4C      SHLD  DMAAD   ;SAVE THE ADDRESS
262   4BB2             DS  10H   ;SPACE FOR SETTING THE DMA ADDRESS
263   4BC2 C9          RET
264                  ;
265                  READ: ;PERFORM READ OPERATION (USUALLY THIS IS SIMILAR TO WRITE
266                  ; SO WE WILL ALLOW SPACE TO SET UP READ COMMAND, THEN USE
267                  ; COMMON CODE IN WRITE)
268   4BC3             DS  10H   ;SET UP READ COMMAND
269   4BD3 C3E64B      JMP WAITIO    ;TO PERFORM THE ACTUAL I/O
270                  ;
271                  WRITE:  ;PERFORM A WRITE OPERATION
272   4BD6             DS  10H   ;SET UP WRITE COMMAND
273                  ;
```

```
   274                  WAITIO: ;ENTER  HERE FROM READ  AND WRITE TO PERFORM THE ACTUAL I/O
   275                  ; OPERATION. RETURN A 00H IN REGISTER A IF THE OPERATION COMPLETES
   276                  ; PROPERLY, AND 0LH IF AN ERROR OCCURS DURING THE READ OR WRITE
   277                  ;
   278                  ; IN THIS CASE, WE HAVE SAVED THE DISK NUMBER IN 'DISKNO' (0, 1)
   279                  ;     THE TRACK NUMBER IN 'TRACK' (0-76)
   280                  ;     THE SECTOR NUMBER IN 'SECTOR' (1-26)
   281                  ;     THE DMA ADDRESS IN 'DMAAD' (0-65535)
   282   4BE6             DS  256   ;SPACE RESERVED FOR I/O DRIVERS
   283   4CE6 3E01        MVI A, 1    ;ERROR CONDITION
   284   4CE8 C9          RET     ;REPLACED WHEN FILLED-IN
   285                  ;
   286                  ; THE REMAINDER OF THE CBIOS IS RESERVED UNINITIALIZED
   287                  ; DATA AREA, AND DOES NOT NEED TO BE A PART OF THE
   288                  ; SYSTEM  MEMORY IMAGE (THE SPACE MUST BE AVAILABLE,
   289                  ; HOWEVER, BETWEEN"BEGDAT" AND"ENDDAT").
   290                  ;
   291   4CE9           TRACK:  DS  2   ;TWO BYTES FOR EXPANSION
   292   4CEB           SECTOR: DS  2   ;TWO BYTES FOR EXPANSION
   293   4CED           DMAAD:  DS  2   ;DIRECT MEMORY ADDRESS
   294   4CEF           DISKNO: DS  1   ;DISK NUMBER 0-15
   295                  ;
   296                  ; SCRATCH RAM AREA FOR BDOS USE
   297   4CF0 =         BEGDAT  EQU $   ;BEGINNING OF DATA AREA
   298   4CF0           DIRBF:  DS  128   ;SCRATCH DIRECTORY AREA
   299   4D70           ALL00:  DS  31    ;ALLOCATION VECTOR 0
   300   4D8F           ALL01:  DS  31    ;ALLOCATION VECTOR 1
   301   4DAE           ALL02:  DS  31    ;ALLOCATION VECTOR 2
   302   4DCD           ALL03:  DS  31    ;ALLOCATION VECTOR 3
   303   4DEC           CHK00:  DS  16    ;CHECK VECTOR 0
   304   4DFC           CHK01:  DS  16    ;CHECK VECTOR 1
   305   4E0C           CHK02:  DS  16    ;CHECK VECTOR 2
   306   4E1C           CHK03:  DS  16    ;CHECK VECTOR 3
   307                  ;
   308   4E2C =         ENDDAT  EQU $   ;END OF DATA AREA
   309   013C =         DATSIZ  EQU $-BEGDAT; ;SIZE OF DATA AREA
   310   4E2C             END
ALL00            4D70    45   299#
ALL01            4D8F    50   300#
ALL02            4DAE    55   301#
ALL03            4DCD    60   302#
BDOS             3C06    10#  156
BEGDAT           4CF0   297#  309
BIAS             0000     8#    9
BIOS             4A00    11#   15
BOOT             4A9C    20    86#
CCP              3400     9#   10    11    16   103   165
CDISK            0004    12#   89   163
CHK00            4DEC    45   303#
CHK01            4DFC    50   304#
CHK02            4E0C    55   305#
CHK03            4E1C    60   306#
CONIN            4B24    23   177#
CONOUT           4B37    24   182#
CONST            4B11    22   172#
DATSIZ           013C   309#
DIRBF            4CF0    44    49    54    59   298#
```

```
DISKNO          4CEF   219   225   294#
DMAAD           4CED   261   293#
DPBASE          4A33    42#  232
DPBLK           4A8D    44    49    54    59    71#
ENDDAT          4E2C   308#
GOCPM           4AEF    90   126   149#
HOME            4B54    28    96   210#
IOBYTE          0003    13#   88
LIST            4B49    25   187#
LISTST          4B4B    35   191#
LOAD1           4ABA   104#  132   146
MSIZE           0014     3#    8
NSECTS          002C    16#   98
PUNCH           4B4D    26   195#
READ            4BC3    33   115   265#
READER          4B4F    27   200#
SECTOR          4CEB   244   292#
SECTRAN         4BA7    36   249#
SELDSK          4B5A    29    95   216#
SETDMA          4BAD    32   112   160   258#
SETSEC          4B92    31   109   242#
SETTRK          4B7D    30   142   213   236#
TRACK           4CE9   238   291#
TRANS           4A73    42    47    52    57    63#
WAITIO          4BE6   269   274#
WBOOT           4AA6    21    92#  117
WBOOTE          4A03    21#  152
WRITE           4BD6    34   271#
```

C. A Skeletal GETSYS/PUTSYS Program

Note

This appendix consists of a cross-reference listing generated by the XREF utility from the results of assembly with MAC.

```
 1                ;   COMBINED GETSYS AND PUTSYS PROGRAMS FROM
 2                ;   SEC 6.4
 3                ;
 4                ; START THE PROGRAMS AT THE BASE OF THE TPA
 5   0100           ORG 0100H
 6
 7   0014 =       MSIZE EQU 20      ;SIZE OF CP/M IN KBYTES
 8
 9                ;"BIAS" IS THE AMOUNT TO ADD TO ADDRESSES FOR &gt; 20K
10                ; (REFERRED TO AS"B" THROUGHOUT THE TEXT)
11   0000 =       BIAS  EQU (MSIZE-20)*1024
12   3400 =       CCP EQU 3400H+BIAS
13   3C00 =       BDOS  EQU CCP+0800H
14   4A00 =       BIOS  EQU CCP+1600H
15
16                ; GETSYS PROGRAMS TRACKS 0 AND 1 TO MEMORY AT 3880H + BIAS
17                ; REGISTER       USAGE
18                ; A   (SCRATCH REGISTER)
19                ; B   TRACK COUNT (0...76)
20                ; C   SECTOR COUNT (1...26)
21                ; D,E   (SCRATCH REGISTER PAIR)
22                ; H,L   LOAD ADDRESS
23                ; SP    SET TO TRACK ADDRESS
24
25                GSTART: ;START OF GETSYS
26   0100 318033    LXI SP,CCP-0080H  ;CONVENIENT PLACE
27   0103 218033    LXI H,CCP-0080H ;SET INITIAL LOAD
28   0106 0600      MVI B,0   ;START WITH TRACK
29                RD$TRK: ;READ NEXT TRACK
30   0108 0E01      MVI C,1   ;EACH TRACK START
31                RD$SEC:
32   010A CD0003    CALL  READ$SEC  ;GET THE NEXT SECTOR
33   010D 118000    LXI D,128   ;OFFSET BY ONE SECTOR
34   0110 19        DAD D   ; (HL=HL+128)
35   0111 0C        INR C   ;NEXT SECTOR
36   0112 79        MOV A,C   ;FETCH SECTOR NUMBER
37   0113 FE1B      CPI 27    ;AND SEE IF LAST
38   0115 DA0A01    JC  RDSEC   ;&lt;, DO ONE MORE
39
40                ;ARRIVE HERE AT END OF TRACK, MOVE TO NEXT TRACK
41
42   0118 04        INR B   ;TRACK = TRACK+1
43   0119 78        MOV A,B   ;CHECK FOR LAST
44   011A FE02      CPI 2   ;TRACK = 2 ?
45   011C DA0801    JC  RD$TRK    ;&lt;, DO ANOTHER
```

```
                ;ARRIVE HERE AT END OF LOAD, HALT FOR LACK OF ANYTHING
                ;BETTER

011F FB           EI
0120 76           HLT

                ; PUTSYS PROGRAM, PLACES MEMORY IMAGE
                ; STARTING AT
                ; 3880H + BIAS BACK TO TRACKS 0 AND 1
                ; START THIS PROGRAM AT THE NEXT PAGE BOUNDARY
0200            ORG ($+0100H) AND 0FF00H
                PUT$SYS:
0200 318033       LXI   SP,CCP-0080H  ;CONVENIENT PLACE
0203 218033       LXI   H,CCP-0080H   ;START OF DUMP
0206 0600         MVI   B,0     ;START WITH TRACK
                WR$TRK:
0208 0605         MVI   B,L     ;START WITH SECTOR
                WR$SEC:
020A CD0004       CALL  WRITE$SEC ;WRITE ONE SECTOR
020D 118000       LXI   D,128     ;LENGTH OF EACH
0210 19           DAD D   ;&lt;HL&gt;=&lt;HL&gt; + 128
0211 0C           INR C   ; &lt;C&gt;=&lt;C&gt; + 1
0212 79           MOV A,C   ;SEE IF
0213 FE1B         CPI   27    ;PAST END OF TRACK
0215 DA0A02       JC    WR$SEC    ;NO, DO ANOTHER

                ;ARRIVE HERE AT END OF TRACK, MOVE TO NEXT TRACK

0218 04           INR B   ;TRACK = TRACK+1
0219 78           MOV A,B   ;SEE IF
021A FE02         CPI 2   ;LAST TRACK
021C DA0802       JC  WR$TRK    ;NO, DO ANOTHER


                ; DONE WITH PUTSYS, HALT FOR LACK OF ANYTHING
                ; BETTER
021F FB           EI
0220 76           HLT


                ;USER SUPPLIED SUBROUTINES FOR SECTOR READ AND WRITE

                ; MOVE TO NEXT PAGE BOUNDARY
0300            ORG ($+0100H) AND 0FF00H

                READ$SEC:
                  ;READ THE NEXT SECTOR
                  ;TRACK IN &lt;B&gt;,
                  ;SECTOR IN &lt;C&gt;
                  ;DMAADDR IN&lt;HL&gt;

0300 C5           PUSH  B
0301 E5           PUSH  H

                ;USER DEFINED READ OPERATION GOES HERE
0302            DS  64
```

```
   103   0342 E1          POP H
   104   0343 C1          POP B
   105   0344 C9          RET
   106
   107   0400             ORG ($+100H) AND 0FF00H ;ANOTHER PAGE
   108                          ; BOUNDARY
   109                 WRITE$SEC:
   110
   111                   ;SAME PARAMETERS AS READ$SEC
   112
   113   0400 C5          PUSH  B
   114   0401 E5          PUSH  H
   115
   116                 ;USER DEFINED WRITE OPERATION GOES HERE
   117   0402             DS  64
   118   0442 E1          POP H
   119   0443 C1          POP B
   120   0444 C9          RET
   121
   122                 ;END OF GETSYS/PUTSYS PROGRAM
   123
   124   0445             END
BDOS             3C00    13#
BIAS             0000    11#   12
BIOS             4A00    14#
CCP              3400    12#   13    14    26    27    59    60
GSTART           0100    25#
MSIZE            0014     7#   11
PUTSYS           0200    58#
RDSEC            010A    31#   38
RDTRK            0108    29#   45
READSEC          0300    32    92#
WRITESEC         0400    65   109#
WRSEC            020A    64#   71
WRTRK            0208    62#   78
```

D. The MDS-800 Cold Start Loader for CP/M 2

Note

This appendix consists of a cross-reference listing generated by the XREF utility from the results of assembly with MAC.

```
 1                    TITLE   'mds cold start loader at 3000h'
 2                  ;
 3                  ; MDS-800 COLD START LOADER FOR CP/M 2.0
 4                  ;
 5                  ; VERSION 2.0 AUGUST, 1979
 6                  ;
 7   0000 =         FALSE EQU 0
 8   FFFF =         TRUE  EQU NOT FALSE
 9   0000 =         TESTING EQU FALSE     ;IF TRUE, THEN GO TO MON80 ON  ERRORS
10                  ;
11                    IF  TESTING
12                  BIAS  EQU 03400H
13                    ENDIF
14                    IF  NOT TESTING
15   0000 =         BIAS  EQU 0000H
16                    ENDIF
17   0000 =         CPMB  EQU BIAS    ;BASE OF DOS LOAD
18   0806 =         BDOS  EQU 806H+BIAS ;ENTRY TO DOS FOR CALLS
19   1880 =         BDOSE EQU 1880H+BIAS  ;END OF DOS LOAD
20   1600 =         BOOT  EQU 1600H+BIAS  ;COLD START ENTRY POINT
21   1603 =         RBOOT EQU BOOT+3    ;WARM START ENTRY POINT
22                  ;
23   3000             ORG 03000H    ;LOADED DOWN FROM HARDWARE BOOT AT 3000H
24                  ;
25   1880 =         BDOSL EQU BDOSE-CPMB
26   0002 =         NTRKS EQU 2   ;NUMBER OF TRACKS TO READ
27   0031 =         BDOSS EQU BDOSL/128 ;NUMBER OF SECTORS IN DOS
28   0019 =         BDOSO EQU 25    ;NUMBER OF BDOS SECTORS ON TRACK 0
29   0018 =         BDOS1 EQU BDOSS-BDOSO ;NUMBER OF SECTORS ON TRACK 1
30                  ;
31   F800 =         MON80 EQU 0F800H    ;INTEL MONITOR BASE
32   FF0F =         RMON80  EQU 0FF0FH    ;RESTART LOCATION FOR MON80
33   0078 =         BASE  EQU 078H    ;'BASE' USED BY CONTROLLER
34   0079 =         RTYPE EQU BASE+1    ;RESULT TYPE
35   007B =         RBYTE EQU BASE+3    ;RESULT BYTE
36   007F =         RESET EQU BASE+7    ;RESET CONTROLLER
37                  ;
38                  ;
39   0078 =         DSTAT EQU BASE    ;DISK STATUS PORT
40   0079 =         ILOW  EQU BASE+1    ;LOW IOPB ADDRESS
41   007A =         IHIGH EQU BASE+2    ;HIGH IOPB ADDRESS
42   00FF =         BSW EQU 0FFH    ;BOOT SWITCH
43   0003 =         RECAL EQU 3H    ;RECALIBRATE SELECTED DRIVE
44   0004 =         READF EQU 4H    ;DISK READ FUNCTION
45   0100 =         STACK EQU 100H    ;USE END OF BOOT FOR STACK
```

```
                ;
                RSTART:
3000 310001       LXI SP,STACK; ;IN CASE OF CALL TO MON80
                ; CLEAR DISK STATUS
3003 DB79         IN  RTYPE
3005 DB7B         IN  RBYTE
                ; CHECK IF BOOT SWITCH IS OFF
                COLDSTART:
3007 DBFF         IN  BSW
3009 E602         ANI 02H   ;SWITCH ON?
300B C20730       JNZ COLDSTART
                ; CLEAR THE CONTROLLER
300E D37F         OUT RESET   ;LOGIC CLEARED
                ;
                ;
3010 0602         MVI B,NTRKS   ;NUMBER OF TRACKS TO READ
3012 214230       LXI H,IOPBO
                ;
                START:
                ;
                ; READ FIRST/NEXT TRACK INTO CPMB
3015 7D           MOV A,L
3016 D379         OUT ILOW
3018 7C           MOV A,H
3019 D37A         OUT IHIGH
301B DB78       WAITO:  IN  DSTAT
301D E604         ANI 4
301F CA1B30       JZ  WAITO
                ;
                ; CHECK DISK STATUS
3022 DB79         IN  RTYPE
3024 E603         ANI 11B
3026 FE02         CPI 2
                ;
                  IF  TESTING
                  CNC RMON80    ;GO TO MONITOR IF 11 OR 10
                  ENDIF
                  IF  NOT TESTING
3028 D20030       JNC RSTART    ;RETRY THE LOAD
                  ENDIF
                ;
302B DB7B         IN  RBYTE   ;I/O COMPLETE, CHECK STATUS
                ; IF NOT  READY, THEN GO TO MON80
302D 17           RAL
302E DC0FFF       CC  RMON80    ;NOT READY BIT SET
3031 1F           RAR     ;RESTORE
3032 E61E         ANI 11110B    ;OVERRUN/ADDR ERR/SEEK/CRC/XXXX
                ;
                  IF  TESTING
                  CNZ RMON80    ;GO TO MONITOR
                  ENDIF
                  IF  NOT TESTING
3034 C20030       JNZ RSTART    ;RETRY THE LOAD
                  ENDIF
                ;
                ;
3037 110700       LXI D,IOPBL   ;LENGTH OF IOPB
```

```
  103   303A 19          DAD D   ;ADDRESSING NEXT IOPB
  104   303B 05          DCR B   ;COUNT DOWN TRACKS
  105   303C C21530      JNZ START
  106                  ;
  107                  ;
  108                  ; JMP TO BOOT TO PRINT INITIAL MESSAGE, AND SET UP JMPS
  109   303F C30016      JMP BOOT
  110                  ;
  111                  ; PARAMETER BLOCKS
  112   3042 80        IOPBO:  DB  80H   ;IOCW, NO UPDATE
  113   3043 04          DB  READF   ;READ FUNCTION
  114   3044 19          DB  BDOSO   ;*SECTORS TO READ ON TRACK 0
  115   3045 00          DB  0   ;TRACK 0
  116   3046 02          DB  2   ;START WITH SECTOR 2 ON TRACK 0
  117   3047 0000        DW  CPMB    ;START AT BASE OF BDOS
  118   0007 =         IOPBL EQU $-IOPBO
  119                  ;
  120   3049 80        IOPB1:  DB  80H
  121   304A 04          DB  READF
  122   304B 18          DB  BDOS1   ;SECTORS TO READ ON TRACK 1
  123   304C 01          DB  1   ;TRACK 1
  124   304D 01          DB  1   ;SECTOR 1
  125   304E 800C        DW  CPMB+BDOSO*128  ;BASE OF SECOND READ
  126                  ;
  127   3050             END
BASE            0078    33#   34    35    36    39    40    41
BDOS            0806    18#
BDOS1           0018    29#  122
BDOSE           1880    19#   25
BDOSL           1880    25#   27
BDOSO           0019    28#   29   114   125
BDOSS           0031    27#   29
BIAS            0000    12#   15#   17    18    19    20
BOOT            1600    20#   21   109
BSW             00FF    42#   54
COLDSTART       3007    53#   56
CPMB            0000    17#   25   117   125
DSTAT           0078    39#   71
FALSE           0000     7#    8     9
IHIGH           007A    41#   70
ILOW            0079    40#   68
IOPB1           3049   120#
IOPBL           0007   102   118#
IOPBO           3042    62   112#  118
MON80           F800    31#
NTRKS           0002    26#   61
RBOOT           1603    21#
RBYTE           007B    35#   51    87
READF           0004    44#  113   121
RECAL           0003    43#
RESET           007F    36#   58
RMON80          FF0F    32#   81    90    95
RSTART          3000    47#   84    98
RTYPE           0079    34#   50    76
STACK           0100    45#   48
START           3015    64#  105
TESTING         0000     9#   11    14    80    83    94    97
```

```
TRUE            FFFF     8#
WAITO           301B    71#   73
```

E. A Skeletal Cold Start Loader

Note

This appendix consists of a cross-reference listing generated by the XREF utility from the results of assembly with MAC.

```
 1                ;THIS IS A SAMPLE COLD START LOADER, WHICH, WHEN
 2                ;MODIFIED
 3                ;RESIDES ON TRACK 00, SECTOR 01 (THE FIRST SECTOR ON THE
 4                ;DISKETTE), WE ASSUME THAT THE CONTROLLER HAS LOADED
 5                ;THIS SECTOR INTO MEMORY UPON SYSTEM START-UP (THIS
 6                ;PROGRAM CAN BE KEYED-IN, OR CAN EXIST IN READ-ONLY
 7                ;MEMORY
 8                ;BEYOND THE ADDRESS SPACE OF THE CP/M VERSION YOU ARE
 9                ;RUNNING). THE COLD START LOADER BRINGS THE CP/M SYSTEM
10                ;INTO MEMORY AT"LOADP" (3400H +"BIAS"). IN A 20K
11                ;MEMORY SYSTEM, THE VALUE OF"BIAS" IS 000H, WITH
12                ;LARGE
13                ;VALUES FOR INCREASED MEMORY SIZES (SEE SECTION 2).
14                ;AFTER
15                ;LOADING THE CP/M SYSTEM, THE COLD START LOADER
16                ;BRANCHES
17                ;TO THE "BOOT" ENTRY POINT OF THE BIOS, WHICH BEGINS AT
18                ; "BIOS" +"BIAS". THE COLD START LOADER IS NOT USED UN-
19                ;TIL THE SYSTEM IS POWERED UP AGAIN, AS LONG AS THE BIOS
20                ;IS NOT OVERWRITTEN. THE ORIGIN IS ASSUMED AT 0000H, AND
21                ;MUST BE CHANGED IF THE CONTROLLER BRINGS THE COLD START
22                ;LOADER INTO ANOTHER AREA, OR IF A READ-ONLY MEMORY
23                ;AREA
24                ;IS USED.
25   0000           ORG 0   ;BASE OF RAM IN
26                         ;CP/M
27   0014 =       MSIZE EQU 20    ;MIN MEM SIZE IN
28                         ;KBYTES
29   0000 =       BIAS  EQU (MSIZE-20)*1024 ;OFFSET FROM 20K
30                         ;SYSTEM
31   3400 =       CCP EQU 3400H+BIAS  ;BASE OF THE CCP
32   4A00 =       BIOS  EQU CCP+1600H ;BASE OF THE BIOS
33   0300 =       BIOSL EQU 0300H   ;LENGTH OF THE BIOS
34   4A00 =       BOOT  EQU BIOS
35   1900 =       SIZE  EQU BIOS+BIOSL-CCP  ;SIZE OF CP/M
36                         ;SYSTEM
37   0032 =       SECTS EQU SIZE/128  ;# OF SECTORS TO LOAD
38                ;
39                ; BEGIN THE LOAD OPERATION
40
41                COLD:
42   0000 010200    LXI B,2   ;B=0, C=SECTOR 2
43   0003 1632      MVI D,SECTS   ;D=# SECTORS TO
44                         ;LOAD
45   0005 210034    LXI H,CCP   ;BASE TRANSFER
```

```
    46                          ;ADDRESS
    47                  LSECT:  ;LOAD THE NEXT SECTOR
    48
    49                  ; INSERT INLINE CODE AT THIS POINT TO
    50                  ; READ ONE 128 BYTE SECTOR FROM THE
    51                  ; TRACK GIVEN IN REGISTER B, SECTOR
    52                  ; GIVEN IN REGISTER C,
    53                  ; INTO THE ADDRESS GIVEN BY &lt;HL&gt;
    54                  ;BRANCH TO LOCATION "COLD" IF A READ ERROR OCCURS
    55                  ;
    56                  ;
    57                  ;
    58                  ;
    59                  ; USER SUPPLIED READ OPERATION GOES
    60                  ; HERE...
    61                  ;
    62                  ;
    63                  ;
    64                  ;
    65  0008 C36B00       JMP PAST$PATCH  ;REMOVE THIS
    66                          ;WHEN PATCHED
    67  000B              DS  60H
    68
    69                  PAST$PATCH:
    70                  ;GO TO NEXT SECTOR IF LOAD IS INCOMPLETE
    71  006B 15           DCR D   ;SECTS=SECTS-1
    72  006C CA004A       JZ  BOOT    ;HEAD. FOR THE BIOS
    73
    74                  ; MORE SECTORS TO LOAD
    75                  ;
    76
    77                  ;WE AREN'T USING A STACK, SO USE &lt;SP&gt; AS SCRATCH
    78                  ;REGISTER
    79                  ; TO HOLD THE LOAD ADDRESS INCREMENT
    80  006F 318000       LXI SP,128    ;128 BYTES PER
    81                          ;SECTOR
    82  0072 39           DAD SP    ;&lt;HL&gt; = &lt;HL&gt; + 128
    83  0073 0C           INR C   ;SECTOR=SECTOR + 1
    84  0074 79           MOV A,C
    85  0075 FE1B         CPI 27    ;LAST SECTOR OF
    86                          ;TRACK?
    87  0077 DA0800       JC  LSECT   ;NO, GO READ
    88                          ;ANOTHER
    89
    90                  ;END OF TRACK, INCREMENT TO NEXT TRACK
    91
    92  007A 0E01         MVI C,1   ;SECTOR = 1
    93  007C 04           INR B   ;TRACK = TRACK + 1
    94  007D C30800       JMP LSECT   ;FOR ANOTHER GROUP
    95  0080              END     ;OF BOOT LOADER
BIAS             0000    29#   31
BIOS             4A00    32#   34    35
BIOSL            0300    33#   35
BOOT             4A00    34#   72
CCP              3400    31#   32    35    45
COLD             0000    41#
LSECT            0008    47#   87    94
```

```
MSIZE           0014    27#   29
PASTPATCH       006B    65    69#
SECTS           0032    37#   43
SIZE            1900    35#   37
```

F. CP/M Disk Definition Library

Note

This file is intended to be included with your CBIOS to provide disk definitions.

```
;       CP/M 2.0 disk re-definition library
;
;       Copyright (c) 1979
;       Digital Research
;       Box 579
;       Pacific Grove, CA
;       93950
;
;       CP/M logical disk drives are defined using the
;       macros given below, where the sequence of calls
;       is:
;
;       disks   n
;       diskdef parameter-list-0
;       diskdef parameter-list-1
;       ...
;       diskdef parameter-list-n
;       endef
;
;       where n is the number of logical disk drives attached
;       to the CP/M system, and parameter-list-i defines the
;       characteristics of the ith drive (i=0,1,...,n-1)
;
;       each parameter-list-i takes the form
;               dn,fsc,lsc,[skf],bls,dks,dir,cks,ofs,[0]
;       where
;       dn      is the disk number 0,1,...,n-1
;       fsc     is the first sector number (usually 0 or 1)
;       lsc     is the last sector number on a track
;       skf     is optional "skew factor" for sector translate
;       bls     is the data block size (1024,2048,...,16384)
;       dks     is the disk size in bls increments (word)
;       dir     is the number of directory elements (word)
;       cks     is the number of dir elements to checksum
;       ofs     is the number of tracks to skip (word)
;       [0]     is an optional 0 which forces 16K/directory entry
;
;       for convenience, the form
;               dn,dm
;       defines disk dn as having the same characteristics as
;       a previously defined disk dm.
;
;       a standard four drive CP/M system is defined by
;               disks   4
;               diskdef 0,1,26,6,1024,243,64,64,2
;       dsk     set     0
```

```
;               rept    3
;       dsk     set     dsk+1
;               diskdef %dsk,0
;               endm
;               endef
;
;       the value of "begdat" at the end of assembly defines the
;       beginning of the uninitialize ram area above the bios,
;       while the value of "enddat" defines the next location
;       following the end of the data area.  the size of this
;       area is given by the value of "datsiz" at the end of the
;       assembly.  note that the allocation vector will be quite
;       large if a large disk size is defined with a small block
;       size.
;
dskhdr  macro   dn
;;      define a single disk header list
dpe&amp;dn: dw      xlt&amp;dn,0000h    ;translate table
        dw      0000h,0000h     ;scratch area
        dw      dirbuf,dpb&amp;dn   ;dir buff,parm block
        dw      csv&amp;dn,alv&amp;dn   ;check, alloc vectors
        endm
;
disks   macro   nd
;;      define nd disks
ndisks  set     nd              ;;for later reference
dpbase  equ     $               ;base of disk parameter blocks
;;        generate the nd elements
dsknxt  set     0
        rept    nd
        dskhdr  %dsknxt
dsknxt  set     dsknxt+1
        endm
        endm
;
dpbhdr  macro   dn
dpb&amp;dn  equ     $               ;disk parm block
        endm
;
ddb     macro   data,comment
;;      define a db statement
        db      data            comment
        endm
;
ddw     macro   data,comment
;;      define a dw statement
        dw      data            comment
        endm
;
gcd     macro   m,n
;;      greatest common divisor of m,n
;;      produces value gcdn as result
;;      (used in sector translate table generation)
gcdm    set     m       ;;variable for m
gcdn    set     n       ;;variable for n
gcdr    set     0       ;;variable for r
        rept    65535
```

```
gcdx    set     gcdm/gcdn
gcdr    set     gcdm - gcdx*gcdn
        if      gcdr = 0
        exitm
        endif
gcdm    set     gcdn
gcdn    set     gcdr
        endm
        endm
;
diskdef macro   dn,fsc,lsc,skf,bls,dks,dir,cks,ofs,k16
;;      generate the set statements for later tables
        if      nul lsc
;;      current disk dn same as previous fsc
dpb&amp;dn  equ     dpb&amp;fsc         ;equivalent parameters
als&amp;dn  equ     als&amp;fsc         ;same allocation vector size
css&amp;dn  equ     css&amp;fsc         ;same checksum vector size
xlt&amp;dn  equ     xlt&amp;fsc         ;same translate table
        else
secmax  set     lsc-(fsc)       ;;sectors 0...secmax
sectors set     secmax+1        ;;number of sectors
als&amp;dn  set     (dks)/8         ;;size of allocation vector
        if      ((dks) mod 8) ne 0
als&amp;dn  set     als&amp;dn+1
        endif
css&amp;dn  set     (cks)/4         ;;number of checksum elements
;;      generate the block shift value
blkval  set     bls/128         ;;number of sectors/block
blkshf  set     0               ;;counts right 0's in blkval
blkmsk  set     0               ;;fills with 1's from right
        rept    16              ;;once for each bit position
        if      blkval=1
        exitm
        endif
;;      otherwise, high order 1 not found yet
blkshf  set     blkshf+1
blkmsk  set     (blkmsk shl 1) or 1
blkval  set     blkval/2
        endm
;;      generate the extent mask byte
blkval  set     bls/1024        ;;number of kilobytes/block
extmsk  set     0               ;;fill from right with 1's
        rept    16
        if      blkval=1
        exitm
        endif
;;      otherwise more to shift
extmsk  set     (extmsk shl 1) or 1
blkval  set     blkval/2
        endm
;;      may be double byte allocation
        if      (dks) &gt; 256
extmsk  set     (extmsk shr 1)
        endif
;;      may be optional [0] in last position
        if      not nul k16
extmsk  set     k16
```

```
        endif
;;      now generate directory reservation bit vector
dirrem  set     dir             ;;# remaining to process
dirbks  set     bls/32          ;;number of entries per block
dirblk  set     0               ;;fill with 1's on each loop
        rept    16
        if      dirrem=0
        exitm
        endif
;;      not complete, iterate once again
;;      shift right and add 1 high order bit
dirblk  set     (dirblk shr 1) or 8000h
        if      dirrem &gt; dirbks
dirrem  set     dirrem-dirbks
        else
dirrem  set     0
        endif
        endm
        dpbhdr  dn              ;;generate equ $
        ddw     %sectors,&lt;;sec per track&gt;
        ddb     %blkshf,&lt;;block shift&gt;
        ddb     %blkmsk,&lt;;block mask&gt;
        ddb     %extmsk,&lt;;extnt mask&gt;
        ddw     %(dks)-1,&lt;;disk size-1&gt;
        ddw     %(dir)-1,&lt;;directory max&gt;
        ddb     %dirblk shr 8,&lt;;alloc0&gt;
        ddb     %dirblk and 0ffh,&lt;;alloc1&gt;
        ddw     %(cks)/4,&lt;;check size&gt;
        ddw     %ofs,&lt;;offset&gt;
;;      generate the translate table, if requested
        if      nul skf
xlt&amp;dn  equ     0               ;no xlate table
        else
        if      skf = 0
xlt&amp;dn  equ     0               ;no xlate table
        else
;;        generate the translate table
nxtsec  set     0               ;;next sector to fill
nxtbas  set     0               ;;moves by one on overflow
        gcd     %sectors,skf
;;      gcdn = gcd(sectors,skew)
neltst  set     sectors/gcdn
;;      neltst is number of elements to generate
;;      before we overlap previous elements
nelts   set     neltst          ;;counter
xlt&amp;dn  equ     $               ;translate table
        rept    sectors         ;;once for each sector
        if      sectors &lt; 256
        ddb     %nxtsec+(fsc)
        else
        ddw     %nxtsec+(fsc)
        endif
nxtsec  set     nxtsec+(skf)
        if      nxtsec &gt;= sectors
nxtsec  set     nxtsec-sectors
        endif
nelts   set     nelts-1
```

```
        if      nelts = 0
nxtbas  set     nxtbas+1
nxtsec  set     nxtbas
nelts   set     neltst
        endif
        endm
        endif   ;;end of nul fac test
        endif   ;;end of nul bls test
        endm
;
defds   macro   lab,space
lab:    ds      space
        endm
;
lds     macro   lb,dn,val
        defds   lb&amp;dn,%val&amp;dn
        endm
;
endef   macro
;;      generate the necessary ram data areas
begdat  equ     $
dirbuf: ds      128             ;directory access buffer
dsknxt  set     0
        rept    ndisks          ;;once for each disk
        lds     alv,%dsknxt,als
        lds     csv,%dsknxt,css
dsknxt  set     dsknxt+1
        endm
enddat  equ     $
datsiz  equ     $-begdat
;;      db 0 at this point forces hex record
        endm
;
```

G. Blocking and Deblocking Algorithms

Note

This appendix consists of a cross-reference listing generated by the XREF utility from the results of assembly with MAC.

```
 1                ;*******************************************************
 2                ;*                                                     *
 3                ;*      SECTOR DEBLOCKING ALGORITHMS FOR CP/M 2.0      *
 4                ;*                                                     *
 5                ;*******************************************************
 6                ;
 7                ; UTILITY MACRO TO COMPUTE SECTOR MASK
 8                SMASK MACRO HBLK
 9                ;;  COMPUTE LOG2(HBLK), RETURN @X AS RESULT
10                ;;  (2 ** @X = HBLK ON RETURN)
11                @Y  SET HBLK
12                @X  SET 0
13                ;;  COUNT RIGHT SHIFTS OF @Y UNTIL = 1
14                  REPT  8
15                  IF  @Y = 1
16                  EXITM
17                  ENDIF
18                ;;  @Y IS NOT 1, SHIFT RIGHT ONE POSITION
19                @Y  SET @Y SHR 1
20                @X  SET @X + 1
21                  ENDM
22                  ENDM
23                ;
24                ;*******************************************************
25                ;*                                                     *
26                ;*         CP/M TO HOST DISK CONSTANTS                 *
27                ;*                                                     *
28                ;*******************************************************
29   0800 =       BLKSIZ  EQU 2048    ;CP/M ALLOCATION SIZE</a>
30   0200 =       HSTSIZ  EQU 512   ;HOST DISK SECTOR SIZE
31   0014 =       HSTSPT  EQU 20    ;HOST DISK SECTORS/TRK
32   0004 =       HSTBLK  EQU HSTSIZ/128  ;CP/M SECTS/HOST BUFF
33   0050 =       CPMSPT  EQU HSTBLK * HSTSPT ;CP/M SECTORS/TRACK
34   0003 =       SECMSK  EQU HSTBLK-1  ;SECTOR MASK
35                  SMASK HSTBLK    ;COMPUTE SECTOR MASK
36   0002 =       SECSHF  EQU @X    ;LOG2(HSTBLK)
37                ;
38                ;*******************************************************
39                ;*                                                     *
40                ;*        BDOS CONSTANTS ON ENTRY TO WRITE             *
41                ;*                                                     *
42                ;*******************************************************
43   0000 =       WRALL EQU 0   ;WRITE TO ALLOCATED
44   0001 =       WRDIR EQU 1   ;WRITE TO DIRECTORY
45   0002 =       WRUAL EQU 2   ;WRITE TO UNALLOCATED
```

```
                ;
                ;*****************************************************
                ;*                                                   *
                ;*  THE BDOS ENTRY POINTS GIVEN BELOW SHOW THE   *
                ;*      CODE WHICH IS RELEVANT TO DEBLOCKING ONLY.   *
                ;*                                                   *
                ;*****************************************************
                ;
                ; DISKDEF MACRO, OR HAND CODED TABLES GO HERE
0000 =          DPBASE  EQU $   ;DISK PARAM BLOCK BASE
                ;
                BOOT:</a>
                WBOOT:
                  ;ENTER HERE ON SYSTEM BOOT TO INITIALIZE
0000 AF           XRA A   ;0 TO ACCUMULATOR
0001 326A01       STA HSTACT    ;HOST BUFFER INACTIVE
0004 326C01       STA UNACNT    ;CLEAR UNALLOC COUNT
0007 C9           RET
                ;
                HOME:</a>
                  ;HOME THE SELECTED DISK
                HOME:
0008 3A6B01       LDA HSTWRT  ;CHECK FOR PENDING WRITE
000B B7           ORA A
000C C21200       JNZ HOMED
000F 326A01       STA HSTACT  ;CLEAR HOST ACTIVE FLAG
                HOMED:
0012 C9           RET
                ;
                SELDSK:
                  ;SELECT DISK
0013 79           MOV A,C   ;SELECTED DISK NUMBER
0014 326101       STA SEKDSK    ;SEEK DISK NUMBER
0017 6F           MOV L,A   ;DISK NUMBER TO HL
0018 2600         MVI H,0
                  REPT  4   ;MULTIPLY BY 16
                  DAD H
                  ENDM
001A+29           DAD H
001B+29           DAD H
001C+29           DAD H
001D+29           DAD H
001E 110000       LXI D,DPBASE  ;BASE OF PARM BLOCK
0021 19           DAD D   ;HL=.DPB(CURDSK)
0022 C9           RET
                ;
                SETTRK:
                  ;SET TRACK GIVEN BY REGISTERS BC
0023 60           MOV H,B
0024 69           MOV L,C
0025 226201       SHLD  SEKTRK    ;TRACK TO SEEK
0028 C9           RET
                ;
                SETSEC:
                  ;SET SECTOR GIVEN BY REGISTER C
0029 79           MOV A,C
002A 326401       STA SEKSEC    ;SECTOR TO SEEK
```

```
103   002D C9          RET
104                    ;
105                    SETDMA:
106                      ;SET DMA ADDRESS GIVEN BY BC
107   002E 60          MOV H,B
108   002F 69          MOV L,C
109   0030 227501      SHLD  DMAADR
110   0033 C9          RET</a>
111                    ;
112                    SECTRAN:
113                      ;TRANSLATE SECTOR NUMBER BC
114   0034 60          MOV H,B
115   0035 69          MOV L,C
116   0036 C9          RET
117                    ;
118                    ;*******************************************************
119                    ;*                                                     *
120                    ;*  THE READ ENTRY POINT TAKES THE PLACE OF      *
121                    ;*  THE PREVIOUS BIOS DEFINTION FOR READ.        *
122                    ;*                                                     *
123                    ;*******************************************************
124                    READ:
125                      ;READ THE SELECTED CP/M SECTOR</a>
126   0037 AF          XRA A
127   0038 326C01      STA UNACNT
128   003B 3E01        MVI A,1
129   003D 327301      STA READOP    ;READ OPERATION
130   0040 327201      STA RSFLAG    ;MUST READ DATA
131   0043 3E02        MVI A,WRUAL
132   0045 327401      STA WRTYPE    ;TREAT AS UNALLOC
133   0048 C3B600      JMP RWOPER    ;TO PERFORM THE READ
134                    ;
135                    ;*******************************************************
136                    ;*                                                     *
137                    ;*  THE WRITE ENTRY POINT TAKES THE PLACE OF     *
138                    ;*  THE PREVIOUS BIOS DEFINTION FOR WRITE.       *
139                    ;*                                                     *
140                    ;*******************************************************
141                    WRITE:
142                      ;WRITE THE SELECTED CP/M SECTOR
143   004B AF          XRA A   ;0 TO ACCUMULATOR
144   004C 327301      STA READOP    ;NOT A READ OPERATION
145   004F 79          MOV A,C   ;WRITE TYPE IN C
146   0050 327401      STA WRTYPE
147   0053 FE02        CPI WRUAL   ;WRITE UNALLOCATED?
148   0055 C26F00      JNZ CHKUNA    ;CHECK FOR UNALLOC
149                    ;
150                    ; WRITE TO UNALLOCATED, SET PARAMETERS
151   0058 3E10        MVI A,BLKSIZ/128  ;NEXT UNALLOC RECS
152   005A 326C01      STA UNACNT
153   005D 3A6101      LDA SEKDSK    ;DISK TO SEEK
154   0060 326D01      STA UNADSK    ;UNADSK = SEKDSK
155   0063 2A6201      LHLD  SEKTRK
156   0066 226E01      SHLD  UNATRK    ;UNATRK = SECTRK
157   0069 3A6401      LDA SEKSEC
158   006C 327001      STA UNASEC    ;UNASEC = SEKSEC
159                    ;
```

```
160                 CHKUNA:
161                   ;CHECK FOR WRITE TO UNALLOCATED SECTOR
162   006F 3A6C01     LDA UNACNT    ;ANY UNALLOC REMAIN?
163   0072 B7         ORA A
164   0073 CAAE00     JZ  ALLOC   ;SKIP IF NOT
165                 ;
166                 ; MORE UNALLOCATED RECORDS REMAIN
167   0076 3D         DCR A   ;UNACNT = UNACNT-1
168   0077 326C01     STA UNACNT
169   007A 3A6101     LDA SEKDSK    ;SAME DISK?
170   007D 216D01     LXI H,UNADSK
171   0080 BE         CMP M   ;SEKDSK = UNADSK?
172   0081 C2AE00     JNZ ALLOC   ;SKIP IF NOT
173                 ;
174                 ; DISKS ARE THE SAME
175   0084 216E01     LXI H,UNATRK
176   0087 CD5301     CALL  SEKTRKCMP ;SEKTRK = UNATRK?
177   008A C2AE00     JNZ ALLOC   ;SKIP IF NOT
178                 ;
179                 ; TRACKS ARE THE SAME
180   008D 3A6401     LDA SEKSEC    ;SAME SECTOR?
181   0090 217001     LXI H,UNASEC
182   0093 BE         CMP M   ;SEKSEC = UNASEC?
183   0094 C2AE00     JNZ ALLOC   ;SKIP IF NOT
184                 ;
185                 ; MATCH, MOVE TO NEXT SECTOR FOR FUTURE REF
186   0097 34         INR M   ;UNASEC = UNASEC+1
187   0098 7E         MOV A,M   ;END OF TRACK?
188   0099 FE50       CPI CPMSPT    ;COUNT CP/M SECTORS
189   009B DAA700     JC  NOOVF   ;SKIP IF NO OVERFLOW
190                 ;
191                 ; OVERFLOW TO NEXT TRACK
192   009E 3600       MVI M,0   ;UNASEC = 0
193   00A0 2A6E01     LHLD  UNATRK
194   00A3 23         INX H
195   00A4 226E01     SHLD  UNATRK    ;UNATRK = UNATRK+1
196                 ;
197                 NOOVF:
198                   ;MATCH FOUND, MARK AS UNNECESSARY READ
199   00A7 AF         XRA A   ;0 TO ACCUMULATOR
200   00A8 327201     STA RSFLAG    ;RSFLAG = 0
201   00AB C3B600     JMP RWOPER    ;TO PERFORM THE WRITE
202                 ;
203                 ALLOC:
204                   ;NOT AN UNALLOCATED RECORD, REQUIRES PRE-READ
205   00AE AF         XRA A   ;0 TO ACCUM
206   00AF 326C01     STA UNACNT    ;UNACNT = 0
207   00B2 3C         INR A   ;1 TO ACCUM
208   00B3 327201     STA RSFLAG    ;RSFLAG = 1
209                 ;
210                 ;*****************************************************
211                 ;*                                                   *
212                 ;*  COMMON CODE FOR READ AND WRITE FOLLOWS       *
213                 ;*                                                   *
214                 ;*****************************************************
215                 RWOPER:
216                   ;ENTER HERE TO PERFORM THE READ/WRITE
```

```
217   00B6 AF          XRA A   ;ZERO TO ACCUM
218   00B7 327101      STA ERFLAG    ;NO ERRORS (YET)
219   00BA 3A6401      LDA SEKSEC    ;COMPUTE HOST SECTOR
220                    REPT  SECSHF
221                    ORA A   ;CARRY = 0
222                    RAR     ;SHIFT RIGHT
223                    ENDM
224   00BD+B7          ORA A   ;CARRY = 0
225   00BE+1F          RAR     ;SHIFT RIGHT
226   00BF+B7          ORA A   ;CARRY = 0
227   00C0+1F          RAR     ;SHIFT RIGHT
228   00C1 326901      STA SEKHST    ;HOST SECTOR TO SEEK
229                  ;
230                  ; ACTIVE HOST SECTOR?
231   00C4 216A01      LXI H,HSTACT  ;HOST ACTIVE FLAG
232   00C7 7E          MOV A,M
233   00C8 3601        MVI M,1   ;ALWAYS BECOMES 1
234   00CA B7          ORA A   ;WAS IT ALREADY?
235   00CB CAF200      JZ  FILHST    ;FILL HOST IF NOT
236                  ;
237                  ; HOST BUFFER ACTIVE, SAME AS SEEK BUFFER?
238   00CE 3A6101      LDA SEKDSK
239   00D1 216501      LXI H,HSTDSK  ;SAME DISK?
240   00D4 BE          CMP M   ;SEKDSK = HSTDSK?
241   00D5 C2EB00      JNZ NOMATCH
242                  ;
243                  ; SAME DISK, SAME TRACK?
244   00D8 216601      LXI H,HSTTRK
245   00DB CD5301      CALL  SEKTRKCMP ;SEKTRK = HSTTRK?
246   00DE C2EB00      JNZ NOMATCH
247                  ;
248                  ; SAME DISK, SAME TRACK, SAME BUFFER?
249   00E1 3A6901      LDA SEKHST
250   00E4 216801      LXI H,HSTSEC  ;SEKHST = HSTSEC?
251   00E7 BE          CMP M
252   00E8 CA0F01      JZ  MATCH   ;SKIP IF MATCH
253                  ;
254                  NOMATCH:
255                    ;PROPER DISK, BUT NOT CORRECT SECTOR
256   00EB 3A6B01      LDA HSTWRT    ;HOST WRITTEN?
257   00EE B7          ORA A
258   00EF C45F01      CNZ WRITEHST  ;CLEAR HOST BUFF
259                  ;
260                  FILHST:
261                    ;MAY HAVE TO FILL THE HOST BUFFER
262   00F2 3A6101      LDA SEKDSK
263   00F5 326501      STA HSTDSK
264   00F8 2A6201      LHLD  SEKTRK
265   00FB 226601      SHLD  HSTTRK
266   00FE 3A6901      LDA SEKHST
267   0101 326801      STA HSTSEC
268   0104 3A7201      LDA RSFLAG    ;NEED TO READ?
269   0107 B7          ORA A
270   0108 C46001      CNZ READHST   ;YES, IF 1
271   010B AF          XRA A   ;0 TO ACCUM
272   010C 326B01      STA HSTWRT    ;NO PENDING WRITE
273                  ;
```

```
                MATCH:
                  ;COPY DATA TO OR FROM BUFFER
010F 3A6401       LDA SEKSEC    ;MASK BUFFER NUMBER
0112 E603         ANI SECMSK    ;LEAST SIGNIF BITS
0114 6F           MOV L,A   ;READY TO SHIFT
0115 2600         MVI H,0   ;DOUBLE COUNT
                  REPT  7   ;SHIFT LEAST 7
                  DAD H
                  ENDM
0117+29           DAD H
0118+29           DAD H
0119+29           DAD H
011A+29           DAD H
011B+29           DAD H
011C+29           DAD H
011D+29           DAD H
                ; HL HAS RELATIVE HOST BUFFER ADDRESS
011E 117701       LXI D,HSTBUF
0121 19           DAD D   ;HL = HOST ADDRESS
0122 EB           XCHG      ;NOW IN DE
0123 2A7501       LHLD  DMAADR    ;GET/PUT CP/M DATA
0126 0E80         MVI C,128   ;LENGTH OF MOVE
0128 3A7301       LDA READOP    ;WHICH WAY?
012B B7           ORA A
012C C23501       JNZ RWMOVE    ;SKIP IF READ
                ;
                ; WRITE OPERATION, MARK AND SWITCH DIRECTION
012F 3E01         MVI A,1
0131 326B01       STA HSTWRT    ;HSTWRT = 1
0134 EB           XCHG      ;SOURCE/DEST SWAP
                ;
                RWMOVE:
                  ;C INITIALLY 128, DE IS SOURCE, HL IS DEST
0135 1A           LDAX  D   ;SOURCE CHARACTER
0136 13           INX D
0137 77           MOV M,A   ;TO DEST
0138 23           INX H
0139 0D           DCR C   ;LOOP 128 TIMES
013A C23501       JNZ RWMOVE
                ;
                ; DATA HAS BEEN MOVED TO/FROM HOST BUFFER
013D 3A7401       LDA WRTYPE    ;WRITE TYPE
0140 FE01         CPI WRDIR   ;TO DIRECTORY?
0142 3A7101       LDA ERFLAG    ;IN CASE OF ERRORS
0145 C0           RNZ     ;NO FURTHER PROCESSING
                ;
                ; CLEAR HOST BUFFER FOR DIRECTORY WRITE
0146 B7           ORA A   ;ERRORS?
0147 C0           RNZ     ;SKIP IF SO
0148 AF           XRA A   ;0 TO ACCUM
0149 326B01       STA HSTWRT    ;BUFFER WRITTEN
014C CD5F01       CALL  WRITEHST
014F 3A7101       LDA ERFLAG
0152 C9           RET
                ;
                ;*****************************************************
                ;*                                                   *
```

```
                ;*  UTILITY SUBROUTINE FOR 16-BIT COMPARE        *
                ;*                                                 *
                ;***************************************************
                SEKTRKCMP:
                  ;HL = .UNATRK OR .HSTTRK, COMPARE WITH SEKTRK
0153 EB           XCHG
0154 216201       LXI H,SEKTRK
0157 1A           LDAX  D   ;LOW BYTE COMPARE
0158 BE           CMP M   ;SAME?
0159 C0           RNZ     ;RETURN IF NOT
                ; LOW BYTES EQUAL, TEST HIGH 1S
015A 13           INX D
015B 23           INX H
015C 1A           LDAX  D
015D BE           CMP M ;SETS FLAGS
015E C9           RET
                ;
                ;***************************************************
                ;*                                                 *
                ;*  WRITEHST PERFORMS THE PHYSICAL WRITE TO      *
                ;*  THE HOST DISK, READHST READS THE PHYSICAL    *
                ;*  DISK.              *
                ;*                                                 *
                ;***************************************************
                WRITEHST:
                  ;HSTDSK = HOST DISK #, HSTTRK = HOST TRACK #,
                  ;HSTSEC = HOST SECT #. WRITE "HSTSIZ" BYTES
                  ;FROM HSTBUF AND RETURN ERROR FLAG IN ERFLAG.
                  ;RETURN ERFLAG NON-ZERO IF ERROR
015F C9           RET
                ;
                READHST:
                  ;HSTDSK = HOST DISK #, HSTTRK = HOST TRACK #,
                  ;HSTSEC = HOST SECT #. READ "HSTSIZ" BYTES
                  ;INTO HSTBUF AND RETURN ERROR FLAG IN ERFLAG.
0160 C9           RET
                ;
                ;***************************************************
                ;*                                                 *
                ;*  UNITIALIZED RAM DATA AREAS         *
                ;*                                                 *
                ;***************************************************
                ;
0161            SEKDSK: DS  1   ;SEEK DISK NUMBER
0162            SEKTRK: DS  2   ;SEEK TRACK NUMBER
0164            SEKSEC: DS  1   ;SEEK SECTOR NUMBER
                ;
0165            HSTDSK: DS  1   ;HOST DISK NUMBER
0166            HSTTRK: DS  2   ;HOST TRACK NUMBER
0168            HSTSEC: DS  1   ;HOST SECTOR NUMBER
                ;
0169            SEKHST: DS  1   ;SEEK SHR SECSHF
016A            HSTACT: DS  1   ;HOST ACTIVE FLAG
016B            HSTWRT: DS  1   ;HOST WRITTEN FLAG
                ;
016C            UNACNT: DS  1   ;UNALLOC REC CNT
016D            UNADSK: DS  1   ;LAST UNALLOC DISK
```

```
   388   016E           UNATRK: DS  2   ;LAST UNALLOC TRACK
   389   0170           UNASEC: DS  1   ;LAST UNALLOC SECTOR
   390                  ;
   391   0171           ERFLAG: DS  1   ;ERROR REPORTING
   392   0172           RSFLAG: DS  1   ;READ SECTOR FLAG
   393   0173           READOP: DS  1   ;1 IF READ OPERATION
   394   0174           WRTYPE: DS  1   ;WRITE OPERATION TYPE
   395   0175           DMAADR: DS  2   ;LAST DMA ADDRESS
   396   0177           HSTBUF: DS  HSTSIZ    ;HOST BUFFER
   397                  ;
   398                  ;*******************************************************
   399                  ;*                                                     *
   400                  ;*  THE ENDEF MACRO INVOCATION GOES HERE       *
   401                  ;*                                                     *
   402                  ;*******************************************************
   403   0377             END
ALLOC            00AE   164   172   177   183   203#
BLKSIZ           0800    29#  151
BOOT             0000    57#
CHKUNA           006F   148   160#
CPMSPT           0050    33#  188
DMAADR           0175   109   294   395#
DPBASE           0000    55#   88
ERFLAG           0171   218   317   326   391#
FILHST           00F2   235   260#
HOME             0008    65#   67#
HOMED            0012    70    72#
HSTACT           016A    61    71   231   383#
HSTBLK           0004    32#   33    34    35
HSTBUF           0177   291   396#
HSTDSK           0165   239   263   378#
HSTSEC           0168   250   267   380#
HSTSIZ           0200    30#   32   396
HSTSPT           0014    31#   33
HSTTRK           0166   244   265   379#
HSTWRT           016B    68   256   272   302   324   384#
MATCH            010F   252   274#
NOMATCH          00EB   241   246   254#
NOOVF            00A7   189   197#
READ             0037   124#
READHST          0160   270   362#
READOP           0173   129   144   296   393#
RSFLAG           0172   130   200   208   268   392#
RWMOVE           0135   298   305#  312
RWOPER           00B6   133   201   215#
SECMSK           0003    34#  277
SECSHF           0002    36#  220
SECTRAN          0034   112#
SEKDSK           0161    78   153   169   238   262   374#
SEKHST           0169   228   249   266   382#
SEKSEC           0164   102   157   180   219   276   376#
SEKTRK           0162    96   155   264   337   375#
SEKTRKCMP        0153   176   245   334#
SELDSK           0013    75#
SETDMA           002E   105#
SETSEC           0029    99#
SETTRK           0023    92#
```

```
UNACNT          016C    62   127   152   162   168   206   386#
UNADSK          016D   154   170   387#
UNASEC          0170   158   181   389#
UNATRK          016E   156   175   193   195   388#
WBOOT           0000    58#
WRALL           0000    43#
WRDIR           0001    44#  316
WRITE           004B   141#
WRITEHST        015F   258   325   355#
WRTYPE          0174   132   146   315   394#
WRUAL           0002    45#  131   147
```

H. Glossary

address Number representing the location of a byte in memory. Within CP/M there are two kinds of addresses: **logical** and **physical**. A **physical** address refers to an absolute and unique location within the computer's memory space. A **logical** address refers to the offset or displacement of a byte in relation to a base location. A standard CP/M program is loaded at address 0100H, the base value; the first instruction of a program has a **physical** address of 0100H and a **logical** address or offset of 0000H.

allocation vector (ALV) An allocation vector is maintained in the BIOS for each logged-in disk drive. A vector consists of a string of bits, one for each block on the drive. The bit corresponding to a particular block is set to one when the block has been allocated and to zero otherwise. The first two bytes of this vector are initialized with the bytes AL0 and AL1 on, thus allocating the directory blocks. CP/M Function 27 returns the allocation vector address.

AL0, AL1 Two bytes in the disk parameter block that reserve data blocks for the directory. These two bytes are copied into the first two bytes of the allocation vector when a drive is logged in. See **allocation vector**.

ALV See **allocation vector**

ambiguous filename Filename that contains either of the CP/M wildcard characters, ? or *, in the primary filename, filetype, or both. When you replace characters in a filename with these wildcard characters, you create an ambiguous filename and can easily reference more than one CP/M file in a single command line.

American Standard Code for Information Interchange See **ASCII**

applications program Program designed to solve a specific problem. Typical applications programs are business accounting packages, word processing (editing) programs and mailing list programs.

archive attribute File attribute controlled by the high-order bit of the t3 byte (FCB + 11) in a directory element. This attribute is set if the file has been archived.

argument Symbol, usually a letter, indicating a place into which you can substitute a number, letter, or name to give an appropriate meaning to the formula in question.

ASCII American Standard Code for Information Interchange. ASCII is a standard set of seven-bit numeric character codes used to represent characters in memory. Each character requires one byte of memory with the high-order bit usually set to zero. Characters can be numbers, letters, and symbols. An ASCII file can be intelligibly displayed on the video screen or printed on paper.

assembler Program that translates assembly language into the binary machine code. Assembly language is simply a set of mnemonics used to designate the instruction set of the CPU. See ASM in Section 3 of this manual.

back-up Copy of a disk or file made for safekeeping, or the creation of the duplicate disk or file.

Basic Disk Operating System See **BDOS**.

BDOS Basic Disk Operating System. The BDOS module of the CP/M operating system provides an interface for a user program to the operating. This interface is in the form of a set of function calls which may be made to the BDOS through calls to location 0005H in page zero. The user program specifies the number of the desired function in register C. User programs running under CP/M should

use BDOS functions for all I/O operations to remain compatible with other CP/M systems and future releases. The BDOS normally resides in high memory directly below the BIOS.

bias Address value which when added to the origin address of your BIOS module produces 1F80H, the address of the BIOS module in the MOVCPM image. There is also a bias value that when added to the BOOT module origin produces 0900H, the address of the BOOT module in the MOVCPM image. You must use these bias values with the R command under DDT or SID when you patch a CP/M system. If you do not, the patched system may fall to function.

binary Base 2 numbering system. A binary digit can have one of two values: 0 or 1. Binary numbers are used in computers because the hardware can most easily exhibit two states: off and on. Generally, a bit in memory represents one binary digit.

Basic Input/Output System See **BIOS**.

BIOS Basic Input/Output System. The BIOS is the only hardware-dependent module of the CP/M system. It provides the BDOS with a set of primitive I/O operations. The BIOS is an assembly language module usually written by the user, hardware manufacturer, or independent software vendor, and is the key to CP/M's portability. The BIOS interfaces the CP/M system to its hardware environment through a standardized jump table at the front of the BIOS routine and through a set of disk parameter tables which define the disk environment. Thus, the BIOS provides CP/M with a completely table-driven I/O system.

BIOS base Lowest address of the BIOS module in memory, that by

definition must be the first entry point in the BIOS jump table.

bit Switch in memory that can be set to on (1) or off (0). Bits are grouped into **bytes**, eight bits to a byte, which is the smallest directly addressable unit in an Intel 8080 or Zilog Z80.

By common convention, the bits in a byte are numbered from right, 0 for the low-order bit, to left, 7 for the high-order bit. Bit values are often represented in hexadecimal notation by grouping the bits from the low-order bit in groups of four. Each group of four bits can have a value from 0 to 15 and thus can easily be represented by one hexadecimal digit.

BLM See **Block Mask**.

block Basic unit of disk space allocation. Each disk drive has a fixed block size (BLS) defined in its **disk parameter block** in the BIOS. A block can consist of 1K, 2K, 4K, 8K, or 16K consecutive bytes. Blocks are numbered relative to zero so that each block is unique and has a byte displacement in a file equal to the block number times the block size.

block mask (BLM) Byte value in the disk parameter block at $\mathrm{DPB}+3$. The block mask is always one less than the number of 128 byte sectors that are in one block. Note that

$$\mathrm{BLM} = \left(2^{\mathrm{BSH}}\right) - 1$$

block shift (BSH) Byte parameter in the disk parameter block at $\mathrm{DPB}+2$. Block shift and block mask (BLM) values are determined by the block size (BLS). Note that

$$\mathrm{BLM} = \left(2^{\mathrm{BSH}}\right) - 1$$

blocking and deblocking algorithm In some disk subsystems the disk sector size is larger than 128 bytes, usually 256, 512, 1024, or 2048 bytes. When the host sector size is larger than 128 bytes, host sectors must be buffered in memory and the 128-byte CP/M sectors must be blocked and deblocked

by adding an additional module, the blocking and deblocking algorithm, between the BIOS disk I/O routines and the actual disk I/O. The host sector size must be an even multiple of 128 bytes for the algorithm to work correctly. The blocking and deblocking algorithm allows the BDOS and BIOS to function exactly as if the entire disk consisted only of 128-byte sectors, as in the standard CP/M installation.

BLS Block size in bytes. See **block**.

boot Process of loading an operating system into memory. A boot program is a small piece of code that is automatically executed when you power-up or reset your computer. The boot program loads the rest of the operating system into memory in a manner similar to a person pulling himself up by his own bootstraps. This process is sometimes called a cold boot or cold start. Bootstrap procedures vary from system to system. The boot program must be customized for the memory size and hardware environment that the operating system manages. Typically, the boot resides on the first sector of the system tracks on your system disk. When executed, the boot loads the remaining sectors of the system tracks into high memory at the location for which the CP/M system has been configured. Finally, the boot transfers execution to the boot entry point in the BIOS jump table so that the system can initialize itself. In this case, the boot program should be placed at 0900H in the SYSGEN image. Alternatively, the boot program may be located in ROM.

bootstrap See **boot**.

BSH See `block shift`

BTREE General purpose file access method that has become the standard organization for indexes in large data base systems. BTREE provides near optimum performance over the full range of file operations, such as insertion, deletion, search, and search next.

buffer Area of memory that temporarily stores data during the transfer of information.

built-in commands Commands that permanently reside in memory. They respond quickly because they are not accessed from a disk.

byte Unit of memory or disk storage containing eight bits. A byte can represent a binary number between 0 and 255, and is the smallest unit of memory that can be addressed directly in 8-bit CPUs such as the Intel 8080 or Zilog Z80.

CCP Console Command Processor. The CCP is a module of the CP/M operating system. It is loaded directly below the BDOS module and interprets and executes commands typed by the console user. Usually these commands are programs that the CCP loads and calls. Upon completion, a command program may return control to the CCP if it has not overwritten it. If it has, the program can reload the CCP into memory by a warm boot operation initiated by either a jump to zero, BDOS system reset (Function 0), or a cold boot. Except for its location in high memory, the CCP works like any other standard CP/M program; that is, it makes only BDOS function calls for its I/O operations.

CCP base Lowest address of the CCP module in memory. This term sometimes refers to the base of the CP/M system in memory, as the CCP is normally the lowest CP/M module in high memory.

checksum vector (csv) Contiguous data area in the BIOS, with one byte for each directory sector to be checked, that is, CKS bytes. See **CKS**. A checksum vector is initialized and maintained for each logged-in drive. Each directory access by the system results in a checksum calculation that is compared with the one in the checksum vector. If there is a discrepancy, the drive is set to Read-Only status. This feature prevents the user from inadvertently switching disks without logging in the new

disk. If the new disk is not logged-in, it is treated the same as the old one, and data on it might be destroyed if writing is done.

CKS Number of directory records to be checked summed on directory accesses. This is a parameter in the disk parameter block located in the BIOS. If the value of CKS is zero, then no directory records are checked. CKS is also a parameter in the diskdef macro library, where it is the actual number of directory elements to be checked rather than the number of directory records.

cold boot See **boot**. Cold boot also refers to a jump to the boot entry. point in the BIOS jump table.

COM Filetype for a CP/M command file. See **command file**.

command CP/M command line. In general, a CP/M command line has three parts: the **command keyword**, **command tail**, and a **carriage return**. To execute a command, enter a CP/M command line directly after the CP/M prompt at the console and press the carriage return or enter key.

command file Executable program file of filetype COM. A command file is a machine language object module ready to be loaded and executed at the absolute address of 0100H. To execute a command file, enter its primary filename as the command keyword in a CP/M command line.

command keyword Name that identifies a CP/M command, usually the primary filename of a file of type COM, or a built-in command. The command keyword precedes the command tail and the carriage return in the command line.

command syntax Statement that defines the correct way to enter a command. The correct structure generally includes the **command keyword**, the **command tail**, and a **carriage return**. A syntax line usually contains symbols that you should replace with actual values when you enter the command.

command tail Part of a command that follows the command keyword in the command line. The command tail can include a drive specification, a filename and filetype, and options or parameters. Some commands do not require a command tail.

CON: Mnemonic that represents the CP/M console device. For example, the CP/M command PIP CON:=TEST.SUB displays the file TEST.SUB on the console device. The explanation of the STAT command tells how to assign the logical device CON: to various physical devices. See **console**.

concatenate Name of the PIP operation that copies two or more separate files into one new file in the specified sequence.

concurrency Execution of two processes or operations simultaneously.

CONIN: BIOS entry point to a routine that reads a character from the console device.

CONOUT: BIOS entry point to a routine that sends a character to the console device.

console Primary input/output device. The console consists of a listing device, such as a screen or teletype, and a keyboard through which the user communicates with the operating system or applications program.

Console Command Processor See CCP

CONST: BIOS entry point to a routine that returns the status of the console device.

control character Non-printing character combination. CP/M interprets some control characters as simple commands such as line editing functions. To enter a control character, hold down the CONTROL key and strike the specified character key.

Control Program for Microcomputers See CP/M

CP/M Control Program for Microcomputers. An operating system that manages computer resources and provides a standard systems interface to software written for a large variety of microprocessor-based computer systems.

CP/M 1.4 compatibility For a CP/M 2 system to be able to read correctly single-density disks produced under a CP/M 1.4 system, the extent mask must be zero and the block size 1K. This is because under CP/M 2 an FCB may contain more than one extent. The number of extents that may be contained by an FCB is $\mathrm{EXM}+1$. The issue of CP/M 1.4 compatibility also concerns random file I/O. To perform random file I/O under CP/M 1.4, you must maintain an FCB for each extent of the file. This scheme is upward compatible with CP/M 2 for files not exceeding 512K bytes, the largest file size supported under CP/M 1.4. If you wish to implement random I/O for files larger than 512K bytes under CP/M 2, you must use the random read and random write functions, BDOS functions 33, 34, and 36. In this case, only one FCB is used, and if CP/M 1.4 compatibility is required, the program must use the return version number function, BDOS Function 12, to determine which method to employ.

CP/M prompt Characters that indicate that CP/M is ready to execute your next command. The CP/M prompt consists of an upper-case letter, A-P, followed by a > character; for example, A>. The letter designates which drive is currently logged in as the default drive. CP/M will search this drive for the command file specified, unless the command is a built-in command or prefaced by a select drive command: for example, B:STAT.

CP/NET Digital Research network operating system enabling microcomputers to obtain access to common resources via a network. CP/NET consists of MP/M masters and CP/M slaves with a network interface between them.

CSV See checksum vector.

cursor One-character symbol that can appear anywhere on the console screen. The cursor indicates the position where the next keystroke at the console will have an effect.

data file File containing information that will be processed by a program.

deblocking See blocking & deblocking algorithm.

default Currently selected disk drive and user number. Any command that does not specify a disk drive or a user number references the default disk drive and user number. When CP/M is first invoked, the default disk drive is drive A, and the default user number is 0.

default buffer Default 128-byte buffer maintained at 0080H in page zero. When the CCP loads a COM file, this buffer is initialized to the command tail; that is, any characters typed after the COM file name are loaded into the buffer. The first byte at 0080H contains the length of the command tail, while the command tail itself begins at 0081H. The command tail is terminated by a byte containing a binary zero value. The I command under DDT and SID initializes this buffer in the same way as the CCP.

default FCB Two default FCBs are maintained by the CCP at 005CH and 006CH in page zero. The first default FCB is initialized from the first delimited field in the command tail. The second default FCB is initialized from the next field in the command tail.

delimiter Special characters that separate different items in a command line; for example, a colon separates the drive specification from the filename. The CCP recognizes the following characters as delimiters: . : = ; < > _, blank, and carriage return. Several CP/M commands also treat the following as delimiter characters: , [ ] ( ) $. It is advisable to avoid the use of delimiter characters and lower-case characters in CP/M filenames.

DIR: Parameter in the diskdef macro library that specifies the number of directory elements on the drive.

DIR attribute File attribute. A file with the DIR attribute can be displayed by a DIR command. The file can be accessed from the default user number and drive only.

DIRBUF: 128-byte scratchpad area for directory operations, usually located at the end of the BIOS. DIRBUF is used by the BDOS during its directory operations. DIRBUF also refers to the two-byte address of this scratchpad buffer in the disk parameter header at DPbase + 8 bytes.

directory Portion of a disk that contains entries for each file on the disk. In response to the DIR command, CP/M displays the filenames stored in the directory. The directory also contains the locations of the blocks allocated to the files. Each file directory element is in the form of a 32-byte FCB, although one file can have several elements, depending on its size. The maximum number of directory elements supported is specified by the drive's disk parameter block value for **DRM**.

directory element Data structure. Each file on a disk has one or more 32-byte directory elements associated with it. There are four directory elements per directory sector. Directory elements can also be referred to as directory FCBs.

directory entry File entry displayed by the DIR command. Sometimes this term refers to a physical directory element.

disk, diskette Magnetic media used for mass storage in a computer system. Programs and data are recorded on the disk in the same way music can be recorded on cassette tape. The CP/M operating system must be initially loaded from disk when the computer is turned on. Diskette refers to smaller capacity removable floppy diskettes, while disk may refer to either a diskette, removable cartridge disk, or fixed hard disk. Hard disk capacities range from five to several hundred megabytes of storage.

diskdef macro library Library of code that when used with MAC, the Digital Research macro assembler, creates disk definition tables such as the **DPB** and **DPH** automatically.

disk drive Peripheral device that reads and writes information on disk. CP/M assigns a letter to each drive under its control. For example, CP/M may refer to the drives in a four-drive system as A, B, C, and D.

disk parameter block (DPB) Data structure referenced by one or more disk parameter headers. The disk parameter block defines disk characteristics in the fields listed below:

SPT is the total number of sectors per track.
BSH is the data allocation block shift factor.
BLM is the data allocation block mask.
EXM is the extent mask determined by BLS and DSM.
DSM is the maximum data block number.
DRM is the maximum number of directory entries–1.
AL0 reserves directory blocks.
AL1 reserves directory blocks.
CKS is the number of directory sectors check summed.
OFF is the number of reserved system tracks.

The address of the disk parameter block is located in the disk parameter header at DPbase + 0AH. CP/M Function 31 returns the **DPB** address. Drives with the same characteristics can use the same

disk parameter header, and thus the same **DPB**. However, drives with different characteristics must each have their own disk parameter header and disk parameter blocks. When the BDOS calls the SELDSK entry point in the BIOS, SELDSK must return the address of the drive's disk parameter header in register HL.

disk parameter header (DPH) Data structure that contains information about the disk drive and provides a scratchpad area for certain BDOS operations. The disk parameter header contains six bytes of scratchpad area for the BDOS, and the following five 2-byte parameters:

XLT is the sector translation table address.
DIRBUF is the directory buffer address.
DPB is the disk parameter block address.
CSV is the checksum vector address.
ALV is the allocation vector address.

Given *n* disk drives, the disk parameter headers are arranged in a table whose first row of 16 bytes corresponds to drive 0, with the last row corresponding to drive $n - 1$.

DKS Parameter in the diskdef macro library specifying the number of data blocks on the drive.

DMA Direct Memory Access. DMA is a method of transferring data from the disk into memory directly. In a CP/M system, the BDOS calls the BIOS entry point READ to read a sector from the disk into the currently selected DMA address. The DMA address must be the address of a 128-byte buffer in memory, either the default buffer at 0080H in page zero, or a user-assigned buffer in the TPA. Similarly, the BDOS calls the BIOS entry point WRITE to write the record at the current DMA address to the disk.

DN Parameter in the diskdef macro library specifying the logical drive number.

DPB See disk parameter block.

DPH See disk parameter header.

DRM 2-byte parameter in the disk parameter block at $\mathrm{DPB} + 7$. **DRM** is one less than the total number of directory entries allowed for the drive. This value is related to **DPB** bytes **AL0** and **AL1**, which allocates up to 16 blocks for directory entries.

DSM 2-byte parameter of the disk parameter block at $\mathrm{DPB} + 5$. **DSM** is the maximum data block number supported by the drive. The product **BLS** times (**DSM**+1) is the total number of bytes held by the drive. This must not exceed the capacity of the physical disk less the reserved system tracks.

editor Utility program that creates and modifies text files. An editor can be used for creation of documents or creation of code for computer programs. The CP/M editor is invoked by typing the command ED next to the system prompt on the console.

EX Extent number field in an FCB. See extent.

executable Ready to be run by the computer. Executable code is a series of instructions that can be carried out by the computer. For example, the computer cannot execute names and addresses, but it can execute a program that prints all those names and addresses on mailing labels.

execute a program Start the processing of executable code.

EXM See extent mask.

extent 16K consecutive bytes in a file. Extents are numbered from `0` to `31`. One extent can contain `1`, `2`, `4`, `8`, or `16` blocks. **EX** is the extent number field of an FCB and is a one-byte field at $\mathrm{FCB}+12$, where FCB labels the first byte in the FCB. Depending on the block size (**BLS**) and the maximum data block number (**DSM**), an FCB can contain `1`, `2`, `4`, `8`, or `16` extents. The **EX** field is normally set to `0` by the user but contains the current extent number during file I/O. The term FCB folding describes FCBs containing more than one extent. In CP/M version 1.4, each FCB contained only one extent. Users attempting to perform random record I/O and maintain CP/M 1.4 compatibility should be aware of the implications of this difference. See CP/M 1.4 compatibility

extent mask (EXM) A byte parameter in the disk parameter block located at $\mathrm{DPB}+3$. The value of **EXM** is determined by the block size (**BLS**) and whether the maximum data block number (**DSM**) exceeds 255. There are $\mathrm{EXM}+1$ extents per directory FCB.

FCB See File Control Block.

file Collection of characters, instructions, or data that can be referenced by a unique identifier. Files are usually stored on various types of media, such as disk, or magnetic tape. A CP/M file is identified by a file specification and resides on disk as a collection of from zero to 65,536 records. Each record is 128 bytes and can contain either binary or ASCII data. Binary files contain bytes of data that can vary in value from 0H to 0FFH. ASCII files contain sequences of character codes delineated by a carriage return and line-feed combination; normally byte values range from 0H to 7FH. The directory maps the file as a series of physical blocks. Although files are defined as a sequence of consecutive logical records, these records can not reside in consecutive sectors on the disk. See also block, directory, extent, record, and sector.

File Control Block (FCB) Structure used for accessing files on disk. Contains the drive, filename, filetype, and other information describing a file to be accessed or created on the disk. A file control block consists of `36` consecutive bytes specified by the user for file I/O functions. FCB can also refer to a directory element in the directory portion of the allocated disk space. These contain the same first `32` bytes of the FCB, but lack the current record and random record number bytes.

filename Name assigned to a file. A filename can include a primary filename of one to eight characters; a filetype of zero to three characters. A period separates the primary filename from the filetype.

file specification Unique file identifier. A complete CP/M file specification includes a disk drive specification followed by a colon, *d*`:`, a primary filename of one to eight characters, a period, and a filetype of zero to three characters. For example, `b:example.tex` is a complete CP/M file specification.

filetype Extension to a filename. A filetype can be from zero to three characters and must be separated from the primary filename by a period. A filetype can tell something about the file. Some programs require that files to be processed have specific filetypes.

floppy disk Flexible magnetic disk used to store information. Floppy disks come in 5 1/4- and 8-inch diameters.

FSC Parameter in the `diskdef` macro library specifying the first physical sector number. This parameter is used to determine **SPT** and build **XLT**.

hard disk Rigid, platter-like, magnetic disk sealed in a container. A hard disk stores more information than a floppy disk.

hardware Physical components of a computer.

hexadecimal notation Notation for base 16 values using the decimal digits and letters A, B, C, D, E, and F to represent the 16 digits. Hexadecimal notation is often used to refer to binary numbers. A binary number can be easily expressed as a hexadecimal value by taking the bits in groups of 4, starting with the least significant bit, and expressing each group as a hexadecimal digit, 0-F. Thus the bit value 1011 becomes 0BH and 10110101 becomes 0B5H.

hex file ASCII-printable representation of a command, machine language file.

hex file format Absolute output of ASM and MAC for the Intel 8080 is a hex format file, containing a sequence of absolute records that give a load address and byte values to be stored, starting at the load address.

HOME: BIOS entry point which sets the disk head of the currently selected drive to the track zero position.

host Physical characteristics of a hard disk drive in a system using the blocking and deblocking algorithm. The term, host, helps distinguish physical hardware characteristics from CP/M's logical characteristics. For example, CP/M sectors are always 128 bytes, although the host sector size can be a multiple of 128 bytes.

input Data going into the computer, usually from an operator typing at the terminal or by a program reading from the disk.

input/output See I/O

interface Object that allows two independent systems to communicate with each other, as an interface between hardware and software in a microcomputer.

I/O Abbreviation for input/output. Usually refers to input/output operations or routines handling the input and output of data in the computer system.

IOBYTE A one-byte field in page zero, currently at location 0003H, that can support a logical-to-physical device mapping for I/O. However, its implementation in your BIOS is purely optional and might or might not be supported in a given CP/M system. The IOBYTE is easily set using the command:

STAT *logical device = physical device*

The CP/M logical devices are CON:, RDR:, PUN:, and LST:; each of these can be assigned to one of four physical devices. The IOBYTE can be initialized by the BOOT entry point of the BIOS and interpreted by the BIOS I/O entry points CONST, CONIN, CONOUT, LIST, PUNCH, and READER. Depending on the setting of the IOBYTE, different I/O drivers can be selected by the BIOS. For example, setting LST:=TTY: might cause LIST output to be directed to a serial port, while setting LST:=LPT: causes LIST output to be directed to a parallel port.

K Abbreviation for kilobyte. See kilobyte.

keyword See command keyword.

kilobyte (K) 1024 bytes or 0400H bytes of memory. This is a standard unit of memory. For example, the Intel 8080 supports up to 64K of memory address space or 65,536 bytes. 1024 kilobytes equal one megabyte, or over one million bytes.

linker Utility program used to combine relocatable object modules into an absolute file ready for execution. For example, LINK-80

creates either a COM or PRL file from relocatable REL files, such as those produced by PL/I-80

LIST: A BIOS entry point to a routine that sends a character to the list device, usually a printer.

list device Device such as a printer onto which data can be listed or printed.

LISTST: BIOS entry point to a routine that returns the ready status of the list device.

loader Utility program that brings an absolute program image into memory ready for execution under the operating system, or a utility used to make such an image. For example, LOAD prepares an absolute COM file from the assembler hex file output that is ready to be executed under CP/M.

logged in Made known to the operating system, in reference to drives. A drive is logged in when it is selected by the user or an executing process. It remains selected or logged in until you change disks in a floppy disk drive or enter CTRL-C at the command level, or until a BDOS Function 0 is executed.

logical Representation of something that might or might not be the same in its actual physical form. For example, a hard disk can occupy one physical drive, yet you can divide the available storage on it to appear to the user as if it were in several different drives. These apparent drives are the logical drives.

logical sector See sector.

logical-to-physical sector translation table See **XLT**.

LSC: diskdef macro library parameter specifying the last physical

sector number.

LST: Logical CP/M list device, usually a printer. The CP/M list device is an output-only device referenced through the LIST and LISTST entry points of the BIOS. The STAT command allows assignment of LST: to one of the physical devices: TTY:, CRT:, LPT:, or UL1:, provided these devices and the IOBYTE are implemented in the LIST and LISTST entry points of your CP/M BIOS module. The CP/NET command NETWORK allows assignment of LST: to a list device on a network master. For example,
PIP LST:=TEST.SUB
prints the file TEST.SUB on the list device.

macro assembler Assembler code translator providing macro processing facilities. Macro definitions allow groups of instructions to be stored and substituted in the source program as the macro names are encountered. Definitions and invocations can be nested and macro parameters can be formed to pass arbitrary strings of text to a specific macro for substitution during expansion.

megabyte Over one million bytes; 1024 kilobytes. See byte and kilobyte.

microprocessor Silicon chip that is the central processing unit (CPU) of the microcomputer. The Intel 8080 and the Zilog Z80 are microprocessors commonly used in CP/M systems.

MOVCPM image Memory image of the CP/M system created by MOVCPM. This image can be saved as a disk file using the SAVE command or placed on the system tracks using the SYSGEN command without specifying a source drive. This image varies, depending on the presence of a one-sector or two-sector boot. If the boot is less than 128 bytes (one sector), the boot begins at 0900H, the CP/M system at 0980H, and the BIOS at 1F80H. Otherwise, the boot is at 0900H, the CP/M system at 1000H, and the BIOS at 2000H. In a CP/M 1.4 system with a one-sector boot, the addresses are the same as for the CP/M 2 system – except that the BIOS begins at 1E80H instead of 1F80H.

MP/M Multi-Programming Monitor control program. A microcomputer operating system supporting multi-terminal access with multi- programming at each terminal.

multi-programming The capability of initiating and executing more than one program at a time. These programs, usually called processes, are time-shared, each receiving a slice of CPU time on a round-robin basis. See concurrency.

nibble One half of a byte, usually the high-order or low-order 4 bits in a byte.

OFF: Two-byte parameter in the disk parameter block at $\mathrm{DPB} + 13$ bytes. This value specifies the number of reserved system tracks. The disk directory begins in the first sector of track OFF.

OFS: diskdef macro library parameter specifying the number of reserved

system tracks. See OFF:.

operating system Collection of programs that supervises the execution of other programs and the management of computer resources. An operating system provides an orderly input/output environment between the computer and its peripheral devices. It enables user-written programs to execute safely. An operating system standardizes the use of computer resources for the programs running under it.

option One of many parameters that can be part of a command tail. Use options to specify additional conditions for a command's execution.

output Data that is sent to the console, disk, or printer.

page 256 consecutive bytes in memory beginning on a page boundary, whose base address is a multiple of 256 (100H) bytes. In hex notation, pages always begin at an address with a least significant byte of zero.

page relocatable program See PRL.

page zero Memory region between 0000H and 0100H used to hold critical system parameters. Page zero functions primarily as an interface region between user programs and the CP/M BDOS module. Note that in non-standard systems this region is the base page of the system and represents the first 256 bytes of memory used by the CP/M system and user programs running under it.

parameter Value in the command tail that provides additional information for the command. Technically, a parameter is a required element of a command.

peripheral devices Devices external to the CPU. For example, terminals, printers, and disk drives are common peripheral devices that are not part of the processor but are used in conjunction with it.

physical Characteristic of computer components, generally hardware, that actually exist. In programs, physical components can be represented by logical components.

primary filename First 8 characters of a filename. The primary filename is a unique name that helps the user identify the file contents. A primary filename contains one to eight characters and can include any letter or number and some special characters. The primary filename follows the optional drive specification and precedes the optional filetype.

PRL Page relocatable program. A page relocatable program is stored on disk with a PRL filetype. Page relocatable programs are easily relocated to any page boundary and thus are suitable for execution in a non-banked MP/M system.

program Series of coded instructions that performs specific tasks when executed by a computer. A program can be written in a processor-specific language or a high-level language that can be implemented on a number of different processors.

prompt Any characters displayed on the video screen to help the user decide what the next appropriate action is. A system prompt is a special prompt displayed by the operating system. The alphabetic character indicates the default drive. Some applications programs have their own special prompts. See CP/M prompt.

PUN Logical CP/M punch device. The punch device is an output-only device accessed through the PUNCH entry point of the BIOS. In certain implementations, PUN: can be a serial device such as a modem.

PUNCH: BIOS entry point to a routine that sends a character to the punch device.

RDR: Logical CP/M reader device. The reader device is an input-only device accessed through the READER entry point in the BIOS. See PUN:.

READ: Entry point in the BIOS to a routine that reads 128 bytes from the currently selected drive, track, and sector into the current DMA address.

READER: Entry point to a routine in the BIOS that reads the next character from the currently assigned reader device.

Read-Only (R/O) Attribute that can be assigned to a disk file or a disk drive. When assigned to a file, the Read-Only attribute allows you to read from that file but not write to it. When assigned to a drive, the Read-Only attribute allows you to read any file on the disk, but prevents you from adding a new file, erasing or changing a file, renaming a file, or writing on the disk. The STAT command can set a file or a drive to Read-Only. Every file and drive is either Read-Only or Read-Write. The default setting for drives and files is Read-Write, but an error in resetting the disk or changing media automatically sets the drive to Read-Only until the error is corrected. See also ROM.

Read-Write (R/W) Attribute that can be assigned to a disk file or a disk drive. The Read-Write attribute allows you to read from and write to a specific Read-Write file or to any file on a disk that is in a drive set to Read-Write. A file or drive can be set to either Read-Only or Read-Write.

record Group of bytes in a file. A physical record consists of 128 bytes and is the basic unit of data transfer between the operating system and the application program. A logical record might vary in length and is used to represent a unit of information. Two 64-byte employee records can be stored in one 128-byte physical record. Records are grouped together to form a file.

recursive-font procedure Code that can call itself during execution.

reentrant procedure Code that can be called by one process while another is already executing it. Thus, reentrant code can be shared between different users. Reentrant procedures must not be self-modifying; that is, they must be pure code and not contain data. The data for reentrant procedures can be kept in a separate data area or placed on the stack.

restart (RST) One-byte call instruction usually used during interrupt sequences and for debugger break pointing. There are eight restart locations, RST 0 through RST 7, whose addresses are given by the product of 8 times the restart number.

R/O See Read-Only

ROM Read-Only Memory. This memory can be read but not written and so is suitable for code and pre-initialized data areas only.

RST See restart.

R/W See Read-Write.

sector In a CP/M system, a sector is always 128 consecutive bytes. A sector is the basic unit of data read and written on the disk by the BIOS. A sector can be one 128-byte record in a file or a sector of the directory. The BDOS always requests a logical sector number between 0 and $(\mathrm{SPT}-1)$. This is typically translated into a physical sector by the BIOS entry point SECTRAN. In some disk subsystems, the disk sector size is larger than 128 bytes, usually a power of two, such as 256, 512, 1024, or 2048 bytes. These disk sectors are always referred to as host sectors in CP/M documentation and should not be confused with other references to sectors, in which cases the CP/M 128-byte sectors should be assumed. When the host sector size is larger than 128 bytes, host sectors must be buffered in memory and the 128-byte CP/M sectors must be blocked and deblocked from them. This can be done by adding an additional module, the blocking and deblocking algorithm, between the BIOS disk I/O routines and the actual disk I/O.

SECTRAN: Entry point to a routine in the BIOS that performs logical-to-physical sector translation for the BDOS.

SELDSK: Entry point to a routine in the BIOS that sets the currently selected drive.

SETDMA: Entry point to a routine in the BIOS that sets the currently selected DMA address. The DMA address is the address of a 128-byte buffer region in memory that is used to transfer data to and from the disk in subsequent reads and writes.

SETSEC: Entry point to a routine in the BIOS that sets the currently selected sector.

SETTRK: Entry point to a routine in the BIOS that sets the currently selected track.

skew factor Factor that defines the logical-to-physical sector number translation in **XLT**. Logical sector numbers are used by the BDOS and range between 0 and $(\mathrm{SPT}-1)$. Data is written in consecutive logical 128-byte sectors grouped in data blocks. The number of sectors per block is given by $\frac{\mathrm{BLS}}{128}$. Physical sectors on the disk media are also numbered consecutively. If the physical sector size is also 128 bytes, a one-to-one relationship exists between logical and physical sectors. The logical-to-physical translation table (**XLT**) maps this relationship, and a skew factor is typically used in generating the table entries. For instance, if the skew factor is 6, **XLT** will be:

Logical:	0	1	2	3	4	5	6	...	25
Physical:	1	7	13	19	25	5	11	...	22

The *skew factor* allows time for program processing without missing the next sector. Otherwise, the system must wait for an entire disk revolution before reading the next logical sector. The skew factor can be varied, depending on hardware speed and application processing overhead. Note that no sector translation is done when the physical sectors are larger than 128 bytes, as sector deblocking is done in this case. See also sector, SKF:, and **XLT**.

SKF: A diskdef macro library parameter specifying the skew factor to be used in building **XLT**. If SKF is zero, no translation table is generated and the **XLT** bytes in the **DPH** will be 0000H.

software Programs that contain machine-readable instructions, as opposed to hardware, which is the actual physical components of a computer.

source file ASCII text file usually created with an editor that is an input file to a system program, such as a language translator or text formatter.

SP Stack pointer. See stack.

spooling Process of accumulating printer output in a file while the printer is busy. The file is printed when the printer becomes free; a program does not have to wait for the slow printing process.

SPT: See sectors per track.

stack Reserved area of memory where the processor saves the return address when a call instruction is received. When a return instruction is encountered, the processor restores the current address on the stack to the program counter. Data such as the contents of the registers can also be saved on the stack. The push instruction places data on the stack and the pop instruction removes it. An item is pushed onto the stack by decrementing the stack pointer (SP) by 2 and writing the item at the SP address. In other words, the stack grows downward in memory.

syntax Format for entering a given command.

SYS: See system attribute.

SYSGEN image Memory image of the CP/M system created by SYSGEN when a destination drive is not specified. This is the same as the MOVCPM image that can be read by SYSGEN if a source drive is not specified. See MOVCPM image.

system attribute (SYS) File attribute. You can give a file the system attribute by using the SYS option in the STAT command or by using the set file attributes function, BDOS Function 12. A file with the SYS attribute is not displayed in response to a DIR command. If you give a file with user number 0 the SYS attribute, you can read and execute that file from any user number on the same drive. Use this feature to make your commonly used programs available under any user number.

system prompt Symbol displayed by the operating system indicating that the system is ready to receive input. See prompt and CP/M prompt.

system tracks Tracks reserved on the disk for the CP/M system. The number of system tracks is specified by the parameter OFF in the disk parameter block (**DPB**). The system tracks for a drive always precede its data tracks. The command SYSGEN copies the CP/M system from the system tracks to memory, and vice versa. The standard SYSGEN utility copies 26 sectors from track 0 and 26 sectors from track 1. When the system tracks contain additional sectors or tracks to be copied, a customized SYSGEN must be used.

terminal See console.

TPA Transient Program Area. Area in memory where user programs run and store data. This area is a region of memory beginning at 0100H and extending to the base of the CP/M system in high memory. The first module of the CP/M system is the CCP, which can be overwritten by a user program. If so, the TPA is extended to the base of the CP/M BDOS module. If the CCP is overwritten, the user program must terminate with either a system reset (Function 0) call or a jump to location zero in page zero. The address of the base of the CP/M BDOS is stored in location 0006H in page zero least significant byte first.

track Data on the disk media is accessed by combination of track and sector numbers. Tracks form concentric rings on the disk; the standard IBM single-density disks have 77 tracks. Each track consists of a fixed number of numbered sectors. Tracks are numbered from zero to one less than the number of tracks on the disk.

Transient Program Area See TPA.

upward compatible Term meaning that a program created for the previously released operating system, or compiler, runs under the newly released version of the same operating system.

USER Term used in CP/M and MP/M systems to distinguish distinct regions of the directory.

user number Number assigned to files in the disk directory so that different users need only deal with their own files and have their own directories, even though they are all working from the same disk. In CP/M, files can be divided into 16 user groups.

utility Tool. Program that enables the user to perform certain operations, such as copying files, erasing files, and editing files. The utilities are created for the convenience of programmers and users.

vector Location in memory. An entry point into the operating system used for making system calls or interrupt handling.

warm start Program termination by a jump to the warm start vector at location 0000H, a system reset (BDOS Function 0), or a CTRL-C typed at the keyboard. A warm start re-initializes the disk subsystem and returns control to the CP/M operating system at the CCP level. The warm start vector is simply a jump to the WBOOT entry point in the BIOS.

WBOOT: Entry point to a routine in the BIOS used when a warm start occurs. A warm start is performed when a user program branches to location 0000H, when the CPU is reset from the front panel, or when the user types CTRL-C. The CCP and BDOS are reloaded from the system tracks of drive A.

wildcard characters Special characters that match certain specified items. In CP/M there are two wildcard characters: ? and *. The ? can be substituted for any single character in a filename, and the * can be substituted for the primary filename, the filetype, or both. By placing wildcard characters in filenames, the user creates an ambiguous filename and can quickly reference one or more files.

word 16-bit or two-byte value, such as an address value. Although the Intel 8080 is an 8-bit CPU, addresses occupy two bytes and are called word values.

WRITE: Entry point to a routine in the BIOS that writes the record at the currently selected DMA address to the currently selected drive, track, and sector.

XLT Logical-to-physical sector translation table located in the BIOS. SECTRAN uses **XLT** to perform logical-to-physical sector number translation. **XLT** also refers to the two-byte address in the disk parameter header at $\mathrm{DPBASE} + 0$. If this parameter is zero, no sector translation takes place. Otherwise this parameter is the address of the translation table.

ZERO PAGE See Page zero.

I. CP/M Error Messages

Messages come from several different sources. CP/M displays error messages when there are errors in calls to the Basic Disk Operating System (BDOS). CP/M also displays messages when there are errors in command lines. Each utility supplied with CP/M has its own set of messages. The following lists CP/M messages and utility messages. One might see messages other than those listed here if one is running an application program. Check the application program's documentation for explanations of those messages.

Message	Meaning
`?`	DDT. This message has four possible meanings: • DDT does not understand the assembly language instruction. • The file cannot be opened. • A checksum error occurred in a HEX file. • The assembler/disassembler was overlaid.
`ABORTED`	`PIP`. You stopped a `PIP` operation by pressing a key.
`ASM` Error Messages	
`D`	Data error: data statement element cannot be placed in specified data area.
`E`	Expression error: expression cannot be evaluated during assembly.
`L`	Label error: label cannot appear in this context (might be duplicate label).
`N`	Not implemented: unimplemented features, such as macros, are trapped.
`O`	Overflow: expression is too complex to evaluate.
`P`	Phase error: label value changes on two passes through assembly.
`R`	Register error: the value specified as a register is incompatible with the code.
`S`	Syntax error: improperly formed expression.
`U`	Undefined label: label used does not exist.
`V`	Value error: improperly formed operand encountered in an expression.
`BAD DELIMITER`	STAT. Check command line for typing errors.

Message	Meaning
`Bad Load`	CCP error message, or `SAVE` error message.
`BDOS Err on` *d*`:`	Basic Disk Operating System error on the designated drive: CP/M replaces *d*: with the drive specification of the drive where the error occurred. This message is followed by one of the four phrases in the situations described below.
`BDOS Err on` *d*`: Bad Sector`	This message appears when CP/M finds no disk in the drive, when the disk is improperly formatted, when the drive latch is open, or when power to the drive is off. Check for one of these situations and try again. This could also indicate a hardware problem or a worn or improperly formatted disk. Press ^C to terminate the program and return to CP/M, or press RETURN to ignore the error.
`BDOS Err on` *d*`: File R/O`	You tried to erase, rename, or set file attributes on a Read-Only file. The file should first be set to Read-Write (R/W) with the command: `STAT` *filespec* `$R/W`.
`BDOS Err on` *d*`: R/O`	Drive has been assigned Read-Only status with a `STAT` command, or the disk in the drive has been changed without being initialized with a ^C. CP/M terminates the current program as soon as you press any key.
`BDOS Err on` *d*`: Select`	CP/M received a command line specifying a nonexistent drive. CP/M terminates the current program as soon as you press any key. Press RETURN or CTRL-C to recover.
`Break "x" at` *c*	ED. "*x*" is one of the symbols described below and *c* is the command letter being executed when the error occurred.
`#`	Search failure. `ED` cannot find the string specified in an `F`, `S`, or `N` command.
`?`	Unrecognized command letter *c*. `ED` does not recognize the indicated command letter, or an `E`, `H`, `Q`, or `O` command is not alone on its command line.

Message	Meaning
O	The file specified in an R command cannot be found.
Break "x" at *c* (continued)	
>	Buffer full. ED cannot put any more characters in the memory buffer, or the string specified in an F, N, or S command is too long.
E	Command aborted. A keystroke at the console aborted command execution.
F	Disk or directory full. This error is followed by either the disk or directory full message. Refer to the recovery procedures listed under these messages.
CANNOT CLOSE DESTINATION FILE *filespec*	PIP. An output file cannot be closed. You should take appropriate action after checking to see if the correct disk is in the drive and that the disk is not write-protected.
Cannot close, R/O CANNOT CLOSE FILES	CP/M cannot write to the file. This usually occurs because the disk is write-protected. ASM. An output file cannot be closed. This is a fatal error that terminates ASM execution. Check to see that the disk is in the drive, and that the disk is not write-protected. DDT. The disk file written by a W command cannot be closed. This is a fatal error that terminates DDT execution. Check if the correct disk is in the drive and that the disk is not write-protected. SUBMIT. This error can occur during SUBMIT file processing. Check if the correct system disk is in the A drive and that the disk is not write-protected. The SUBMIT job can be restarted after rebooting CP/M.
CANNOT READ	PIP. PIP cannot read the specified source. Reader cannot be implemented.
CANNOT WRITE	PIP. The destination specified in the PIP command is illegal. You probably specified an input device as a destination.

Message	Meaning
`Checksum Error`	PIP. A HEX record checksum error was encountered. The HEX record that produced the error must be corrected, probably by recreating the HEX file.
`CHECKSUM ERROR LOAD ADDRESS`	LOAD. File contains incorrect data. Regenerate HEX file from the source.
`Command Buffer Overflow`	SUBMIT. The SUBMIT buffer allows up to 2048 characters in the input file.
`Command too long`	SUBMIT. A command in the SUBMIT file cannot exceed 125 characters.
`CORRECT ERROR, TYPE RETURN OR CTRL-Z`	PIP. A HEX record checksum was encountered during the transfer of a HEX file. The HEX file with the checksum error should be corrected, probably by recreating the HEX file.
`DESTINATION IS R/O, DELETE (Y/N)?`	PIP. The destination file specified in a PIP command already exists and it is Read-Only. If you type Y, the destination file is deleted before the file copy is done.
`Directory Full`	ED. There is not enough directory space for the file being written to the destination disk. You can use the OX*filespec* command to erase any unnecessary files on the disk without leaving the editor. SUBMIT. There is not enough directory space to write the $$$.SUB file used for processing SUBMITs. Erase some files or select a new disk and retry.
`Disk Full`	ED. There is not enough disk space for the output file. This error can occur on the W, E, H, or X commands. If it occurs with X command, you can repeat the command prefixing the filename with a different drive.
`DISK READ ERROR`	

Message	Meaning
	PIP. The input disk file specified in a PIP command cannot be read properly. This is usually the result of an unexpected end-of-file. Correct the problem in your file.
DISK WRITE ERROR	DDT. A disk write operation cannot be successfully performed during a W command, probably due to a full disk. You should either erase some unnecessary files or get another disk with more space. PIP. A disk write operation cannot be successfully performed during a PIP command, probably due to a full disk. You should either erase some unnecessary files or get another disk with more space and execute PIP again. SUBMIT. The SUBMIT program cannot write the $$$.SUB file to the disk. Erase some files, or select a new disk and try again.
ERROR: BAD PARAMETER	PIP. You entered an illegal parameter in a PIP command. Retype the entry correctly.
ERROR: CANNOT OPEN SOURCE, LOAD ADDRESS *hhhh*	LOAD. Displayed if LOAD cannot find the specified file or if no filename is specified.
ERROR: CANNOT CLOSE FILE, LOAD ADDRESS *hhhh*	LOAD. Caused by an error code returned by a BDOS function call. Disk might be write-protected.
ERROR: CANNOT OPEN SOURCE, LOAD ADDRESS *hhhh*	LOAD. Cannot find source file. Check disk directory.
ERROR: DISK READ, LOAD ADDRESS *hhhh*	LOAD. Caused by an error code returned by a BDOS function call.
ERROR: DISK WRITE, LOAD ADDRESS *hhhh*	LOAD. Destination disk is full.
ERROR: INVERTED LOAD ADDRESS, LOAD ADDRESS *hhhh*	LOAD. The address of a record was too far from the address of the previously-processed record. This is an internal limitation of LOAD, but it can be circumvented. Use DDT to read

Message	Meaning
	the HEX file into memory, then use a SAVE command to store the memory image file on disk.
`ERROR: NO MORE DIRECTORY SPACE, LOAD ADDRESS` *hhhh*	
	LOAD. Disk directory is full.
`Error on line` *nnn message*	
	SUBMIT. The SUBMIT program displays its messages in the format shown above, where *nnn* represents the line number of the SUBMIT file. Refer to the *message* following the line number.
`FILE ERROR`	
	ED. Disk or directory is full, and ED cannot write anything more on the disk. This is a fatal error, so make sure there is enough space on the disk to hold a second copy of the file before invoking ED.
`FILE EXISTS`	
	You have asked CP/M to create or rename a file using a file specification that is already assigned to another file. Either delete the existing file or use another file specification. REN. The new name specified is the name of a file that already exists. You cannot rename a file with the name of an existing file. If you want to replace an existing file with a newer version of the same file, either rename or erase the existing file, or use the PIP utility.
`File exists, erase it`	
	ED. The destination filename already exists when you are placing the destination file on a different disk than the source. It should be erased or another disk selected to receive the output file.
`** FILE IS READ/ONLY **`	
	ED. The file specified in the command to invoke ED has the Read-Only attribute. ED can read the file so that the user can examine it, but ED cannot change a Read-Only file.
`File Not Found`	
	CP/M cannot find the specified file. Check that you have entered the correct drive specification or that you have the correct disk in the drive.

Message	Meaning
	ED. ED cannot find the specified file. Check that you have entered the correct drive specification or that you have the correct disk in the drive.
File Not Found (continued)	STAT. STAT cannot find the specified file. The message might appear if you omit the drive specification. Check to see if the correct disk is in the drive.
FILE NOT FOUND	PIP. An input file that you have specified does not exist.
Filename required	ED. You typed the ED command without a filename. Reenter the ED command followed by the name of the file you want to edit or create.
hhhh??=*dd*	DDT. The ?? indicates DDT does not know how to represent the hexadecimal value *dd* encountered at address *hhhh* in 8080 assembly language. *dd* is not an 8080 machine instruction opcode.
Insufficient Memory	DDT. There is not enough memory to load the file specified in an R or E command.
Invalid Assignment	STAT. You specified an invalid drive or file assignment, or misspelled a device name. This error message might be followed by a list of the valid file assignments that can follow a filename. If an invalid drive assignment was attempted the message Use: d:=RO is displayed, showing the proper syntax for drive assignments.
Invalid control character	SUBMIT. The only valid control characters in the SUBMIT files of the type SUB are ^A through ^Z. Note that in a SUBMIT file the control character is represented by typing the circumflex, ^, not by pressing the control key.
INVALID DIGIT	PIP. An invalid HEX digit has been encountered while reading a HEX file. The HEX file with the invalid HEX digit should be corrected, probably by recreating the HEX file.

Message	Meaning
`Invalid Disk Assignment`	`STAT`. Might appear if you follow the drive specification with anything except `=R/O`.
`INVALID DISK SELECT`	CP/M received a command line specifying a nonexistent drive, or the disk in the drive is improperly formatted. CP/M terminates the current program as soon as you press any key.
`INVALID DRIVE NAME (Use A, B, C, or D)`	`SYSGEN`. `SYSGEN` recognizes only drives `A`, `B`, `C`, and `D` as valid destinations for system generation.
`Invalid File Indicator`	`STAT`. Appears if you do not specify `RO`, `RW`, `DIR`, or `SYS`.
`INVALID FORMAT`	`PIP`. The format of your `PIP` command is illegal. See the description of the `PIP` command.
`INVALID HEX DIGIT`	`LOAD`. File contains incorrect HEX digit.
`INVALID MEMORY SIZE`	`MOVCPM`. Specify a value less than `64K` or your computer's actual memory size.
`INVALID SEPARATOR`	`PIP`. You have placed an invalid character for a separator between two input filenames.
`INVALID USER NUMBER`	`PIP`. You have specified a user number greater than `15`. User numbers are in the range `0` to `15`.
n`?`	

Message	Meaning
	USER. You specified a number greater than fifteen for a user area number. For example, if you type USER 18<cr>, the screen displays 18?.
NO DIRECTORY SPACE	
	ASM. The disk directory is full. Erase some files to make room for PRN and HEX files. The directory can usually hold only 64 filenames.
NO DIRECTORY SPACE *filespec*	
	PIP. There is not enough directory space for the output file. You should either erase some unnecessary files or get another disk with more directory space and execute PIP again.
NO FILE	
	DIR, ERA, REN, PIP. CP/M cannot find the specified file, or no files exist. ASM. The indicated source or include file cannot be found on the indicated drive. DDT. The file specified in an R or E command cannot be found on the disk.
NO INPUT FILE PRESENT ON DISK	
	DUMP. The file you requested does not exist.
No memory	
	There is not enough (buffer?) memory available for loading the program specified.
NO SOURCE FILE ON DISK	
	SYSGEN. SYSGEN cannot find CP/M either in CPMxx.com form or on the system tracks of the source disk.
NO SOURCE FILE PRESENT	
	ASM. The assembler cannot find the file you specified. Either you mistyped the file specification in your command line, or the filetype is not ASM.
NO SPACE	
	SAVE. Too many files are already on the disk, or no room is left on the disk to save the information.

Message	Meaning
`No SUB file Present`	`SUBMIT`. For `SUBMIT` to operate properly, you must create a file with filetype of `SUB`. The `SUB` file contains usual CP/M commands. Use one command per line.
`NOT A CHARACTER SOURCE`	`PIP`. The source specified in your `PIP` command is illegal. You have probably specified an output device as a source.
`** NOT DELETED **`	`PIP`. `PIP` did not delete the file, which might have had the `R/O` attribute.
`NOT FOUND`	`PIP`. `PIP` cannot find the specified file.
`OUTPUT FILE WRITE ERROR`	`ASM`. You specified a write-protected disk as the destination for the `PRN` and `HEX` files, or the disk has no space left. Correct the problem before assembling your program.
`Parameter error`	`SUBMIT`. Within the `SUBMIT` file of type `SUB`, valid parameters are `$0` through `$9`.
`PARAMETER ERROR, TYPE RETURN TO IGNORE`	`SYSGEN`. If you press RETURN, `SYSGEN` proceeds without processing the invalid parameter.
`QUIT NOT FOUND`	`PIP`. The string argument to a `Q` parameter was not found in your input file.
`Read error`	`TYPE`. An error occurred when reading the file specified in the type command. Check the disk and try again. The `STAT` *filespec* command can diagnose trouble.
`READER STOPPING`	`PIP`. Reader operation interrupted.

Message	Meaning
`Record Too Long`	`PIP`. `PIP` cannot process a record longer than 128 bytes.
`Requires CP/M 2.0 or later`	`XSUB`. `XSUB` requires the facilities of CP/M 2.0 or newer version.
`Requires CP/M 2.0 or new for operation`	`PIP`. This version of `PIP` requires the facilities of CP/M 2.0 or newer version.
`START NOT FOUND`	`PIP`. The string argument to an `S` parameter cannot be found in the source file.
`SOURCE FILE INCOMPLETE`	`SYSGEN`. `SYSGEN` cannot use your CP/M source file.
`SOURCE FILE NAME ERROR`	`ASM`. When you assemble a file, you cannot use the wildcard characters `*` and `?` in the filename. Only one file can be assembled at a time.
`SOURCE FILE READ ERROR`	`ASM`. The assembler cannot understand the information in the file containing the assembly-language program. Portions of another file might have been written over your assembly-language file, or information was not properly saved on the disk. Use the `TYPE` command to locate the error. Assembly-language files contain the letters, symbols, and numbers that appear on your keyboard. If your screen displays unrecognizable output or behaves strangely, you have found where computer instructions have crept into your file.
`SYNCHRONIZATION ERROR`	`MOVCPM`. The `MOVCPM` utility is being used with the wrong CP/M system.
`"SYSTEM" FILE NOT ACCESSIBLE`	You tried to access a file set to `SYS` with the `STAT` command.
`** TOO MANY FILES **`	

Message	Meaning
	STAT. There is not enough memory for STAT to sort the files specified, or more than 512 files were specified.
UNEXPECTED END OF HEX FILE	PIP. An end-of-file was encountered prior to a termination HEX record. The HEX file without a termination record should be corrected, probably by recreating the HEX file.
Unrecognized Destination	PIP. Check command line for valid destination.
Use: STAT d:=RO	STAT. An invalid STAT drive command was given. The only valid drive assignment in STAT is STAT *d*:=RO.
VERIFY ERROR	PIP. When copying with the V option, PIP found a difference when rereading the data just written and comparing it to the data in its memory buffer. Usually this indicates a failure of either the destination disk or drive.
WRONG CP/M VERSION (REQUIRES 2.0)	
XSUB ACTIVE	SUBMIT. XSUB has been invoked.
XSUB ALREADY PRESENT	SUBMIT. XSUB is already active in memory.
Your input?	If CP/M cannot find the command you specified, it returns the command name you entered followed by a question mark. Check that you have typed the command line correctly, or that the command you requested exists as a .COM file on the default or specified disk.

Table 9-45: CP/M Error Messages

Notes on the Zcim Project Edition

The Zcim Project goal is to improve the accessibility of legacy Z80 CP/M era content. As part of the project, Jim Burlingame (*jb@samplx.org*) created a *Typst* version of this manual.

You can find out more about the project at its site z80cim.org. The sources of this document are available on GitHub.

The source material is from the Tim Olmstead Memorial Digital Research CP/M Library

- Bondwell CP/M Operating System Manual
- CP/M 2.2 MANUAL
- CP/M 2.2 PDF MANUAL
- CP/M 2.2 TEX Manual
- DDT - The CP/M Debugger
- ED - The CP/M Editor
- The CPM Assembler in PDF format.

The contents of the manual were edited using the Typst.app site.

The figures for Section 2 (*The CP/M Editor*) were recreated as SVG files using Inkscape.

License